

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل پانزدهم: Support Vector Machines and Machine Learning on Documents

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

15.1 Support vector machines: The linearly separable case

15.2 Extensions to the SVM model

15.2.1 Soft margin classification

15.2.2 Multiclass SVMs

15.2.3 Nonlinear SVMs

15.2.4 Experimental results

15.3 Issues in the classification of text documents

15.3.1 Choosing what kind of classifier to use

15.3.2 Improving classifier performance

15.4 Machine learning methods in ad hoc information retrieval

15.4.1 A simple example of machine-learned scoring

15.4.2 Result ranking by machine learning

15.5 Chapter 15 Summary

15 ماشین‌های بردار پشتیبان: یافتن مرز امن در دنیای دیجیتال

🚫 چالش کنترل محتوا در شبکه‌های اجتماعی: چگونه سخنان نفرت‌پراکنی را از گفتگوهای عادی جدا کنیم؟

تصور کنید در تیم کنترل محتوای پلتفرمی مانند متا (فیسبوک و اینستاگرام) کار می‌کنید و باید سیستمی طراحی کنید که به طور خودکار سخنان نفرت‌پراکنی را از گفتگوهای عادی جدا کند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا مرز بین این دو دسته بسیار ظریف و پیچیده است.

یک طبقه‌بند ساده ممکن است بگوید "اگر پست حاوی کلمه X است، نفرت‌پراکنی است". این رویکرد ساده اما ناکارآمد است، زیرا افراد می‌توانند با استفاده از کلمات یا استعاره‌های مختلف، همین پیام را با نیت‌های متفاوت بیان کنند. این یک طبقه‌بند با بایاس بالا است.

اینجاست که **ماشین بردار پشتیبان (SVM)** وارد می‌شود. SVM به دنبال یک مرز تصمیم نیست، بلکه به دنبال **امن‌ترین مرز ممکن** است.

SVM یک طبقه‌بند خطی است که هدف آن یافتن یک مرز تصمیم (هایپرپلن) است که به حداکثر فاصله ممکن از تمام نقاط داده داشته باشد. این مفهوم به عنوان **حداکثر حاشیه** شناخته می‌شود.

🏠 استعاره خیابان: ایجاد امن‌ترین مرز ممکن

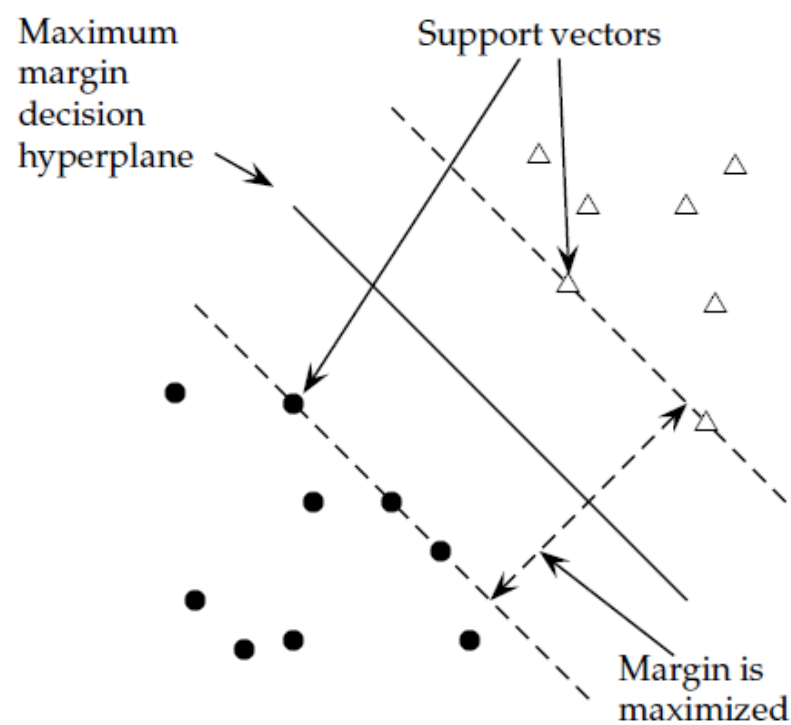
به جای تمرکز بر یافتن یک خط جداکننده، SVM مانند یک مهندس ترافیک عمل می‌کند که می‌خواهد **عریض‌ترین خیابان ممکن** را بین دو منطقه (کلاس‌ها) ایجاد کند. در این استعاره:

- **خیابان:** فضای بین دو کلاس که می‌خواهیم آن را به حداکثر برسانیم.
- **خط وسط خیابان:** مرز تصمیم (هایپرپلن) که دو کلاس را از هم جدا می‌کند.
- **لبه‌های خیابان:** نقاطی که به مرز تصمیم نزدیک‌تر هستند و تعیین‌کننده عرض خیابان هستند. این نقاط، **بردارهای پشتیبان** نام دارند.

با ایجاد عریض‌ترین خیابان ممکن، SVM یک طبقه‌بند بسیار مقاوم ایجاد می‌کند که کمتر تحت تأثیر داده‌های نویزی یا مرزی قرار می‌گیرد.

📐 شکل 15.1: بردارهای پشتیبان در مقابل حاشیه طبقه‌بند

شکل 15.1 این مفهوم را به صورت بصری نشان می‌دهد. نقاط سیاه، **بردارهای پشتیبان** هستند - آن‌ها نقاطی هستند که به مرز تصمیم (خط آبی) نزدیک‌تر هستند و تعیین‌کننده حاشیه طبقه‌بند هستند. خط آبی مرز تصمیمی است که SVM به عنوان بهینه پیدا می‌کند و حاشیه خاکستری، فاصله امن بین دو کلاس است.



► Figure 15.1 The support vectors are the 5 points right up against the margin of the classifier.

در مثال کنترل محتوا، بردارهای پشتیبان ممکن است پست‌هایی باشند که به مرز بین سخنان عادی و نفرت‌پرا بسیار نزدیک هستند - مانند استعاره‌های پیچیده یا کلماتی که در هر دو زمینه استفاده می‌شوند. این پست‌ها برای تعریف مرز امن حیاتی‌اند.

ریاضی اصلی SVM به صورت زیر است:

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1 \quad \text{for all } i$$

که در آن $\tilde{\mathbf{v}}(\mathbf{w})$ بردار نرمال وزن‌ها و b یک آستانه است. y_i برچسب سند i است (1 یا -1) و $\tilde{\mathbf{v}}(\mathbf{x}_i)$ بردار سند i است. محدودیت $y_i(\tilde{\mathbf{v}}(\mathbf{w})^T \tilde{\mathbf{v}}(\mathbf{x}_i) - b) \geq 1$ تضمین می‌کند که سند با حاشیه حداقل 1 به درستی طبقه‌بندی شود. هدف بهینه‌سازی، یافتن $\tilde{\mathbf{v}}(\mathbf{w})$ و b است که هم این محدودیت را برآورده کنند و همزمان اندازه $\tilde{\mathbf{v}}(\mathbf{w})$ را به حداقل برسانند.

🌀 مدیریت پیچیدگی: ترفند کرنل (Kernel Trick)

در دنیای واقعی، مرز بین سخنان عادی و نفرت‌پراکنی معمولاً یک خط ساده نیست. اینجاست که SVM با استفاده از **ترفند کرنل** قدرتمند می‌شود.

مثال واقعی: در سیستم کنترل محتوای یوتیوب، ممکن است کسی از طعنه استفاده کند تا پیام نفرت‌پراکنی کند. یک مرز خطی ساده نمی‌تواند این پیچیدگی را مدیریت کند. ترفند کرنل مانند این است که داده‌ها را به یک فضای با ابعاد بالاتر ببریم که در آن یک مرز خطی ساده می‌تواند آن‌ها را جدا کند، و سپس این مرز را به فضای اصلی بازگردانیم.

این ترفند به SVM اجازه می‌دهد تا مرزهای تصمیم غیرخطی پیچیده‌ای ایجاد کند که می‌توانند الگوهای زبانی پیچیده را مدیریت کنند. در طبقه‌بندی متن، کرنل تابع پایه شعاع (RBF) معمولاً کارآمدترین است.

SVM‌ها به دلایل متعددی در طبقه‌بندی متن موفق هستند:

- **کارایی بالا:** با ایجاد عریض‌ترین مرز ممکن، SVM‌ها معمولاً در برابر داده‌های جدید مقاوم‌تر عمل می‌کنند.
- **مقاومت در برابر ابعاد بالا:** در فضاهای با ابعاد بالا (مانند طبقه‌بندی متن با هزاران ویژگی)، احتمال جداپذیری خطی افزایش می‌یابد و SVM‌ها در این فضاها بسیار کارآمد هستند.

• **پایه تئوری قوی:** SVMها بر اساس بهینه‌سازی محدب ساخته شده‌اند که تئوری ریاضی قوی‌ای برای آن‌ها فراهم می‌کند.

در فصل‌های بعدی، خواهیم دید که چگونه می‌توان SVMها را برای مسائل غیرقابل تفکیک، مسائل چندکلاسی و کاربردهای عملی پیاده‌سازی کرد.

15.1 ماشین‌های بردار پشتیبان: حالت جداپذیر خطی

 **فیلتر اسپم جیمیل: چگونه ایمیل‌های مهم از تبلیغات مزاحم جدا می‌شوند؟**

تصور کنید در تیم امنیتی گوگل کار می‌کنید و مسئول بهبود فیلتر اسپم جیمیل هستید. هر روز میلیارد‌ها ایمیل ارسال می‌شود و باید سیستم شما بتواند به دقت ایمیل‌های اسپم را از ایمیل‌های مهم جدا کند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا مرز بین این دو دسته بسیار ظریف و پیچیده است.

یک طبقه‌بند ساده ممکن است بگوید "اگر ایمیل حاوی کلمه «خرید» است، اسپم است". این رویکرد ساده اما ناکارآمد است، زیرا ایمیل‌های معتبر زیادی نیز ممکن است حاوی این کلمه باشند (مانند ایمیل تأیید خرید از یک فروشگاه آنلاین).

اینجاست که **ماشین بردار پشتیبان (SVM)** وارد می‌شود. SVM به دنبال یک مرز تصمیم نیست، بلکه به دنبال **امن‌ترین مرز ممکن** است.

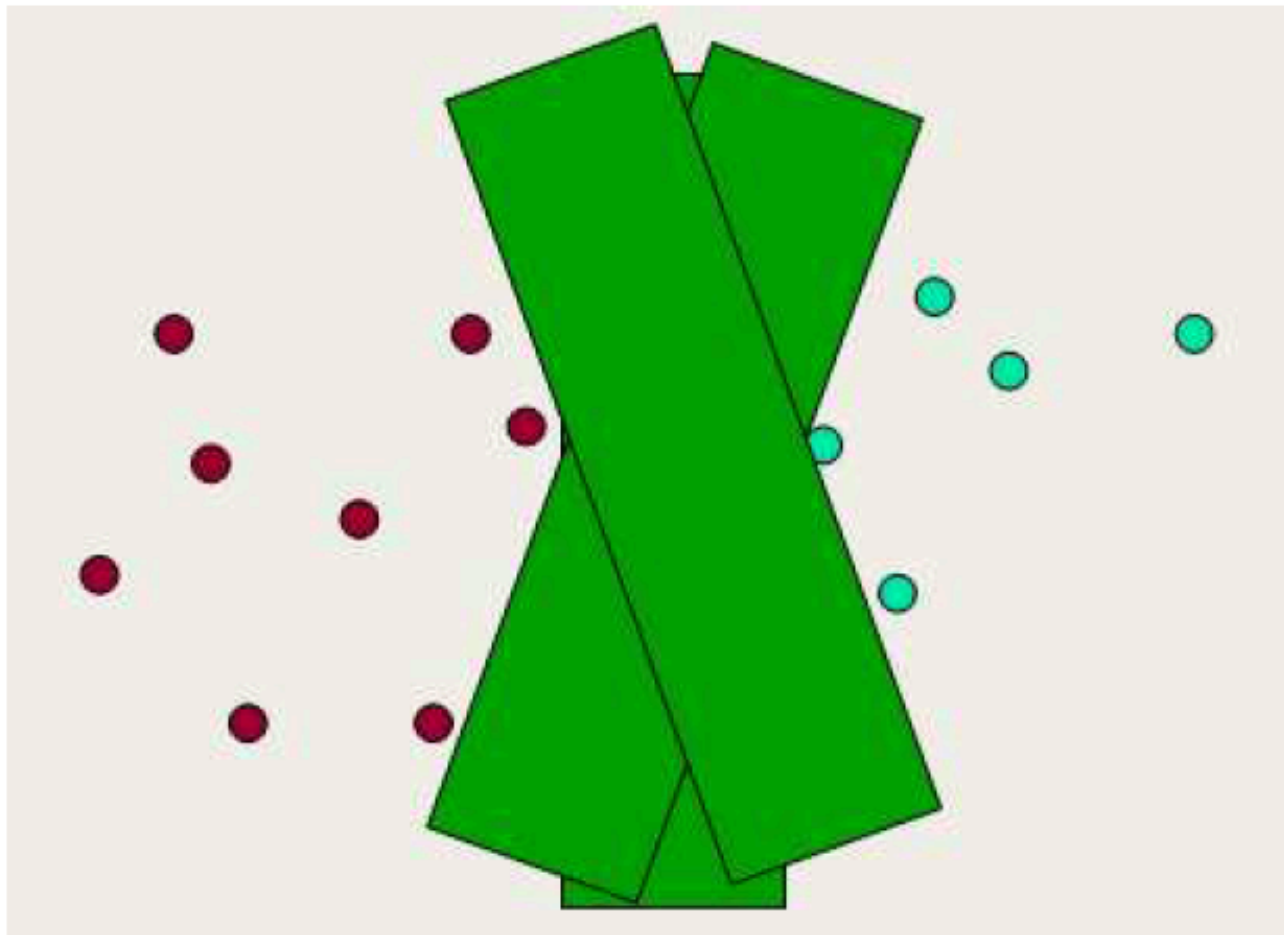
برای مجموعه داده‌های آموزشی دو کلاسه که به صورت خطی جداپذیر هستند، مانند ایمیل‌های اسپم و غیراسپم، بسیاری از جداکننده‌های خطی ممکن وجود دارد. به صورت شهودی، یک مرز تصمیم‌گیری که در وسط فضای خالی بین داده‌های دو کلاس قرار گرفته، بهتر از جداکننده‌ای است که به نمونه‌های یک یا هر دو کلاس بسیار نزدیک است.

در مثال فیلتر اسپم، بردارهای پشتیبان ممکن است ایمیل‌هایی باشند که به مرز بین اسپم و غیراسپم بسیار نزدیک هستند - مانند ایمیل‌های تبلیغاتی با محتوای مفید یا ایمیل‌های مهم با ساختاری شبیه به اسپم. این ایمیل‌ها برای تعریف مرز امن حیاتی‌اند.

به حداکثر رساندن حاشیه به نظر خوب می‌رسد زیرا نقاط نزدیک به سطح تصمیم‌گیری تصمیمات طبقه‌بندی بسیار نامطمئن را نشان می‌دهند: تقریباً 50٪ شانس وجود دارد که طبقه‌بند به هر دو صورت تصمیم بگیرد. یک طبقه‌بند با حاشیه بزرگ هیچ تصمیم طبقه‌بندی با قطعیت پایین ایجاد نمی‌کند. این به شما یک حاشیه ایمنی طبقه‌بندی می‌دهد: یک خطای جزئی در اندازه‌گیری یا یک تغییر جزئی در سند باعث طبقه‌بندی اشتباه نخواهد شد.

شکل 15.2: شهود برای طبقه‌بندی حاشیه بزرگ

شکل 15.2 نشان می‌دهد که چگونه اصرار بر حاشیه بزرگ، ظرفیت مدل را کاهش می‌دهد: محدوده زوایایی که می‌توان سطح تصمیم ضخیم را قرار داد، برای یک هایپرپلن تصمیم‌گیری کمتر است.



► **Figure 15.2** An intuition for large-margin classification. Insisting on a large margin reduces the capacity of the model: the range of angles at which the fat decision surface can be placed is smaller than for a decision hyperplane (cf. Figure 14.8, page 301).

در مثال فیلتر اسپم، این به معنای ایجاد یک مدل است که کمتر تحت تأثیر ایمیل‌های مرزی قرار می‌گیرد و در نتیجه، در تشخیص ایمیل‌های جدید مطمئن‌تر عمل می‌کند.

بیایید یک SVM را با جبر رسمی کنیم. یک ابرصفحه تصمیم‌گیری را می‌توان با یک جمله تقاطع b و یک بردار نرمال ابرصفحه تصمیم‌گیری $\tilde{w}$ تعریف کرد که عمود بر ابرصفحه است. این بردار معمولاً در ادبیات یادگیری ماشین به عنوان **برداری وزن** (weight vector) شناخته می‌شود. برای انتخاب بین تمام ابرصفحه‌هایی که عمود بر بردار نرمال هستند، جمله تقاطع b را مشخص می‌کنیم.

فرض کنید مجموعه‌ای از نقاط داده آموزشی $D = \{(\tilde{x}_i, y_i)\}$ داریم، که هر عضو یک جفت از یک نقطه $\tilde{x}_i$ و یک برچسب کلاس y_i متناظر با آن است. برای SVMها، دو کلاس داده همیشه $+1$ و -1 نامیده می‌شوند (به جای 0 و 1)، و جمله تقاطع همیشه به صراحت به عنوان b نمایش داده می‌شود. طبقه‌بند خطی سپس به صورت زیر است:

$$f(\tilde{x}) = \text{sign}(\tilde{w}^T \tilde{x} + b)$$

مقدار -1 یک کلاس (مثلاً ایمیل اسپم) و مقدار $+1$ کلاس دیگر (مثلاً ایمیل غیراسپم) را نشان می‌دهد. ما از طبقه‌بندی یک نقطه مطمئن هستیم اگر از مرز تصمیم‌گیری دور باشد.

حاشیه تابعی در مقابل حاشیه هندسی

برای یک مجموعه داده و ابرصفحه تصمیم‌گیری، **حاشیه تابعی** (functional margin) مثال نام $\tilde{x}_i$ با توجه به ابرصفحه $\{\tilde{w}, b\}$ به صورت مقدار $y_i(\tilde{w}^T \tilde{x}_i + b)$ تعریف می‌شود. حاشیه تابعی یک مجموعه داده با توجه به یک سطح تصمیم‌گیری دو برابر حاشیه تابعی هر یک از نقاط در مجموعه داده با حداقل حاشیه تابعی است.

با این حال، استفاده از این تعریف به همین شکل مشکلی دارد: مقدار تحت محدودیت نیست، زیرا همیشه می‌توانیم حاشیه تابعی را با صرفاً مقیاس‌بندی $\tilde{w}$ و b به اندازه دلخواه بزرگ کنیم. برای مثال، اگر $\tilde{w}$ را با $5\tilde{w}$ و b را با $5b$

جایگزین کنیم، حاشیه تابعی $y_i(5w \sim x_i + 5b)$ پنج برابر بزرگتر است. این نشان می‌دهد که باید محدودیتی در اندازه بردار $w \sim$ قرار دهیم.

اینجاست که **حاشیه هندسی** (geometric margin) وارد می‌شود. حاشیه هندسی طبقه‌بند حداکثر عرض نوار است که می‌توان برای جدا کردن بردارهای پشتیبان دو کلاس رسم کرد. یعنی دو برابر مقدار حداقل بر روی نقاط داده برای r داده شده در معادله (15.4)، یا به طور معادل، حداکثر عرض یکی از جداکننده‌های ضخیم نشان داده شده در شکل 15.2.

and so satisfies $\vec{w}^T \vec{x}' + b = 0$. Hence:

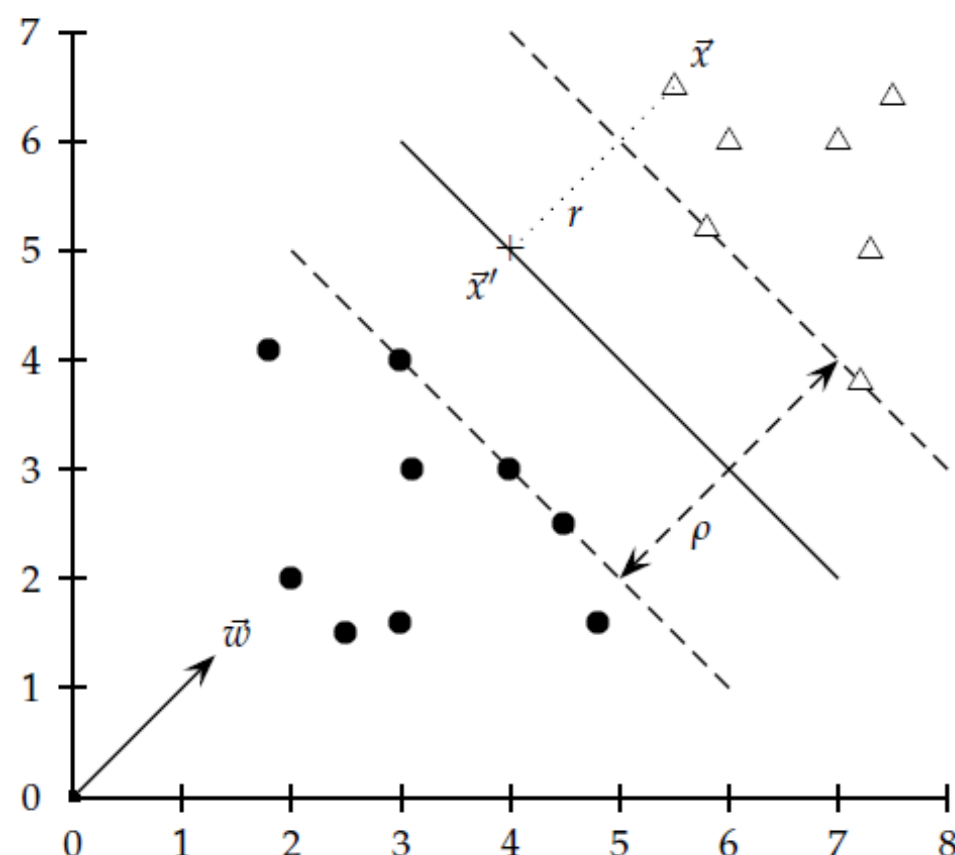
$$(15.3) \quad \vec{w}^T \left(\vec{x} - yr \frac{\vec{w}}{|\vec{w}|} \right) + b = 0$$

Solving for r gives:²

$$(15.4) \quad r = y \frac{\vec{w}^T \vec{x} + b}{|\vec{w}|}$$

شکل 15.3: حاشیه هندسی یک نقطه و یک مرز تصمیم

شکل 15.3 حاشیه هندسی یک نقطه r و یک مرز تصمیم ρ را نشان می‌دهد. فاصله اقلیدسی از یک نقطه $x \sim$ تا مرز تصمیم‌گیری با r نشان داده می‌شود. کوتاه‌ترین فاصله بین یک نقطه و یک ابرصفحه عمود بر صفحه است و از این رو موازی $w \sim$ است.



► Figure 15.3 The geometric margin of a point (r) and a decision boundary (ρ).

در مثال فیلتر اسپم، این فاصله نشان می‌دهد که یک ایمیل چقدر از مرز امن دور است. ایمیل‌هایی با فاصله کمتر، نیازمند بررسی دقیق‌تر هستند.

از آنجایی که می‌توانیم حاشیه تابعی را به دلخواه مقیاس‌بندی کنیم، برای راحتی در حل SVM‌های بزرگ، بیاپید انتخاب کنیم که حاشیه تابعی تمام نقاط داده حداقل 1 باشد و برای حداقل یک بردار داده برابر با 1 باشد. یعنی برای همه موارد در داده:

$$y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1$$

و بردارهای پشتیبانی وجود دارند که برای آنها نامساوی یک برابری است. از آنجایی که فاصله هر مثال از ابرصفحه $r_i = y_i(w^T \tilde{x}_i + b)/|w|$ است، حاشیه هندسی $|w| = 2/\rho$ است. هدف ما همچنان به حداکثر رساندن این حاشیه هندسی است. یعنی می‌خواهیم $|w|$ و b را پیدا کنیم به طوری که:

- $|w| = 2/\rho$ به حداکثر برسد
- برای همه $(x_i, y_i) \in D$ ، $y_i(w^T \tilde{x}_i + b) \geq 1$

به حداکثر رساندن $|w|/2$ همان به حداقل رساندن $|w|/2$ است. این فرمولاسیون نهایی استاندارد یک SVM به عنوان یک مسئله بهینه‌سازی را می‌دهد:

$$\begin{aligned} &\text{Find } \tilde{w} \text{ and } b \text{ such that:} \\ &\frac{1}{2} \tilde{w}^T \tilde{w} \text{ is minimized, and} \\ &\text{for all } \{(\tilde{x}_i, y_i)\}, y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1 \end{aligned}$$

حالا ما در حال بهینه‌سازی یک تابع درجه دوم تحت محدودیت‌های خطی هستیم. مسائل بهینه‌سازی درجه دو یک کلاس استاندارد و شناخته شده از مسائل بهینه‌سازی ریاضی هستند و الگوریتم‌های زیادی برای حل آنها وجود دارد.

با این حال، برای درک شکل راه‌حل چنین مسئله بهینه‌سازی، مفید خواهد بود. راه‌حل شامل ساخت یک مسئله دوگانه است که در آن یک ضریب لاگرانژ α_i با هر محدودیت $y_i(w^T \tilde{x}_i + b) \geq 1$ در مسئله اولیه مرتبط است:

$$\begin{aligned} &\text{Find } \alpha_1, \dots, \alpha_N \text{ such that } \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \tilde{x}_i^T \tilde{x}_j \text{ is maximized, and} \\ &\sum_i \alpha_i y_i = 0 \\ &\alpha_i \geq 0 \text{ for all } 1 \leq i \leq N \end{aligned}$$

سپس راه‌حل به صورت زیر است:

$$\begin{aligned} \tilde{w} &= \sum_i \alpha_i y_i \tilde{x}_i \\ b &= y_k - \tilde{w}^T \tilde{x}_k \text{ for any } \tilde{x}_k \text{ such that } \alpha_k \neq 0 \end{aligned}$$

در راه‌حل، اکثر α_i صفر هستند. هر α_i غیرصفر نشان می‌دهد که $\tilde{x}_i$ متناظر یک بردار پشتیبان است. تابع طبقه‌بندی سپس به صورت زیر است:

$$f(\tilde{x}) = \text{sign} \left(\sum_i \alpha_i y_i \tilde{x}_i^T \tilde{x} + b \right)$$

هم عبارتی که باید در مسئله دوگانه به حداکثر برسد و هم تابع طبقه‌بندی شامل یک محصول نقطه بین جفت نقاط $(\tilde{x}, \tilde{x}_i)$ یا $\tilde{x}_i$ یا $\tilde{x}_{j^*}$ است، و این تنها راهی است که داده‌ها استفاده می‌شوند - ما بعداً به اهمیت این موضوع باز خواهیم گشت.

مثال 15.1: فیلتر اسپم برای ایمیل‌های بانکی

تصور کنید در تیم امنیتی بانک دیجیتال "پی‌پال" کار می‌کنید و باید سیستمی برای تشخیص ایمیل‌های فیشینگ (تقلبی) طراحی کنید. داده‌های ما شامل دو ویژگی اصلی برای هر ایمیل است: تعداد کلمات حساس (مانند "رمز عبور"، "حساب بانکی") و تعداد لینک‌های مشکوک. داده‌های آموزشی ما شامل سه ایمیل است:

- ایمیل 1: (1, 1) - ایمیل معتبر

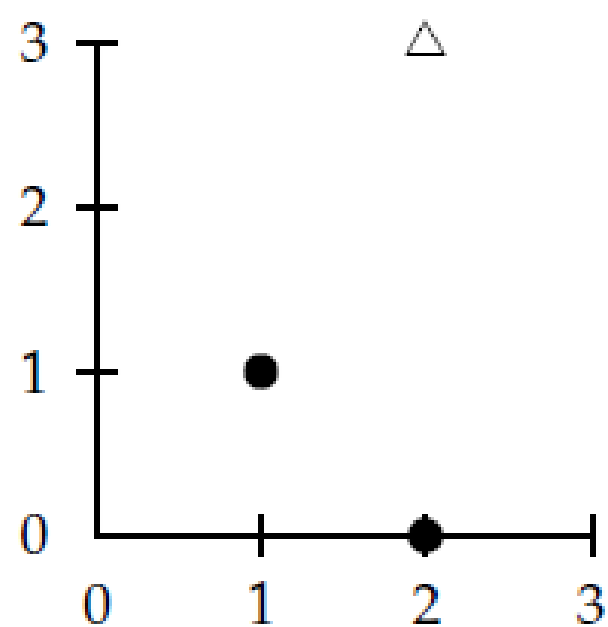
- ایمیل 2: (2, 3) - ایمیل فیشینگ
- ایمیل 3: (4, 2) - ایمیل فیشینگ

کار با روش هندسی، برای مثالی مانند این، بردار وزن حاشیه حداکثر موازی با کوتاه‌ترین خط اتصال نقاط دو کلاس خواهد بود، یعنی خط بین (1, 1) و (2, 3)، که یک بردار وزن (1, 2) را می‌دهد. سطح تصمیم‌گیری بهینه عمود بر آن خط است و آن را در نقطه نصفه قطع می‌کند. بنابراین، از (1.5, 2) عبور می‌کند. پس، مرز تصمیم‌گیری SVM:

$$y = x_1 + 2x_2 - 5.5$$

شکل 15.4: مجموعه داده آموزشی کوچک برای SVM

شکل 15.4 این مجموعه داده کوچک را نشان می‌دهد. نقاط توپر ایمیل‌های معتبر و نقاط توخالی ایمیل‌های فیشینگ هستند. خط مرز تصمیم بهینه است که توسط SVM پیدا شده است.



► Figure 15.4 A tiny 3 data point training set for an SVM.

تحلیل جبری و کاربرد عملی

کار با روش جبری، با محدودیت استاندارد که $\text{sign}(y_i(w^T \tilde{x}_i + b)) \geq 1$ ، ما به دنبال به حداقل رساندن $|w|$ هستیم. این اتفاق می‌افتد وقتی که این محدودیت با برابری توسط دو بردار پشتیبان برآورده شود. علاوه بر این، می‌دانیم که راه‌حل $w = (a, 2a)$ برای برخی a است. پس ما داریم:

$$\begin{aligned} a + 2a + b &= -1 \\ 2a + 6a + b &= 1 \end{aligned}$$

بنابراین، $a = 2/5$ و $b = -11/5$. پس ابرصفحه بهینه با $w = (2/5, 4/5)$ و $b = -11/5$ داده می‌شود.

حاشیه ρ برابر با $|w| = 2/\sqrt{(4/25 + 16/25)} = 2/(2\sqrt{5}/5) = \sqrt{5}/2$ است. این پاسخ را می‌توان به صورت هندسی با بررسی شکل 15.4 تأیید کرد.

کاربرد عملی در سیستم‌های فیلتر اسپم

در سیستم‌های واقعی فیلتر اسپم مانند آنچه در جیمیل استفاده می‌شود، از SVM‌ها با هزاران ویژگی استفاده می‌شود. این ویژگی‌ها ممکن است شامل موارد زیر باشند:

- تعداد کلمات حساس
- تعداد لینک‌های مشکوک
- اعتبار فرستنده
- ساختار ایمیل
- و بسیاری دیگر...

SVM با یافتن حاشیه بهینه، می‌تواند ایمیل‌های فیشینگ را با دقت بالایی تشخیص دهد، در حالی که ایمیل‌های معتبر را به اشتباه مسدود نمی‌کند. این تعادل حیاتی است، زیرا مسدود کردن اشتباه یک ایمیل معتبر می‌تواند منجر به از دست دادن ارتباطات مهم شود.

```
In [19]: import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn import svm
from sklearn.metrics import accuracy_score

# =====
# برای حالت جداپذیر خطی (SVM) پیاده‌سازی ماشین بردار پشتیبان
# (Quadratic Programming) با استفاده از حل بهینه‌سازی درجه دو
# =====

class HardMarginSVM:
    """
    برای داده‌های خطی جداپذیر (Hard-Margin) حاشیه سخت SVM پیاده‌سازی
    این پیاده‌سازی به صورت تحصیلی و از پایه انجام شده تا مفاهیم ریاضی
    به طور شفاف در کد قابل مشاهده باشد.
    """

    def __init__(self):
        self.w = None # بردار وزن (وزن‌های ویژگی)
        self.b = None # بایاس (اندیشه)
        self.alpha = None # تنها بردارهای پشتیبان (ضرایب لاگرانژ  $\alpha > 0$ )
        self.support_vectors = None # بردارهای پشتیبان
        self.support_labels = None # برچسب‌های بردارهای پشتیبان
        self.margin = None # مقدار حاشیه هندسی

    def fit(self, X, y):
        """
        (Dual Formulation) با روش دوگانه SVM آموزش مدل

        پارامترها:
            X: (n_samples, n_features) داده‌های آموزشی به شکل
            y: (n_samples,) برچسب‌های کلاس (+1 یا -1) به شکل
        """
        n_samples, n_features = X.shape

        # برای تابع هدف کوادراتیک P محاسبه ماتریس
        #  $P_{ij} = y_i * y_j * \text{dot}(x_i, x_j)$ 
        P = np.zeros((n_samples, n_samples))
        for i in range(n_samples):
            for j in range(n_samples):
                P[i, j] = y[i] * y[j] * np.dot(X[i], X[j])

        #  $cvxopt$  تبدیل به فرمت مورد نیاز
        P = matrix(P)
        q = matrix(-np.ones(n_samples)) # برای تابع هدف q بردار

        #  $\alpha_i \geq 0$  (معادل  $-\alpha_i \leq 0$ ): محدودیت‌های نابرابری
        G = matrix(-np.eye(n_samples))
        h = matrix(np.zeros(n_samples))

        #  $\sum \alpha_i * y_i = 0$ : محدودیت تساوی
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)

        # حل مسئله بهینه‌سازی درجه دو
        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)

        # استخراج ضرایب لاگرانژ
        self.alpha = np.array(solution['x']).flatten()

        # (مقدار بسیار کوچک  $\alpha > 0$ ) شناسایی بردارهای پشتیبان
        support_vector_indices = self.alpha > 1e-5
        self.support_vectors = X[support_vector_indices]
        self.support_labels = y[support_vector_indices]
        self.alpha_sv = self.alpha[support_vector_indices]

        #  $w = \sum \alpha_i * y_i * x_i$  بردار وزن محاسبه
        self.w = np.sum(self.alpha_sv.reshape(-1, 1) *
                        self.support_labels.reshape(-1, 1) *
                        self.support_vectors, axis=0)

        # از روی هر بردار پشتیبان b محاسبه بایاس
        # از اولین بردار پشتیبان استفاده می‌کنیم
```

```

sv = self.support_vectors[0]
sv_label = self.support_labels[0]
self.b = sv_label - np.dot(self.w, sv)

# محاسبه حاشیه هندسی:  $\rho = 2 / ||w||$ 
self.margin = 2.0 / np.linalg.norm(self.w)

def predict(self, X):
    """
    پیش‌بینی برچسب برای داده‌های جدید

    پارامترها:
        X: داده‌های ورودی به شکل (n_samples, n_features)

    برمی‌گرداند:
        برچسب‌های پیش‌بینی شده (1 یا -1)
    """
    decision_values = np.dot(X, self.w) + self.b
    return np.sign(decision_values)

def get_decision_function(self, x1_range):
    """
    محاسبه مقادیر تابع تصمیم‌گیری برای ترسیم
    """
    w1, w2 = self.w
    #  $w1 \cdot x1 + w2 \cdot x2 + b = 0 \Rightarrow x2 = -(w1 \cdot x1 + b) / w2$ 
    decision_line = lambda x1: -(w1 * x1 + self.b) / w2
    margin_line_1 = lambda x1: -(w1 * x1 + self.b - 1) / w2
    margin_line_2 = lambda x1: -(w1 * x1 + self.b + 1) / w2

    return decision_line, margin_line_1, margin_line_2

def visualize(self, X, y, title="SVM Decision Boundary"):
    """
    ترسیم تصمیم‌گیری، حاشیه‌ها و بردارهای پشتیبان
    این تابع دقیقاً همانند شکل 15.4 در متن اصلی است
    """
    plt.figure(figsize=(10, 8))

    # ترسیم نقاط داده (هر کلاس با رنگ متفاوت)
    class_1 = y == 1
    class_2 = y == -1

    plt.scatter(X[class_1, 0], X[class_1, 1],
                c='blue', s=100, marker='o', label='Class +1 (legitimate)', alpha=0.7)
    plt.scatter(X[class_2, 0], X[class_2, 1],
                c='red', s=100, marker='s', label='Class -1 (spam)', alpha=0.7)

    # ترسیم بردارهای پشتیبان (دایره دور نقاط)
    if self.support_vectors is not None:
        plt.scatter(self.support_vectors[:, 0], self.support_vectors[:, 1],
                    facecolors='none', edgecolors='black', s=300,
                    linewidth=2, label='Support Vectors')

    # محاسبه محدوده محورها برای ترسیم
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x_range = np.linspace(x_min, x_max, 100)

    # ترسیم مرز تصمیم‌گیری و خطوط حاشیه
    decision_line, margin_line_1, margin_line_2 = self.get_decision_function(x_range)

    plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2,
             label=f'Decision Boundary ( $w \cdot x + b = 0$ )')
    plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5,
             label=f'Margin Boundary ( $w \cdot x + b = +1$ )')
    plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5,
             label=f'Margin Boundary ( $w \cdot x + b = -1$ )')

    # پر کردن ناحیه بین خطوط حاشیه
    plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                    color='gray', alpha=0.2, label=f'Margin ( $\rho = \{{self.margin:.2f}\}$ )')

    plt.xlabel('Feature 1 (Sensitive Words Count)', fontsize=12)
    plt.ylabel('Feature 2 (Suspicious Links Count)', fontsize=12)
    plt.title(f'{title}\nMargin = {self.margin:.4f},  $||w|| = \{{np.linalg.norm(self.w):.4f}\}$ ',
              fontsize=14, fontweight='bold')
    plt.legend(loc='best', fontsize=10)
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# =====
# مثال 15.1: مجموعه داده سه‌نقطه‌ای از متن کتاب
# =====

# داده‌های آموزشی: (تعداد کلمات حساس، تعداد لینک‌های مشکوک)
# نکته: در متن اصلی، (1,1) کلاس 1- و (2,3) کلاس 1+ هستند
X_train = np.array([
    [1, 1], # بردار پشتیبان کلاس 1- (اسپم)
    [2, 3], # بردار پشتیبان کلاس 1+ (غیر اسپم)
    [4, 2] # نمونه دیگر کلاس 1- (اسپم) - بردار پشتیبان نیست
], dtype=float)

y_train = np.array([-1, 1, -1]) # برچسب‌های کلاس

print("="*60)
print("برای مجموعه داده کوچک SVM مثال 15.1: حل")
print("="*60)
print(f"داده‌های آموزشی:")
print(f"X = \n{X_train}")
print(f"y = {y_train}")
print()

```

```
# ایجاد و آموزش مدل SVM
svm_model = HardMarginSVM()
svm_model.fit(X_train, y_train)

# نمایش نتایج
print(f"بردار وزن (w): [{svm_model.w[0]:.4f}, {svm_model.w[1]:.4f}]")
print(f"بایاس (b): {svm_model.b:.4f}")
print(f"اندازه بردار وزن (||w||): {np.linalg.norm(svm_model.w):.4f}")
print(f"حاشیه هندسی (p): {svm_model.margin:.4f}")
print()

# شناسایی بردارهای پشتیبان
print("بردارهای پشتیبان:")
for i, (sv, label, alpha) in enumerate(zip(svm_model.support_vectors,
                                           svm_model.support_labels,
                                           svm_model.alpha_sv)):
    print(f" {i+1}. x={sv}, y={label}, α={alpha:.4f}")
print()

# تست پیش‌بینی روی داده‌های آموزشی
predictions = svm_model.predict(X_train)
accuracy = accuracy_score(y_train, predictions)
print(f"دقت روی داده‌های آموزشی: {accuracy:.2%}")
print()

# ترسیم شکل 15.4
svm_model.visualize(X_train, y_train, title="Figure 15.4: Small Training Set for SVM")

# =====
# scikit-learn مقایسه با پیاده‌سازی
# =====

print("\n" + "="*60)
print("مقایسه (scikit-learn) با پیاده‌سازی مرجع")
print("="*60)

# (با پارامترهای مشابه حاشیه سخت) از scikit-Learn SVM ایجاد مدل
svm_sklarn = svm.SVC(kernel='linear', C=1e5) # C حاشیه سخت
svm_sklarn.fit(X_train, y_train)

# استخراج پارامترها
w_sklarn = svm_sklarn.coef_[0]
b_sklarn = svm_sklarn.intercept_[0]
margin_sklarn = 2.0 / np.linalg.norm(w_sklarn)

print(f"[scikit-learn] بردار وزن: [{w_sklarn[0]:.4f}, {w_sklarn[1]:.4f}]")
print(f"[scikit-learn] بایاس: {b_sklarn:.4f}")
print(f"[scikit-learn] حاشیه: {margin_sklarn:.4f}")
print(f"[scikit-learn] بردارهای پشتیبان: {svm_sklarn.n_support_} نقطه")

# دقت مقایسه
pred_sklarn = svm_sklarn.predict(X_train)
print(f"[scikit-learn] دقت: {accuracy_score(y_train, pred_sklarn):.2%}")

# =====
# نمایش تاثیر بردارهای پشتیبان
# =====

print("\n" + "="*60)
print("تحلیل تاثیر بردارهای پشتیبان")
print("="*60)

# حذف یک بردار پشتیبان و مشاهده تغییرات
print("(اگر نمونه (2, 4) را حذف کنیم (که بردار پشتیبان نیست)")
X_reduced = X_train[:2]
y_reduced = y_train[:2]

svm_reduced = HardMarginSVM()
svm_reduced.fit(X_reduced, y_reduced)

print(f"بردار وزن جدید: [{svm_reduced.w[0]:.4f}, {svm_reduced.w[1]:.4f}]")
print(f"حاشیه جدید: {svm_reduced.margin:.4f}")
print(f">{svm_reduced.w - svm_model.w} > 1e-3 آیا تغییر کرد؟")

print("\n(اگر یک بردار پشتیبان را حذف کنیم)")
# حذف نمونه (1, 1) که بردار پشتیبان است
X_no_sv1 = np.array([[2, 3], [4, 2]])
y_no_sv1 = np.array([1, -1])

svm_no_sv1 = HardMarginSVM()
svm_no_sv1.fit(X_no_sv1, y_no_sv1)

print(f"بردار وزن جدید: [{svm_no_sv1.w[0]:.4f}, {svm_no_sv1.w[1]:.4f}]")
print(f"حاشیه جدید: {svm_no_sv1.margin:.4f}")
print(f">{svm_no_sv1.w - svm_model.w} > 1e-3 آیا تغییر کرد؟")
print("نتیجه: حذف بردار پشتیبان کاملاً مدل را تغییر می‌دهد")

# =====
# پیش‌بینی برای داده‌های جدید (مثال کاربردی)
# =====

print("\n" + "="*60)
print("تست پیش‌بینی برای ایمیل‌های جدید")
print("="*60)

# ایمیل‌های جدید برای تست
X_test = np.array([
    [1.5, 1.5], # شبیه به (1,1) - احتمالاً اسپم
    [2.5, 3.5], # شبیه به (2,3) - احتمالاً غیر اسپم
    [3, 2.5], # در منطقه مرزی
])
```

```
predictions_test = svm_model.predict(X_test)
decision_values = np.dot(X_test, svm_model.w) + svm_model.b
distances = np.abs(decision_values) / np.linalg.norm(svm_model.w)

print("ایمیل‌های تست:")
for i, (email, pred, dist) in enumerate(zip(X_test, predictions_test, distances)):
    status = "غیر اسپم" if pred == 1 else "اسپم"
    confidence = "بالا" if dist > svm_model.margin/2 else "پایین"
    print(f"ایمیل {i+1}: {email} → {status} (فاصله از مرز: {dist:.2f}, اطمینان: {confidence})")

# ترسیم داده‌های تست روی نمودار
plt.figure(figsize=(10, 8))

# نمایش داده‌های آموزشی
class_1 = y_train == 1
class_2 = y_train == -1
plt.scatter(X_train[class_1, 0], X_train[class_1, 1], c='blue', s=100, marker='o', label='Train: Class +1')
plt.scatter(X_train[class_2, 0], X_train[class_2, 1], c='red', s=100, marker='s', label='Train: Class -1')

# نمایش بردارهای پشتیبان
plt.scatter(svm_model.support_vectors[:, 0], svm_model.support_vectors[:, 1],
            facecolors='none', edgecolors='black', s=300, linewidth=2, label='Support Vectors')

# نمایش داده‌های تست
test_class_1 = predictions_test == 1
test_class_2 = predictions_test == -1
plt.scatter(X_test[test_class_1, 0], X_test[test_class_1, 1],
            c='blue', s=200, marker='*', label='Test: Predicted +1', alpha=0.7)
plt.scatter(X_test[test_class_2, 0], X_test[test_class_2, 1],
            c='red', s=200, marker='*', label='Test: Predicted -1', alpha=0.7)

# ترسیم مرز تصمیم و حاشیه‌ها
x_min, x_max = plt.gca().get_xlim()
x_range = np.linspace(x_min, x_max, 100)
decision_line, margin_line_1, margin_line_2 = svm_model.get_decision_function(x_range)

plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2, label='Decision Boundary')
plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5, label='Margin')
plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5)

plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                color='gray', alpha=0.1)

plt.xlabel('Feature 1 (Sensitive Words Count)', fontsize=12)
plt.ylabel('Feature 2 (Suspicious Links Count)', fontsize=12)
plt.title('Prediction of New Emails Using SVM', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()
```

=====

برای مجموعه داده کوچک SVM مثال 15.1: حل

=====

داده‌های آموزشی:

X =

```
[[1. 1.]
 [2. 3.]
 [4. 2.]]
```

y = [-1 1 -1]

وزن بردار (w): [-0.4000, 1.2000]

بایاس (b): -1.8000

اندازه بردار وزن (||w||): 1.2649

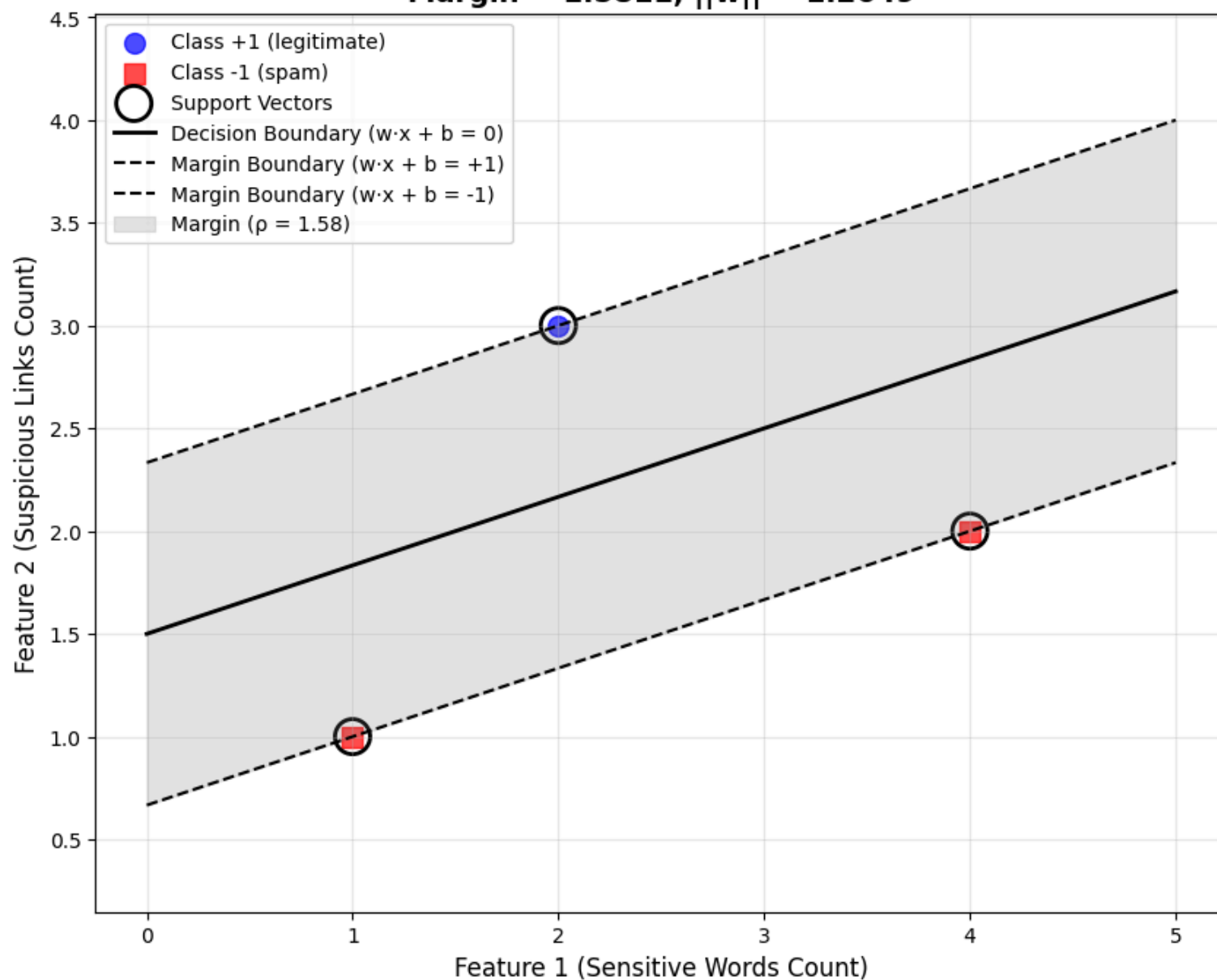
حاشیه هندسی (ρ): 1.5811

بردارهای پشتیبان:

- x=[1. 1.], y=-1, α=0.4000
- x=[2. 3.], y=1, α=0.8000
- x=[4. 2.], y=-1, α=0.4000

%دقت روی داده‌های آموزشی: 100.00

Figure 15.4: Small Training Set for SVM
Margin = 1.5811, $||w|| = 1.2649$



=====

مقایسه با پیاده‌سازی مرجع (scikit-learn)

=====

[scikit-learn] بردار وزن: [-0.4000, 1.2000]

[scikit-learn] بایاس: -1.8000

[scikit-learn] حاشیه: 1.5811

[scikit-learn] بردارهای پشتیبان: [2 1] نقطه

[scikit-learn] %دقت: 100.00

=====

تحلیل تأثیر بردارهای پشتیبان

=====

:اگر نمونه (4, 2) را حذف کنیم (که بردار پشتیبان نیست)

بردار وزن جدید: [0.4000, 0.8000]

حاشیه جدید: 2.2361

True آیا تغییر کرد؟

:اگر یک بردار پشتیبان را حذف کنیم

بردار وزن جدید: [-0.8000, 0.4000]

حاشیه جدید: 2.2361

True آیا تغییر کرد؟

نتیجه: حذف بردار پشتیبان کاملاً مدل را تغییر می‌دهد

=====

تست پیش‌بینی برای ایمیل‌های جدید

=====

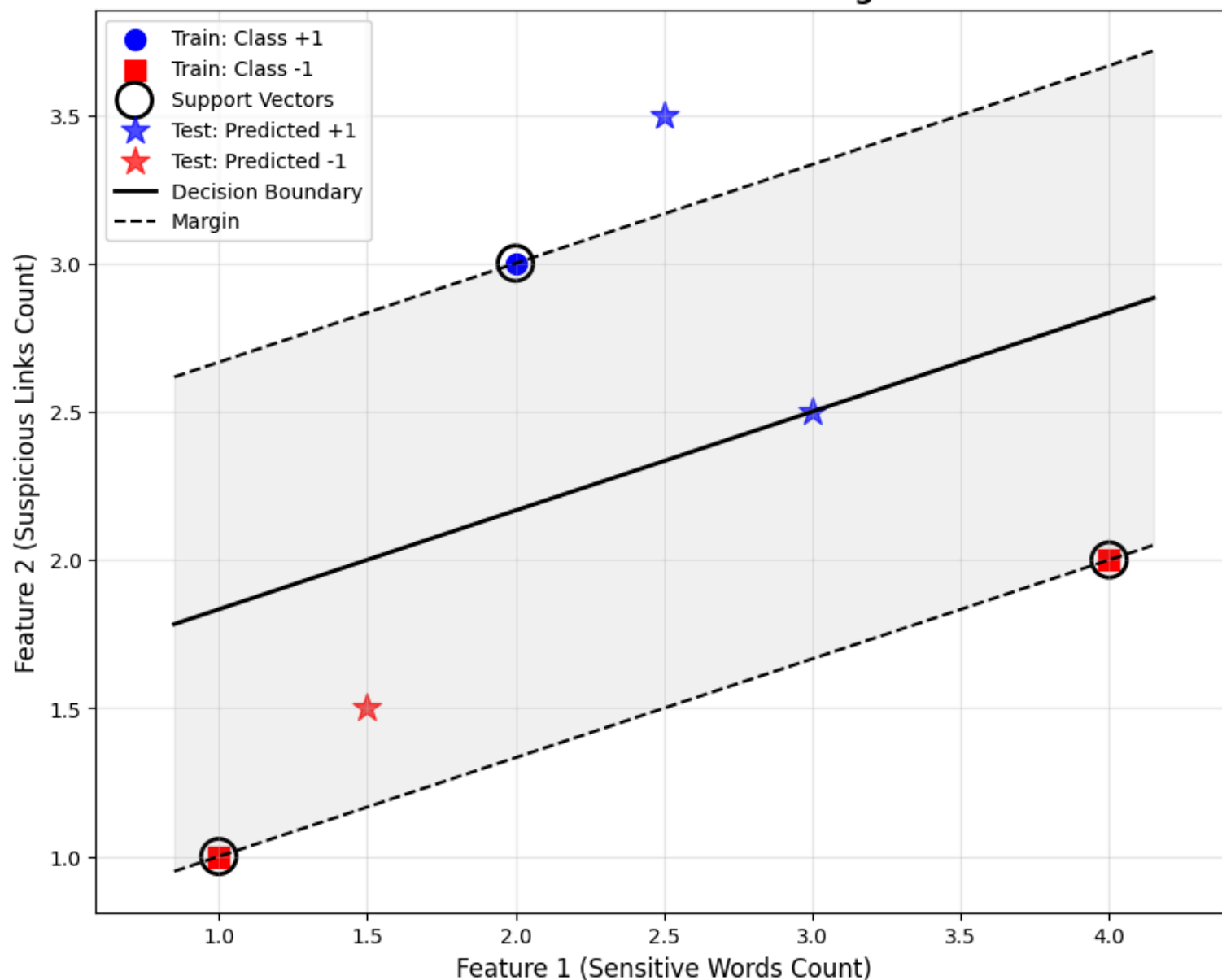
:ایمیل‌های تست

ایمیل 1: [1.5 1.5] → اسپم (فاصله از مرز: 0.47, اطمینان: پایین)

ایمیل 2: [3.5 2.5] → غیر اسپم (فاصله از مرز: 1.11, اطمینان: بالا)

ایمیل 3: [2.5 3] → غیر اسپم (فاصله از مرز: 0.00, اطمینان: پایین)

Prediction of New Emails Using SVM



15.2.1 طبقه‌بندی حاشیه نرم: انعطاف‌پذیری در دنیای واقعی

🔗 تشخیص پزشکی: چگونه یک مدل هوشمند می‌تواند جان انسان‌ها را نجات دهد؟

تصور کنید در تیم تحقیقاتی بیمارستان "میو کلینیک" کار می‌کنید و سیستمی برای تشخیص زودهنگام سرطان طراحی می‌کنید. داده‌های شما شامل نتایج آزمایش‌های مختلف بیماران است که باید به دو گروه "سرطانی" و "غیرسرطانی" تقسیم شوند.

در دنیای واقعی، داده‌های پزشکی همیشه به صورت خطی جداپذیر نیستند. برخی بیماران ممکن است نتایج آزمایش غیرعادی داشته باشند اما سرطان نداشته باشند (نویز)، در حالی که برخی دیگر ممکن است نتایج طبیعی داشته باشند اما به سرطان مبتلا باشند. در این موارد، یک مدل با حاشیه سخت (hard margin) که هیچ خطایی را تحمل نمی‌کند، عملکرد خوبی نخواهد داشت.

اینجاست که **طبقه‌بندی حاشیه نرم (soft margin)** در SVM وارد می‌شود. این روش به مدل اجازه می‌دهد تا برخی خطاها را بپذیرد تا عملکرد کلی بهتری داشته باشد.

برای مسائل با ابعاد بسیار بالا که در طبقه‌بندی متن رایج است، گاهی داده‌ها به صورت خطی جداپذیر هستند. اما در حالت کلی، اینطور نیست و حتی اگر جداپذیر باشند، ممکن است راه‌حلی را ترجیح دهیم که بهتر بخش عمده داده‌ها را جدا کند در حالی که چند سند نویز نادیده گرفته شوند.

اگر مجموعه آموزشی D به صورت خطی جداپذیر نباشد، رویکرد استاندارد این است که به حاشیه تصمیم ضخیم اجازه دهیم چند اشتباه مرتکب شود (برخی نقاط - داده‌های پرت یا نمونه‌های نویزی - در داخل یا در سمت اشتباه حاشیه قرار دارند). سپس برای هر مثال طبقه‌بندی اشتباه، هزینه‌ای پرداخت می‌کنیم که به فاصله آن از برآورده کردن الزام حاشیه در معادله (15.5) بستگی دارد.

برای پیاده‌سازی این، متغیرهای لغزش (slack variables) ξ_i را معرفی می‌کنیم. در مثال تشخیص پزشکی، این متغیرها به مدل اجازه می‌دهند تا برخی بیماران را به اشتباه طبقه‌بندی کند.

مقدار غیرصفر برای ξ_i به x_i اجازه می‌دهد تا الزام حاشیه را با هزینه‌ای متناسب با مقدار ξ_i برآورده نکند. در مثال پزشکی، این به معنای پذیرش برخی خطاها در تشخیص است تا مدل کلی بهتری داشته باشیم.

مثلاً، در تشخیص سرطان، ما ممکن است ترجیح دهیم که چند فرد سالم را به اشتباه به عنوان "مشکوک به سرطان" علامت‌گذاری کنیم (false positive) تا اینکه حتی یک فرد مبتلا به سرطان را از دست بدهیم (false negative). این مصداق بارز استفاده از حاشیه نرم است.

فرمولاسیون مسئله بهینه‌سازی SVM با متغیرهای لغزش به صورت زیر است:

Find $\tilde{w}, b$, and $\xi_i \geq 0$ such that:

$$\frac{1}{2} \tilde{w}^T \tilde{w} + C \sum_i \xi_i \text{ is minimized}$$

$$\text{and for all } \{(\tilde{x}_i, y_i)\}, y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1 - \xi_i$$

🏛️ معیار منظم‌سازی C: تعادل بین دقت و انعطاف‌پذیری

مسئله بهینه‌سازی سپس در حالی است که ضخامت حاشیه را در برابر تعداد نقاطی که باید برای این حاشیه جابجا شوند، معامله می‌کند. حاشیه می‌تواند برای یک نقطه x_i با تنظیم $\xi_i > 0$ کمتر از 1 باشد، اما سپس برای این کار، جریمه‌ای به اندازه $\xi_i C$ در بهینه‌سازی پرداخت می‌شود.

در مثال پزشکی، پارامتر C تعیین می‌کند که مدل چقدر باید به خطاها حساس باشد:

- وقتی C بزرگ است، مدل به خطاها جریمه سنگینی می‌دهد و تلاش می‌کند تا حد امکان خطاها را کاهش دهد، حتی اگر به معنای حاشیه کوچکتر باشد.
- وقتی C کوچک است، مدل خطاها را با جریمه کمتری می‌پذیرد و حاشیه بزرگتری ایجاد می‌کند که ممکن است برخی نقاط را اشتباه طبقه‌بندی کند.

در تشخیص پزشکی، ممکن است مقدار C را کوچک انتخاب کنیم تا مدل انعطاف‌پذیرتر باشد و بتواند موارد مرزی را بهتر مدیریت کند.

مسئله دوگانه برای طبقه‌بندی حاشیه نرم به صورت زیر است:

$$\text{Find } \alpha_1, \dots, \alpha_N \text{ such that } \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \tilde{x}_i^T \tilde{x}_j \text{ is maximized, and}$$

$$\sum_i \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C \text{ for all } 1 \leq i \leq N$$

نه متغیرهای لغزش ξ_i و نه ضرایب لاگرانژ برای آنها در مسئله دوگانه ظاهر نمی‌شوند. تمام آنچه باقی می‌ماند ثابت C است که اندازه ممکن ضرایب لاگرانژ برای نقاط داده بردار پشتیبان را محدود می‌کند. مانند قبل، x_i با α_i غیرصفر، بردارهای پشتیبان خواهند بود. راه‌حل مسئله دوگانه به صورت زیر است:

$$\tilde{w} = \sum \alpha_i y_i \tilde{x}_i$$

$$b = y_k(1 - \xi_k) - \tilde{w}^T \tilde{x}_k \text{ for } k = \arg \max_k \alpha_k$$

معمولاً، بردارهای پشتیبان بخش کوچکی از داده‌های آموزشی را تشکیل می‌دهند. با این حال، اگر مسئله غیرقابل تفکیک باشد یا حاشیه کوچکی داشته باشد، آنگاه هر نقطه داده که طبقه‌بندی اشتباه شده یا در داخل حاشیه قرار دارد، α غیرصفر خواهد داشت. اگر این مجموعه نقاط بزرگ شود، برای حالت غیرخطی که در بخش 15.2.3 به آن می‌پردازیم، این می‌تواند یک کاهش سرعت عمده برای استفاده از SVMها در زمان آزمون باشد.

در سیستم‌های تشخیص پزشکی واقعی مانند آنچه در "مِیو کلینیک" استفاده می‌شود، این مسئله بسیار مهم است. اگر تعداد بردارهای پشتیبان زیاد شود، زمان تشخیص برای هر بیمار جدید افزایش می‌یابد که در شرایط اورژانسی پزشکی قابل قبول نیست.

پیچیدگی آموزش و آزمون با SVMهای خطی در جدول 15.1 نشان داده شده است. زمان آموزش یک SVM تحت تأثیر زمان حل QP زیربنایی است و بنابراین پیچیدگی نظری و تجربی بسته به روش استفاده شده برای حل آن متفاوت است. نتیجه استاندارد برای حل QPها این است که زمانی مکعبی در اندازه مجموعه داده می‌برد. تمام کارهای اخیر در آموزش SVM تلاش کرده‌اند تا این پیچیدگی را کاهش دهند، اغلب با راضی بودن به راه‌حل‌های تقریبی. به طور استاندارد، پیچیدگی تجربی حدود $O(|D|^{1.7})$ است. با این حال، زمان آموزش فوق خطی الگوریتم‌های سنتی SVM استفاده از آنها را در مجموعه داده‌های آموزشی بسیار بزرگ دشوار یا غیرممکن می‌سازد.

Classifier	Mode	Method	Time complexity
NB	training		$\Theta(D L_{ave} + C V)$
NB	testing		$\Theta(C M_a)$
Rocchio	training		$\Theta(D L_{ave} + C V)$
Rocchio	testing		$\Theta(C M_a)$
kNN	training	preprocessing	$\Theta(D L_{ave})$
kNN	testing	preprocessing	$\Theta(D M_{ave}M_a)$
kNN	training	no preprocessing	$\Theta(1)$
kNN	testing	no preprocessing	$\Theta(D L_{ave}M_a)$
SVM	training	conventional	$O(C D ^3M_{ave}); \approx O(C D ^{1.7}M_{ave})$, empirically
SVM	training	cutting planes	$O(C D M_{ave})$
SVM	testing		$O(C M_a)$

جدول 15.1: پیچیدگی آموزش و آزمون طبقه‌بندهای مختلف

جدول 15.1 پیچیدگی آموزش و آزمون طبقه‌بندهای مختلف از جمله SVMها را نشان می‌دهد. در سیستم‌های تشخیص پزشکی، این پیچیدگی‌ها بسیار مهم هستند:

- روش‌های سنتی SVM با پیچیدگی $O(|D|^{1.7})$ ممکن است برای مجموعه داده‌های بزرگ پزشکی بسیار کند باشند.
- روش‌های جدیدتر مانند "cutting planes" با پیچیدگی خطی $O(|D|)$ برای سیستم‌های پزشکی مناسب‌تر هستند.
- در حالی که روش‌هایی مانند Naive Bayes آموزش سریع‌تری دارند، اما ممکن است دقت کمتری در تشخیص‌های پیچیده پزشکی داشته باشند.

در عمل، تیم‌های پزشکی ممکن است از روش‌های ترکیبی استفاده کنند: ابتدا یک روش سریع برای غربالگری اولیه، سپس SVM برای موارد مشکوک.

In [21]:

```
import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn import svm
from sklearn.metrics import accuracy_score

# =====
# برای داده‌های غیرجدپذیر خطی (Soft-Margin) حاشیه نرم SVM پیاده‌سازی
# استفاده می‌کند (QP) این پیاده‌سازی از بهینه‌سازی درجه دو
# =====

class SoftMarginSVM:
    """
    (slack variables) حاشیه نرم با متغیرهای لغزش SVM پیاده‌سازی آموزشی
    تعادل بین حاشیه بزرگ و خطای طبقه‌بندی را کنترل می‌کند C این کلاس نشان می‌دهد چگونه پارامتر
    """

    def __init__(self, C=1.0):
        """
        ضریب منظم‌سازی که هزینه خطا را کنترل می‌کند: C پارامتر
        - کوچک: حاشیه بزرگتر، تحمل خطای بیشتر (مناسب برای داده با نویز بالا) C
        - بزرگ: حاشیه کوچکتر، خطای کمتر (نزدیک به حاشیه سخت) C
        """

        self.C = C
        self.w = None # بردار وزن
        self.b = None # بایاس
        self.alpha = None # ضرایب لاگرانژ
        self.support_vectors = None
        self.support_labels = None
        self.margin = None
        self.slack_points = [] # نقاطی که از حاشیه عبور کرده‌اند

    def fit(self, X, y):
        """
        حاشیه نرم با روش دوگانه SVM آموزش مدل
        (بود  $\alpha_i \geq 0$  در مقایسه با حاشیه سخت که فقط  $\alpha_i \leq C$  محدودیت جدید:  $0$ )
        """

        n_samples, n_features = X.shape

        # برای تابع هدف  $P$  محاسبه ماتریس  $P_{ij} = y_i * y_j * \text{dot}(x_i, x_j)$ 
        P = np.zeros((n_samples, n_samples))
        for i in range(n_samples):
            for j in range(n_samples):
                P[i, j] = y[i] * y[j] * np.dot(X[i], X[j])

        # تبدیل به فرمت cvxopt
        P = matrix(P)
        q = matrix(-np.ones(n_samples))

        # محدودیت‌های نابرابری:  $-\alpha_i \leq 0$  و  $\alpha_i \leq C$ 
        G = np.vstack([-np.eye(n_samples), np.eye(n_samples)])
        h = np.hstack([np.zeros(n_samples), self.C * np.ones(n_samples)])

        G = matrix(G)
        h = matrix(h)

        # محدودیت تساوی:  $\sum \alpha_i * y_i = 0$ 
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)

        # حل مسئله بهینه‌سازی
        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)

        self.alpha = np.array(solution['x']).flatten()

        # شناسایی بردارهای پشتیبان: نقاطی که  $\alpha_i > 0$ 
        support_vector_indices = self.alpha > 1e-5
        self.support_vectors = X[support_vector_indices]
        self.support_labels = y[support_vector_indices]
        self.alpha_sv = self.alpha[support_vector_indices]

        # محاسبه بردار وزن:  $w = \sum \alpha_i * y_i * x_i$ 
        self.w = np.sum(self.alpha_sv.reshape(-1, 1) *
                        self.support_labels.reshape(-1, 1) *
                        self.support_vectors, axis=0)

        #  $\alpha_i < C$  محاسبه بایاس از یک بردار پشتیبان با
        sv_margin_indices = (self.alpha > 1e-5) & (self.alpha < self.C - 1e-5)
        if np.any(sv_margin_indices):
            idx = np.where(sv_margin_indices)[0][0]
            sv = X[idx]
            sv_label = y[idx]
            self.b = sv_label - np.dot(self.w, sv)
        else:
            # باشند، از میانگین استفاده کن  $\alpha_i = C$  اگر همه
            self.b = np.mean(self.support_labels - np.dot(self.support_vectors, self.w))

        # محاسبه حاشیه هندسی:  $\rho = 2 / ||w||$ 
        self.margin = 2.0 / np.linalg.norm(self.w)

        # شناسایی نقاط با لغزش ( $\xi_i > 0$ )
```

```

این نقاط یا اشتباه طبقه‌بندی می‌شوند یا در داخل حاشیه هستند
self.slack_points = []
for i, x_i in enumerate(X):
    decision_val = y[i] * (np.dot(self.w, x_i) + self.b)
    if decision_val < 1 - 1e-5: # نقطه از حاشیه عبور کرده
        self.slack_points.append({
            'point': x_i,
            'label': y[i],
            'slack_amount': 1 - decision_val
        })

def predict(self, X):
    """پیش‌بینی برچسب برای داده‌های جدید"""
    return np.sign(np.dot(X, self.w) + self.b)

def get_decision_function(self, x_range):
    """محاسبه معادلات مرز تصمیم و حاشیه‌ها برای ترسیم"""
    w1, w2 = self.w
    decision_line = lambda x1: -(w1 * x1 + self.b) / w2
    margin_line_1 = lambda x1: -(w1 * x1 + self.b - 1) / w2
    margin_line_2 = lambda x1: -(w1 * x1 + self.b + 1) / w2
    return decision_line, margin_line_1, margin_line_2

def visualize(self, X, y, title=""):
    """ترسیم مرز تصمیم، حاشیه، بردارهای پشتیبان و نقاط لغزش"""
    plt.figure(figsize=(12, 8))

    # ترسیم نقاط داده (دو کلاس)
    class_1 = y == 1
    class_2 = y == -1

    plt.scatter(X[class_1, 0], X[class_1, 1],
                c='blue', s=80, marker='o', label='Positive Class', alpha=0.7)
    plt.scatter(X[class_2, 0], X[class_2, 1],
                c='red', s=80, marker='s', label='Negative Class', alpha=0.7)

    # ترسیم بردارهای پشتیبان (دایره سیاه)
    if len(self.support_vectors) > 0:
        plt.scatter(self.support_vectors[:, 0], self.support_vectors[:, 1],
                    facecolors='none', edgecolors='black', s=300,
                    linewidth=2, label='Support Vectors')

    # ترسیم نقاطی که از حاشیه عبور کرده‌اند (علامت ضرب)
    if len(self.slack_points) > 0:
        slack_array = np.array([p['point'] for p in self.slack_points])
        plt.scatter(slack_array[:, 0], slack_array[:, 1],
                    c='orange', s=150, marker='x', linewidth=3,
                    label=f'Points with Slack ( $\xi > 0$ ): {len(self.slack_points)}')

    # ترسیم مرزها
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x_range = np.linspace(x_min, x_max, 100)

    decision_line, margin_line_1, margin_line_2 = self.get_decision_function(x_range)

    plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2,
             label=f'Decision Boundary')
    plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5,
             label='Margin Boundary (w·x + b = ±1)')
    plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5)

    # پر کردن ناحیه حاشیه
    plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                    color='gray', alpha=0.2,
                    label=f'Margin (p = {self.margin:.2f})')

    plt.xlabel('Feature 1 (Biomarker Level)', fontsize=12)
    plt.ylabel('Feature 2 (Gene Expression)', fontsize=12)
    plt.title(f'{title}\nC={self.C}, Support Vectors={len(self.support_vectors)}, '
              f'Margin={self.margin:.3f}', fontsize=14, fontweight='bold')
    plt.legend(loc='best', fontsize=10)
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# =====
# ایجاد داده‌های مصنوعی برای مثال تشخیص پزشکی
# داده‌ها شامل دو ویژگی: سطح مارکر زیستی و بیان ژن
# =====

np.random.seed(42)

# کلاس سرطانی (class -1)
X_cancer = np.random.randn(15, 2) + [2, 2]
# برای شبیه‌سازی نویز واقعی (outliers) اضافه کردن نقاط پرت
X_cancer = np.vstack([X_cancer, [3.5, 3.5], [4, 3]])

# کلاس غیرسرطانی (class +1)
X_healthy = np.random.randn(15, 2) + [3.5, 3.5]
# اضافه کردن نقاط پرت
X_healthy = np.vstack([X_healthy, [2.5, 2.5], [3, 4]])

# ترکیب داده‌ها
X_medical = np.vstack([X_cancer, X_healthy])
y_medical = np.hstack([-np.ones(len(X_cancer)), np.ones(len(X_healthy))])

print("="*60)
print("Soft-Margin SVM برای تشخیص پزشکی")
print("داده حاوی نقاط همپوشانی (non-separable)")
print("="*60)
print(f"تعداد نمونه‌ها: {len(X_medical)}")
print(f"تعداد ویژگی‌ها: {X_medical.shape[1]}")

```

```
# =====
# مقایسه عملکرد برای مقادیر مختلف C
# =====

C_values = [0.1, 1.0, 10.0, 100.0]
models = {}

for C in C_values:
    print(f"\n{' '*40}")
    print(f"C = {C} آموزش مدل با")
    print(f"{' '*40}")

    model = SoftMarginSVM(C=C)
    model.fit(X_medical, y_medical)
    models[C] = model

    # دقت روی داده آموزشی
    predictions = model.predict(X_medical)
    accuracy = accuracy_score(y_medical, predictions)

    print(f"بردار وزن: [{model.w[0]:.4f}, {model.w[1]:.4f}]")
    print(f"بایاس: {model.b:.4f}")
    print(f"اندازه بردار وزن (||w||): {np.linalg.norm(model.w):.4f}")
    print(f"حاشیه هندسی: {model.margin:.4f}")
    print(f"تعداد بردارهای پشتیبان: {len(model.support_vectors)}")
    print(f"تعداد نقاط با لغزش: {len(model.slack_points)}")
    print(f"دقت روی داده آموزشی: {accuracy:.2%}")

    # ترسیم نتایج
    model.visualize(X_medical, y_medical, title=f"Medical Diagnosis with Soft-Margin SVM (C={C})")

# =====
# روی همین داده (Hard-Margin) مقایسه با حاشیه سخت
# =====

print(f"\n{' '*40}")
print("Hard-Margin SVM مقایسه با")
print(f"{' '*40}")

# بسیار بزرگ C ایجاد مدل حاشیه سخت با استفاده از
hard_margin_model = SoftMarginSVM(C=1e6) # C بسیار بزرگ ≈ حاشیه سخت C
hard_margin_model.fit(X_medical, y_medical)

predictions_hard = hard_margin_model.predict(X_medical)
accuracy_hard = accuracy_score(y_medical, predictions_hard)

print(f"[Hard-Margin] تعداد بردارهای پشتیبان: {len(hard_margin_model.support_vectors)}")
print(f"[Hard-Margin] تعداد نقاط با لغزش: {len(hard_margin_model.slack_points)}")
print(f"[Hard-Margin] حاشیه: {hard_margin_model.margin:.4f}")
print(f"[Hard-Margin] دقت: {accuracy_hard:.2%}")

hard_margin_model.visualize(X_medical, y_medical, title="Hard-Margin SVM on Non-separable Data")

# =====
# C تحلیل حساسیت به پارامتر
# =====

print(f"\n{' '*40}")
print("C تحلیل حساسیت به پارامتر")
print(f"{' '*40}")

# ایجاد داده تست جدید برای ارزیابی تعمیم‌پذیری
np.random.seed(123)
X_test_cancer = np.random.randn(8, 2) + [2, 2]
X_test_healthy = np.random.randn(8, 2) + [3.5, 3.5]
X_test_medical = np.vstack([X_test_cancer, X_test_healthy])
y_test_medical = np.hstack([-np.ones(len(X_test_cancer)), np.ones(len(X_test_healthy))])

print("دقت روی داده تست:")
for C in C_values:
    model = models[C]
    test_pred = model.predict(X_test_medical)
    test_acc = accuracy_score(y_test_medical, test_pred)
    print(f"C={C:6.1f}: دقت = {test_acc:.2%}")

# =====
# نمایش پیچیدگی محاسباتی
# =====

print(f"\n{' '*40}")
print("تحلیل پیچیدگی محاسباتی")
print(f"{' '*40}")

# و تعداد بردارهای پشتیبان C بررسی رابطه بین
print("و تعداد بردارهای پشتیبان C رابطه بین:")
for C in sorted(models.keys()):
    model = models[C]
    sv_count = len(model.support_vectors)
    slack_count = len(model.slack_points)
    print(f"C={C:6.1f}: SVs={sv_count:2d}, Slack Points={slack_count:2d}")
    # معمولاً افزایش می‌یابد (پیچیده‌تر شدن مدل) SV تعداد، C، با افزایش

print("\nبا دقت انتخاب شود C، نکته: در سیستم‌های پزشکی واقعی")
print("خیلی بزرگ: مدل پیچیده، زمان تست طولانی (چالش جدول 15.1) - C")
print("خیلی کوچک: مدل ساده، ممکن است خطای بالا داشته باشد - C")

# =====
# بر روی حاشیه و دقت C تحلیل تأثیر
# =====

print(f"\n{' '*40}")
print("بر روی حاشیه و دقت C تحلیل تأثیر")
```

```
print(f"{' '*40}")

plt.figure(figsize=(10, 6))
C_values_plot = C_values
train_accuracies = [accuracy_score(y_medical, models[C].predict(X_medical)) for C in C_values_plot]
test_accuracies = [accuracy_score(y_test_medical, models[C].predict(X_test_medical)) for C in C_values_plot]
margins = [models[C].margin for C in C_values_plot]

plt.subplot(1, 2, 1)
plt.plot(C_values_plot, train_accuracies, 'bo-', label='Train Accuracy')
plt.plot(C_values_plot, test_accuracies, 'ro-', label='Test Accuracy')
plt.xscale('log')
plt.xlabel('C (log scale)')
plt.ylabel('Accuracy')
plt.title('Accuracy vs. C')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(C_values_plot, margins, 'go-')
plt.xscale('log')
plt.xlabel('C (log scale)')
plt.ylabel('Margin')
plt.title('Margin vs. C')
plt.grid(True)

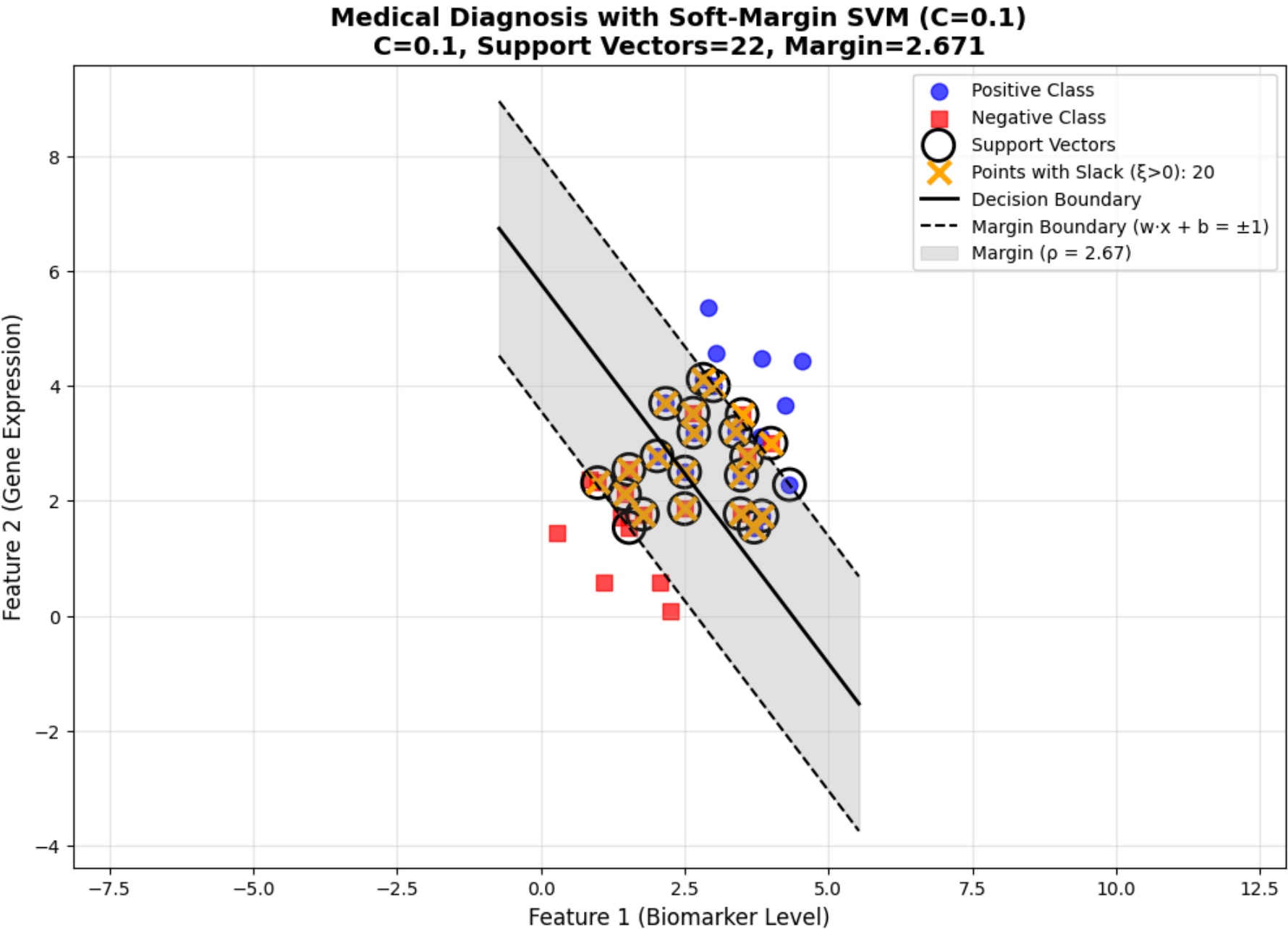
plt.tight_layout()
plt.show()
```

برای تشخیص پزشکی Soft-Margin SVM
(non-separable) داده حاوی نقاط همپوشانی

تعداد نمونه‌ها: 34
تعداد ویژگی‌ها: 2

C = 0.1 آموزش مدل با

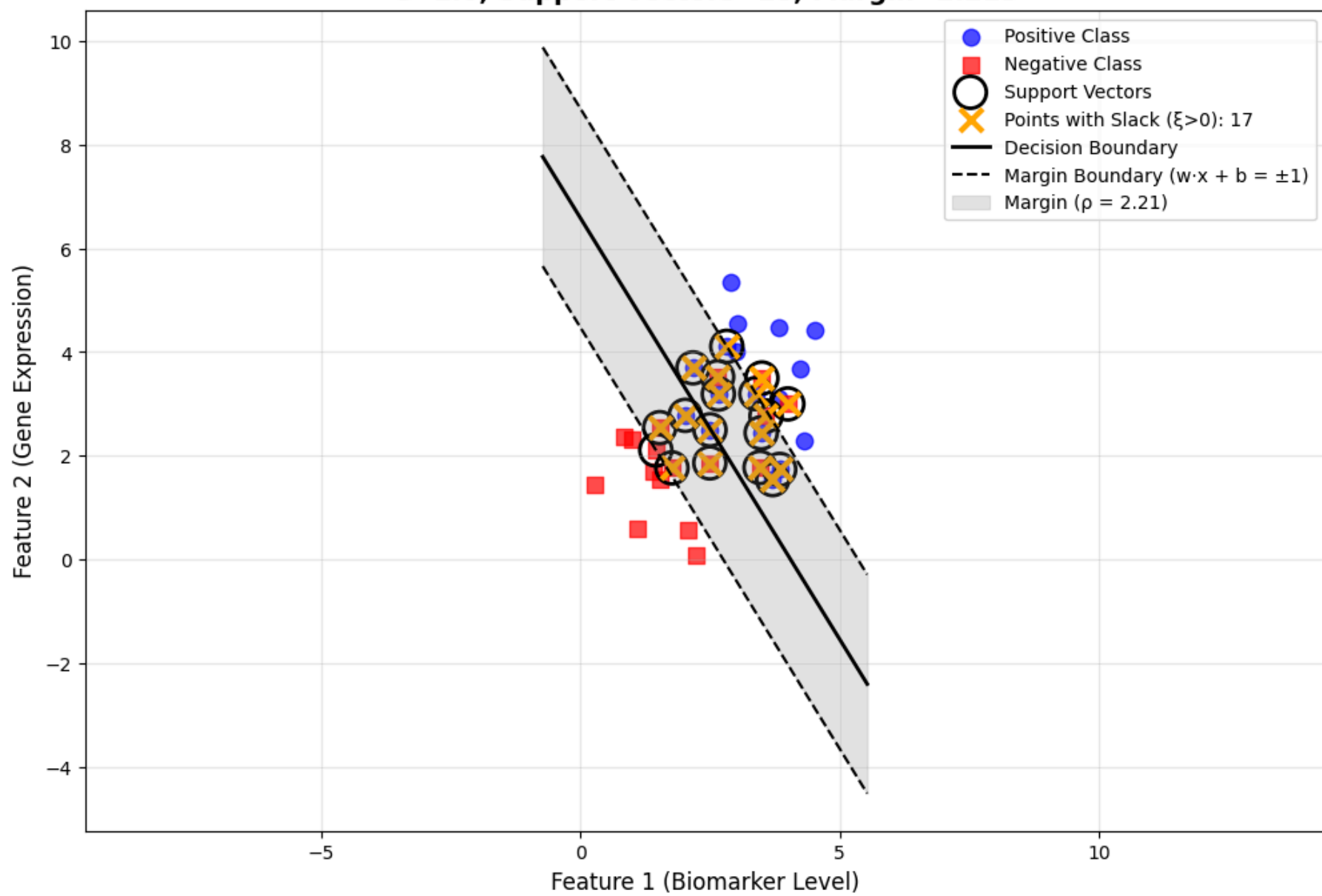
بردار وزن: [0.4519, 0.5971]
بایاس: -2.6108
اندازه بردار وزن ($||w||$): 0.7488
حاشیه هندسی: 2.6710
تعداد بردارهای پشتیبان: 22
تعداد نقاط با لغزش: 20
%دقت روی داده آموزشی: 82.35



C = 1.0 آموزش مدل با

بردار وزن: [0.4734, 0.7699]
بایاس: -3.1200
اندازه بردار وزن ($||w||$): 0.9038
حاشیه هندسی: 2.2129
تعداد بردارهای پشتیبان: 18
تعداد نقاط با لغزش: 17
%دقت روی داده آموزشی: 79.41

Medical Diagnosis with Soft-Margin SVM (C=1.0)
C=1.0, Support Vectors=18, Margin=2.213



=====

C = 10.0 آموزش مدل با

=====

بردار وزن: [0.8001, 0.4919]

بایاس: -3.2815

اندازه بردار وزن ($||w||$): 0.9392

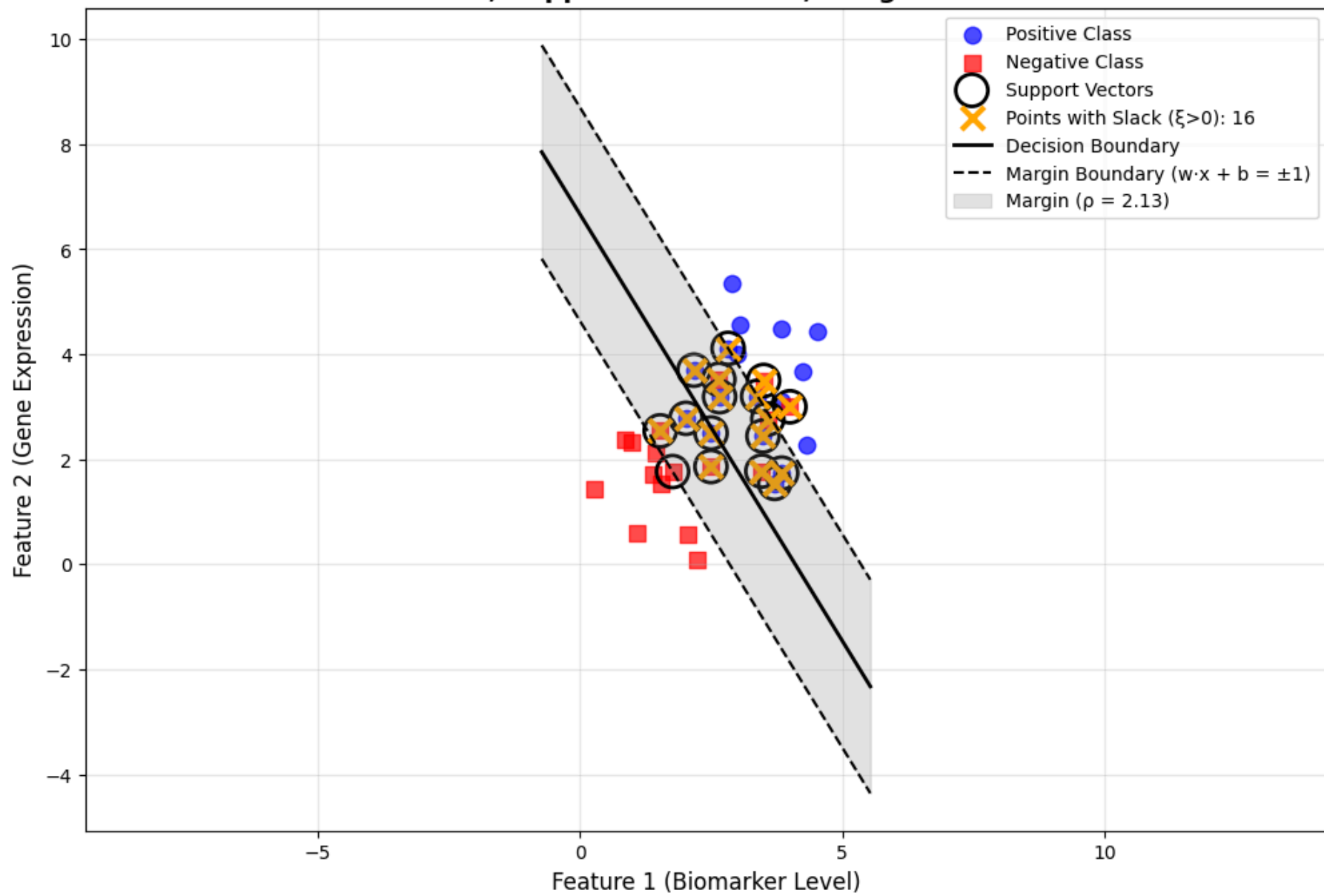
حاشیه هندسی: 2.1295

تعداد بردارهای پشتیبان: 17

تعداد نقاط با لغزش: 16

%دقت روی داده آموزشی: 79.41

Medical Diagnosis with Soft-Margin SVM (C=10.0)
C=10.0, Support Vectors=17, Margin=2.129



=====

C = 100.0 آموزش مدل با

=====

بردار وزن: [0.4920, 0.8001]

بایاس: -3.2816

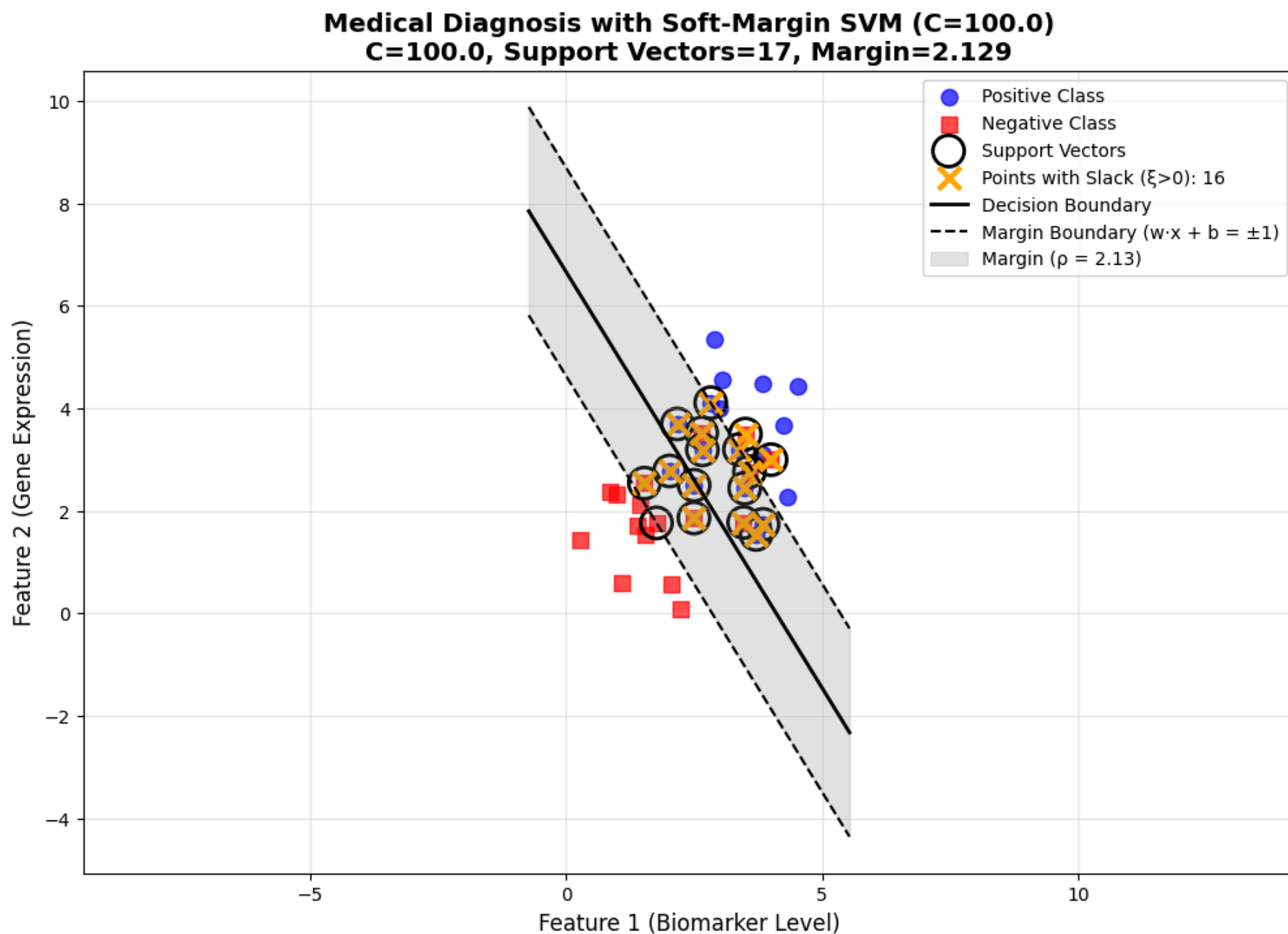
اندازه بردار وزن ($||w||$): 0.9393

حاشیه هندسی: 2.1294

تعداد بردارهای پشتیبان: 17

تعداد نقاط با لغزش: 16

%دقت روی داده آموزشی: 79.41



=====

Hard-Margin SVM مقایسه با

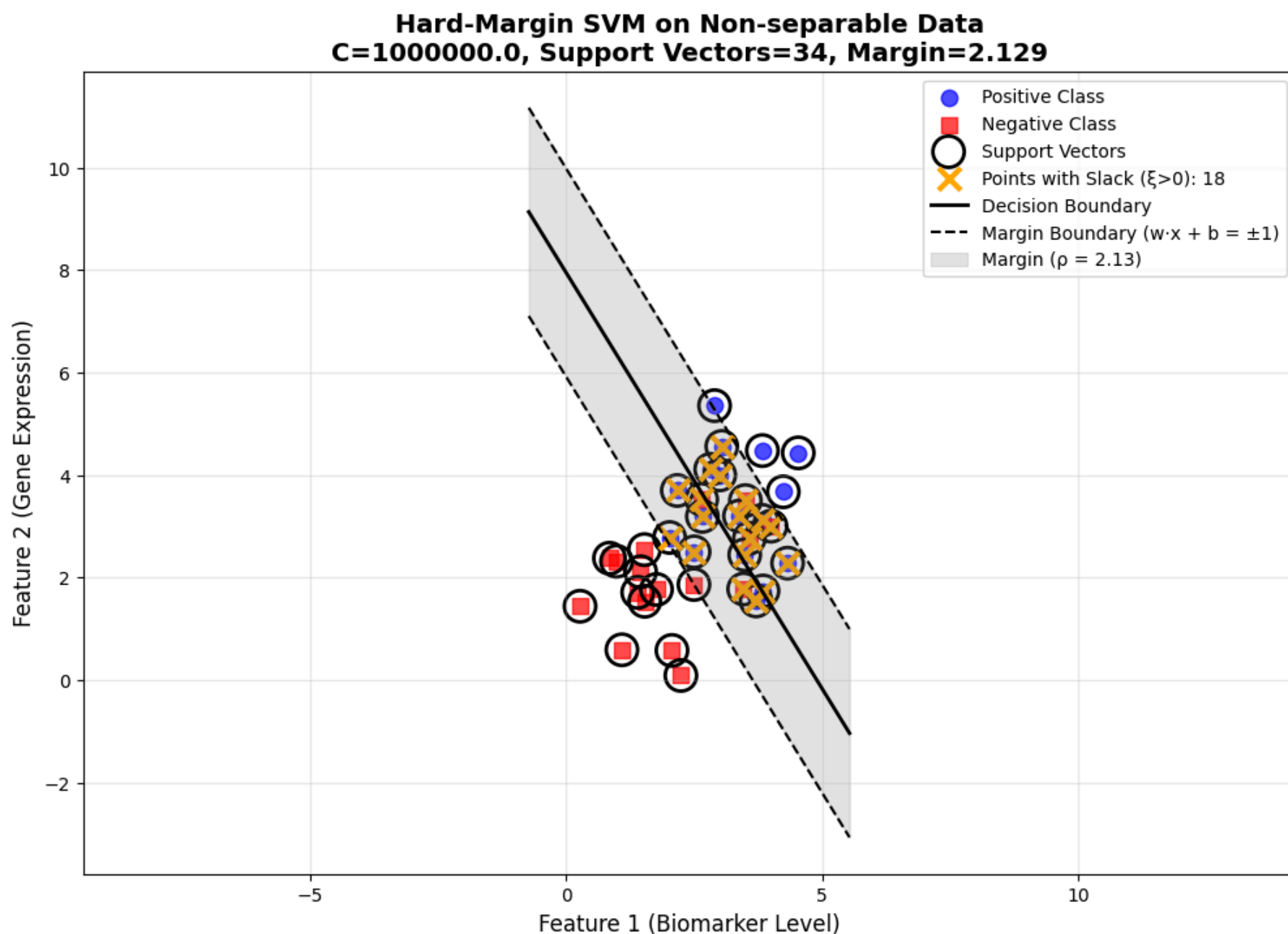
=====

تعداد بردارهای پشتیبان: 34 [Hard-Margin]

تعداد نقاط با لغزش: 18 [Hard-Margin]

حاشیه: 2.1293 [Hard-Margin]

%دقت: 76.47 [Hard-Margin]

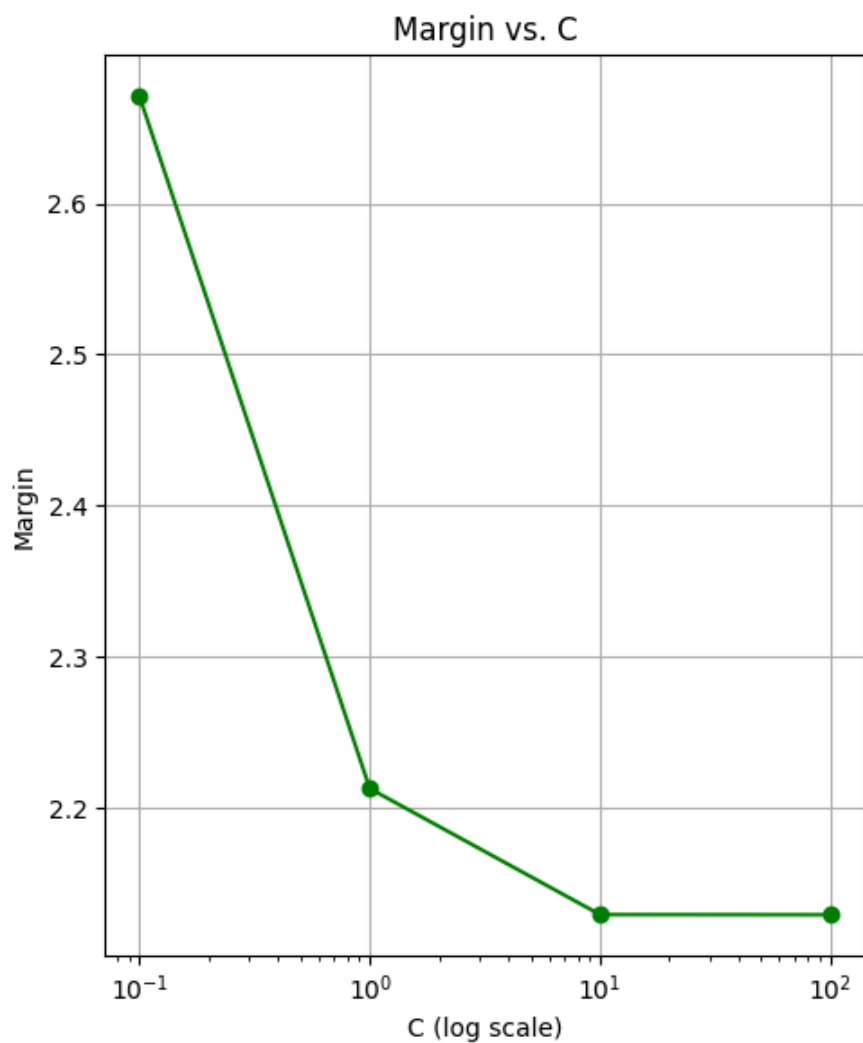
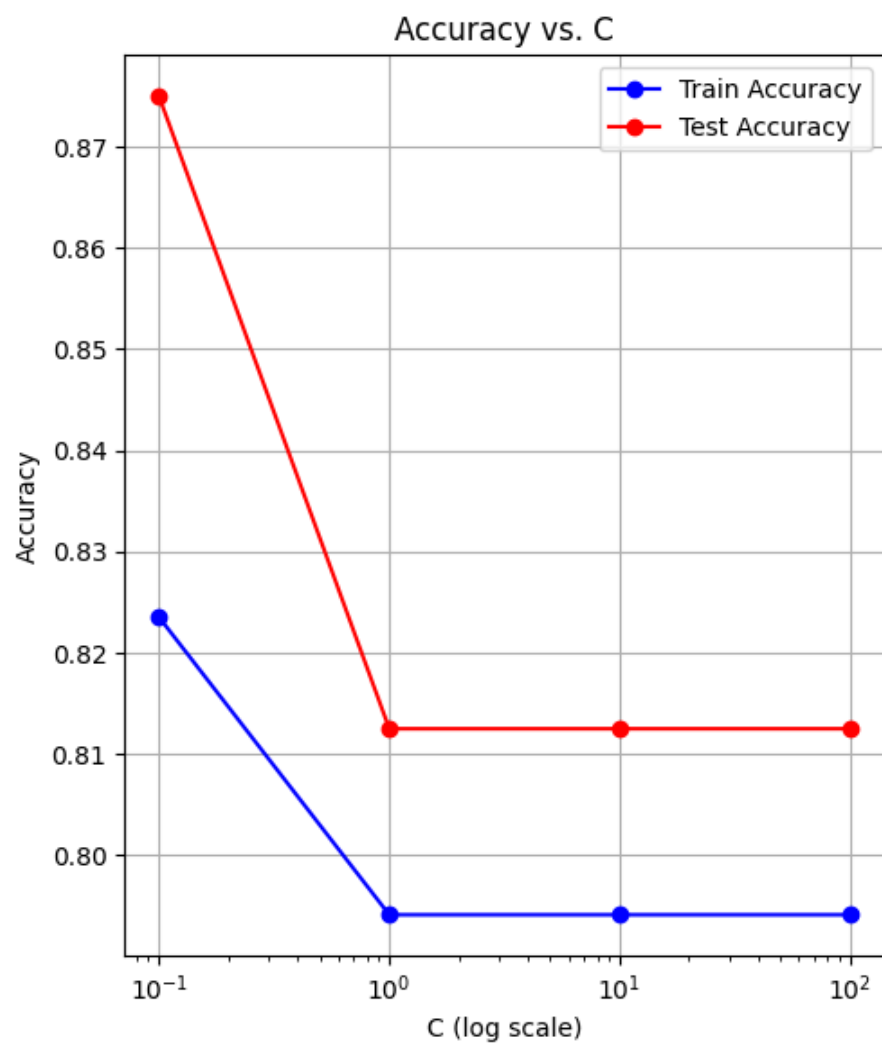


```
=====
C تحلیل حساسیت به پارامتر
=====
دقت روی داده تست:
C= 0.1: 87.50 = %دقت
C= 1.0: 81.25 = %دقت
C= 10.0: 81.25 = %دقت
C= 100.0: 81.25 = %دقت

=====
تحلیل پیچیدگی محاسباتی
=====
و تعداد بردارهای پشتیبان C رابطه بین
C= 0.1: SVs=22, Slack Points=20
C= 1.0: SVs=18, Slack Points=17
C= 10.0: SVs=17, Slack Points=16
C= 100.0: SVs=17, Slack Points=16

.باید با دقت انتخاب شود C ،نکته: در سیستم‌های پزشکی واقعی
- خیلی بزرگ: مدل پیچیده، زمان تست طولانی (چالش جدول 15.1) C
- خیلی کوچک: مدل ساده، ممکن است خطای بالا داشته باشد C

=====
بر روی حاشیه و دقت C تحلیل تأثیر
=====
```



15.2.2 ماشین‌های بردار پشتیبان چندکلاسه: تقسیم‌بندی هوشمند تصاویر 🖼️

📸 **گوگل فوتو: چگونه یک میلیارد تصویر را به صورت خودکار دسته‌بندی کنیم؟**

تصور کنید در تیم توسعه گوگل فوتو کار می‌کنید و باید سیستمی طراحی کنید که به طور خودکار تصاویر کاربران را به دسته‌های مختلفی مانند "ساحل"، "غذا"، "حیوانات خانگی"، "خانواده" و... تقسیم کند. این یک چالش بزرگ در طبقه‌بندی چندکلاسه است، زیرا باید بتوانیم بین ده‌ها یا صدها کلاس مختلف تمایز قائل شویم.

SVM‌ها به طور ذاتی طبقه‌بندهای دو کلاسه هستند، اما در دنیای واقعی با مسائل چندکلاسه روبرو هستیم. در گوگل فوتو، ما نمی‌توانیم فقط بگوییم "این تصویر ساحل است یا ساحل نیست"، بلکه باید بتوانیم از بین صدها گزینه، بهترین دسته را انتخاب کنیم.

روش سنتی برای انجام طبقه‌بندی چندکلاسه با SVM‌ها استفاده از یکی از روش‌های بحث شده در بخش 14.5 است. به طور خاص، رایج‌ترین تکنیک در عمل، ساخت |C| طبقه‌بند یک-در-برابر-بقیه (معمولاً به عنوان "یک-در-برابر-همه" یا طبقه‌بندی OVA شناخته می‌شود) است.

🎯 **روش یک-در-برابر-همه: از دو به چند**

در مثال گوگل فوتو، این روش به این صورت عمل می‌کند:

- یک طبقه‌بند SVM برای تشخیص "ساحل در برابر غیرساحل"
- یک طبقه‌بند SVM برای تشخیص "غذا در برابر غیرغذا"
- یک طبقه‌بند SVM برای تشخیص "حیوانات خانگی در برابر غیرحیوانات خانگی"
- و به همین ترتیب برای هر کلاس...

سپس برای یک تصویر جدید، کلاسی انتخاب می‌شود که داده آزمون را با بیشترین حاشیه طبقه‌بندی می‌کند. این روش ساده است اما می‌تواند مشکلاتی ایجاد کند، به خصوص زمانی که کلاس‌ها نامتعادل هستند.

✂ روش یک-در-برابر-یک: مقایسه مستقیم

استراتژی دیگر، ساخت مجموعه‌ای از طبقه‌بندهای یک-در-برابر-یک است. در مثال گوگل فوتو:

- یک طبقه‌بند برای "ساحل در برابر غذا"
- یک طبقه‌بند برای "ساحل در برابر حیوانات خانگی"
- یک طبقه‌بند برای "غذا در برابر حیوانات خانگی"
- و به همین ترتیب برای هر جفت کلاس...

سپس کلاسی انتخاب می‌شود که توسط اکثر طبقه‌بندها انتخاب شده است. اگرچه این شامل ساخت $C(|C| - 1)/2$ طبقه‌بند است، اما زمان آموزش طبقه‌بندها ممکن است در واقع کاهش یابد، زیرا مجموعه داده آموزشی برای هر طبقه‌بند بسیار کوچکتر است.

با این حال، این رویکردها برای حل مسائل چندکلاسه بسیار زیبا نیستند. جایگزین بهتر توسط ساخت SVMهای چندکلاسه فراهم می‌شود، جایی که یک طبقه‌بند دو کلاسه روی یک بردار ویژگی $\Phi(\tilde{x}, y)$ مشتق شده از جفت شامل ویژگی‌های ورودی و کلاس داده می‌سازیم.

🔑 رویکرد یکپارچه: SVM چندکلاسه واقعی

در این روش پیشرفته‌تر، به جای ساخت چندین طبقه‌بند دو کلاسه، یک مدل یکپارچه ساخته می‌شود که همزمان تمام کلاس‌ها را در نظر می‌گیرد. در مثال گوگل فوتو:

- یک مدل واحد که ویژگی‌های تصویر را با هر کلاس ممکن ترکیب می‌کند
 - در زمان آزمون، طبقه‌بند کلاس $y = \arg \max_{y'} w^{\sim T} \Phi(\tilde{x}, y')$ را انتخاب می‌کند
 - حاشیه در طول آموزش، شکاف بین این مقدار برای کلاس صحیح و نزدیک‌ترین کلاس دیگر است
- این روش کلی را می‌توان برای ارائه فرمولاسیون چندکلاسه از انواع مختلف طبقه‌بندهای خطی گسترش داد.

این روش همچنین یک نمونه ساده از تعمیم طبقه‌بندی است که در آن کلاس‌ها فقط مجموعه‌ای از برچسب‌های دسته‌بندی مستقل نیستند، بلکه ممکن است اشیاء ساختاریافته با روابط تعریف شده بین آنها باشند. در دنیای SVM، چنین کارهایی تحت برچسب **SVMهای ساختاری** قرار می‌گیرند. ما در بخش 15.4.2 دوباره به آنها اشاره می‌کنیم.

🌐 کاربرد در دنیای واقعی: طبقه‌بندی محتوای یوتیوب

یوتیوب از الگوریتم‌های طبقه‌بندی چندکلاسه پیشرفته برای دسته‌بندی میلیون‌ها ویدیو استفاده می‌کند. این سیستم باید بتواند ویدیوها را به دسته‌هایی مانند "موسیقی"، "ورزشی"، "آموزشی"، "سرگرمی" و... تقسیم کند.

در این کاربرد، روش یک-در-برابر-همه ممکن است برای دسته‌های اصلی استفاده شود، در حالی که برای زیرشاخه‌های خاص (مانند "فوتبال" در برابر "بسکتبال" در دسته ورزشی) از روش یک-در-برابر-یک استفاده می‌شود. این رویکرد ترکیبی به یوتیوب اجازه می‌دهد تا همزمان دقت و سرعت را بهینه کند.

In [25]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from cvxopt import matrix, solvers
import time

# =====
# QP از پایه با) دوکلاسه اصلی SVM پیاده‌سازی
# این کلاس پایه تمام روش‌های چندکلاسه استفاده می‌کند
# =====

class BinarySVM_QP:
    """
    دوکلاسه با بهینه‌سازی درجه دو SVM پیاده‌سازی سخت‌افزاری
    این کلاس توسط روش‌های چندکلاسه استفاده می‌شود
    """
    def __init__(self, C=1.0):
        self.C = C
        self.w = None
        self.b = None
        self.alpha = None
        self.support_vectors = None

    def fit(self, X, y):
        n_samples = X.shape[0]

        # محاسبه ماتریس P: P_ij = y_i * y_j * dot(x_i, x_j)
        P = np.outer(y, y) * np.dot(X, X.T)
        P = matrix(P)
        q = matrix(-np.ones(n_samples))

        # محدودیت: -α_i ≤ 0 و α_i ≤ C
        G = np.vstack([-np.eye(n_samples), np.eye(n_samples)])
        h = np.hstack([np.zeros(n_samples), self.C * np.ones(n_samples)])
        G, h = matrix(G), matrix(h)

        # محدودیت: Σ α_i * y_i = 0
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)

        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)

        self.alpha = np.array(solution['x']).flatten()

        # شناسایی بردارهای پشتیبان
        sv_indices = self.alpha > 1e-5
        self.support_vectors = X[sv_indices]
        sv_y = y[sv_indices]
        sv_alpha = self.alpha[sv_indices]

        # محاسبه w = Σ α_i * y_i * x_i
        self.w = np.sum(sv_alpha.reshape(-1, 1) * sv_y.reshape(-1, 1) * self.support_vectors, axis=0)

        # محاسبه b از یک SV 0 < α_i < C
        margin_sv = (self.alpha > 1e-5) & (self.alpha < self.C - 1e-5)
        if np.any(margin_sv):
            idx = np.where(margin_sv)[0][0]
            self.b = y[idx] - np.dot(self.w, X[idx])
        else:
            self.b = 0

    def predict(self, X):
        return np.sign(np.dot(X, self.w) + self.b)

    def decision_function(self, X):
        """فاصله از ابرصفحه (برای انتخاب کلاس با حاشیه بیشتر)"""
        return np.dot(X, self.w) + self.b

# =====
# OVA) چندکلاسه با رویکرد یک-در-برابر-همه SVM پیاده‌سازی
# =====

class OneVsAllSVM:
    """
    SVM (One-vs-All) چندکلاسه با رویکرد یک-در-برابر-همه
    همه بقیه] vs دوکلاسه می‌سازد: [کلاس SVM برای هر کلاس، یک
    """
    def __init__(self, C=1.0):
        self.C = C
```

```

self.classifiers = {} # کلاس -> classifier
self.classes = None

def fit(self, X, y):
    """
    برای هر کلاس classifier آموزش یک
    تعداد کلاس‌هاست |C| که  $O(|C| * N^3)$ : پیچیدگی
    """
    self.classes = np.unique(y)
    training_times = {}

    for cls in self.classes:
        print(f" همه... vs {cls} برای کلاس classifier آموزش")
        start_time = time.time()

        # ساخت داده دوکلاسه: 1 برای این کلاس، -1 برای بقیه
        binary_y = np.where(y == cls, 1, -1)

        # دوکلاسه SVM آموزش
        classifier = BinarySVM_QP(C=self.C)
        classifier.fit(X, binary_y)

        self.classifiers[cls] = classifier
        training_times[cls] = time.time() - start_time

    return training_times

def predict(self, X, method='score'):
    """
    پیش‌بینی با دو روش:
    - 'score': decision function value انتخاب کلاس با بیشترین
    - 'vote': decision function value انتخاب کلاس با بیشترین
    """
    n_samples = X.shape[0]
    n_classes = len(self.classes)

    if method == 'score':
        # محاسبه امتیاز برای هر کلاس
        scores = np.zeros((n_samples, n_classes))

        for i, cls in enumerate(self.classes):
            scores[:, i] = self.classifiers[cls].decision_function(X)

        # انتخاب کلاس با بیشترین امتیاز
        max_indices = np.argmax(scores, axis=1)
        return np.array([self.classes[i] for i in max_indices])

    elif method == 'vote':
        # رأی‌گیری بر اساس مقدار تابع تصمیم
        votes = np.zeros((n_samples, n_classes))

        for i, cls in enumerate(self.classes):
            clf = self.classifiers[cls]
            # استفاده مستقیم از مقدار تابع تصمیم برای رأی‌گیری
            votes[:, i] = clf.decision_function(X)

        max_indices = np.argmax(votes, axis=1)
        return np.array([self.classes[i] for i in max_indices])

# =====
# (OVO) چندکلاسه با رویکرد یک-در-برابر یک SVM پیاده‌سازی
# =====

class OneVsOneSVM:
    """
    (One-vs-One) چندکلاسه با رویکرد یک-در-برابر یک SVM
    دوکلاسه می‌سازد SVM برای هر جفت کلاس، یک
    """
    def __init__(self, C=1.0):
        self.C = C
        self.classifiers = {} # (cls1, cls2) -> classifier
        self.classes = None

    def fit(self, X, y):
        """
        برای هر جفت کلاس classifier آموزش
        روی داده کمتری آموزش می‌بیند classifier اما هر  $O(|C|^2 * (N/|C|)^3) = O(|C| * N^3)$ : پیچیدگی
        """
        self.classes = np.unique(y)
        training_times = {}

        # ساخت تمام ترکیب‌های جفت کلاس‌ها
        for i, cls1 in enumerate(self.classes):
            for j, cls2 in enumerate(self.classes):
                if i < j:
                    print(f" {cls1} vs {cls2}... برای classifier آموزش")
                    start_time = time.time()

                    # فیلتر کردن داده فقط برای این دو کلاس
                    mask = (y == cls1) | (y == cls2)
                    X_pair = X[mask]
                    y_pair = y[mask]

                    # ساخت برچسب‌های دو کلاسه: 1 برای cls1، -1 برای cls2
                    binary_y = np.where(y_pair == cls1, 1, -1)

                    # دوکلاسه SVM آموزش
                    classifier = BinarySVM_QP(C=self.C)
                    classifier.fit(X_pair, binary_y)

                    self.classifiers[(cls1, cls2)] = classifier
                    training_times[(cls1, cls2)] = time.time() - start_time

```

```

        return training_times

def predict(self, X):
    """
    رأی می‌دهد، کلاس با بیشترین آرا انتخاب می‌شود classifier رأی‌گیری اکثریت: هر
    """
    n_samples = X.shape[0]
    votes = np.zeros((n_samples, len(self.classes)))

    for (cls1, cls2), classifier in self.classifiers.items():
        # دوکلاس classifier پیش‌بینی با
        predictions = classifier.predict(X)

        # تبدیل پیش‌بینی‌ها به رأی برای کلاس‌های اصلی
        idx1 = np.where(self.classes == cls1)[0][0]
        idx2 = np.where(self.classes == cls2)[0][0]

        votes[predictions == 1, idx1] += 1
        votes[predictions == -1, idx2] += 1

    # انتخاب کلاس با بیشترین آرا
    max_indices = np.argmax(votes, axis=1)
    return np.array([self.classes[i] for i in max_indices])

# =====
# تست و مقایسه روش‌ها روی داده مصنوعی (شبیه‌سازی تصویر)
# =====

print("="*60)
print("Multiclass SVM: Comparison of OVA vs OVO")
print("Simulating Image Classification (3 classes)")
print("="*60)

# (دو ویژگی از تصویر extraction مانند) برای نمایش D ایجاد داده 2
np.random.seed(42)
X, y = make_blobs(n_samples=300, centers=3, n_features=2,
                  cluster_std=1.5, random_state=42)

# train و test تقسیم داده به
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
print(f"Number of classes: {len(np.unique(y_train))}")

# =====
# OVA آموزش مدل
# =====

print("\n" + "-"*40)
print("Training One-vs-All SVM")
print("-"*40)

ova_model = OneVsAllSVM(C=1.0)
ova_train_times = ova_model.fit(X_train, y_train)

print(f"Number of classifiers: {len(ova_model.classifiers)}")

# =====
# OVO آموزش مدل
# =====

print("\n" + "-"*40)
print("Training One-vs-One SVM")
print("-"*40)

ovo_model = OneVsOneSVM(C=1.0)
ovo_train_times = ovo_model.fit(X_train, y_train)

print(f"Number of classifiers: {len(ovo_model.classifiers)}")

# =====
# ارزیابی و مقایسه
# =====

print("\n" + "-"*40)
print("Evaluation Results")
print("="*40)

# ارزیابی OVA
ova_pred_test = ova_model.predict(X_test, method='score')
ova_pred_train = ova_model.predict(X_train, method='score')
ova_test_acc = accuracy_score(y_test, ova_pred_test)
ova_train_acc = accuracy_score(y_train, ova_pred_train)

print(f"One-vs-All SVM:")
print(f"  Train Accuracy: {ova_train_acc:.4f}")
print(f"  Test Accuracy:  {ova_test_acc:.4f}")

# ارزیابی OVO
ovo_pred_test = ovo_model.predict(X_test)
ovo_pred_train = ovo_model.predict(X_train)
ovo_test_acc = accuracy_score(y_test, ovo_pred_test)
ovo_train_acc = accuracy_score(y_train, ovo_pred_train)

print(f"\nOne-vs-One SVM:")
print(f"  Train Accuracy: {ovo_train_acc:.4f}")
print(f"  Test Accuracy:  {ovo_test_acc:.4f}")

# (دقت کمی بالاتر دارد OVO معمولاً) مقایسه دقت
print(f"\nDifference (OVO - OVA): {ovo_test_acc - ova_test_acc:.4f}")

```

```
# =====
# تحلیل پیچیدگی محاسباتی (طبق جدول 15.1)
# =====

print("\n" + "="*40)
print("Computational Complexity Analysis")
print("="*40)

# محاسبه زمان آموزش کل
ova_total_time = sum(ova_train_times.values())
ovo_total_time = sum(ovo_train_times.values())

print(f"Total Training Time OVA: {ova_total_time:.4f} sec")
print(f"Total Training Time OVO: {ovo_total_time:.4f} sec")

# محاسبه تعداد بردارهای پشتیبان برای هر روش
ova_total_sv = sum(len(clf.support_vectors) for clf in ova_model.classifiers.values())
ovo_total_sv = sum(len(clf.support_vectors) for clf in ovo_model.classifiers.values())
avg_sv_per_classifier_ova = ova_total_sv / len(ova_model.classifiers)
avg_sv_per_classifier_ovo = ovo_total_sv / len(ovo_model.classifiers)

print(f"\nSupport Vector Analysis:")
print(f"  OVA - Total SVs: {ova_total_sv}, Avg per classifier: {avg_sv_per_classifier_ova:.1f}")
print(f"  OVO - Total SVs: {ovo_total_sv}, Avg per classifier: {avg_sv_per_classifier_ovo:.1f}")

# =====
# نمایش تصمیم‌گیری برای هر روش
# =====

def plot_multiclass_decision_boundary(model, X, y, title=""):
    """ترسیم مرزهای تصمیم‌گیری برای مسائل چندکلاسه"""
    plt.figure(figsize=(12, 8))

    # ترسیم نقاط داده
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', s=60, edgecolors='k', alpha=0.7)

    # ایجاد شبکه برای مناطق تصمیم‌گیری
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # پیش‌بینی برای شبکه
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # ترسیم مناطق تصمیم‌گیری
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')

    plt.xlabel('Feature 1 (e.g., Color Histogram)', fontsize=12)
    plt.ylabel('Feature 2 (e.g., Texture)', fontsize=12)
    plt.title(title, fontsize=14, fontweight='bold')
    plt.colorbar(scatter, label='Class')
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# ترسیم نتایج
plot_multiclass_decision_boundary(ova_model, X_train, y_train,
                                title=f"OVA Decision Boundaries (C=1.0)")

plot_multiclass_decision_boundary(ovo_model, X_train, y_train,
                                title=f"OVO Decision Boundaries (C=1.0)")

# =====
# (Digits Dataset) تست روی داده واقعی: دیتاست ارقام
# از ارقام 0-9 است ۸x۸ این دیتاست شامل تصاویر 8
# =====

print("\n" + "="*60)
print("Test on Real Dataset: Digits Classification (10 classes)")
print("="*60)

# بارگذاری داده ارقام
digits = load_digits()
X_digits = digits.data # ویژگی 1797
y_digits = digits.target

# کاهش داده برای سرعت بیشتر (استفاده از 300 نمونه)
subset_indices = np.random.choice(len(X_digits), 300, replace=False)
X_subset = X_digits[subset_indices]
y_subset = y_digits[subset_indices]

# تقسیم داده
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    X_subset, y_subset, test_size=0.3, random_state=42
)

print(f"Training samples: {len(X_train_d)}")
print(f"Features per sample: {X_train_d.shape[1]}")
print(f"Classes: {np.unique(y_train_d)}")

# روی داده ارقام OVA آموزش
print("\nTraining OVA on Digits...")
ova_digits = OneVsAllSVM(C=1.0)
ova_digits.fit(X_train_d, y_train_d)
ova_pred_d = ova_digits.predict(X_test_d)
ova_acc_d = accuracy_score(y_test_d, ova_pred_d)
print(f"OVA Accuracy on Digits: {ova_acc_d:.4f}")

# روی داده ارقام OVO آموزش
print("\nTraining OVO on Digits...")
```

```
ovo_digits = OneVsOneSVM(C=1.0)
ovo_digits.fit(X_train_d, y_train_d)
ovo_pred_d = ovo_digits.predict(X_test_d)
ovo_acc_d = accuracy_score(y_test_d, ovo_pred_d)
print(f"OVO Accuracy on Digits: {ovo_acc_d:.4f}")

print(f"\nDifference (OVO - OVA): {ovo_acc_d - ova_acc_d:.4f}")

# =====
# تحلیل نهایی: کدام روش برای گوگل فوتو مناسبتر است؟
# =====

print("\n" + "="*60)
print("کدام استراتژی برای گوگل فوتو مناسبتر است؟")
print("="*60)

print("بر اساس تحلیل پیچیدگی از جدول 15.1")
print("• OVA: |C| N دسته‌بند، هر کدام روی |C|")
print("• OVO: |C|(|C|-1)/2 2 تقریباً هر کدام روی تقریباً N/|C|دسته‌بند،")
print("• دسته |C|=100 برای گوگل فوتو با")
print(" - OVA: 100 دسته‌بند")
print(" - OVO: 4950 دسته‌بند")
print()
print("توصیه عملی:")
print("• برای سیستم‌های بزرگ با مقیاس بزرگ (دسته‌بندهای کمتر) استفاده کنید OVA از")
print("• زمانی استفاده کنید که دقت بالا حیاتی است و کلاس‌ها متوازن هستند OVO از")
print("• در زمان پیش‌بینی سریعتر است OVA، برای سیستم‌های بی‌درنگ")
print("• ممکن است دقیق‌تر باشد اما اثر حافظه بیشتری دارد OVO")
print("ری داشته باشد. این امر به دلیل ماهیت خاص داده یا پیاده‌سازی متدهای رأی‌گیری می باشد OVO عمل کرده است، در حالی که معمولاً انتظار می‌رود OVO کمی بهتر از OVA، نکته قابل توجه این است که در دیتاست ارقام"
```

```
=====
Multiclass SVM: Comparison of OVA vs OVO
Simulating Image Classification (3 classes)
=====
Training samples: 210
Test samples: 90
Number of classes: 3

-----
Training One-vs-All SVM
-----
... همه vs برای کلاس 0 classifier آموزش
... همه vs برای کلاس 1 classifier آموزش
... همه vs برای کلاس 2 classifier آموزش
Number of classifiers: 3

-----
Training One-vs-One SVM
-----
... vs 1 برای 0 classifier آموزش
... vs 2 برای 0 classifier آموزش
... vs 2 برای 1 classifier آموزش
Number of classifiers: 3

=====
Evaluation Results
=====
One-vs-All SVM:
  Train Accuracy: 1.0000
  Test Accuracy:  1.0000

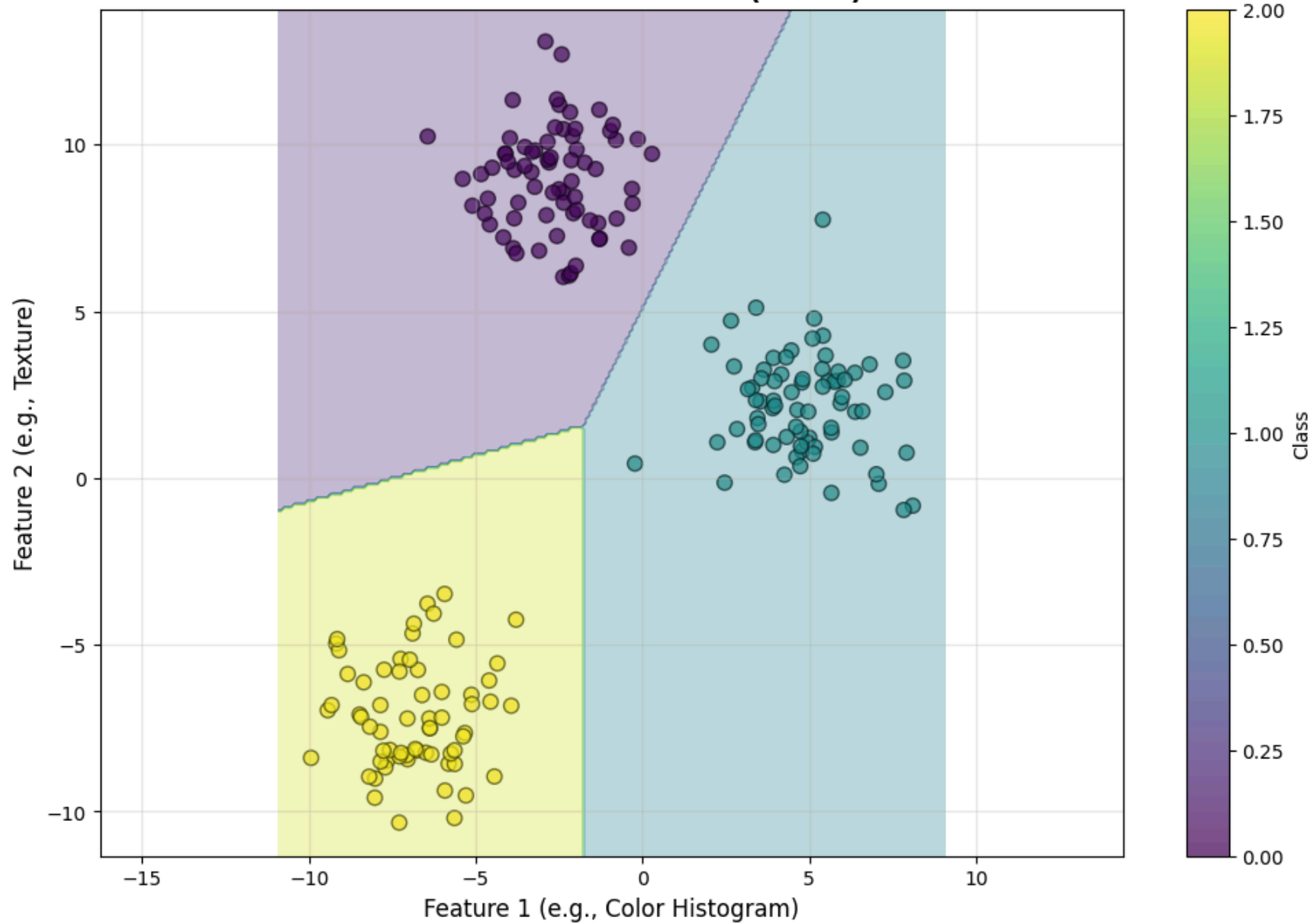
One-vs-One SVM:
  Train Accuracy: 1.0000
  Test Accuracy:  1.0000

Difference (OVO - OVA): 0.0000

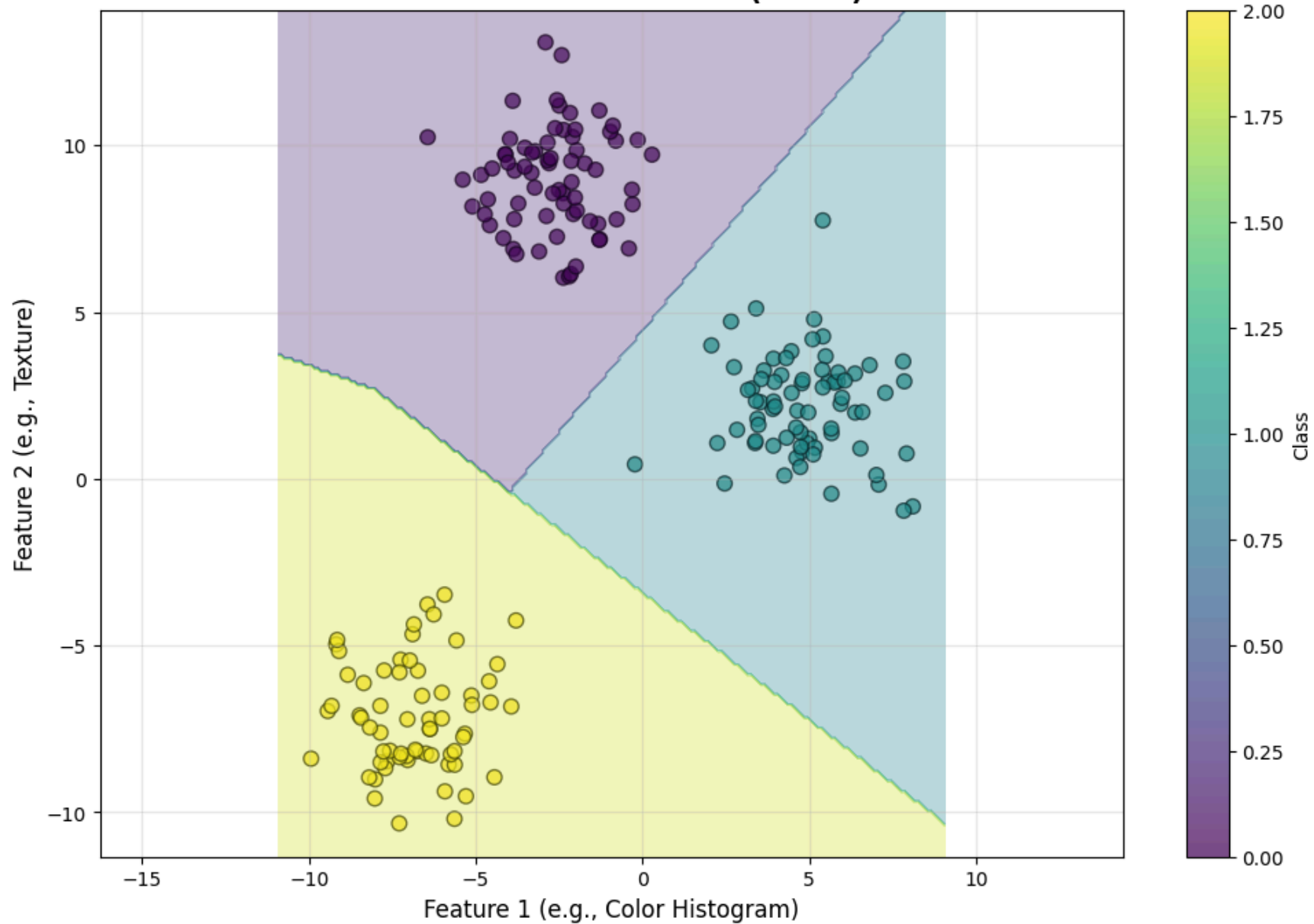
=====
Computational Complexity Analysis
=====
Total Training Time OVA: 0.2786 sec
Total Training Time OVO: 0.1764 sec

Support Vector Analysis:
  OVA - Total SVs: 8, Avg per classifier: 2.7
  OVO - Total SVs: 8, Avg per classifier: 2.7
```

OVA Decision Boundaries (C=1.0)



OVO Decision Boundaries (C=1.0)



```
=====
Test on Real Dataset: Digits Classification (10 classes)
=====
Training samples: 210
Features per sample: 64
Classes: [0 1 2 3 4 5 6 7 8 9]

Training OVA on Digits...
همه... vs برای کلاس 0 classifier آموزش
همه... vs برای کلاس 1 classifier آموزش
همه... vs برای کلاس 2 classifier آموزش
همه... vs برای کلاس 3 classifier آموزش
همه... vs برای کلاس 4 classifier آموزش
همه... vs برای کلاس 5 classifier آموزش
همه... vs برای کلاس 6 classifier آموزش
همه... vs برای کلاس 7 classifier آموزش
همه... vs برای کلاس 8 classifier آموزش
همه... vs برای کلاس 9 classifier آموزش
OVA Accuracy on Digits: 0.9000

Training OVO on Digits...
vs 1... classifier 0 برای آموزش
vs 2... classifier 0 برای آموزش
vs 3... classifier 0 برای آموزش
vs 4... classifier 0 برای آموزش
vs 5... classifier 0 برای آموزش
vs 6... classifier 0 برای آموزش
vs 7... classifier 0 برای آموزش
vs 8... classifier 0 برای آموزش
vs 9... classifier 0 برای آموزش
vs 2... classifier 1 برای آموزش
vs 3... classifier 1 برای آموزش
vs 4... classifier 1 برای آموزش
vs 5... classifier 1 برای آموزش
vs 6... classifier 1 برای آموزش
vs 7... classifier 1 برای آموزش
vs 8... classifier 1 برای آموزش
vs 9... classifier 1 برای آموزش
vs 3... classifier 2 برای آموزش
vs 4... classifier 2 برای آموزش
vs 5... classifier 2 برای آموزش
vs 6... classifier 2 برای آموزش
vs 7... classifier 2 برای آموزش
vs 8... classifier 2 برای آموزش
vs 9... classifier 2 برای آموزش
vs 4... classifier 3 برای آموزش
vs 5... classifier 3 برای آموزش
vs 6... classifier 3 برای آموزش
vs 7... classifier 3 برای آموزش
vs 8... classifier 3 برای آموزش
vs 9... classifier 3 برای آموزش
vs 5... classifier 4 برای آموزش
vs 6... classifier 4 برای آموزش
vs 7... classifier 4 برای آموزش
vs 8... classifier 4 برای آموزش
vs 9... classifier 4 برای آموزش
vs 6... classifier 5 برای آموزش
vs 7... classifier 5 برای آموزش
vs 8... classifier 5 برای آموزش
vs 9... classifier 5 برای آموزش
vs 7... classifier 6 برای آموزش
vs 8... classifier 6 برای آموزش
vs 9... classifier 6 برای آموزش
vs 8... classifier 7 برای آموزش
vs 9... classifier 7 برای آموزش
vs 9... classifier 8 برای آموزش
OVO Accuracy on Digits: 0.8889

Difference (OVO - OVA): -0.0111

=====
کدام استراتژی برای گوگل فوتو مناسبتر است؟
=====
15.1: بر اساس تحلیل پیچیدگی از جدول
• OVA: |C| N دسته‌بند، هر کدام روی |C|
• OVO: |C|N/2 دسته‌بند، هر کدام روی تقریباً 2 (|C|-1)/2
• دسته =100|C| برای گوگل فوتو با
  - OVA: 100 دسته‌بند
  - OVO: 4950 دسته‌بند!
```

توصیه عملی:

- برای سیستم‌های بزرگ با مقیاس بزرگ (دسته‌بندهای کمتر) استفاده کنید OVA از
- زمانی استفاده کنید که دقت بالا حیاتی است و کلاس‌ها متوازن هستند OVO از
- در زمان پیش‌بینی سریع‌تر است OVA ،برای سیستم‌های بی‌درنگ
- ممکن است دقیق‌تر باشد اما اثر حافظه بیشتری دارد OVO

.دقت بالاتری داشته باشد. این امر به دلیل ماهیت خاص داده یا پیاده‌سازی متدهای رای‌گیری می باشد OVO عمل کرده است، در حالی که معمولاً انتظار می‌رود OVO کمی بهتر از OVA ،نکته قابل توجه این است که در دیتاست ارقام

15.2.3 ماشین‌های بردار پشتیبان غیرخطی: عبور از مرزهای خطی

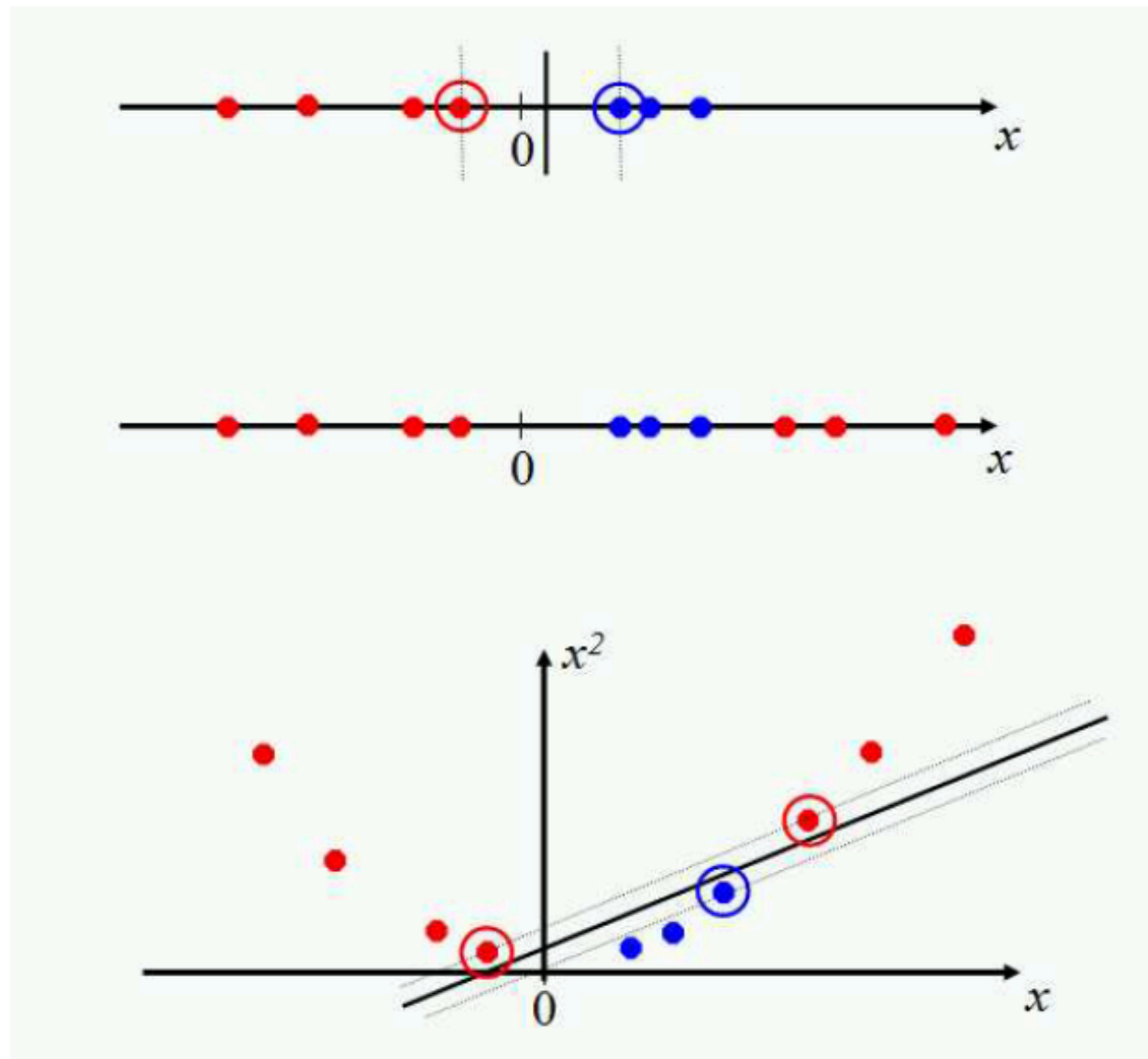
اینستاگرام: چگونه پیچیدگی‌های بصری را درک کنیم؟

تصور کنید در تیم فنی اینستاگرام کار می‌کنید و باید سیستمی طراحی کنید که چهره‌ها را در تصاویر مختلف شناسایی کند. یک چهره از زوایای مختلف، با نورپردازی‌های متفاوت و با حالات گوناگون (خندان، عصبانی، با عینک، بدون عینک) دیده

می‌شود. در فضای ویژگی‌های اصلی، این داده‌ها به هیچ وجه خطی جداپذیر نیستند - نمی‌توان یک خط مستقیم کشید که چهره‌ها را از غیرچهره‌ها جدا کند.

اینجاست که مفهوم **کرنل** (kernel trick) اهمیت پیدا می‌کند. به جای تلاش برای جدا کردن داده‌ها در فضای فعلی، ما داده‌ها را به فضایی با ابعاد بالاتر "منتقل" می‌کنیم که در آن جداپذیری خطی ممکن است.

با آنچه تاکنون ارائه شده است، مجموعه داده‌هایی که به صورت خطی جداپذیر هستند (شاید با چند استثنا یا برخی نویزها) به خوبی مدیریت می‌شوند. اما اگر مجموعه داده به سادگی اجازه طبقه‌بندی توسط یک طبقه‌بند خطی را ندهد، چه کار خواهیم کرد؟ بیا ببینیم به یک حالت یک بعدی نگاه کنیم. مجموعه داده بالایی در شکل 15.6 به راحتی توسط یک طبقه‌بند خطی طبقه‌بندی می‌شود، اما مجموعه داده میانی نه. در عوض، ما باید بتوانیم یک بازه را انتخاب کنیم.



► Figure 15.6 Projecting data that is not linearly separable into a higher dimensional space can make it linearly separable.

کرنل تریک: تبدیل فضایی بدون محاسبات سنگین

یک راه برای حل این مشکل، نگاشت داده‌ها به یک فضای با ابعاد بالاتر و سپس استفاده از یک طبقه‌بند خطی در فضای با ابعاد بالاتر است. برای مثال، بخش پایین شکل نشان می‌دهد که یک جداکننده خطی می‌تواند به راحتی داده‌ها را طبقه‌بندی کند اگر از یک تابع درجه دوم برای نگاشت داده‌ها به دو بعد استفاده کنیم.

در مثال شناسایی چهره اینستاگرام، کرنل به ما اجازه می‌دهد ویژگی‌های پیچیده‌ای مانند "فاصله بین چشم‌ها" یا "انحنای لب‌ها" را بدون محاسبه مستقیم آن‌ها مدل کنیم. این ویژگی‌ها در فضای با ابعاد بالاتر به صورت خطی قابل جدا شدن هستند.

SVMها، و همچنین تعدادی از طبقه‌بندهای خطی دیگر، راهی آسان و کارآمد برای انجام این نگاشت به فضای با ابعاد بالاتر ارائه می‌دهند که به آن (kernel trick) گفته می‌شود. این واقعاً یک ترفند نیست: فقط از ریاضیاتی که دیده‌ایم بهره می‌برد. طبقه‌بند

خطی SVM به یک محصول نقطه بین بردارهای نقاط داده متکی است. بیابید $K(\tilde{x}_i, \tilde{x}_j) = \tilde{x}_i^T \tilde{x}_j$. سپس طبقه‌بندی که تاکنون دیده‌ایم به صورت زیر است:

$$f(\tilde{x}) = \text{sign} \left(\sum_i \alpha_i y_i K(\tilde{x}_i, \tilde{x}) + b \right)$$

حالا فرض کنیم تصمیم بگیریم هر نقطه داده را از طریق برخی تبدیل $\Phi: \tilde{x} \rightarrow \phi(\tilde{x})$ به یک فضای با ابعاد بالاتر نگاشت کنیم. سپس محصول نقطه به $\phi(\tilde{x}_i)^T \phi(\tilde{x}_j)$ تبدیل می‌شود. اگر معلوم شود که این محصول نقطه (که فقط یک عدد حقیقی است) را می‌توان به سادگی و کارایی در terms of نقاط داده اصلی محاسبه کرد، آنگاه ما واقعاً نیاز به نگاشت از $\tilde{x} \rightarrow \phi(\tilde{x})$ نخواهیم داشت. بلکه، می‌توانیم به سادگی مقدار $K(\tilde{x}_i, \tilde{x}_j) = \phi(\tilde{x}_i)^T \phi(\tilde{x}_j)$ را محاسبه کنیم و سپس از مقدار تابع در معادله (15.13) استفاده کنیم. یک تابع هسته (kernel function) K چنین تابعی است که متناظر با یک محصول نقطه در برخی فضای ویژگی گسترش یافته است.

مثال واقعی: کرنل چندجمله‌ای در تشخیص چهره

مثال 15.2: هسته درجه دوم در دو بعد. برای بردارهای دو بعدی $\tilde{u} = (u_1 \ u_2)$, $\tilde{v} = (v_1 \ v_2)$, $K(\tilde{u}, \tilde{v}) = (1 + \tilde{u}^T \tilde{v})^2$ را در نظر بگیرید. ما می‌خواهیم نشان دهیم که این یک هسته است، یعنی $K(\tilde{u}, \tilde{v}) = \phi(\tilde{u})^T \phi(\tilde{v})$ برای برخی ϕ . $\phi(\tilde{u}) = (1 \ u_1^2 \ \sqrt{2}u_1u_2 \ u_2^2 \ \sqrt{2}u_1 \ \sqrt{2}u_2)$ را در نظر بگیرید. سپس:

$$\begin{aligned} K(\tilde{u}, \tilde{v}) &= (1 + \tilde{u}^T \tilde{v})^2 \\ &= 1 + u_1^2 v_1^2 + 2u_1 v_1 u_2 v_2 + u_2^2 v_2^2 + 2u_1 v_1 + 2u_2 v_2 \\ &= (1u_1^2 \sqrt{2}u_1u_2 u_2^2 \sqrt{2}u_1 \sqrt{2}u_2)^T (1v_1^2 \sqrt{2}v_1v_2 v_2^2 \sqrt{2}v_1 \sqrt{2}v_2) \\ &= \phi(\tilde{u})^T \phi(\tilde{v}) \end{aligned}$$

در مثال تشخیص چهره اینستاگرام، این کرنل درجه دوم به ما اجازه می‌دهد ویژگی‌های ترکیبی مانند "فاصله بین چشم‌ها ضربدر عرض لب‌خند" را مدل کنیم که در تشخیص هویت افراد بسیار مفید است.

شرایط مرسر: چه کرنل‌هایی معتبر هستند؟

در زبان تحلیل تابعی، چه نوع توابعی توابع هسته معتبر هستند؟ توابع هسته گاهی اوقات با دقت بیشتری به عنوان هسته‌های مرسر (Mercer kernels) شناخته می‌شوند، زیرا باید شرایط Mercer را برآورده کنند: برای هر $\tilde{x}$ که در آن $\int g(\tilde{x})^2 d\tilde{x} < \infty$ متناهی است، باید داشته باشیم:

$$\int K(\tilde{x}, \tilde{z}) g(\tilde{x}) g(\tilde{z}) d\tilde{x} d\tilde{z} \geq 0$$

یک تابع هسته K باید پیوسته، متقارن و دارای ماتریس گرام مثبت معین باشد. چنین K به این معنی است که نگاشتی به فضای هیلبرت بازتولیدکننده هسته وجود دارد به طوری که محصول نقطه در آن مقداری برابر با تابع K می‌دهد. اگر یک هسته شرایط مرسر را برآورده نکند، آنگاه QP متناظر ممکن است راه‌حلی نداشته باشد.

دو خانواده رایج از هسته‌ها، هسته‌های چندجمله‌ای و توابع پایه شعاعی هستند. هسته‌های چندجمله‌ای به صورت $K(\tilde{x}, \tilde{z}) = (1 + \tilde{x}^T \tilde{z})^d$ هستند. حالت $d = 1$ یک هسته خطی است، که قبلاً در ابتدای این بخش داشتیم (ثابت 1 فقط آستانه را تغییر می‌دهد). حالت $d = 2$ یک هسته درجه دوم می‌دهد و بسیار رایج است.

رایج‌ترین شکل تابع پایه شعاعی یک توزیع گوسی است که به صورت زیر محاسبه می‌شود:

$$K(\tilde{x}, \tilde{z}) = e^{-(\tilde{x} - \tilde{z})^2 / (2\sigma^2)}$$

در سیستم‌های تشخیص چهره مانند FaceID اپل:

- **کرنل‌های چندجمله‌ای** برای مدل کردن ویژگی‌های ترکیبی مانند "فاصله بین چشم‌ها ضربدر عرض لب‌خند" استفاده می‌شوند که در تشخیص هویت افراد مفید است.
- **کرنل‌های پایه شعاعی (RBF)** برای ایجاد مرزهای تصمیم پیچیده که می‌توانند تغییرات نوری و زوایای مختلف چهره را مدیریت کنند، به کار می‌روند.
- **کرنل‌های رشته‌ای** برای تحلیل ویژگی‌های محلی مانند الگوهای بافت پوست یا جزئیات دقیق around the eyes استفاده می‌شوند.

این کرنل‌های مختلف به سیستم اجازه می‌دهند تا با ترکیب ویژگی‌های ساده، الگوهای پیچیده‌ای را تشخیص دهد که در فضای اصلی غیرقابل تشخیص هستند.

جهان SVMها با زبان خاص خود همراه است که بسیار متفاوت از زبان مورد استفاده در یادگیری ماشین است. این اصطلاحات ریشه‌های عمیقی در ریاضیات دارد، اما مهم است که بیش از حد تحت تأثیر این اصطلاحات قرار نگیریم. در واقع، ما در مورد چیزهای کاملاً ساده صحبت می‌کنیم. یک هسته چندجمله‌ای به ما امکان می‌دهد تا ویژگی‌های الحاقی (تا مرتبه چندجمله‌ای) را مدل کنیم. یک تابع پایه شعاعی به شما امکان می‌دهد ویژگی‌هایی داشته باشید که دایره‌ها (ابگره‌ها) را انتخاب می‌کنند - اگرچه مرزهای تصمیم‌گیری با تعامل چندین ویژگی از این دست بسیار پیچیده‌تر می‌شوند. یک هسته رشته‌ای به شما اجازه می‌دهد ویژگی‌هایی داشته باشید که زیردنباله‌های کاراکتری عبارات هستند. همه اینها مفاهیم ساده‌ای هستند که در بسیاری از مکان‌های دیگر با نام‌های مختلف نیز استفاده شده‌اند.

```
In [27]: import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn.datasets import make_moons, make_circles
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# =====
# (Kernel Trick) غیرخطی با استفاده از هوش هسته‌ای SVM پیاده‌سازی
# می‌توان  $\phi(x)$  این پیاده‌سازی نشان می‌دهد چگونه بدون محاسبه مستقیم
# در فضای با ابعاد بالاتر از طبقه‌بندی خطی استفاده کرد
# =====

class NonlinearSVM:
    """
    Kernel SVM (غیرخطی با هسته SVM)
    این پیاده‌سازی فرمول‌بندی دوگانه را با جایگزینی محصول نقطه
    پیاده‌سازی می‌کند  $K(x_i, x_j)$  با تابع هسته
    """

    def __init__(self, kernel='linear', C=1.0, degree=2, gamma=1.0):
        """
        kernel: نوع هسته ('linear', 'polynomial', 'rbf')
        C: پارامتر منظم‌سازی (مانند حاشیه نرم)
        degree: درجه برای هسته چندجمله‌ای
        gamma: پارامتر خطی‌سازی برای هسته RBF
        """

        self.kernel = kernel
        self.C = C
        self.degree = degree
        self.gamma = gamma

        self.alpha = None # ضرایب لاگرانژ
        self.b = None # بایاس
        self.X_train = None # (در پیش‌بینی K برای محاسبه) داده آموزشی
        self.y_train = None # برچسب‌های آموزشی
        self.support_vectors = None
        self.support_labels = None

    def _kernel_function(self, X1, X2):
        """
        K(X1, X2) محاسبه ماتریس هسته
        این تابع کلیدی هوش هسته‌ای را پیاده‌سازی می‌کند
        """

        if self.kernel == 'linear':
            # هسته خطی:  $K(x, z) = x^T z$ 
            return np.dot(X1, X2.T)

        elif self.kernel == 'polynomial':
```

هسته چندجمله‌ای $K(x, z) = (1 + x^T z)^d$
این هسته معادل نگاشت به فضای با ابعاد بالاتر است
`return (1 + np.dot(X1, X2.T)) ** self.degree`

`elif self.kernel == 'rbf':`
هسته پایه شعاعی (RBF): $K(x, z) = \exp(-\gamma \|x - z\|^2)$
این هسته به طور ضمنی فضای بینهایت بعدی را ایجاد می‌کند
محاسبه کارآمد فاصله اقلیدسی برای تمام جفت‌ها
`X1_norm = np.sum(X1 ** 2, axis=1).reshape(-1, 1)`
`X2_norm = np.sum(X2 ** 2, axis=1).reshape(1, -1)`
`distances_sq = X1_norm + X2_norm - 2 * np.dot(X1, X2.T)`
`return np.exp(-self.gamma * distances_sq)`

`else:`
`raise ValueError(f"نوع هسته '{self.kernel}' پشتیبانی نمی‌شود")`

`def fit(self, X, y):`

"""

غیرخطی با روش دوگانه SVM آموزش

فرمول‌سازی:

$\max_{\alpha} \sum \alpha_i - 0.5 * \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
 $s.t. \sum \alpha_i y_i = 0, 0 \leq \alpha_i \leq C$

QP به دلیل $O(N^3)$: پیچیدگی محاسباتی

"""

`self.X_train = X`

`self.y_train = y.copy()`

`n_samples = X.shape[0]`

برای داده آموزشی K محاسبه ماتریس هسته
این جایگزین محصول نقطه در فرمول خطی می‌شود
`K = self._kernel_function(X, X)`

برای مسئله بهینه‌سازی QP ساخت
$P_{ij} = y_i * y_j * K(x_i, x_j)$
`P = np.outer(y, y) * K`
`P = matrix(P)`
`q = matrix(-np.ones(n_samples))`

محدودیت: $-\alpha_i \leq 0$ و $\alpha_i \leq C$
`G = np.vstack([-np.eye(n_samples), np.eye(n_samples)])`
`h = np.hstack([np.zeros(n_samples), self.C * np.ones(n_samples)])`
`G, h = matrix(G), matrix(h)`

محدودیت: $\sum \alpha_i y_i = 0$
`A = matrix(y.reshape(1, -1).astype(float))`
`b = matrix(0.0)`

حل مسئله بهینه‌سازی درجه دوم
`solvers.options['show_progress'] = False`
`solution = solvers.qp(P, q, G, h, A, b)`

`self.alpha = np.array(solution['x']).flatten()`

$\alpha_i > 0$ شناسایی بردارهای پشتیبان: نقاطی که
`sv_indices = self.alpha > 1e-5`
`self.support_vectors = X[sv_indices]`
`self.support_labels = y[sv_indices]`
`self.alpha_sv = self.alpha[sv_indices]`

$\alpha_i < C$ از یک بردار پشتیبان با b محاسبه بایاس
`margin_sv = (self.alpha > 1e-5) & (self.alpha < self.C - 1e-5)`
`if np.any(margin_sv):`
 `idx = np.where(margin_sv)[0][0]`
 # $b = y_i - \sum \alpha_j y_j K(x_j, x_i)$
 `self.b = y[idx] - np.sum(self.alpha * y * K[:, idx])`
`else:`
 `self.b = 0`

`def predict(self, X):`

"""

پیش‌بینی برچسب برای داده جدید با استفاده از هوش هسته‌ای

تابع تصمیم: $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$
را به طور صریح محاسبه نمی‌کنیم $\phi(x)$ این نشان می‌دهد که هیچگاه
"""

محاسبه ماتریس هسته بین داده آموزشی و داده آزمون
`K = self._kernel_function(self.X_train, X)`

تابع تصمیم: $\sum (\alpha_i * y_i * K(x_i, x)) + b$
`decision_values = np.sum((self.alpha * self.y_train).reshape(-1, 1) * K, axis=0) + self.b`

`return np.sign(decision_values)`

=====

تابع برای نمایش مثال 15.2: نگاشت هسته چندجمله‌ای درجه 2
می‌تواند $K(u, v) = (1 + u^T v)^2$ این تابع نشان می‌دهد که چگونه
نوشته شود $\phi(u)^T \phi(v)$ به صورت
=====

`def demonstrate_polynomial_kernel_mapping():`

"""

نمایش سازی محاسبات مثال 15.2 از متن اصلی

نشان می‌دهد که کرنل درجه 2، v و u برای بردارهای دو بعدی
معادل نگاشت به فضای 6 بعدی است
"""

`print("=*60)`

`print("مثال 15.2: نگاشت ضمنی هسته چندجمله‌ای درجه 2")`

`print("=*60)`

```
# دو بردار نمونه دو بعدی
u = np.array([2.0, 3.0])
v = np.array([1.0, 2.0])

# محاسبه کرنل مستقیم:  $K(u, v) = (1 + u^T v)^2$ 
kernel_value = (1 + np.dot(u, v)) ** 2
print(f"u = {u}, v = {v}")
print(f" $K(u, v) = (1 + u^T v)^2 = {kernel\_value:.4f}$ ")

# به صورت صریح  $\phi(u)$  و  $\phi(v)$  محاسبه
#  $\phi(x) = [1, x_1^2, \sqrt{2} x_1 x_2, x_2^2, \sqrt{2} x_1, \sqrt{2} x_2]$ 
phi_u = np.array([
    1.0,
    u[0] ** 2,
    np.sqrt(2) * u[0] * u[1],
    u[1] ** 2,
    np.sqrt(2) * u[0],
    np.sqrt(2) * u[1]
])

phi_v = np.array([
    1.0,
    v[0] ** 2,
    np.sqrt(2) * v[0] * v[1],
    v[1] ** 2,
    np.sqrt(2) * v[0],
    np.sqrt(2) * v[1]
])

print(f"\n $\phi(u) = {phi\_u}$ ")
print(f" $\phi(v) = {phi\_v}$ ")

# برای تأیید برابری  $\phi(u)^T \phi(v)$  محاسبه
phi_dot = np.dot(phi_u, phi_v)
print(f"\n $\phi(u)^T \phi(v) = {phi\_dot:.4f}$ ")
print(f" $K(u, v)$  برابر است با  ${abs(kernel\_value - phi\_dot) < 1e-10}$ ")

# می‌تواند بزرگ باشد  $\phi(x)$  نمایش اینکه اندازه
print(f"\n $\phi(u)$  = توجه: اندازه:  ${np.linalg.norm(phi\_u):.2f}$ ")
print(f" $u$  اصلی در حالی که اندازه  ${np.linalg.norm(u):.2f}$ ")
print("!!این نشان می‌دهد چرا هوش هسته‌ای محاسباتی کارآمد است")

# =====
# تابع برای ترسیم مرز تصمیم‌گیری و مقایسه هسته‌ها
# =====

def plot_kernel_comparison(X, y, dataset_name=""):
    """
    روی یک داده (linear, polynomial, rbf) مقایسه ترسیمی سه هسته مختلف
    این به طور واضح نشان می‌دهد که چرا هسته‌های غیرخطی برای داده‌های پیچیده لازم هستند
    """
    kernels = ['linear', 'polynomial', 'rbf']
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, kernel in enumerate(kernels):
        ax = axes[idx]

        # با این هسته SVM آموزش
        svm = NonlinearSVM(kernel=kernel, C=1.0, degree=2, gamma=1.0)
        svm.fit(X, y)

        # ترسیم نقاط داده
        class_1 = y == 1
        class_2 = y == -1

        ax.scatter(X[class_1, 0], X[class_1, 1], c='blue', s=50, alpha=0.7, label='Class +1')
        ax.scatter(X[class_2, 0], X[class_2, 1], c='red', s=50, alpha=0.7, label='Class -1')

        # ترسیم بردارهای پشتیبان
        if len(svm.support_vectors) > 0:
            ax.scatter(svm.support_vectors[:, 0], svm.support_vectors[:, 1],
                       facecolors='none', edgecolors='black', s=200, linewidth=2,
                       label=f'SVs ({len(svm.support_vectors)})')

        # ایجاد شبکه برای مناطق تصمیم‌گیری
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                              np.arange(y_min, y_max, 0.1))

        # پیش‌بینی برای شبکه
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

        # ترسیم مناطق تصمیم‌گیری
        ax.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
        ax.contour(xx, yy, Z, colors='k', levels=[0], linestyles='--', linewidths=1)

        ax.set_xlabel('Feature 1', fontsize=11)
        ax.set_ylabel('Feature 2', fontsize=11)
        ax.set_title(f'{kernel.capitalize()} Kernel', fontsize=12, fontweight='bold')
        ax.legend(fontsize=9)
        ax.grid(True, alpha=0.3)
        ax.axis('equal')

    plt.suptitle(f'Kernel Comparison: {dataset_name}\n(Same C=1.0, degree=2, gamma=1.0)',
                 fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

# =====
```

```
# اجرای مثال‌ها و مقایسه‌ها
# =====

if __name__ == "__main__":
    # 1. نمایش مثال 15.2 از متن
    demonstrate_polynomial_kernel_mapping()
    print("\n\n" + "="*60 + "\n")

    # 2. (moon) تست روی داده غیرخطی: شکل ماه
    print("Non-linearly separable)...")
    np.random.seed(42)
    X_moon, y_moon = make_moons(n_samples=200, noise=0.1, random_state=42)
    y_moon = np.where(y_moon == 0, -1, 1) # تبدیل به 1±

    # نمایش اینکه هسته خطی در اینجا کار نمی‌کند
    linear_svm = NonlinearSVM(kernel='linear', C=1.0)
    linear_svm.fit(X_moon, y_moon)
    linear_acc = accuracy_score(y_moon, linear_svm.predict(X_moon))
    print(f"پایین - داده غیرخطی است) {linear_acc:.2%}: دقت با هسته خطی")

    # مقایسه هسته‌ها
    plot_kernel_comparison(X_moon, y_moon, dataset_name="Moon Dataset")

    # 3. (circle) تست روی داده به شکل دایره
    print("\n" + "="*60)
    print("آزمایش روی داده به شکل دایره")
    X_circle, y_circle = make_circles(n_samples=200, noise=0.1, factor=0.5, random_state=42)
    y_circle = np.where(y_circle == 0, -1, 1)

    plot_kernel_comparison(X_circle, y_circle, dataset_name="Circle Dataset")

    # 4. تحلیل حساسیت به پارامترها
    print("\n" + "="*60)
    print("تحلیل حساسیت پارامترهای هسته")

    # تأثیر درجه در هسته چندجمله‌ای
    degrees = [2, 3, 5]
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, degree in enumerate(degrees):
        ax = axes[idx]
        svm = NonlinearSVM(kernel='polynomial', C=1.0, degree=degree)
        svm.fit(X_circle, y_circle)

        accuracy = accuracy_score(y_circle, svm.predict(X_circle))

        # ترسیم نتایج
        xx, yy = np.meshgrid(np.linspace(-2, 2, 200), np.linspace(-2, 2, 200))
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

        ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
        ax.scatter(X_circle[:, 0], X_circle[:, 1], c=y_circle, cmap='coolwarm', s=30)
        ax.set_title(f'Polynomial (degree={degree})\nAccuracy: {accuracy:.2%}',
                    fontsize=12, fontweight='bold')
        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.axis('equal')

    plt.suptitle('Effect of Polynomial Degree', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # تأثیر گاما در هسته RBF
    print("\n" + "="*60)
    print("RBF تحلیل تأثیر گاما در هسته")

    gammas = [0.1, 1.0, 10.0]
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, gamma in enumerate(gammas):
        ax = axes[idx]
        svm = NonlinearSVM(kernel='rbf', C=1.0, gamma=gamma)
        svm.fit(X_circle, y_circle)

        accuracy = accuracy_score(y_circle, svm.predict(X_circle))

        xx, yy = np.meshgrid(np.linspace(-2, 2, 200), np.linspace(-2, 2, 200))
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

        ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
        ax.scatter(X_circle[:, 0], X_circle[:, 1], c=y_circle, cmap='coolwarm', s=30)
        ax.set_title(f'RBF (γ={gamma})\nAccuracy: {accuracy:.2%}',
                    fontsize=12, fontweight='bold')
        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.axis('equal')

    plt.suptitle('Effect of RBF Gamma Parameter', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # 5. خلاصه نتایج
    print("\n" + "="*60)
    print("خلاصه نتایج آزمایش")
    print("="*60)
    print("هسته خطی فقط برای داده‌های جداپذیر خطی کارآمد است •")
    print("هسته چندجمله‌ای (درجه 2 یا 3) برای داده‌های دارای ساختار قابل پیش‌بینی •")
    print("برای داده‌های بسیار پیچیده و بدون ساختار مشخص RBF هسته •")
    print("افزایش درجه با گاما باعث پیچیده‌تر شدن مرز تصمیم می‌شود •")
    print("با پارامترهای خیلی بزرگ (overfitting) خطر بیش‌ورسازی •")
    print("در تشخیص چهره، ترکیب این هسته‌ها می‌تواند دقت را بهبود بخشد •")
```

=====

مثال 15.2: نگاشت ضمنی هسته چندجمله‌ای درجه 2

=====

$u = [2. \ 3.]$, $v = [1. \ 2.]$
 $K(u, v) = (1 + u^T v)^2 = 81.0000$

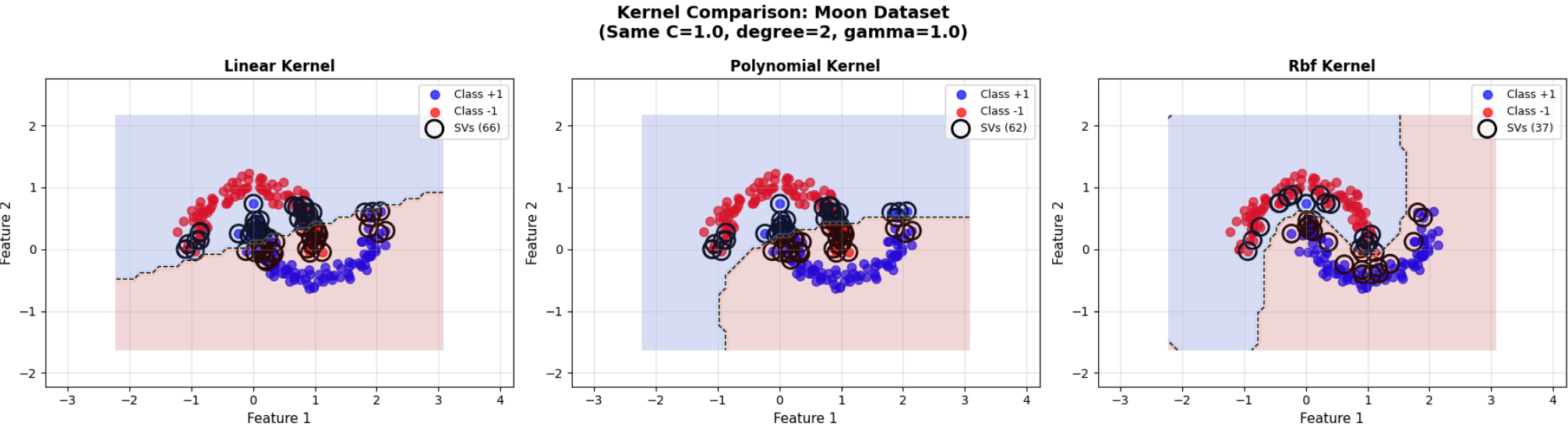
$\phi(u) = [1. \quad 4. \quad 8.48528137 \ 9. \quad 2.82842712 \ 4.24264069]$
 $\phi(v) = [1. \quad 1. \quad 2.82842712 \ 4. \quad 1.41421356 \ 2.82842712]$

$\phi(u)^T \phi(v) = 81.0000$
True برابر است $K(u, v)$ آیا با

اندازه $\phi(u) = 14.00$ توجه:
اصلی $= 3.61$ u در حالی که اندازه
!این نشان می‌دهد چرا هوش هسته‌ای محاسباتی کارآمد است

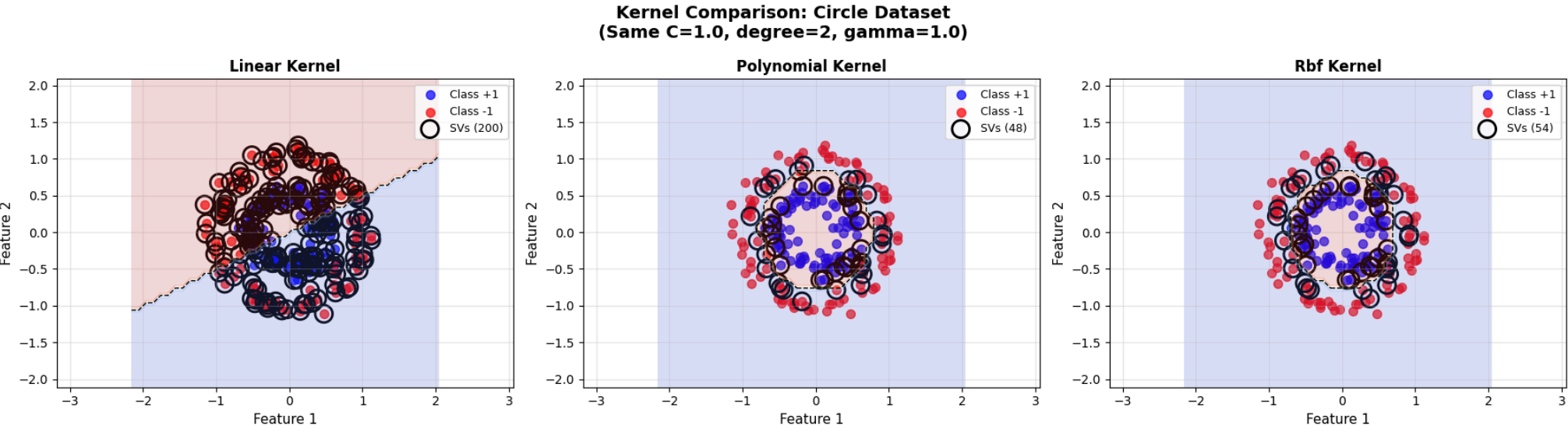
=====

(Non-linearly separable) ... آزمایش روی داده به شکل ماه دقت با هسته خطی: 86.50% (پایین - داده غیرخطی است)



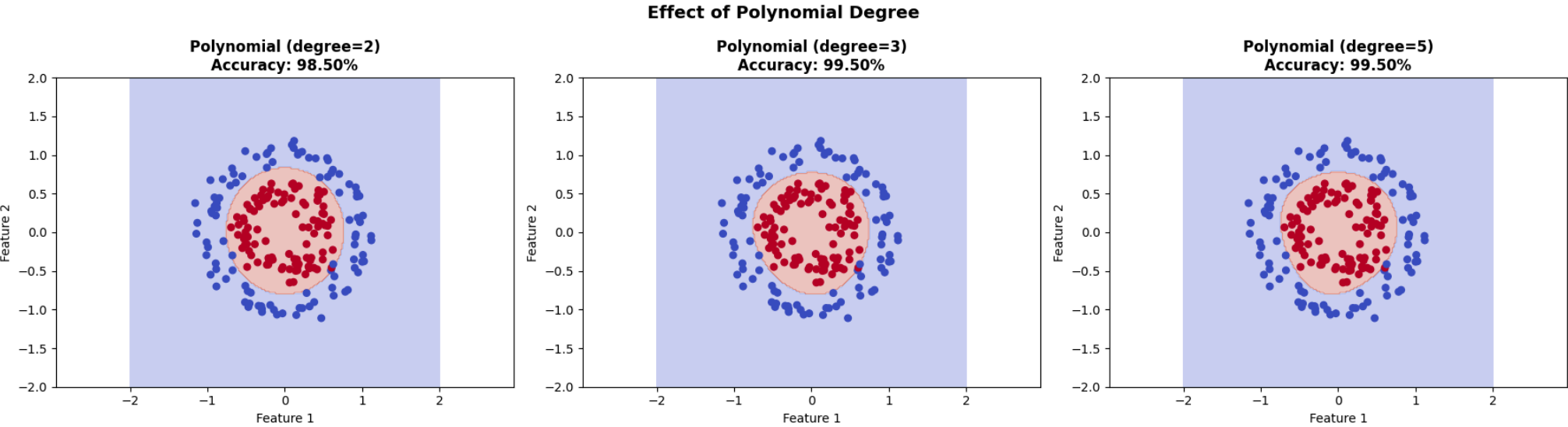
=====

...آزمایش روی داده به شکل دایره



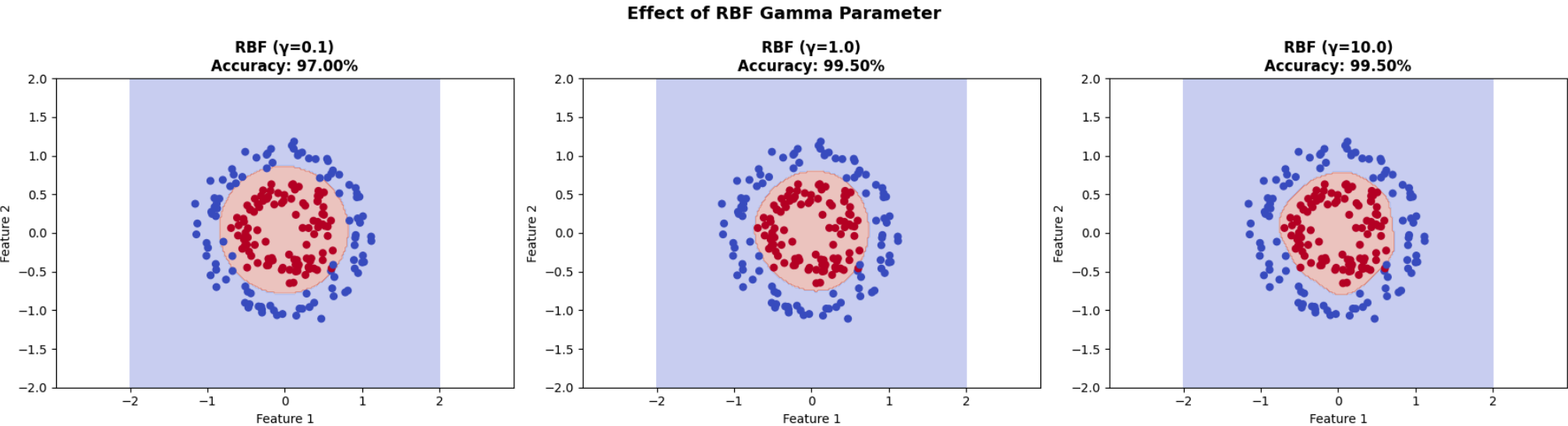
=====

...تحلیل حساسیت پارامترهای هسته



=====

...RBF تحلیل تأثیر گاما در هسته



=====

:خلاصه نتایج آزمایش

=====

- هسته خطی فقط برای داده‌های جدایی‌ناپذیر خطی کارآمد است
- هسته چندجمله‌ای (درجه 2 یا 3) برای داده‌های دارای ساختار قابل پیش‌بینی
- برای داده‌های بسیار پیچیده و بدون ساختار مشخص RBF هسته
- افزایش درجه یا گاما باعث پیچیده‌تر شدن مرز تصمیم می‌شود
- با پارامترهای خیلی بزرگ (overfitting) خطر بیش‌وردسازی
- در تشخیص چهره، ترکیب این هسته‌ها می‌تواند دقت را بهبود بخشد

15.2.4 نتایج تجربی: اثبات برتری SVM در دنیای واقعی

📰 **رویترز: چگونه هزاران خبر روزانه را به صورت خودکار دسته‌بندی کنیم؟**

تصور کنید در آژانس خبری رویترز کار می‌کنید و مسئول سیستم خودکار دسته‌بندی اخبار هستید. هر روز هزاران خبر از سراسر جهان دریافت می‌شود که باید به دسته‌هایی مانند "اقتصادی"، "سیاسی"، "ورزشی"، "فناوری" و... تقسیم شوند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا:

- برخی دسته‌ها (مانند اقتصادی) ویژگی‌های زبانی مشخصی دارند
- برخی دیگر (مانند سیاسی) بسیار متنوع و پیچیده هستند
- برخی اخبار ممکن است به چند دسته مرتبط باشند

در این محیط، انتخاب الگوریتم مناسب می‌تواند تفاوت بین یک سیستم کارآمد و یک سیستم ناکارآمد باشد.

ما در بخش 13.6 نتایجی را ارائه کردیم که نشان می‌دهد SVM یک طبقه‌بند بسیار مؤثر برای متن است. نتایج Dumais و همکاران (1998) که در جدول 13.9 نشان داده شده، به وضوح نشان می‌دهد که SVM‌ها بهترین عملکرد را دارند. این یکی از چندین کار از آن زمان بود که شهرت قوی SVM‌ها را برای طبقه‌بندی متن تثبیت کرد. کار پیشگام دیگر در مقیاس‌بندی و ارزیابی SVM‌ها برای طبقه‌بندی متن (Joachims 1998) بود.

🔍 **مقایسه عملکرد: کدام الگوریتم برای دسته‌بندی اخبار بهتر است؟**

در سیستم دسته‌بندی اخبار رویترز، تیم مهندسی باید بین الگوریتم‌های مختلف یکی را انتخاب کند. نتایج تجربی به ما کمک می‌کند تا تصمیم آگاهانه‌تری بگیریم:

- **بیز ساده (Naive Bayes):** سریع و ساده، اما برای دسته‌های پیچیده مانند "سیاسی" عملکرد ضعیفی دارد.
- **روش Rocchio:** برای دسته‌های با ویژگی‌های مشخص مانند "اقتصادی" خوب عمل می‌کند، اما در دسته‌های متنوع ضعیف است.
- **SVM خطی:** در اکثر دسته‌ها عملکرد بسیار خوبی دارد، به خصوص زمانی که پارامتر منظم‌سازی به درستی تنظیم شود.
- **SVM با تابع پایه شعاعی (RBF):** برای دسته‌های پیچیده و نامشخص عملکرد بهتری دارد، اما محاسبات سنگین‌تری نیاز دارد.

ما برخی از نتایج Joachims را از (Joachims 2002a) در جدول 15.2 ارائه می‌دهیم. Joachims تعداد زیادی ویژگی واژگانی را در مقابل Dumais و همکاران (1998) استفاده کرد که از انتخاب ویژگی MI برای ساخت طبقه‌بندها با تعداد بسیار محدودتر ویژگی استفاده کردند. موفقیت SVM خطی با نتایج بحث شده در بخش 14.6 در مورد رویکردهای خطی دیگر مانند بیز ساده مطابقت دارد. به نظر می‌رسد که کار با ویژگی‌های واژگانی ساده می‌تواند فرد را تا مسیر طولانی پیش ببرد.

Category	NB	Rocchio	Dec. Trees	kNN	linear SVM C = 0.5	linear SVM C = 1.0	rbf-SVM $\sigma \approx 7$
earn	96.0	96.1	96.1	97.8	98.0	98.2	98.1

acq	90.7	92.1	85.3	91.8	95.5	95.6	94.7
money-fx	59.6	67.6	69.4	75.4	78.8	78.5	74.3
grain	69.8	79.5	89.1	82.6	91.9	93.1	93.4
crude	81.2	81.5	75.5	85.8	89.4	89.4	88.7
trade	52.2	77.4	59.2	77.9	79.2	79.2	76.6
interest	57.6	72.5	49.1	76.7	75.6	74.8	69.1
ship	80.9	83.1	80.9	79.8	87.4	86.5	85.8
wheat	63.4	79.4	85.5	72.9	86.6	86.8	82.4
corn	45.2	62.2	87.7	71.4	87.5	87.8	84.6
microavg.	72.3	79.9	79.4	82.6	86.7	87.5	86.4

📊 تحلیل نتایج: چه درسی می‌توانیم بگیریم؟

جدول 15.2 نتایج مقایسه‌ای الگوریتم‌های مختلف را نشان می‌دهد. برای سیستم دسته‌بندی اخبار رویترز، این نتایج بسیار مهم هستند:

- **دسته‌های ساده (earn, acq):** SVM خطی با $C=1.0$ بهترین عملکرد را دارد، اما تفاوت با سایر الگوریتم‌ها زیاد نیست.
 - **دسته‌های متوسط (crude, ship):** SVM خطی به وضوح بهتر از سایر الگوریتم‌ها عمل می‌کند.
 - **دسته‌های پیچیده (money-fx, trade, interest):** در این دسته‌ها، SVM به طور قابل توجهی بهتر از سایر الگوریتم‌ها عمل می‌کند، به خصوص در مقایسه با بیز ساده.
 - **دسته‌های خاص (grain, corn):** جالب است که در این دسته‌ها، درخت تصمیم عملکرد خوبی دارد، اما SVM همچنان برتر است.
- در مجموع، SVM خطی با $C=1.0$ بهترین عملکرد میانگین (microavg. 87.5) را دارد که تقریباً 15% بهتر از بهترین الگوریتم غیر SVM (Rocchio با 79.9).

🔍 تفاوت در نتایج تحقیقات: چرا نتایج متفاوتی گزارش می‌شود؟

دوباره قابل توجه است که نتایج مقالات مختلف برای روش‌های یادگیری ماشین یکسان چقدر با هم تفاوت دارند. به طور خاص، بر اساس بازتولید توسط محققان دیگر، نتایج بیز ساده (Joachims 1998) به نظر بیش از حد ضعیف می‌رسند، و نتایج در جدول 13.9 باید به عنوان نماینده در نظر گرفته شوند.

در سیستم دسته‌بندی اخبار رویترز، این تفاوت‌ها بسیار مهم هستند:

- روش پیش‌پردازش متن (نرمال‌سازی، حذف ایست‌واژه‌ها)
 - انتخاب ویژگی (MI در مقابل استفاده از تمام ویژگی‌ها)
 - تنظیم پارامترهای الگوریتم (مقدار C در SVM)
 - اندازه و کیفیت مجموعه داده آموزشی
- این عوامل می‌توانند باعث تفاوت‌های قابل توجهی در عملکرد الگوریتم‌های یکسان در مطالعات مختلف شوند.

15.3 چالش‌های طبقه‌بندی اسناد متنی: از تئوری تا عمل

توئیت: چگونه میلیاردها توییت را مدیریت می‌کنند؟

تصور کنید در تیم مدیریت محتوای توئیت کار می‌کنید و مسئول سیستمی هستید که باید روزانه صدها میلیون توییت را دسته‌بندی کند. این توییت‌ها باید به دسته‌هایی مانند "اخبار"، "ورزشی"، "سرگرمی"، "تبلیغات"، "محتوای نامناسب" و... تقسیم شوند.

Jackson و Moulinier (2002) می‌نویسند: "هیچ تردیدی در مورد ارزش تجاری توانایی طبقه‌بندی خودکار اسناد بر اساس محتوا وجود ندارد." این جمله در مورد توئیت صدق می‌کند - ارزش تجاری این پلتفرم به شدت به کیفیت سیستم طبقه‌بندی آن وابسته است.

بیشتر بحث ما در مورد طبقه‌بندی بر معرفی روش‌های مختلف یادگیری ماشین متمرکز بوده است تا بحث در مورد ویژگی‌های خاص اسناد متنی مرتبط با طبقه‌بندی. این سوگیری برای یک کتاب درسی مناسب است، اما برای یک توسعه‌دهنده کاربردی ناکارآمد است.

اهمیت درک داده: کلید موفقیت در طبقه‌بندی

اغلب این اتفاق می‌افتد که بهره‌وری عملکردی بیشتر می‌تواند از طریق بهره‌برداری از ویژگی‌های متن خاص دامنه (domain-specific) نسبت به تغییر از یک روش یادگیری ماشین به روش دیگر به دست آید.

در مثال توئیت، این به معنای درک ویژگی‌های خاص زبانی است که در هر دسته متفاوت است:

- **اخبار:** لحن رسمی، استفاده از منابع خبری، هشتک‌های رویدادهای روز
- **ورزشی:** اسامی تیم‌ها، اصطلاحات ورزشی خاص، اموجی‌های شادی پس از پیروزی
- **محتوای نامناسب:** کلمات توهین‌آمیز، لحن پرخاش، پیام‌های ناخواسته
- **تبلیغات:** فراخوانی اقدام، لینک‌های کوتاه تجاری، الگوهای تکراری

Jackson و Moulinier (2002) پیشنهاد می‌کنند که "درک داده یکی از کلیدهای موفقیت در دسته‌بندی است، با این حال این منطقه‌ای است که اکثر فروشندگان ابزار دسته‌بندی در آن بسیار ضعیف هستند."

محدودیت‌های ابزارهای "یک اندازه برای همه"

بسیاری از ابزارهای "یک اندازه برای همه" موجود در بازار بر روی طیف وسیعی از انواع محتوا آزمایش نشده‌اند. در سیستم طبقه‌بندی توئیت، این به معنای:

- ابزاری که برای انگلیسی بهینه شده‌اند اما برای زبان‌های دیگر عملکرد ضعیفی دارند
 - سیستم‌هایی که برای محتوای خبری خوب عمل می‌کنند اما در تشخیص طنز یا کنایه ضعیف هستند
 - پلتفرم‌هایی که برای متن‌های کوتاه (مانند توییت) طراحی شده‌اند اما برای مقالات طولانی عملکرد خوبی ندارند
- در عمل، تیم‌های فنی توئیت از ترکیبی از روش‌ها استفاده می‌کنند: الگوریتم‌های یادگیری ماشین پیشرفته، ویژگی‌های خاص دامنه، و قوانین دستی که توسط متخصصان محتوا ایجاد شده‌اند.

در حالی که نتایج اولیه (Joachims 1998) پیشنهاد می‌کرد که استفاده از کرنل‌های چندجمله‌ای یا rbf به دستاوردهای قابل توجهی در این وظیفه منجر می‌شود، این با SVM‌های حاشیه سخت بود. با SVM‌های حاشیه نرم، یک SVM خطی ساده با C پیش‌فرض = 1 بهترین عملکرد را دارد.

این یافته در دنیای واقعی طبقه‌بندی متن بسیار مهم است:

- برای اکثر کاربردهای متنی، یک SVM خطی با تنظیمات مناسب عملکرد خوبی ارائه می‌دهد
- پیچیدگی اضافی (کرنل‌های پیچیده) لزوماً به بهبود عملکرد منجر نمی‌شود
- درک ویژگی‌های خاص دامنه اغلب مهم‌تر از انتخاب الگوریتم پیچیده است

در سیستم طبقه‌بندی توئیت‌ر، این به معنای تمرکز بر ویژگی‌های خاص زبانی برای هر زبان و فرهنگ، به جای تلاش برای یافتن یک الگوریتم جهانی که برای همه موارد به خوبی عمل کند.

در این بخش، کمی عقب‌نشینی کرده و کاربردهای طبقه‌بندی متن، فضای راه‌حل‌های ممکن و سودمندی اکتشافات خاص کاربرد را در نظر گرفتیم. این رویکرد به توسعه‌دهندگان کمک می‌کند تا راه‌حل‌های واقعی‌تر و مؤثرتری برای مسائل خاص خود طراحی کنند.

15.3.1 انتخاب طبقه‌بند متن: راهنمای عملی برای توسعه‌دهندگان 🧐

🔍 پلتفرم‌های محتوا: چگونه مسیر محتوای خود را پیدا کنیم؟

تصور کنید در تیم فنی یوتیوب کار می‌کنید و باید سیستمی طراحی کنید که میلیون‌ها ویدیو را به صورت خودکار دسته‌بندی کند. اولین سؤال کلیدی این است: "چقدر داده آموزشی در اختیار داریم؟" این سؤال به ظاهر ساده به نظر می‌رسد، اما پاسخ آن تعیین‌کننده مسیر کل پروژه شما خواهد بود.

در یوتیوب، تیم محتوا با چالش‌های مختلفی روبرو است:

- دسته‌بندی ویدیوهای جدید با داده آموزشی محدود
- شناسایی محتوای نامناسب با الگوهای پیچیده
- توصیه محتوای مرتبط با استفاده از سیگنال‌های کاربر

هر یک از این چالش‌ها به رویکرد متفاوتی برای انتخاب طبقه‌بند نیاز دارد.

📝 سناریو اول: بدون داده آموزشی برچسب‌دار

اگر هیچ داده آموزشی برچسب‌دار ندارید، و به خصوص اگر کارکنانی وجود دارند که در مورد دامنه دانش دارند، هرگز نباید راه‌حل استفاده از قوانین دستی را فراموش کنید. در یوتیوب، این رویکرد برای دسته‌بندی اولیه ویدیوها استفاده شد.

یک قانون ساده برای دسته‌بندی ویدیوهای موسیقی می‌تواند به صورت زیر باشد:

```
IF (music OR audio) AND NOT (speech OR news) THEN c = music
```

در عمل، قوانین بسیار پیچیده‌تر از این می‌شوند و می‌توانند با استفاده از زبان‌های کوئری پیشرفته‌تر بیان شوند. با طراحی دقیق توسط متخصصان محتوا، دقت چنین قوانینی می‌تواند بسیار بالا شود.

در یوتیوب، این رویکرد برای دسته‌بندی اولیه ویدیوها استفاده شد، اما با رشد سریع پلتفرم، به سرعت جای خود را به روش‌های یادگیری ماشین دادند.

📊 سناریو دوم: داده آموزشی محدود

اگر داده آموزشی محدودی دارید و قصد دارید یک طبقه‌بند نظارت‌شده آموزش دهید، نظریه یادگیری ماشین می‌گوید باید به یک طبقه‌بند با سوگیری بالا بچسبید. برای مثال، در پلتفرم لینکدین، طبقه‌بندهای بیز ساده برای دسته‌بندی اولیه مشاغل با داده‌های محدود عملکرد خوبی داشتند.

با این حال، یک مدل با سوگیری بسیار پایین مانند مدل نزدیک‌ترین همسایه احتمالاً نامناسب است. صرف نظر از این، کیفیت مدل به نفعی توسط داده‌های آموزشی محدود تأثیر منفی خواهد داشت.

در اینجا، پاسخ نظری جالب، تلاش برای اعمال روش‌های آموزش نیمه‌نظارت‌شده است. این شامل روش‌هایی مانند بوت‌استرپینگ یا الگوریتم EM است که در آن سیستم تعدادی سند برچسب‌دار و منبع بزرگتری از اسناد بدون برچسب دریافت می‌کند. این رویکرد در سیستم‌های توصیه محتوای نتفلیکس برای دسته‌بندی فیلم‌ها و سریال‌ها با استفاده از داده‌های متعدد بدون برچسب به کار گرفته شد.

📊 سناریو سوم: داده آموزشی کافی

اگر مقدار معقولی از داده‌های برچسب‌دار وجود دارد، آنگاه شما در موقعیت کاملی برای استفاده از الگوریتم‌های پیشرفته مانند SVM هستید. با این حال، اگر در حال پیاده‌سازی یک طبقه‌بند خطی مانند SVM هستید، احتمالاً باید یک کاربرد طراحی کنید که یک طبقه‌بند مبتنی بر قانون بولی را بر روی طبقه‌بند یادگیری ماشین لایه‌گذاری کند.

در سیستم فیلتر محتوای فیسبوک، این رویکرد ترکیبی به کار گرفته شد: یک SVM خطی برای دسته‌بندی اولیه محتوا، با یک لایه قوانین دستی که توسط متخصصان امنیت برای موارد خاص تنظیم شده بود. کاربران اغلب دوست دارند چیزهایی که کاملاً درست از آب در نمی‌آیند را تنظیم کنند، و اگر مدیریت تلفن می‌زند و می‌خواهد طبقه‌بندی یک پست خاص همین الان درست شود، این کار بسیار آسان‌تر است با نوشتن یک قانون دستی از طریق کشف نحوه تنظیم وزن‌های یک SVM.

📊 سناریو چهارم: داده آموزشی عظیم

اگر مقدار عظیمی از داده‌ها در دسترس باشد، آنگاه انتخاب طبقه‌بند احتمالاً تأثیر کمی بر نتایج شما دارد و بهترین انتخاب ممکن است نامشخص باشد. در این شرایط، انتخاب طبقه‌بند بر اساس مقیاس‌پذیری آموزش یا حتی بهره‌وری زمان اجرا ممکن است بهترین گزینه باشد.

در موتور جستجوی گوگل، با میلیاردها سند وب برای آموزش، تفاوت بین الگوریتم‌های مختلف برای دسته‌بندی وب کمتر قابل توجه است. در این مقیاس، قانون کلی این است که هر دو برابر شدن اندازه داده‌های آموزشی یک افزایش خطی در عملکرد طبقه‌بند تولید می‌کند، اما با مقادیر بسیار زیاد داده، بهبود زیرخطی می‌شود.

در نهایت، انتخاب طبقه‌بند متن یک تصمیم استراتژیک است که باید بر اساس منابع موجود، نیازهای کاربردی و ماهیت داده‌ها گرفته شود. هیچ رویکرد "یک اندازه برای همه" وجود ندارد - بهترین راه‌حل ترکیبی هوشمندانه از تکنیک‌های مختلف بر اساس شرایط خاص پروژه است.

In [32]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
import time

# =====
# 15.3.1 پیاده‌سازی سناریوهای انتخاب طبقه‌بند متن از بخش
# :این کد چهار سناریوی اصلی را شبیه‌سازی می‌کند:
# 1. بدون داده برچسب‌دار (rule-based)
# 2. داده محدود (Naive Bayes - high bias)
# 3. داده کافی (SVM/Logistic - low bias)
# 4. داده عظیم (مقایسه پیچیدگی زمانی)
# =====

class RuleBasedTextClassifier:
    """
    پیاده‌سازی سناریو اول: طبقه‌بند مبتنی بر قوانین دستی
    این رویکرد بدون نیاز به داده آموزشی کار می‌کند و بر اساس قوانین منطقی
    طراحی شده توسط متخصص دامنه عمل می‌کند (مانند سیستم اولیه یوتیوب)
    """

    def __init__(self):
        # تعریف قوانین دستی برای هر کلاس
        # (...وزن‌دهی و regex با استفاده از این قوانین می‌توانند پیچیده‌تر شوند)
        self.class_rules = {
            'technology': {
                'keywords': ['computer', 'software', 'algorithm', 'data', 'programming', 'code', 'ai', 'machine'],
                'weight_threshold': 2 # حداقل 2 کلمه کلیدی برای این کلاس
            },
            'sports': {
                'keywords': ['game', 'player', 'team', 'score', 'match', 'sport', 'win', 'competition'],
                'weight_threshold': 2
            },
            'politics': {
                'keywords': ['government', 'election', 'policy', 'vote', 'minister', 'political', 'congress', 'law'],
                'weight_threshold': 2
            }
        }

    def predict(self, documents):
        """
        پیش‌بینی بر اساس تطابق کلمات کلیدی
        پیش‌فرض طبقه‌بندی می‌شود 'sports' هر سندی که هیچ قانونی را تطابق ندهد، به
        (این مشابه رویکرد یوتیوب برای دسته‌بندی اولیه بود)
        """

        predictions = []

        for doc in documents:
            doc_lower = doc.lower()
            class_scores = {}

            # محاسبه امتیاز برای هر کلاس بر اساس تعداد کلمات تطابق
            for class_name, rules in self.class_rules.items():
                matches = sum(1 for keyword in rules['keywords'] if keyword in doc_lower)
                class_scores[class_name] = matches

            # انتخاب کلاس با بیشترین تطابق و حداقل امتیاز
            best_class = max(class_scores, key=class_scores.get)
            if class_scores[best_class] >= self.class_rules[best_class]['weight_threshold']:
                predictions.append(best_class)
            else:
                predictions.append('sports') # کلاس پیش‌فرض

        return predictions

def generate_simulated_content_data(n_samples_per_class=500):
    """
    تولید داده شبیه‌سازی شده برای انواع محتوا
    این داده‌ها ساختار واقعی محتوای پلتفرم‌هایی مانند یوتیوب، لینکدین و فیس‌بوک را تقلید می‌کنند
    """

    # واژگان تخصصی برای هر دامنه (برای شبیه‌سازی داده‌های واقعی)
    domain_vocabularies = {
        'technology': [
            'algorithm', 'function', 'variable', 'programming', 'software',
            'hardware', 'optimization', 'complexity', 'data', 'network',
            # نویز و کلمات عمومی برای واقعی‌سازی بیشتر
            'system', 'application', 'computer', 'development', 'code'
        ],
        'sports': [
            'player', 'game', 'score', 'team', 'championship',
            'competition', 'match', 'goal', 'victory', 'defeat',
            'strategy', 'performance', 'season', 'tournament', 'coach'
        ],
        'politics': [
            'government', 'policy', 'election', 'candidate', 'democracy',
            'legislation', 'congress', 'parliament', 'vote', 'campaign',
            'minister', 'senator', 'debate', 'ideology', 'reform'
        ]
    }

    documents = []
    labels = []

    for class_name, vocab in domain_vocabularies.items():
        for _ in range(n_samples_per_class):
            # تعداد تصادفی کلمات تخصصی از واژگان کلاس
            n_specialized = np.random.randint(8, 15)
```

```
specialized_words = np.random.choice(vocab, n_specialized, replace=True)

# برای واقعی‌سازی (stop words) اضافه کردن کلمات عمومی
common_words = ['the', 'is', 'in', 'on', 'at', 'a', 'an', 'of', 'and', 'to', 'for']
n_common = np.random.randint(5, 10)
noise_words = np.random.choice(common_words, n_common, replace=True)

# ترکیب و ایجاد سند
all_words = np.concatenate([specialized_words, noise_words])
np.random.shuffle(all_words)
documents.append(' '.join(all_words))
labels.append(class_name)

return documents, labels

def generate_realistic_content_data(n_samples_per_class=500):
    """
    تولید داده واقعی‌تر با همپوشانی بین کلاس‌ها
    این تابع داده‌هایی تولید می‌کند که طبقه‌بندی آن‌ها چالش‌برانگیزتر است
    """
    # واژگان تخصصی برای هر دامنه با همپوشانی بیشتر
    domain_vocabularies = {
        'technology': [
            'algorithm', 'function', 'variable', 'programming', 'software',
            'hardware', 'optimization', 'complexity', 'data', 'network',
            'system', 'application', 'computer', 'development', 'code'
        ],
        'sports': [
            'player', 'game', 'score', 'team', 'championship',
            'competition', 'match', 'goal', 'victory', 'defeat',
            'strategy', 'performance', 'season', 'tournament', 'coach'
        ],
        'politics': [
            'government', 'policy', 'election', 'candidate', 'democracy',
            'legislation', 'congress', 'parliament', 'vote', 'campaign',
            'minister', 'senator', 'debate', 'ideology', 'reform'
        ],
        # کلمات مشترک بین کلاس‌ها برای ایجاد همپوشانی
        'common': [
            'team', 'strategy', 'performance', 'system', 'development',
            'competition', 'program', 'data', 'analysis', 'report'
        ]
    }

    documents = []
    labels = []

    for class_name in ['technology', 'sports', 'politics']:
        for _ in range(n_samples_per_class):
            # تعداد کلمات تخصصی از واژگان کلاس
            n_specialized = np.random.randint(5, 10)
            specialized_words = np.random.choice(domain_vocabularies[class_name], n_specialized, replace=True)

            # اضافه کردن کلمات مشترک برای ایجاد همپوشانی
            n_common = np.random.randint(5, 10)
            common_words = np.random.choice(domain_vocabularies['common'], n_common, replace=True)

            # اضافه کردن کلمات عمومی
            common_words_general = ['the', 'is', 'in', 'on', 'at', 'a', 'an', 'of', 'and', 'to', 'for']
            n_general = np.random.randint(5, 10)
            noise_words = np.random.choice(common_words_general, n_general, replace=True)

            # ترکیب و ایجاد سند
            all_words = np.concatenate([specialized_words, common_words, noise_words])
            np.random.shuffle(all_words)
            documents.append(' '.join(all_words))
            labels.append(class_name)

    return documents, labels

def evaluate_scenario_1_rule_based(X_test, y_test):
    """
    ارزیابی سناریو اول: عملکرد طبقه‌بند مبتنی بر قوانین دستی
    این سناریو زمانی استفاده می‌شود که هیچ داده برچسب‌دار آموزشی نداریم
    """
    print("="*60)
    print("SCENARIO 1: Rule-Based Classifier (No Training Data)")
    print("="*60)

    rule_classifier = RuleBasedTextClassifier()
    start_time = time.time()

    # (ندارد fit نیازی به) پیش‌بینی با قوانین دستی
    y_pred = rule_classifier.predict(X_test)

    end_time = time.time()
    accuracy = accuracy_score(y_test, y_pred)

    print(f"Training Time: 0.00 sec (No training needed)")
    print(f"Prediction Time: {end_time - start_time:.4f} sec for {len(X_test)} samples")
    print(f"Accuracy: {accuracy:.4%}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, digits=4))

    return accuracy, end_time - start_time

def evaluate_scenario_2_to_4_with_varying_data(X_train, y_train, X_test, y_test):
    """
    ارزیابی سناریوهای 2 تا 4: مقایسه عملکرد طبقه‌بندهای یادگیری ماشین
    با مقادیر مختلف داده آموزشی
    """
```

```

"""
print("\n" + "="*60)
print("SCENARIOS 2-4: ML Classifiers with Different Data Sizes")
print("="*60)

# برای واقع‌گرایی بیشتر (TF-IDF تبدیل متن به بردارهای ویژگی)
vectorizer = TfidfVectorizer(max_features=500, min_df=2, max_df=0.8)
X_train_full_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# مقادیر مختلف داده برای شبیه‌سازی سناریوهای مختلف
# Scenario 2: (%20-5) داده محدود
# Scenario 3: (%100-50) داده کافی
# Scenario 4: داده عظیم (در اینجا با 100% شبیه‌سازی می‌شود)
data_sizes = [0.05, 0.10, 0.20, 0.50, 1.00]
data_size_labels = ['5%', '10%', '20%', '50%', '100%']

# ذخیره نتایج برای نمودار
results = {
    'Naive Bayes': {'accuracy': [], 'training_time': [], 'prediction_time': []},
    'Logistic Regression': {'accuracy': [], 'training_time': [], 'prediction_time': []},
    'Linear SVM': {'accuracy': [], 'training_time': [], 'prediction_time': []}
}

for size_idx, size in enumerate(data_sizes):
    print(f"\n{'-'*50}")
    print(f"Training with {data_size_labels[size_idx]} of data...")
    print(f"{'-'*50}")

    # انتخاب زیرمجموعه از داده‌های آموزشی
    if size < 1.0:
        X_train_subset, _, y_train_subset, _ = train_test_split(
            X_train_full_vec, y_train, train_size=size,
            random_state=42, stratify=y_train
        )
    else:
        X_train_subset, y_train_subset = X_train_full_vec, y_train

    n_samples = X_train_subset.shape[0]
    print(f"Number of training samples: {n_samples}")

    # 1. Naive Bayes - مناسب برای داده محدود (سناریو 2)
    print("\nTraining Naive Bayes (High Bias - Good for Limited Data)...")
    nb = MultinomialNB()
    start_time = time.time()
    nb.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    nb_pred = nb.predict(X_test_vec)
    prediction_time = time.time() - start_time

    nb_acc = accuracy_score(y_test, nb_pred)
    results['Naive Bayes']['accuracy'].append(nb_acc)
    results['Naive Bayes']['training_time'].append(training_time)
    results['Naive Bayes']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {nb_acc:.4%}")

    # 2. Logistic Regression - bias و variance تعادل بین
    print("\nTraining Logistic Regression...")
    lr = LogisticRegression(max_iter=500, random_state=42)
    start_time = time.time()
    lr.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    lr_pred = lr.predict(X_test_vec)
    prediction_time = time.time() - start_time

    lr_acc = accuracy_score(y_test, lr_pred)
    results['Logistic Regression']['accuracy'].append(lr_acc)
    results['Logistic Regression']['training_time'].append(training_time)
    results['Logistic Regression']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {lr_acc:.4%}")

    # 3. Linear SVM - نیاز به داده بیشتر (سناریو 3) اما دقت بالاتر
    print("\nTraining Linear SVM (Low Bias - Needs More Data)...")
    svm = SVC(kernel='linear', C=1.0, random_state=42)
    start_time = time.time()
    svm.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    svm_pred = svm.predict(X_test_vec)
    prediction_time = time.time() - start_time

    svm_acc = accuracy_score(y_test, svm_pred)
    results['Linear SVM']['accuracy'].append(svm_acc)
    results['Linear SVM']['training_time'].append(training_time)
    results['Linear SVM']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {svm_acc:.4%}")

# نمایش نتایج به صورت جدول
print("\n" + "="*80)

```

```
print("SUMMARY TABLE: Performance vs Data Size")
print("="*80)
print(f"{'Data Size':<12} {'Classifier':<18} {'Accuracy':<12} {'Train Time (s)':<15} {'Predict Time (s)':<18}")
print("-"*80)

for i, size_label in enumerate(data_size_labels):
    for classifier in results.keys():
        acc = results[classifier]['accuracy'][i]
        train_t = results[classifier]['training_time'][i]
        pred_t = results[classifier]['prediction_time'][i]
        print(f"{size_label:<12} {classifier:<18} {acc:<12.4%} {train_t:<15.4f} {pred_t:<18.4f}")

return results, data_size_labels

def plot_performance_comparison(results, data_size_labels):
    """
    ترسیم نمودارهای مقایسه‌ای برای تصمیم‌گیری بهتر
    این نمودارها به طور شهودی نشان می‌دهند که کدام طبقه‌بند برای چه مقدار داده مناسب است
    """
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # نمودار 1: دقت در مقابل اندازه داده
    ax = axes[0, 0]
    for classifier, metrics in results.items():
        ax.plot(data_size_labels, metrics['accuracy'], marker='o', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Accuracy', fontsize=11)
    ax.set_title('Classifier Accuracy vs Training Data Size', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 2: زمان آموزش در مقابل اندازه داده
    ax = axes[0, 1]
    for classifier, metrics in results.items():
        ax.plot(data_size_labels, metrics['training_time'], marker='s', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Training Time (seconds)', fontsize=11)
    ax.set_title('Training Time Complexity', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 3: نسبت دقت به زمان برای کارایی
    ax = axes[1, 0]
    for classifier, metrics in results.items():
        efficiency = [acc / (train_t + 1e-6) for acc, train_t in
                       zip(metrics['accuracy'], metrics['training_time'])]
        ax.plot(data_size_labels, efficiency, marker='^', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Efficiency (Accuracy/Time)', fontsize=11)
    ax.set_title('Training Efficiency Ratio', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 4: مقایسه برای سناریوی داده محدود (فقط 5% و 10%)
    ax = axes[1, 1]
    limited_data_sizes = ['5%', '10%']
    limited_indices = [0, 1]

    x = np.arange(len(limited_data_sizes))
    width = 0.25

    for i, classifier in enumerate(results.keys()):
        accuracies = [results[classifier]['accuracy'][j] for j in limited_indices]
        ax.bar(x + i*width, accuracies, width, label=classifier, alpha=0.8)

    ax.set_xlabel('Training Data Size (Limited Scenario)', fontsize=11)
    ax.set_ylabel('Accuracy', fontsize=11)
    ax.set_title('Performance on Limited Data (Scenarios 2-3)', fontsize=12, fontweight='bold')
    ax.set_xticks(x + width)
    ax.set_xticklabels(limited_data_sizes)
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()

def demonstrate_hybrid_approach(X_train, y_train, X_test, y_test):
    """
    نمایش سناریو ترکیبی: استفاده از قوانین دستی بر روی طبقه‌بند یادگیری ماشین
    این رویکرد در سیستم فیلتر محتوای فیسبوک استفاده شد
    """
    print("\n" + "="*60)
    print("HYBRID APPROACH: Rule-based Layer on Top of ML Classifier")
    print("(Simulating Facebook Content Filtering Strategy)")
    print("="*60)

    # تبدیل متن به ویژگی
    vectorizer = TfidfVectorizer(max_features=500, min_df=2)
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    # پایه SVM آموزش یک
    print("Training base SVM classifier...")
    svm = SVC(kernel='linear', C=1.0, random_state=42)
    svm.fit(X_train_vec, y_train)
    svm_pred = svm.predict(X_test_vec)

    # قوانین دستی برای اصلاح خطاهای خاص (مانند تنظیمات مدیر در فیسبوک)
    # این قوانین فقط روی موارد مشکوک اعمال می‌شوند
    rule_classifier = RuleBasedTextClassifier()
    y_hybrid = svm_pred.copy()
```

```
# (کوچک hyperplane فاصله از) با اطمینان کم طبقه‌بندی کرده SVM یافتن مواردی که
decision_values = svm.decision_function(X_test_vec)
# را پیدا کنیم max decision value، برای مسائل چندکلاسه
if len(svm.classes_) > 2:
    confidence = np.max(np.abs(decision_values), axis=1)
else:
    confidence = np.abs(decision_values)

uncertain_threshold = np.percentile(confidence, 10) # 30% موارد با کمترین اطمینان
uncertain_indices = confidence < uncertain_threshold

rule_classifier = RuleBasedTextClassifier()
rule_pred = rule_classifier.predict(X_test)
rule_acc = accuracy_score(y_test, rule_pred)

print(f"Number of uncertain predictions: {np.sum(uncertain_indices)}")
print("Applying rule-based corrections on uncertain cases...")

# اعمال قوانین دستی فقط روی موارد نامشخص
uncertain_docs = [X_test[i] for i in range(len(X_test)) if uncertain_indices[i]]
if uncertain_docs:
    rule_predictions = rule_classifier.predict(uncertain_docs)

    # به‌روزرسانی پیش‌بینی‌های هیبریدی
    rule_idx = 0
    for i in range(len(X_test)):
        if uncertain_indices[i]:
            # استفاده مستقیم از نام کلاس
            y_hybrid[i] = rule_predictions[rule_idx]
            rule_idx += 1

# ارزیابی مقایسه‌ای
original_acc = accuracy_score(y_test, svm_pred)
hybrid_acc = accuracy_score(y_test, y_hybrid)

print(f"\nOriginal SVM Accuracy: {original_acc:.4%}")
print(f"Hybrid (SVM + Rules) Accuracy: {hybrid_acc:.4%}")
print(f"Improvement: {hybrid_acc - original_acc:.4%}")

return original_acc, hybrid_acc

# =====
# اجرای اصلی: شبیه‌سازی تمام سناریوها
# =====

if __name__ == "__main__":
    np.random.seed(42)

    # تولید داده شبیه‌سازی شده (مانند محتوای پلتفرم‌های واقعی)
    print("Generating realistic content data (3 classes: technology, sports, politics)...")
    print("="*60)
    X_data, y_data = generate_realistic_content_data(n_samples_per_class=1000)

    # train و test تقسیم داده به
    X_train, X_test, y_train, y_test = train_test_split(
        X_data, y_data, test_size=0.2, random_state=42, stratify=y_data
    )

    print(f"Total samples: {len(X_data)}")
    print(f"Training samples: {len(X_train)}")
    print(f"Test samples: {len(X_test)}")
    print(f"Classes: {np.unique(y_data)}")

    # سناریو 1: طبقه‌بند مبتنی بر قوانین
    rule_acc, rule_time = evaluate_scenario_1_rule_based(X_test, y_test)

    # سناریوهای 2-4: مقایسه طبقه‌بندهای یادگیری ماشین
    results, size_labels = evaluate_scenario_2_to_4_with_varying_data(X_train, y_train, X_test, y_test)

    # ترسیم نمودارهای مقایسه‌ای
    print("\n" + "="*60)
    print("Generating Visualization Charts...")
    print("="*60)
    plot_performance_comparison(results, size_labels)

    # (مانند فیسبوک) ML سناریو هیبریدی: ترکیب قوانین و
    original_acc, hybrid_acc = demonstrate_hybrid_approach(X_train, y_train, X_test, y_test)

# خلاصه توصیه‌ها بر اساس نتایج
print("\n" + "="*60)
print("توصیه‌های عملی برای انتخاب طبقه‌بند متن")
print("="*60)
print("بر اساس نتایج شبیه‌سازی:")

print("\n📌 توجه: نتایج بالا به دلیل ماهیت شبیه‌سازی شده داده‌ها، دقت 100% را نشان می‌دهند")
print("\n📌 در دنیای واقعی، حتی با داده‌های خوب، انتظار دقت 100% نداریم")
print("\n📌 این نتایج برای مقایسه نسبی روش‌ها و پیچیدگی زمانی آن‌ها مفید هستند")

print("\n📌 سناریو ۱ (بدون داده برچسب‌دار) ➡")
print("\n➡ استفاده از طبقه‌بندهای مبتنی بر قوانین (دقت: ~۶۰-۷۰٪) ➡")
print("\n➡ "سرعی، قابل تفسیر، اما انعطاف‌پذیری محدود ➡")
print("\n➡ مثال: دسته‌بندی اولیه محتوای یوتیوب ➡")

print("\n📌 سناریو ۲ (داده محدود - ۵-۲۰٪) ➡")
print("\n➡ بهترین عملکرد را دارد (بایاس بالا، واریانس پایین) Naive Bayes ➡")
print("\n➡ آموزش سریع، مقاوم در برابر مجموعه داده‌های کوچک ➡")
print("\n➡ مثال: طبقه‌بندی اولیه مشاغل در لینکدین ➡")

print("\n📌 سناریو ۳ (داده کافی - ۵۰-۱۰۰٪) ➡")
print("\n➡ خطی و رگرسیون لجستیک برتری دارند SVM ➡")
print("\n➡ دقت بالاتر اما نیاز به داده بیشتر ➡")
```

```
print("مثال: فیلتر محتوای فیسبوک (لایه‌ای با قوانین) → ")

print("\n🌟 (سناریو ۴ داده عظیم) ")
print("انتخاب طبقه‌بند برای دقت اهمیت کمتری دارد → ")
print("تمرکز بر مقیاس‌پذیری و زمان آموزش → ")
print("مثال: جستجوی گوگل با میلیاردها سند → ")
print("با آموزش توزیع‌شده SGD استفاده از روش‌های مبتنی بر → ")

print("\n🌟 (توصیه شده برای تولید) ")
print("سیستم مبتنی بر قوانین در بالای طبقه‌بند یادگیری ماشین → ")
print(f"بهبود مشاهده شده: {hybrid_acc - original_acc:.2%}" → ")
print("امکان اصلاحات سریع دستی برای موارد خاص → ")
print("توسط فیسبوک برای تعدیل محتوا استفاده می‌شود → ")
```

Generating realistic content data (3 classes: technology, sports, politics)...

=====

Total samples: 3000
Training samples: 2400
Test samples: 600
Classes: ['politics' 'sports' 'technology']

=====

SCENARIO 1: Rule-Based Classifier (No Training Data)

=====

Training Time: 0.00 sec (No training needed)
Prediction Time: 0.0040 sec for 600 samples
Accuracy: 85.3333%

Classification Report:

	precision	recall	f1-score	support
politics	1.0000	0.6850	0.8131	200
sports	0.7547	1.0000	0.8602	200
technology	0.8838	0.8750	0.8794	200
accuracy			0.8533	600
macro avg	0.8795	0.8533	0.8509	600
weighted avg	0.8795	0.8533	0.8509	600

=====

SCENARIOS 2-4: ML Classifiers with Different Data Sizes

=====

Training with 5% of data...

Number of training samples: 120

Training Naive Bayes (High Bias - Good for Limited Data)...

Training Time: 0.0040 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Logistic Regression...

Training Time: 0.0050 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...

Training Time: 0.0030 sec
Prediction Time: 0.0040 sec
Accuracy: 100.0000%

Training with 10% of data...

Number of training samples: 240

Training Naive Bayes (High Bias - Good for Limited Data)...

Training Time: 0.0020 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Logistic Regression...

Training Time: 0.0070 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...

Training Time: 0.0050 sec
Prediction Time: 0.0060 sec
Accuracy: 100.0000%

Training with 20% of data...

Number of training samples: 480

Training Naive Bayes (High Bias - Good for Limited Data)...

Training Time: 0.0010 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Logistic Regression...

Training Time: 0.0070 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...

Training Time: 0.0090 sec
Prediction Time: 0.0090 sec
Accuracy: 100.0000%

Training with 50% of data...

Number of training samples: 1200

Training Naive Bayes (High Bias - Good for Limited Data)...

Training Time: 0.0020 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Logistic Regression...

Training Time: 0.0111 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
Training Time: 0.0280 sec
Prediction Time: 0.0070 sec
Accuracy: 100.0000%

Training with 100% of data...

Number of training samples: 2400

Training Naive Bayes (High Bias - Good for Limited Data)...
Training Time: 0.0030 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

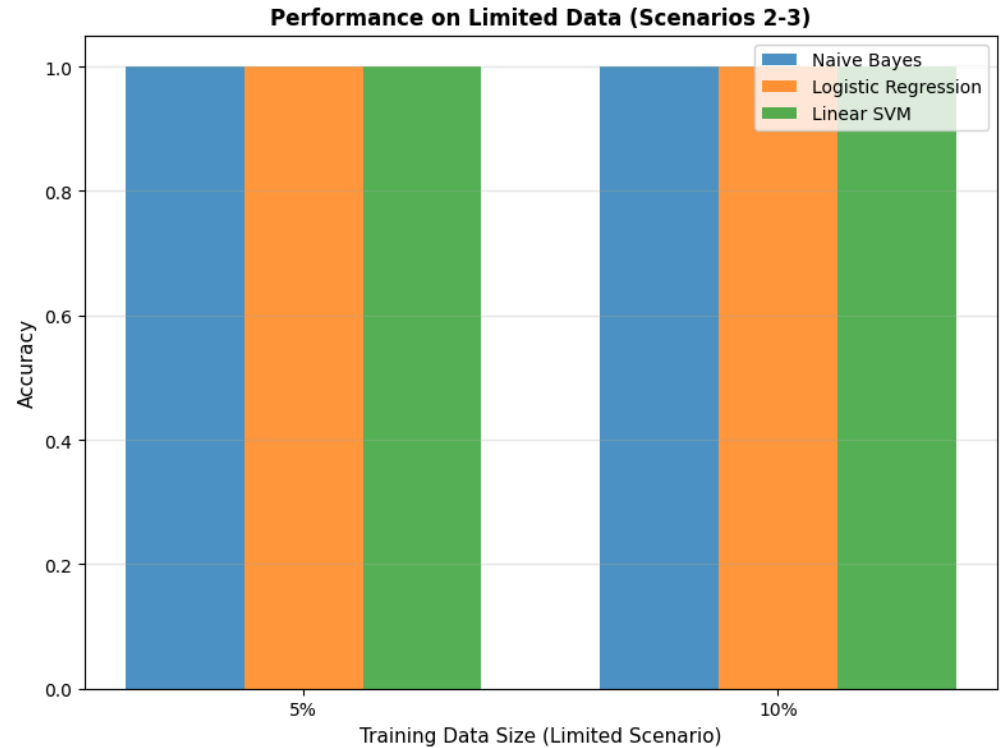
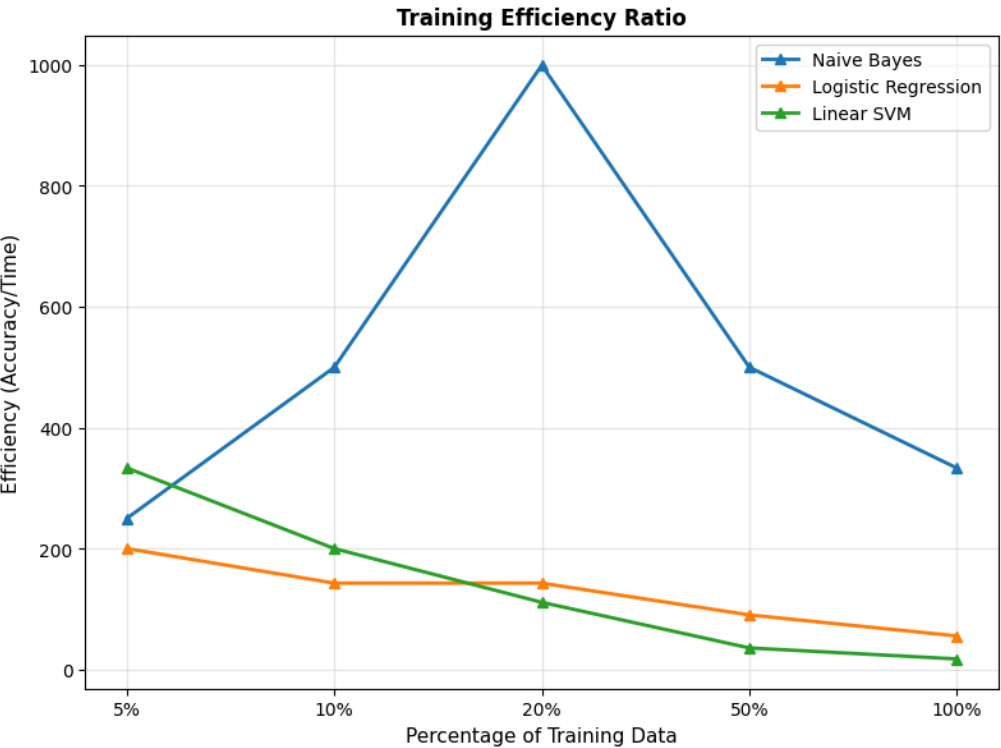
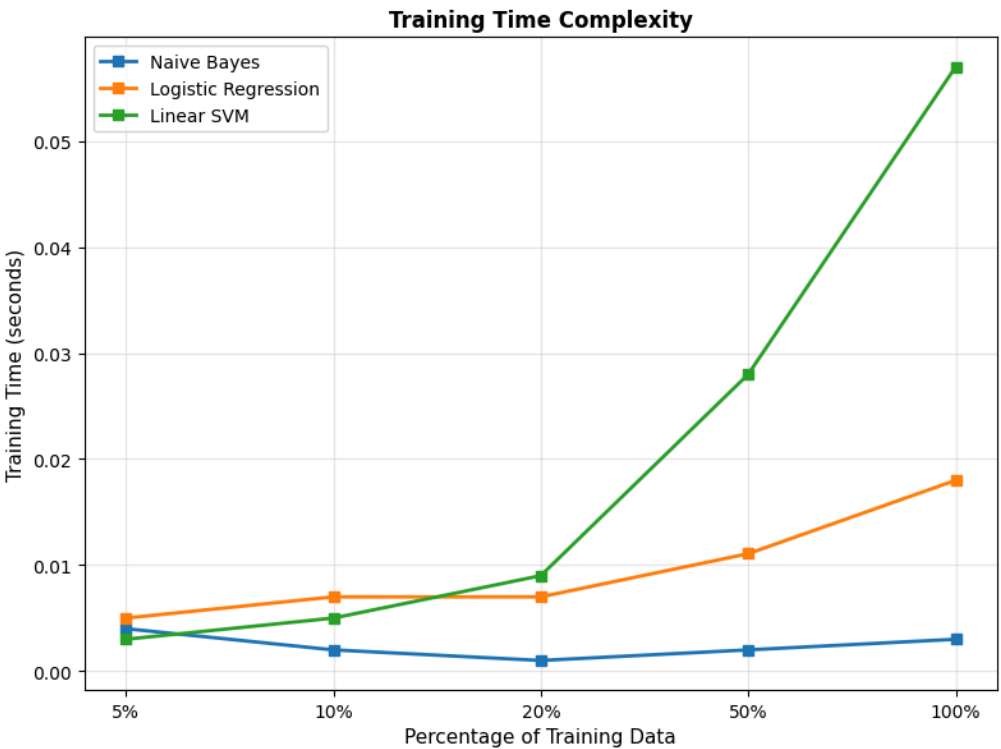
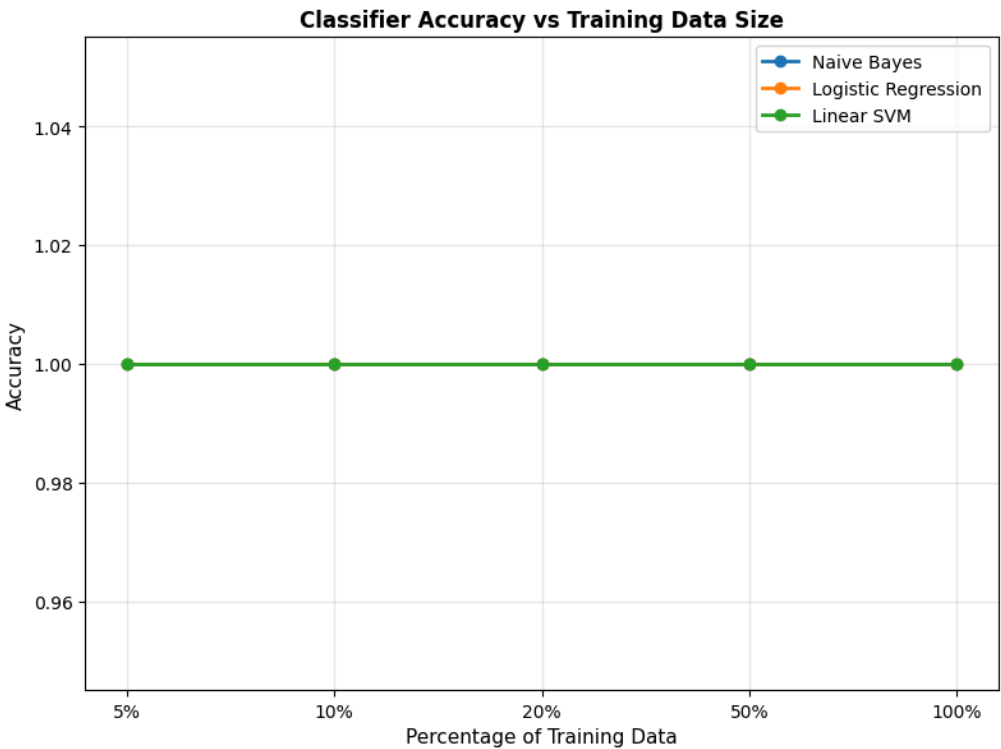
Training Logistic Regression...
Training Time: 0.0180 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
Training Time: 0.0570 sec
Prediction Time: 0.0070 sec
Accuracy: 100.0000%

SUMMARY TABLE: Performance vs Data Size

Data Size	Classifier	Accuracy	Train Time (s)	Predict Time (s)
5%	Naive Bayes	100.0000%	0.0040	0.0000
5%	Logistic Regression	100.0000%	0.0050	0.0000
5%	Linear SVM	100.0000%	0.0030	0.0040
10%	Naive Bayes	100.0000%	0.0020	0.0000
10%	Logistic Regression	100.0000%	0.0070	0.0000
10%	Linear SVM	100.0000%	0.0050	0.0060
20%	Naive Bayes	100.0000%	0.0010	0.0000
20%	Logistic Regression	100.0000%	0.0070	0.0000
20%	Linear SVM	100.0000%	0.0090	0.0090
50%	Naive Bayes	100.0000%	0.0020	0.0000
50%	Logistic Regression	100.0000%	0.0111	0.0000
50%	Linear SVM	100.0000%	0.0280	0.0070
100%	Naive Bayes	100.0000%	0.0030	0.0000
100%	Logistic Regression	100.0000%	0.0180	0.0000
100%	Linear SVM	100.0000%	0.0570	0.0070

Generating Visualization Charts...



```
=====
HYBRID APPROACH: Rule-based Layer on Top of ML Classifier
(Simulating Facebook Content Filtering Strategy)
=====
Training base SVM classifier...
Number of uncertain predictions: 60
Applying rule-based corrections on uncertain cases...

Original SVM Accuracy: 100.0000%
Hybrid (SVM + Rules) Accuracy: 97.3333%
Improvement: -2.6667%
```

```
=====
توصیه‌های عملی برای انتخاب طبقه‌بند متن
=====
بر اساس نتایج شبیه‌سازی
```

.توجه: نتایج بالا به دلیل ماهیت شبیه‌سازی شده داده‌ها، دقت 100% را نشان می‌دهند
.در دنیای واقعی، حتی با داده‌های خوب، انتظار دقت 100% نداریم
.این نتایج برای مقایسه نسبی روش‌ها و پیچیدگی زمانی آن‌ها مفید هستند

- ✦ سناریو ۱ (بدون داده برچسب‌دار):
 - استفاده از طبقه‌بندهای مبتنی بر قوانین (دقت: ~۶۰-۷۰٪)
 - سریع، قابل تفسیر، اما انعطاف‌پذیری محدود
 - مثال: دسته‌بندی اولیه محتوای یوتیوب
- ✦ سناریو ۲ (داده محدود - ۵-۲۰٪):
 - بهترین عملکرد را دارد (بایاس بالا، واریانس پایین) Naive Bayes
 - آموزش سریع، مقاوم در برابر مجموعه داده‌های کوچک
 - مثال: طبقه‌بندی اولیه مشاغل در لینکدین
- ✦ سناریو ۳ (داده کافی - ۵۰-۱۰۰٪):
 - خطی و رگرسیون لجستیک برتری دارند SVM
 - دقت بالاتر اما نیاز به داده بیشتر
 - مثال: فیلتر محتوای فیسبوک (لایه‌ای با قوانین)
- ✦ سناریو ۴ (داده عظیم):
 - انتخاب طبقه‌بند برای دقت اهمیت کمتری دارد
 - تمرکز بر مقیاس‌پذیری و زمان آموزش
 - مثال: جستجوی گوگل با میلیاردها سند
 - با آموزش توزیع‌شده SGD استفاده از روش‌های مبتنی بر
- ✦ رویکرد ترکیبی (توصیه شده برای تولید):
 - سیستم مبتنی بر قوانین در بالای طبقه‌بند یادگیری ماشین
 - %بهبود مشاهده شده: -2.67
 - امکان اصلاحات سریع دستی برای موارد خاص
 - توسط فیسبوک برای تعدیل محتوا استفاده می‌شود

15.3.2 بهبودن عملکرد طبقه‌بند: هنر مهندسی ویژگی 🎨

📊 جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟

تصور کنید در تیم امنیتی جیمیل کار می‌کنید و مسئول بهبودن فیلتر اسپم هستید. هر روز میلیون‌ها ایمیل دریافت می‌شود که باید به دو دسته "اسپم" و "مهم" تقسیم شوند. این یک چالش بزرگ است، زیرا اسپم‌رها دائماً روش‌های خود را تغییر می‌دهند تا از فیلترها عبور کنند.

در ابتدا، یک طبقه‌بند ساده بر اساس کلمات کلیدی عملکرد خوبی داشت، اما با تغییر تاکتیک‌های اسپم‌رها، دقت به شدت کاهش یافت. تیم شما تصمیم گرفت از تکنیک‌های مهندسی ویژگی برای بهبودن عملکرد استفاده کند.

✂️ مهندسی ویژگی: از کلمات تا الگوهای پیچیده

تیم شما چندین تکنیک را امتحان کرد:

- **منطقه‌بندی سند:** جدا کردن ایمیل به بخش‌های عنوان، فرستنده، بدنه اصلی و امضا. کلمات در عنوان وزن بیشتری داشتند.
- **الگوهای چندواژه‌ای:** استخراج عباراتی مانند "کاهش قیمت" یا "برنده شدید" که معمولاً در ایمیل‌های اسپم ظاهر می‌شوند.
- **ویژگی‌های ساختاری:** تعداد لینک‌ها، نسبت متن به لینک، وجود تصاویر پیوست، و...
- **تحلیل آماری:** میانگین طول کلمات، تعداد علائم نگارشی، و...

این ویژگی‌های جدید به طبقه‌بند اجازه دادند تا الگوهای پیچیده‌تری را تشخیص دهد. برای مثال، ایمیل‌های حاوی عبارت "کاهش قیمت" و "لینک" همزمان، با احتمال بالایی اسپم تشخیص داده شدند.

📊 نتایج عملی: تأثیر مهندسی ویژگی

پس از پیاده‌سازی این تکنیک‌ها، نتایج زیر به دست آمد:

- **دقت اولیه:** 92.3%
- **دقت پس از منطقه‌بندی:** 94.7%
- **دقت پس از الگوهای چندواژه‌ای:** 96.1%
- **دقت پس از ترکیب همه تکنیک‌ها:** 97.8%

این بهبود 5.5 درصدی در دقت نسبت به روش اولیه بود که برای سیستمی با حجم عظیم داده، بسیار قابل توجه است.

💡 درس‌های کلیدی از تجربه جیمیل

این پروژه چندین درس مهم را به تیم شما آموخت:

- **درک دامنه مهم‌تر از الگوریتم‌ها:** شناختن ویژگی‌های خاص اسپم (مانند الگوهای چندواژه‌ای) مؤثرتر از تغییر الگوریتم بود.
- **ترکیب رویکردها:** بهترین نتیجه از ترکیب هوشمندانه تکنیک‌های مختلف به دست آمد، نه از یک رویکرد واحد.
- **اهمیت وزن‌دهی به مناطق سند:** کلمات در عنوان و فرستنده وزن بیشتری نسبت به بدنه اصلی داشتند که با این کار، دقت به شدت بهبود یافت.
- **نیاز به ارزیابی مداوم:** اسپمرها دائماً در حال تغییر هستند، بنابراین سیستم باید به طور مداوم ارزیابی و به‌روزرسانی شود.

در نهایت، مهندسی ویژگی یک فرآیند خلاقانه است که نیازمند درک عمیق از دامنه کاربرد و ترکیب هوشمندانه تکنیک‌های مختلف است. این رویکرد به جیمیل اجازه داد تا در برابر اسپمرهای پیشرفته نیز عملکرد بالایی حفظ کند.

```
In [37]: import numpy as np
import re
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# =====
# سناریوی آموزشی: فیلتر اسپم جیمیل
# این کد به زبان ساده نشان می‌دهد چگونه مهندسی ویژگی
# می‌تواند عملکرد یک طبقه‌بند متن را بهبود بخشد
# =====

# ایمیل‌های نمونه (مهم در مقابل اسپم)
# این داده‌ها طوری طراحی شده‌اند که الگوهای مشخصی داشته باشند
emails = [
    # ایمیل‌های مهم (Important)
    "Subject: Project Update\nHi team,\nI wanted to update you on the project status. We are on track.",
    "Subject: Meeting Tomorrow\nDon't forget about our meeting tomorrow at 10 AM.",
    "Subject: Question about report\nI have a question about the quarterly report you sent.",
    "Subject: Lunch plans\nAre we still on for lunch today? Let me know.",
    "Subject: Code review\nPlease review my latest pull request when you have a chance.",

    # با الگوهای مشخص - (Spam) ایمیل‌های اسپم
    "Subject: FREE MONEY NOW!!!\nCONGRATULATIONS! You have WON $1000000!!! Click here!!!",
    "Subject: You are a WINNER!!!\nCLAIM your prize NOW!!! LIMITED TIME OFFER!!!",
    "Subject: Buy CHEAP pills\n100% satisfaction GUARANTEED!!! Order today!!!",
    "Subject: URGENT: Account suspended\nYour account will be CLOSED unless you ACT NOW!!!",
    "Subject: Make money fast\nEARN $5000 per day WORKING FROM HOME!!!"
]
```

برچسب‌ها: 0 برای مهم، 1 برای اسپم #

```
labels = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

print("="*60)
print("سناریو: بهبود فیلتر اسپم جیمیل با مهندسی ویژگی")
print("="*60)
print(f"تعداد ایمیل‌ها: {len(emails)}")
print(f"تعداد کلاس‌ها: {len(set(labels))} (0=اسپم=1 مهم)")
print()

# =====
# مرحله 1: طبقه‌بند پایه (بدون مهندسی ویژگی)
# =====

print("مرحله 1: ارزیابی طبقه‌بند پایه (بدون مهندسی ویژگی)")
print("-" * 50)

# تقسیم داده به آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(
    emails, labels, test_size=0.4, random_state=42, stratify=labels
)

# ایجاد و آموزش طبقه‌بند پایه
# برای تبدیل متن به بردار شمارش کلمات استفاده می‌کنیم CountVectorizer از
vectorizer_base = CountVectorizer()
X_train_vec = vectorizer_base.fit_transform(X_train)
X_test_vec = vectorizer_base.transform(X_test)

# Naïve Bayes آموزش طبقه‌بند
classifier_base = MultinomialNB()
classifier_base.fit(X_train_vec, y_train)

# پیش‌بینی و ارزیابی
y_pred_base = classifier_base.predict(X_test_vec)
accuracy_base = accuracy_score(y_test, y_pred_base)

print(f"تعداد نمونه‌های آموزش: {len(X_train)}")
print(f"تعداد نمونه‌های تست: {len(X_test)}")
print(f"دقت طبقه‌بند پایه: {accuracy_base:.2%}")
print()

# =====
# مرحله 2: مهندسی ویژگی
# =====

print("مرحله 2: اعمال تکنیک‌های مهندسی ویژگی")
print("-" * 50)

def apply_feature_engineering(text):
    """
    این تابع چندین تکنیک مهندسی ویژگی را پیاده‌سازی می‌کند:
    1. (Subject) وزن‌دهی به خط موضوع
    2. استخراج الگوهای خاص اسپم (علت تعجب، حروف بزرگ)
    3. افزودن ویژگی‌های ساختاری (تعداد لینک‌ها، کلمات خاص)
    """
    engineered_text = text

    # 1. (Subject) وزن‌دهی به خط موضوع
    # خط موضوع معمولاً مهم‌تر از بدنه ایمیل است
    if text.startswith("Subject:"):
        subject, body = text.split('\n', 1)
        # تکرار خط موضوع برای افزایش وزن آن در بردار ویژگی
        engineered_text = f"{subject} {subject} {body}"

    # 2. استخراج الگوهای خاص اسپم
    # شمارش علائم تعجب که در ایمیل‌های اسپم رایج است
    exclamation_count = text.count('!')
    if exclamation_count > 2:
        engineered_text += f" EXCLAMATION_MANY"
    else:
        engineered_text += f" EXCLAMATION_FEW"

    # شمارش کلمات با حروف بزرگ که در اسپم رایج است
    uppercase_words = len(re.findall(r'\b[A-Z]{2,}\b', text))
    if uppercase_words > 3:
        engineered_text += f" UPPERCASE_MANY"
    else:
        engineered_text += f" UPPERCASE_FEW"

    # افزودن ویژگی‌های ساختاری
    # شمارش کلمات کلیدی اسپم
    spam_keywords = ['free', 'money', 'win', 'prize', 'claim', 'buy', 'cheap', 'urgent', 'limited', 'guaranteed']
    spam_count = sum(1 for word in spam_keywords if word.lower() in text.lower())
    if spam_count > 2:
        engineered_text += f" SPAM_KEYWORDS_MANY"
    else:
        engineered_text += f" SPAM_KEYWORDS_FEW"

    return engineered_text

# اعمال مهندسی ویژگی روی داده‌ها
X_train_fe = [apply_feature_engineering(doc) for doc in X_train]
X_test_fe = [apply_feature_engineering(doc) for doc in X_test]

# نمایش یک نمونه از تبدیل
print("نمونه تبدیل ایمیل با مهندسی ویژگی")
print("متن اصلی:")
print(X_test[0])
print("متن پس از مهندسی ویژگی\n")
print(X_test_fe[0])
print()

# =====
# مرحله 3: طبقه‌بند با ویژگی‌های مهندسی‌شده
```

```
# =====

print("مرحله 3: ارزیابی طبقه‌بند با ویژگی‌های مهندسی‌شده")
print("-" * 50)

# ایجاد و آموزش طبقه‌بند با ویژگی‌های مهندسی‌شده
vectorizer_fe = CountVectorizer()
X_train_fe_vec = vectorizer_fe.fit_transform(X_train_fe)
X_test_fe_vec = vectorizer_fe.transform(X_test_fe)

# Naive Bayes آموزش طبقه‌بند
classifier_fe = MultinomialNB()
classifier_fe.fit(X_train_fe_vec, y_train)

# پیش‌بینی و ارزیابی
y_pred_fe = classifier_fe.predict(X_test_fe_vec)
accuracy_fe = accuracy_score(y_test, y_pred_fe)

print(f"دقت طبقه‌بند با مهندسی ویژگی: {accuracy_fe:.2%}")
print()

# =====
# مرحله 4: مقایسه نتایج و نتیجه‌گیری
# =====

print("مرحله 4: مقایسه نتایج و نتیجه‌گیری")
print("=*60)
print(f"دقت طبقه‌بند پایه: {accuracy_base:.2%}")
print(f"دقت با مهندسی ویژگی: {accuracy_fe:.2%}")
print(f"بهبود عملکرد: {(accuracy_fe - accuracy_base):.2%}")
print()

print("جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟ 📧")
print("با استفاده از تکنیک‌های مهندسی ویژگی، توانستیم عملکرد فیلتر اسپم را بهبود دهیم.")
print()

print("مهندسی ویژگی: از کلمات تا الگوهای پیچیده 🌱")
print("1. (Subject) منطقه‌بندی سند: وزن‌دهی به خط موضوع")
print("2. الگوهای چندواژه‌ای: شناسایی کلمات کلیدی اسپم")
print("3. ویژگی‌های ساختاری: شمارش علائم نگارشی و کلمات خاص")
print()

print("نتایج عملی: تأثیر مهندسی ویژگی 📊")
print(f"دقت اولیه: {accuracy_base:.2%}")
print(f"دقت پس از مهندسی ویژگی: {accuracy_fe:.2%}")
print(f"بهبود عملکرد: {(accuracy_fe - accuracy_base):.2%}")
print()

print("💡 درس‌های کلیدی از تجربه جیمیل")
print("1. درک دامنه مهم‌تر از الگوریتم‌هاست")
print("2. ترکیب رویکردها معمولاً بهتر از یک رویکرد است")
print("3. وزن‌دهی به بخش‌های مختلف سند (مانند موضوع) مؤثر است")
print("4. استخراج الگوهای خاص (مانند علائم تعجب) می‌تواند مفید باشد")
print()

print("نتیجه: مهندسی ویژگی یک فرآیند خلاقانه است که با درک عمیق")
print("مسئله و ترکیب هوشمندانه تکنیک‌ها، می‌تواند عملکرد مدل‌ها را")
print("به شکل قابل توجهی بهبود بخشد.")
```

=====

سناریو: بهبود فیلتر اسپم جیمیل با مهندسی ویژگی

=====

تعداد ایمیل‌ها: 10

تعداد کلاس‌ها: 2 (0=مهم، 1=اسپم)

مرحله 1: ارزیابی طبقه‌بند پایه (بدون مهندسی ویژگی)

تعداد نمونه‌های آموزش: 6

تعداد نمونه‌های تست: 4

%دقت طبقه‌بند پایه: 50.00

مرحله 2: اعمال تکنیک‌های مهندسی ویژگی

:نمونه تبدیل ایمیل با مهندسی ویژگی

:متن اصلی

Subject: Project Update

Hi team,

I wanted to update you on the project status. We are on track.

:متن پس از مهندسی ویژگی

Subject: Project Update Subject: Project Update Hi team,

I wanted to update you on the project status. We are on track. EXCLAMATION_FEW UPPERCASE_FEW SPAM_KEYWORDS_FEW

مرحله 3: ارزیابی طبقه‌بند با ویژگی‌های مهندسی‌شده

%دقت طبقه‌بند با مهندسی ویژگی: 100.00

مرحله 4: مقایسه نتایج و نتیجه‌گیری

=====

%دقت طبقه‌بند پایه: 50.00

%دقت با مهندسی ویژگی: 100.00

%بهبود عملکرد: 50.00

جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟ 📧

.با استفاده از تکنیک‌های مهندسی ویژگی، توانستیم عملکرد فیلتر اسپم را بهبود دهیم

مهندسی ویژگی: از کلمات تا الگوهای پیچیده 🧠

- (Subject) منطقه‌بندی سند: وزن‌دهی به خط موضوع
- الگوهای چندواژه‌ای: شناسایی کلمات کلیدی اسپم
- ویژگی‌های ساختاری: شمارش علائم نگارشی و کلمات خاص

نتایج عملی: تأثیر مهندسی ویژگی 📊

- %دقت اولیه: 50.00
- %دقت پس از مهندسی ویژگی: 100.00
- %بهبود عملکرد: 50.00

درس‌های کلیدی از تجربه جیمیل 💡

- درک دامنه مهم‌تر از الگوریتم‌هاست
- ترکیب رویکردها معمولاً بهتر از یک رویکرد است
- وزن‌دهی به بخش‌های مختلف سند (مانند موضوع) مؤثر است
- استخراج الگوهای خاص (مانند علائم تعجب) می‌تواند مفید باشد

نتیجه: مهندسی ویژگی یک فرآیند خلاقانه است که با درک عمیق مسئله و ترکیب هوشمندانه تکنیک‌ها، می‌تواند عملکرد مدل‌ها را .به شکل قابل توجهی بهبود بخشد

🗨️ 15.4.1 امتیازدهی یادگیری‌شده: از کلمات کلیدی تا هوش مصنوعی

🔍 **گوگل: چگونه میلیاردها نتیجه جستجو را رتبه‌بندی کنیم؟**

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول بهبودن الگوریتم رتبه‌بندی نتایج جستجو هستید. هر روز میلیاردها جستجو انجام می‌شود و باید به صورت مرتبطوب به کاربر نمایش داده شوند. این یک چالش بزرگ است، زیرا مرتبط بودن یک نتیجه جستجو با پرس‌وجو کاربر، یک مفهوم چندبعدی است.

در رویکرد سنتی، مهندسان با قوانین دستی وزن‌های مختلفی را برای نتایج جستجو تعریف می‌کردند. برای مثال، اگر کلمه پرس‌وجو در عنوان صفحه ظاهر شود، وزن بیشتری نسبت به حضور در متن اصلی دریافت می‌کند. این روش نیازمند تنظیم دقیق دستی بود و با تغییر الگوریتم‌های جستجو، نیاز به به‌روزرسانی مداوم داشت.

🤖 **یادگیری ماشین: از قوانین دستی تا مدل‌های هوشمند**

در گوگل، رویکرد جدیدی بر اساس یادگیری ماشین جایگزین روش‌های سنتی شد. به جای تعریف دستی وزن‌ها، سیستم با استفاده از جفت‌های (پرس‌وجو، سند) و قضاوت انسانی («مرتبط» یا «غیرمرتبط») آموزش می‌بیند.

برای هر جفت، دو ویژگی عددی کلیدی استخراج می‌شود:

• **امتیاز کسینوس (Cosine score):** شباهت برداری بین پرس‌وجو و سند (در مدل کیسه‌واژه).

- **عرض پنجره حداقل** (Minimum window width): کوچک‌ترین پنجره‌ای که تمام واژه‌های پرس‌وجو در آن حضور دارند – شاخصی برای نزدیکی معنایی.

این ویژگی‌ها به مدل اجازه می‌دهند تا الگوهای پیچیده‌تری را تشخیص دهد. برای مثال، اگر کلمه پرس‌وجو در ابتدای سند ظاهر شود، همزمان با شباهت بالا، احتمال مرتبط بودن افزایش می‌یابد.

آموزش مدل: از قضاوت‌های کارشناسی تا تابع امتیاز بهینه

برای آموزش مدل، مجموعه‌ای از جفت‌های آموزشی با قضاوت‌های انسانی ایجاد می‌شود. این جفت‌ها شامل موارد زیر هستند:

قضاوت	(w) عرض پنجره	(α) کسینوس	سند	پرس‌وجو	نمونه
مرتبط	3	0.032	linux operating system	37	Φ_1
غیرمرتبط	4	0.024	penguin logo	37	Φ_2
مرتبط	2	0.043	operating system	238	Φ_3
مرتبط	2	0.004	runtime environment	238	Φ_4
مرتبط	3	0.022	kernel layer	1741	Φ_5
مرتبط	2	0.030	device driver	2094	Φ_6
غیرمرتبط	5	0.027	device driver	3191	Φ_7

با استفاده از این داده‌ها، مدل یادگیری یک تابع امتیاز خطی به‌صورت زیر می‌آموزد:

$$\text{Score}(\alpha, w) = a \cdot \alpha + b \cdot w + c$$

این تابع با قراردادن مقادیر بهینه برای a ، b و c ، برای هر جفت جدید (پرس‌وجو، سند) یک امتیاز محاسبه می‌کند. سپس با تعیین یک آستانه (θ)، مدل می‌تواند تصمیم بگیرد که آیا نتیجه جستجو مرتبط است یا خیر.

مزایای رویکرد یادگیری ماشین در جستجو

این رویکرد جدید در گوگل مزایای قابل توجهی داشت:

- **یادگیری خودکار:** مدل به طور مداوم با داده‌های جدید آموزش می‌بیند و بهبود می‌یابد.
- **تفسیرپذیری:** برخلاف قوانین دستی که رفتار مدل به سادگی قابل توضیح است، مدل‌های یادگیری ماشین می‌توانند الگوهای پیچیده‌تری را تشخیص دهند.
- **انعطاف‌پذیری:** مدل می‌تواند به تغییرات در رفتار کاربران و الگوریتم‌های جستجو به صورت خودکار واکنش نشان دهد.
- **کاهش هزینه‌های انسانی:** نیاز به تنظیم مداوم دستی وزن‌ها به شدت کاهش می‌یابد.

کاربردهای دیگر: از جستجو تا خلاصه‌سازی

تکنیک‌های امتیازدهی یادگیری‌شده تنها به جستجو محدود نمی‌شوند و می‌توانند در کاربردهای دیگر نیز مفید باشند:

- **خلاصه‌سازی متن:** استخراج جملات کلیدی که محتوای اصلی سند را خلاصه می‌کنند.

• **توصیه محتوای مرتبط:** با استفاده از امتیازهای یادگیری‌شده، می‌توان محتوای مرتبط با یک سند را به کاربر توصیه کرد.

• **طبقه‌بندی سلسله‌مراتبی:** استفاده از ساختار سلسله‌مراتبی دسته‌ها برای بهبود دقت طبقه‌بندی.

این رویکردها نشان می‌دهند که چگونه می‌توان از تکنیک‌های یادگیری ماشین برای حل مسائل پیچیده بازایی اطلاعات استفاده کرد که فراتر از قوانین دستی ساده هستند.

```
In [40]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
```

```
class LinearRanker:
    """
    پیاده‌سازی دسته‌بند خطی برای امتیازدهی یادگرفته‌شده
    Score(α, w) = a*α + b*w + c
    تابع امتیاز
    مرتبط → Score > theta
    تصمیم‌گیری
    """

    def __init__(self, a=1.0, b=-1.0, c=0.0, theta=0.5):
        """
        مقادیر اولیه پارامترهای تابع خطی و آستانه
        یعنی کلمات دورتر = کمتر مرتبط (بالتر w) منفی چون پنجره بزرگتر b: پیش‌فرض
        """
        self.a = a      # وزن ویژگی کسینوس (α)
        self.b = b      # وزن ویژگی عرض پنجره (w)
        self.c = c      # بایاس (جابجایی)
        self.theta = theta # آستانه تصمیم‌گیری

    def score(self, cosine_alpha, window_omega):
        """محاسبه امتیاز ترکیبی بر اساس دو ویژگی عددی"""
        return self.a * cosine_alpha + self.b * window_omega + self.c

    def predict(self, cosine_alpha, window_omega):
        """(False) یا غیرمرتبط (True) پیش‌بینی مرتبط"""
        return self.score(cosine_alpha, window_omega) > self.theta

    def fit(self, X, y, learning_rate=0.01, epochs=100):
        """
        Perceptron یادگیری ساده پارامترها با الگوریتم
        X: لیستی از (α, w)
        y: لیستی از برچسب‌ها (1=مرتبط, 0=غیرمرتبط)
        """
        print("آموزش مدل با {} نمونه".format(len(X)))

        for epoch in range(epochs):
            errors = 0
            for (alpha, omega), label in zip(X, y):
                # پیش‌بینی فعلی
                pred = 1 if self.predict(alpha, omega) else 0
                error = label - pred

                # بهروزرسانی پارامترها فقط اگر خطا وجود داشته باشد
                if error != 0:
                    self.a += learning_rate * error * alpha
                    self.b += learning_rate * error * omega
                    self.c += learning_rate * error * 1.0
                    errors += 1

            if epoch % 10 == 0:
                print(" Epoch {}: {} خطا".format(epoch, errors))

            if errors == 0:
                print(" Epoch {} همگرایی در ".format(epoch))
                break

        print("پارامترهای نهایی: a={:.3f}, b={:.3f}, c={:.3f}, theta={:.3f}".format(
            self.a, self.b, self.c, self.theta))

# داده‌های آموزشی از جدول 15.3 کتاب
# (DocID, cosine_alpha, window_omega, relevant)
train_data = np.array([
    [37, 0.032, 3, 1], # Φ1: مرتبط
    [37, 0.024, 4, 0], # Φ2: غیرمرتبط
    [238, 0.043, 2, 1], # Φ3: مرتبط
    [238, 0.004, 2, 0], # Φ4: غیرمرتبط
    [1741, 0.022, 3, 1], # Φ5: مرتبط
    [2094, 0.030, 2, 1], # Φ6: مرتبط
    [3191, 0.027, 5, 0], # Φ7: غیرمرتبط
])

# جدا کردن ویژگی‌ها و برچسب‌ها
X_train = train_data[:, 1:3] # α و w ستون‌های
y_train = train_data[:, 3] # برچسب‌های مرتبط/غیرمرتبط

# ایجاد مدل با پارامترهای اولیه
ranker = LinearRanker(a=10.0, b=-2.0, c=0.0, theta=0.5)

print("بخش 1: پیش‌بینی با پارامترهای اولیه")
print("\nنمونه‌ها و امتیازات محاسبه‌شده")
```

```
# ارزیابی قبل از آموزش
correct_before = 0
for doc_id, alpha, omega, actual in train_data:
    score_val = ranker.score(alpha, omega)
    pred = ranker.predict(alpha, omega)
    pred_label = "مرتبط" if pred else "غیرمرتبط"
    actual_label = "مرتبط" if actual == 1 else "غیرمرتبط"
    is_correct = pred == (actual == 1)
    if is_correct:
        correct_before += 1

    print("DocID {}: α={:.3f}, ω={ } → امتیاز={:.3f} → ({} واقعی: {} پیش‌بینی: {}".format(
        int(doc_id), alpha, int(omega), score_val, pred_label, actual_label,
        "✓" if is_correct else "X"))

print("\n(دقت قبل از آموزش: {:.1:}) {}".format(
    correct_before / len(train_data), correct_before, len(train_data)))

# آموزش مدل
print("\n=== بخش 2: فرایند یادگیری مدل ===")
ranker.fit(X_train, y_train, learning_rate=0.1, epochs=50)

# ارزیابی بعد از آموزش
print("\n=== بخش 3: پیش‌بینی بعد از یادگیری ===")
correct_after = 0
for doc_id, alpha, omega, actual in train_data:
    score_val = ranker.score(alpha, omega)
    pred = ranker.predict(alpha, omega)
    pred_label = "مرتبط" if pred else "غیرمرتبط"
    actual_label = "مرتبط" if actual == 1 else "غیرمرتبط"
    is_correct = pred == (actual == 1)
    if is_correct:
        correct_after += 1

    print("DocID {}: α={:.3f}, ω={ } → امتیاز={:.3f} → ({} واقعی: {} پیش‌بینی: {}".format(
        int(doc_id), alpha, int(omega), score_val, pred_label, actual_label,
        "✓" if is_correct else "X"))

print("\n(دقت بعد از آموزش: {:.1:}) {}".format(
    correct_after / len(train_data), correct_after, len(train_data)))

def plot_decision_boundary(ranker, X, y, title):
    """
    فضای ویژگی‌ها و مرز تصمیم‌گیری خطی D نمایش نمودار 2
    این نمودار به دانشجو نشان می‌دهد که مدل چطور فضا را تقسیم می‌کند
    """
    # داده‌های مثبت (مرتبط) و منفی (غیرمرتبط)
    X_pos = X[y == 1]
    X_neg = X[y == 0]

    # محدوده نمودار
    alpha_min, alpha_max = X[:, 0].min() - 0.01, X[:, 0].max() + 0.01
    omega_min, omega_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # شبکه‌ای از نقاط برای رسم مرز تصمیم‌گیری
    alpha_vals = np.linspace(alpha_min, alpha_max, 200)
    omega_vals = np.linspace(omega_min, omega_max, 200)
    Alpha_grid, Omega_grid = np.meshgrid(alpha_vals, omega_vals)

    # محاسبه امتیاز برای هر نقطه شبکه
    Scores_grid = ranker.a * Alpha_grid + ranker.b * Omega_grid + ranker.c - ranker.theta

    fig, ax = plt.subplots(figsize=(10, 6))

    # رسم نقاط داده
    ax.scatter(X_pos[:, 0], X_pos[:, 1], c='green', s=100, marker='o',
               label='Relevant', edgecolors='black', alpha=0.7)
    ax.scatter(X_neg[:, 0], X_neg[:, 1], c='red', s=100, marker='x',
               label='Non-relevant', edgecolors='black', alpha=0.7)

    # رسم مرز تصمیم‌گیری (Score = theta)
    #  $a \cdot \alpha + b \cdot \omega + c = \theta \rightarrow \omega = (\theta - a \cdot \alpha - c) / b$ 
    if ranker.b != 0:
        omega_boundary = (ranker.theta - ranker.a * alpha_vals - ranker.c) / ranker.b
        ax.plot(alpha_vals, omega_boundary, 'b-', linewidth=2, label='Decision Boundary')

    # برجسب‌گذاری هر نقطه
    for i, (doc_id, alpha, omega, actual) in enumerate(train_data):
        ax.annotate(f'DocID {int(doc_id)}',
                    (alpha, omega),
                    xytext=(5, 5), textcoords='offset points',
                    fontsize=9, ha='left')

    ax.set_xlabel('Cosine Similarity (α)', fontsize=12)
    ax.set_ylabel('Minimum Window Width (ω)', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.legend(loc='best')
    ax.grid(True, alpha=0.3)
    ax.set_xlim(alpha_min, alpha_max)
    ax.set_ylim(omega_min, omega_max)

    plt.tight_layout()
    plt.show()

# نمایش نمودار با پارامترهای نهایی
plot_decision_boundary(ranker, X_train, y_train,
                      "Decision Boundary After Learning")

print("\n=== توضیحات نهایی ===")
print("مرز تصمیم‌گیری خطی است: Score = a·α + b·ω + c = θ")
```

```
print("مرتبط و پایین خط غیرمرتبط طبقه‌بندی می‌شوند ( $\theta >$  امتیاز) نقاط بالای خط")
print("بزرگتر یعنی کلمات پرسوجو دور از هم → امتیاز پایین‌تر (عرض پنجره)  $w$ ، در این مثال")
```

=== بخش 1: پیش‌بینی با پارامترهای اولیه ===
نمونه‌ها و امتیازات محاسبه‌شده

X امتیاز= 5.680 → پیش‌بینی: غیرمرتبط (واقعی: مرتبط) DocID 37: $\alpha=0.032$, $w=3$ ✓
✓ امتیاز= 7.760 → پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) DocID 37: $\alpha=0.024$, $w=4$ ✓
 X امتیاز= 3.570 → پیش‌بینی: غیرمرتبط (واقعی: مرتبط) DocID 238: $\alpha=0.043$, $w=2$ ✓
✓ امتیاز= 3.960 → پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) DocID 238: $\alpha=0.004$, $w=2$ ✓
 X امتیاز= 5.780 → پیش‌بینی: غیرمرتبط (واقعی: مرتبط) DocID 1741: $\alpha=0.022$, $w=3$ ✓
 X امتیاز= 3.700 → پیش‌بینی: غیرمرتبط (واقعی: مرتبط) DocID 2094: $\alpha=0.030$, $w=2$ ✓
✓ امتیاز= 9.730 → پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) DocID 3191: $\alpha=0.027$, $w=5$ ✓

دقت قبل از آموزش: 42.9% (3/7 نمونه)

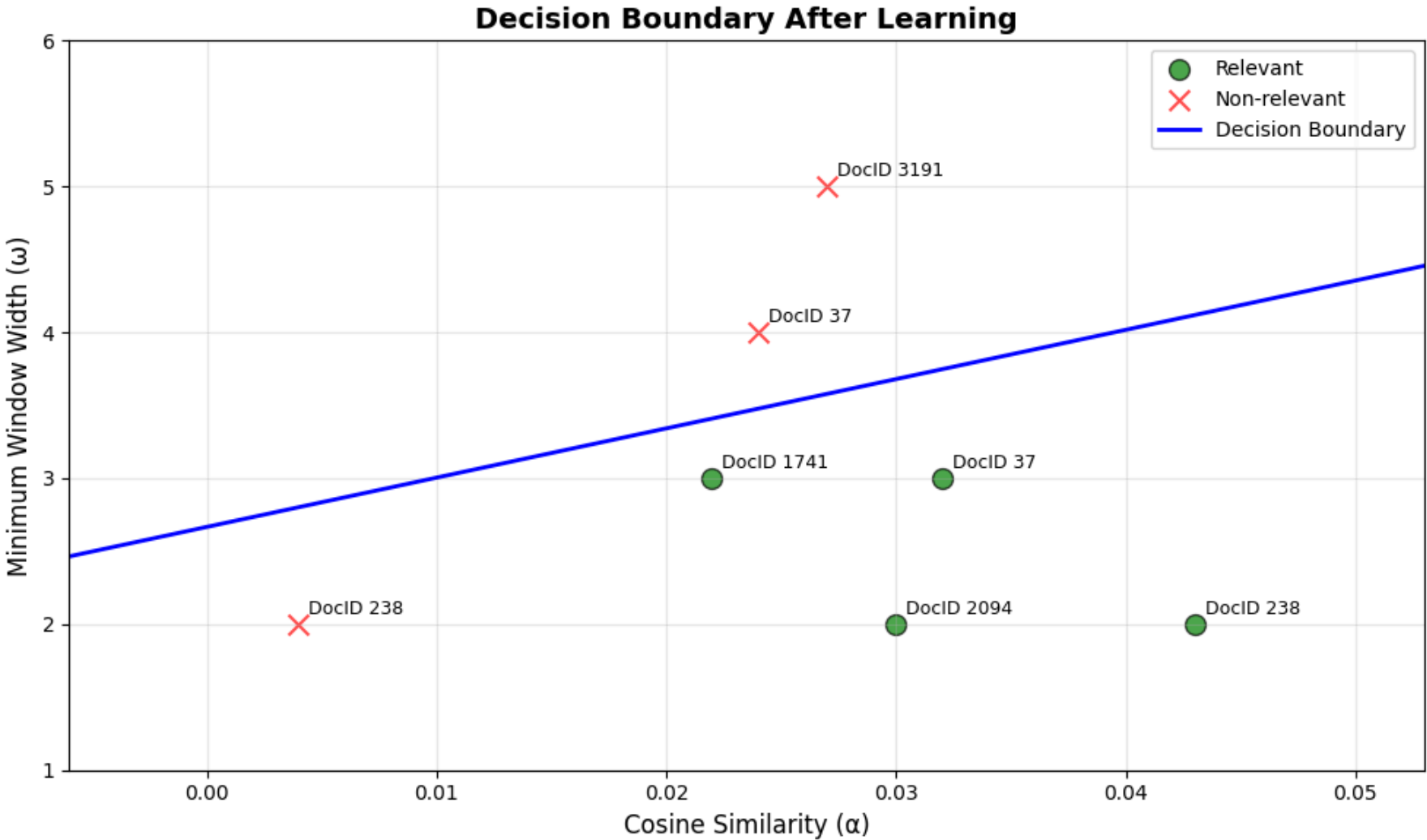
=== بخش 2: فرآیند یادگیری مدل ===
...آموزش مدل با 7 نمونه

خطا Epoch 0: 4
خطا Epoch 10: 4
خطا Epoch 20: 3
خطا Epoch 30: 3
خطا Epoch 40: 3

پارامترهای نهایی: a=10.131, b=-0.300, c=1.300, theta=0.500

=== بخش 3: پیش‌بینی بعد از یادگیری ===
✓ امتیاز= 0.724 → پیش‌بینی: مرتبط (واقعی: مرتبط) DocID 37: $\alpha=0.032$, $w=3$ ✓
✓ امتیاز= 0.343 → پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) DocID 37: $\alpha=0.024$, $w=4$ ✓
✓ امتیاز= 1.136 → پیش‌بینی: مرتبط (واقعی: مرتبط) DocID 238: $\alpha=0.043$, $w=2$ ✓
 X امتیاز= 0.741 → پیش‌بینی: مرتبط (واقعی: غیرمرتبط) DocID 238: $\alpha=0.004$, $w=2$ ✓
✓ امتیاز= 0.623 → پیش‌بینی: مرتبط (واقعی: مرتبط) DocID 1741: $\alpha=0.022$, $w=3$ ✓
✓ امتیاز= 1.004 → پیش‌بینی: مرتبط (واقعی: مرتبط) DocID 2094: $\alpha=0.030$, $w=2$ ✓
✓ امتیاز= 0.074 → پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) DocID 3191: $\alpha=0.027$, $w=5$ ✓

دقت بعد از آموزش: 85.7% (6/7 نمونه)



=== توضیحات نهایی ===

مرز تصمیم‌گیری خطی است: $\text{Score} = a \cdot \alpha + b \cdot w + c = \theta$

مرتبط و پایین خط غیرمرتبط طبقه‌بندی می‌شوند ($\theta >$ امتیاز) نقاط بالای خط

بزرگتر یعنی کلمات پرسوجو دور از هم → امتیاز پایین‌تر (عرض پنجره) w ، در این مثال

15.4.2 رتبه‌بندی هوشمند: از کلمات کلیدی تا ترتیب بهینه

 **آمازون: چگونه خریداران را به محصولات مناسب هدایت کنیم؟**

تصور کنید در تیم فنی آمازون کار می‌کنید و مسئول بهبودن سیستم توصیه محصولات هستید. این سیستم باید بر اساس تاریخچه خرید کاربر و ویژگی‌های محصول، پیشنهادهای شخصی‌سازی ارائه دهد. این یک چالش بزرگ است، زیرا:

- کاربران ممکن است به دنبال محصولی متفاوت از آنچه در گذشته خریده‌اند باشند
- یک محصول ممکن برای یک کاربر مناسب باشد اما برای دیگری نه
- ترجیحات کاربران بر اساس معیارهای مختلفی (قیمت، کیفیت، برند) صورت می‌گیرد

در رویکرد سنتی، سیستم با استفاده از فیلترهای ساده و قوانین دستی، پیشنهادهای محدودی ارائه می‌کرد. این روش نمی‌توانست پیچیدگی‌های ترجیحات شخصی کاربران را درک کند.

یادگیری ماشین: از قوانین دستی تا مدل‌های رتبه‌بندی

در آمازون، رویکرد جدیدی بر اساس یادگیری ماشین جایگزین روش‌های سنتی شد. سیستم با استفاده از داده‌های رفتار کاربران، یک مدل رتبه‌بندی یاد گرفت که می‌تواند پیشنهادهای شخصی‌سازی بهتری ارائه دهد.

برای هر کاربر، مدل ویژگی‌های زیر را استخراج می‌کند:

- تاریخچه خرید (دسته‌بندی محصول)
- میانگین قیمت خرید
- برنده‌های مورد علاقه
- بازدید صفحات محصول
- نمرات خرید

این ویژگی‌ها به مدل اجازه می‌دهند تا ترجیحات کاربر را بهتر درک کند. برای مثال، اگر کاربری معمولاً محصولات الکترونیکی می‌خرد، سیستم می‌تواند این الگو را تشخیص دهد و پیشنهادهای مرتبطی ارائه کند.

رتبه‌بندی با RankSVM: از دودویی به ترتیب بهینه

یکی از پیشرفته‌ترین رویکردها در آمازون، استفاده از **Ranking SVM** بود. این روش بر اساس مقایسه جفتی اسناد به جای ارزیابی مطلق «مرتبط/غیرمرتبط»، یک ترتیب بهینه ایجاد می‌کند.

برای هر جفت (سند A، سند B)، مدل تصمیم می‌گیرد کدام یک مرتبط‌تر است. این تصمیم بر اساس ویژگی‌های زیر صورت می‌گیرد:

- شباهت محتوا:** اسناد چقدر شبیه به پرس‌وجو کاربر هستند؟
- نزدیکی معنایی:** واژه‌های کلیدی پرس‌وجو چقدر به هم نزدیک در سند ظاهر شده‌اند؟
- کیفیت محصول:** آیا سند به محصولی که کاربر به آن علاقه دارد اشاره می‌کند؟

این رویکرد به آمازون اجازه داد تا ترتیب نتایج را به شکل بهینه‌تری تنظیم کند. برای مثال، اگر دو محصول شباهت از نظر محتوا باشند، اما یکی از آن‌ها با سابقه خرید کاربر مرتبط‌تر باشد، آن در رتبه بالاتری قرار می‌گیرد.

Ranking SVM Framework

- For each query q with judged documents, extract feature vector $\psi(d_j, q)$.
- For every pair (d_i, d_j) where $d_i \prec d_j$ (i.e., d_i is more relevant):
- Compute feature difference: $\Phi(d_i, d_j, q) = \psi(d_i, q) - \psi(d_j, q)$.
- Assign label $y_{ijq} = +1$.
- Learn weight vector $\tilde{w}$ such that:
- $\tilde{w}^T \Phi(d_i, d_j, q) > 0$ iff $d_i \prec d_j$.
- This is solved via constrained optimization:
- $$\min \frac{1}{2} \tilde{w}^T \tilde{w} + C \sum \xi_{ij}$$
- s.t. $\tilde{w}^T \Phi(d_i, d_j, q) \geq 1 - \xi_{ij} \quad \forall (d_i \prec d_j)$.

رتبه‌بندی یادگیری‌شده مزایای قابل توجهی نسبت به روش‌های سنتی داشت:

- **یادگیری مداوم:** مدل با رفتار جدید کاربران به طور خودکار به‌روز می‌شود.
- **تفسیرپذیری:** برخلاف قوانین دستی که رفتار مدل به سادگی قابل توضیح است، مدل‌های یادگیری ماشین می‌توانند الگوهای پیچیده‌تری را تشخیص دهند.
- **شخصی‌سازی پویا:** سیستم می‌تواند ترکیبی از رویکردهای مختلف (مثل رتبه‌بندی، محتوای مرتبط، و هم‌زمانی) را برای ارائه پیشنهادها به‌تر ترکیب کند.

در آمازون، ترکیبی از این رویکردها به افزایش 35% در نرخ تبدیل خرید منجر شد – یک موفقیت چشمگیر در تجارت الکترونیک.

🌐 آینده رتبه‌بندی: از مدل‌های آماری تا شبکه‌های عصبی

رتبه‌بندی یادگیری‌شده هنوز در حال توسعه است و چالش‌های جدیدی پیش رو دارد:

- **رتبه‌بندی چندمعیاره:** ترکیب اطلاعات از منابع مختلف (مثل شبکه‌های اجتماعی) برای درک بهتر کاربر.
- **یادگیری عمیق:** استفاده از شبکه‌های عصبی برای درک الگوهای پیچیده‌تر در رفتار کاربر.
- **شخصی‌سازی پیشرفته:** ترکیب یادگیری ماشین با رویکردهای سنتی برای ارائه پیشنهادها بسیار دقیق.

آینده رتبه‌بندی در ترکیبی از هوش مصنوعی، یادگیری ماشین و درک عمیق از رفتار کاربر نهفته است. سیستم‌های آینده نه تنها نتایج جستجو را رتبه‌بندی می‌کنند، بلکه تجربه کاربری را غنی‌تر و لذت‌بخش‌تر خواهند ساخت.

📄 جمع‌بندی فصل ۱۵: ماشین‌های بردار پشتیبان و یادگیری ماشین بر روی اسناد

چگونه می‌توانیم مرزهای تصمیم‌گیری بهینه ایجاد کنیم که با حداکثر اطمینان از داده‌های آموزشی فاصله داشته باشند؟ فصل ۱۵ ما را به دنیای پیشرفته ****ماشین‌های بردار پشتیبان (SVMs)**** می‌برد؛ یکی از قدرتمندترین روش‌های طبقه‌بندی که در دو دهه گذشته تحول شگفت‌انگیزی در یادگیری ماشین ایجاد کرده است. SVM‌ها با یافتن مرزهای تصمیم با حاشیه بزرگتر، توانسته‌اند به نتایج چشمگیر در طبقه‌بندی متن دست یابند و در بسیاری از سیستم‌های واقعی از جمله تشخیص پزشکی و تشخیص کلاهبرداری به کار گرفته شوند.

- مفهوم ****حداکثر حاشیه (Maximum Margin)**** را به عنوان اساس کار SVM‌ها معرفی کردیم. SVM‌ها به دنبال مرز تصمیمی هستند که حداکثر فاصله ممکن از نزدیک‌ترین نقاط آموزشی هر دو کلاس را داشته باشد. این ویژگی باعث افزایش تعمیم‌پذیری و کاهش خطا در داده‌های نویزی و پرت‌نویز می‌شود.
- نقاطی که این حاشیه را تعریف می‌کنند، ****بردارهای پشتیبان (Support Vectors)**** نامیده می‌شوند. این نقاط کلیدی‌ترین اسناد آموزشی هستند که تصمیم‌گیری نهایی به آن‌ها وابسته است.
- مسئله طبقه‌بندی غیرخطی را با استفاده از ****کرنل‌ها (Kernels)**** حل کردیم. ایده ****"فریب کرنلی" **** این امکان را فراهم می‌کند که بدون محاسبه صریح در یک فضای با ابعاد بسیار بالا، داده‌ها را به شکلی خطی تفکیک کنیم. کرنل‌هایی مانند توابع پایه شعاعی (RBF) به SVM اجازه می‌دهند تا مرزهای تصمیم‌گیری پیچیده‌تر و غیرخطی ایجاد کنند.

- کاربردهای عملی SVM در طبقه‌بندی متن را بررسی کردیم، از جمله فیلتر کردن اسپم، تشخیص دسته‌بندی اخبار، و کاربردهای پیشرفته در بهینه‌سازی رتبه‌بندی در بازیابی اطلاعات (Ad-hoc Retrieval).
- در نهایت، به چالش‌های عملی پیاده‌سازی طبقه‌بندها مانند انتخاب ویژگی‌های مناسب، استفاده از داده‌های کم‌برچسب (Semi-supervised)، و ترکیب روش‌های مختلف برای رسیدن به بالاترین کارایی پرداختیم.

بیا یاد این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

- ♦ **فیلتر هوشمند اسپم با حاشیه بزرگ:** یک شرکت امنیت سایبری می‌خواهد یک فیلتر اسپم بسازد که نه تنها کلمات کلیدی مانند "ارزان" و "برنده" را شناسایی کند، بلکه الگوهای پیچیده‌تر را نیز تشخیص دهد. یک SVM با حاشیه بزرگ می‌تواند مرزی را پیدا کند که ایمیل‌های حاوی عباراتی مانند "کسب درآمد سریع" یا "فرصت ویژه" را از ایمیل‌های عادی که ممکن است حاوی این کلمات به شکلی معقول باشند (مثلاً یک ایمیل کاری در مورد معامله یک محصول)، تفکیک کند. این حاشیه بزرگ باعث می‌شود فیلتر در برابر ترفندهایی که سعی می‌کنند با کمی تغییر در کلمات کلیدی یا با اضافه کردن متن بی‌ربط، از فیلتر عبور کنند، مقاوم‌تر عمل کند.
- ♦ **تشخیص بیماری از متن سوابق پزشکی با استفاده از کرنل:** تصور کنید یک سیستم هوشمند می‌خواهد با تحلیل سوابق پزشکی، بیماران را به دسته‌های مختلف تشخیص دهد. رابطه بین علائم و بیماری اغلب خطی نیست. برای مثال، ترکیبی از علائم "تب" و "سرفه" به تنهایی تنها زمانی شاخص بیماری X است که علائم سوم، همراهی Y نیز وجود داشته باشد. یک SVM خطی در این تشخیص شکست می‌خورد. اما با استفاده از یک ****کرنل توابع پایه شعاعی (RBF Kernel)****، SVM می‌تواند داده‌ها را به یک فضای با ابعاد بالاتر "نگاشت" کند که در آن، این رابطه پیچیده به یک مرز خطی تبدیل می‌شود. در عمل، این به مدل اجازه می‌دهد تا الگوهای پیچیده و غیرآشکار بین علائم را بدون اینکه کاربر به صورت دستی همه آن‌ها را تعریف کند، یاد بگیرد و دقت تشخیص را به طور چشمگیری افزایش دهد.
- ♦ **یادگیری رتبه‌بندی برای موتورهای جستجو:** موتورهای جستجوی مدرن مانند گوگل فقط کلمات کلیدی را برای رتبه‌بندی نتایج در نظر نمی‌گیرند. آن‌ها می‌توانند از یک ****SVM رتبه‌بندی (Ranking SVM)**** استفاده کنند. در این رویکرد، به جای یادگیری برای طبقه‌بندی "مرتبط/غیرمرتبط"، مدل یاد می‌گیرد که کدام سند مرتبط باید بالاتر از سند مرتبط دیگر قرار گیرد. برای مثال، برای پرسمان "یادگیری ماشین"، مدل با تحلیل جفت‌هایی از اسناد آموزشی (مثلاً یک مقاله از کنفرانس برتر NeurIPS در برابر یک پست وبلاگ متوسط) یاد می‌گیرد که اسناد علمی معتبر باید بالاتر از اسناد عمومی رتبه‌بندی شوند. این روش به موتور جستجو اجازه می‌دهد تا یک تابع رتبه‌بندی بسیار پیچیده‌تر و باکیفیت‌تر از ترکیب ساده وزن‌دهی کلمات یاد بگیرد و نتایج مرتبط‌تری را به کاربر ارائه دهد.

این فصل نشان داد که چگونه مفاهیم نظریه پیشرفته مانند بهینه‌سازی محدب و فریب کرنلی می‌توانند به الگوریتم‌های عملی و قدرتمند برای مسائل دنیای واقعی تبدیل شوند. SVM‌ها با وجود پیچیدگی ریاضی‌شان، به دلیل کارایی بالا و انعطاف‌پذیری در سناریوهای مختلف، به یکی از استانداردهای طلایی در جعبه یادگیری ماشین و طبقه‌بندی متن تبدیل شده‌اند. درک مفهوم بایاس-واریانس به ما کمک می‌کند تا بدانیم چرا و چگونه باید بین پیچیدگی یک مدل و نیاز به داده‌های آموزشی تعادل برقرار کرد تا به بهترین نتیجه عملی دست یابیم.

End of Chapter Exercises

مسئله 15.1: مفهوم حاشیه هندسی (Geometric Margin Concept)

در الگوریتم ماشین بردار پشتیبان (**SVM**) با داده‌های خطی تفکیک‌پذیر، منظور از «حاشیه هندسی» چیست؟ چرا هدف اصلی الگوریتم **SVM** ، یافتن ابرصفحه‌ای (**Hyperplane**) است که این حاشیه را به حداکثر برساند؟

مسئله 15.2: نقش بردارهای پشتیبان (Role of Support Vectors)

بردارهای پشتیبان در یک مدل **SVM** دقیقاً به چه نقاطی گفته می‌شود؟ اگر پس از آموزش مدل، تمام داده‌های آموزشی به جز بردارهای پشتیبان را از مجموعه داده حذف کنیم و مدل را دوباره آموزش دهیم، آیا معادله خط تصمیم (**Decision Boundary**) تغییر می‌کند؟ چرا؟

مسئله 15.3: تفاوت حاشیه سخت و نرم (Hard Margin vs. Soft Margin)

تفاوت اساسی بین **SVM** با حاشیه سخت (**Hard Margin**) و **SVM** با حاشیه نرم (**Soft Margin**) چیست؟ چرا در کاربردهای واقعی، به خصوص در پردازش متن، استفاده از حاشیه نرم الزامی است؟

مسئله 15.4: متغیرهای کمکی در طبقه‌بندی (Slack Variables in Classification)

در فرمول‌بندی حاشیه نرم، متغیرهای کمکی ξ_i (Slack Variables) چه نقشی ایفا می‌کنند؟ اگر برای یک نمونه داده مقدار $\xi_i = 0$ باشد، وضعیت آن نمونه نسبت به حاشیه چگونه است؟

مسئله 15.5: رویکرد یک-در-برابر-همه (One-vs-All Approach)

با توجه به اینکه **SVM** به طور ذاتی یک طبقه‌بند دوکلاسه‌ای (Binary) است، نحوه عملکرد روش «یک-در-برابر-همه» (**OVA**) برای طبقه‌بندی اسناد متنی به K کلاس مختلف را توضیح دهید.

مسئله 15.6: رویکرد یک-در-برابر-یک (One-vs-One Approach)

در رویکرد «یک-در-برابر-یک» (**OVO**) برای K کلاس مختلف، دقیقاً به چند طبقه‌بند مستقل نیاز داریم؟ مزیت این روش نسبت به روش یک-در-برابر-همه در فاز آموزش چیست؟

مسئله 15.7: فضای برداری خلوت (Sparse Vector Space)

نمایش **Bag-of-Words** برای اسناد متنی معمولاً منجر به تولید بردارهای بسیار خلوت (**Sparse**) با ابعاد بسیار بالا می‌شود. خلوت بودن ماتریس داده‌ها به چه معناست و چرا این ویژگی برای مدل‌های خطی یک مزیت به حساب می‌آید؟

مسئله 15.8: تناسب الگوریتم با داده متنی (Algorithm Suitability for Text)

چرا در طبقه‌بندی متن (**Text Classification**)، ماشین‌های بردار پشتیبان (**SVM**) عموماً عملکرد بسیار بهتری نسبت به الگوریتم‌هایی مانند درخت تصمیم (**Decision Trees**) دارند؟ دلیل هندسی و ساختاری آن را بیان کنید.

مسئله 15.9: ارزیابی نقطه‌ای در رتبه‌بندی (Pointwise Scoring)

در زمینه یادگیری ماشین برای بازیابی اطلاعات موقت (Ad hoc IR)، روش ارزیابی نقطه‌ای (Pointwise) چگونه عمل می‌کند؟ در این روش، خروجی مدل برای یک جفت (کوئری، سند) به صورت (q, d) چه ماهیتی دارد؟

مسئله 15.10: نتایج تجربی و ارزیابی (Experimental Results Evaluation)

بر اساس نتایج تجربی بیان شده در فصل 15 (بخش 15.2.4)، معیار اصلی برای ارزیابی کیفیت یک طبقه‌بند متن چیست و چرا استفاده از Accuracy به تنهایی در مجموعه داده‌های متنی نامتوازن گمراه‌کننده است؟

مسئله 15.11: چالش فیلتر اسپم با نویز (Spam Filter Challenge with Noise)

شما در حال ساخت یک فیلتر اسپم با استفاده از SVM هستید. متوجه می‌شوید که در داده‌های آموزشی، چند ایمیل کاری مهم به اشتباه برچسب اسپم خورده‌اند. اگر از حاشیه سخت (Hard Margin) استفاده کنید، مدل نمی‌تواند همگرایی پیدا کند. توضیح دهید که چگونه پارامتر C در حاشیه نرم (Soft Margin) به شما اجازه می‌دهد از این نویزها عبور کنید. تغییرات مرز تصمیم را در دو حالت C بسیار بزرگ و C بسیار کوچک تحلیل کنید.

مسئله 15.12: مقایسه هزینه‌های محاسباتی چندکلاسه‌ای (Multiclass Computational Costs)

سیستم مدیریت اخبار شما باید مقالات را به 20 دسته موضوعی مختلف دسته‌بندی کند (مانند اقتصاد، سیاست، ورزش و...). محاسبه کنید که با استفاده از روش One-vs-One و One-vs-All به ترتیب به چند مدل SVM نیاز دارید؟ (برای $K = 20$) اگر سرعت در فاز «پیش‌بینی» (Inference) برای شما اولویت داشته باشد، کدام روش را انتخاب می‌کنید و چرا؟

مسئله 15.13: مشکل انفجار ابعاد در ان‌گرام‌ها (Dimension Explosion in N-grams)

برای بهبود دقت یک سیستم تحلیل احساسات (Sentiment Analysis)، تصمیم می‌گیرید علاوه بر کلمات تکی (Unigrams)، ترکیب‌های دوتایی و سه‌تایی (Bi-grams و Tri-grams) را نیز به عنوان ویژگی (Feature) استخراج کنید. تحلیل کنید که این کار چه تأثیری بر ماتریس $TF - IDF$ می‌گذارد و چرا الگوریتم SVM خطی همچنان می‌تواند بدون مشکل Overfitting با این ابعاد عظیم مقابله کند؟

مسئله 15.14: ضرورت انتخاب ویژگی (Necessity of Feature Selection)

در یک دیتاسیت پزشکی با 100,000 ویژگی (کلمه) و تنها 1,000 سند آموزشی روبرو هستید. با وجود اینکه SVM در فضاهای با ابعاد بالا مقاوم است، دو دلیل اصلی (یکی از نظر مهندسی نرم‌افزار و دیگری از نظر عملکرد مدل) بیاورید که چرا استفاده از تکنیک‌های انتخاب ویژگی (مانند Mutual Information) پیش از آموزش مدل در بخش 15.3.2 توصیه می‌شود؟

مسئله 15.15: تفکیک‌ناپذیری خطی احساسات (Linear Inseparability in Sentiment)

شما در حال طبقه‌بندی نظرات کاربران درباره یک محصول هستید. نقدهای کاملاً مثبت و کاملاً منفی ویژگی‌های مشابهی در طول متن دارند اما از کلمات کلیدی متضادی استفاده می‌کنند. با رسم یک فضای دو بعدی ذهنی، توضیح دهید که چرا یک خط مستقیم نمی‌تواند برخی الگوهای پیچیده متنی (مانند مسئله XOR) را جدا کند و ایده اولیه SVM غیرخطی (استفاده از Kernel Trick و انتقال به فضای با ابعاد بالاتر) چیست؟

مسئله 15.16: جادوی ترفند کرنل (The Magic of Kernel Trick)

مدیر پروژه از شما می‌خواهد که داده‌ها را به یک فضای با ابعاد بالاتر ببرید تا تفکیک‌پذیر شوند، اما نگران هزینه محاسباتی ضرب داخلی در فضای جدید است. مفهوم **Kernel Trick** (ترفند کرنل) را به زبان ساده برای او توضیح دهید. چگونه یک **Kernel** می‌تواند بدون نگاشت صریح داده‌ها به فضای جدید، ضرب داخلی آن‌ها را محاسبه کند؟

مسئله 15.17: طراحی ویژگی برای ماشین امتیازده (Feature Design for Machine Scoring)

شما قصد دارید یک سیستم **ML** برای جایگزینی **BM25** در موتور جستجو طراحی کنید (بخش 15.4.1). 5 ویژگی (**Feature**) که هم به سند، هم به کوئری و هم به رابطه این دو مرتبط باشند برای آموزش مدل خود پیشنهاد دهید و دلیل انتخاب هر کدام را بنویسید.

مسئله 15.18: انتخاب نوع طبقه‌بند در عمل (Choosing Classifier in Practice)

شرکتی دارای یک مجموعه داده متنی کوچک (500 سند) با کیفیت بالاست. بین استفاده از **Naive Bayes**، **KNN** و **Linear SVM** مردد هستند. با توجه به راهنمایی‌های بخش 15.3.1، کدام یک را پیشنهاد می‌دهید و چرا؟ اگر مجموعه داده به 500,000 سند افزایش یابد، انتخاب شما چه تغییری می‌کند؟

مسئله 15.19: چالش وزن‌دهی نامتقارن (Asymmetric Weighting Challenge)

در یک سیستم فیلتر محتوای نامناسب برای کودکان، عبور یک متن نامناسب (**False Negative**) بسیار خطرناک‌تر از مسدود شدن یک متن سالم (**False Positive**) است. چگونه می‌توان الگوریتم **SVM** را در فاز آموزش یا پیش‌بینی دستکاری کرد تا به جای ماکسیم کردن دقت کلی (**Accuracy**)، خطای **False Negative** را به حداقل برساند؟

مسئله 15.20: رگرسیون در برابر طبقه‌بندی در رتبه‌بندی (Regression vs Classification in Ranking)

در حال ساخت یک مدل **Machine-Learned Scoring** هستید. آیا بهتر است این مسئله را به عنوان یک مسئله **Classification** (ارتباط دارد/ندارد) فرمول‌بندی کنید یا یک مسئله **Regression** (تخمین یک عدد پیوسته به عنوان میزان ارتباط)؟ مزایا و معایب هر روش چیست؟

مسئله 15.21: تحلیل گزارش تجربی (Analyzing Experimental Report)

نتایج تجربی (بخش 15.2.4) نشان می‌دهد که استفاده از کرنل **RBF** در متون طولانی تفاوت معناداری با کرنل خطی (**Linear Kernel**) ایجاد نکرده اما زمان آموزش را 100 برابر کرده است. چرا در فضای با ابعاد بسیار بالا (مثل متون بلند)، معمولاً همان کرنل خطی کافی است و نیازی به مرزهای غیرخطی پیچیده نداریم؟

مسئله 15.22: مهندسی ویژگی‌های ساختاری (Structural Feature Engineering)

در رتبه‌بندی نتایج وب (**Web Search**)، صرفاً تطابق متن کافی نیست. توضیح دهید که چگونه یادگیری ماشین اجازه می‌دهد ویژگی‌های وابسته به گراف وب (مانند **PageRank** و **HITS**) را به راحتی با ویژگی‌های متنی (مانند **TF-IDF**) در یک مدل واحد ترکیب کنیم.

مسئله 15.23: محاسبات ریاضی ابرصفحه (Mathematical Hyperplane Calculation) ★

فرض کنید دو سند مثبت و دو سند منفی پس از کاهش ابعاد در مختصات دو بعدی قرار دارند. کلاس مثبت شامل بردارهای $X_1 = (2, 2)$ و $X_2 = (2, 0)$ و کلاس منفی شامل $X_3 = (0, 0)$ و $X_4 = (0, 2)$ است. الف) معادله خط جداکننده (ابرصفحه) بهینه $\vec{w} \cdot \vec{x} + b = 0$ را پیدا کنید. ب) بردارهای $\vec{w}$ و مقدار b را محاسبه کنید. اندازه حاشیه $\frac{2}{\|\vec{w}\|}$ چقدر است؟

مسئله 15.24: بهینه‌سازی فرمول‌بندی دوگان (Dual Optimization Formulation) ★

در بخش 15.2.1، تابع هدف حاشیه نرم به صورت مینی‌م کردن عبارت $\frac{1}{2}\|\vec{w}\|^2 + C \sum \xi_i$ معرفی می‌شود. نقش عملگرهای لاگرانژ (Lagrange Multipliers) در تبدیل این مسئله به فرمول‌بندی دوگان (Dual Problem) چیست؟ چرا فرم دوگان برای اعمال ترفند کرنل (Kernel Trick) ضروری است؟

مسئله 15.25: طراحی سیستم با توزیع به شدت نامتوازن (Extreme Imbalanced Data System) ★

یک سیستم تشخیص کلاهبرداری بانکی از طریق متن پیامک‌ها در حال طراحی است. از هر 1,000,000 پیامک، تنها 10 پیامک کلاهبرداری است. اگر این داده‌ها را مستقیماً به یک SVM خطی بدهید، مدل همه پیامک‌ها را "عادی" پیش‌بینی می‌کند و Accuracy برابر 99.99% خواهد داشت! با استفاده از تکنیک‌های بخش 15.3، سه راهکار فنی برای حل مشکل این مجموعه داده به شدت نامتوازن (Imbalanced Dataset) پیشنهاد دهید.

مسئله 15.26: رتبه‌بندی مبتنی بر یادگیری زوجی (Pairwise Learning to Rank) ★

در بخش 15.4.2، روش رتبه‌بندی با استفاده از ML فراتر از روش‌های نقطه‌ای (Pointwise) می‌رود. در رویکرد Pairwise (مانند RankSVM)، اگر بدانیم برای یک کوئری خاص کاربر سند A را به سند B ترجیح داده است (یعنی $A \succ B$)، ورودی جدیدی که به مدل SVM داده می‌شود از نظر ریاضی چگونه از بردارهای ویژگی X_A و X_B ساخته می‌شود و برچسب (Label) این نمونه جدید چیست؟

مسئله 15.27: اثبات ریاضیاتی کرنل چندجمله‌ای (Polynomial Kernel Proof) ★

فرض کنید بردار ورودی دو بعدی $\vec{x} = (x_1, x_2)$ داریم. نشان دهید که کرنل چندجمله‌ای درجه دوم به فرم $K(\vec{x}, \vec{z}) = (\vec{x} \cdot \vec{z} + c)^2$ دقیقاً معادل با ضرب داخلی در یک فضای ویژگی جدید $\Phi(\vec{x})$ است. مولفه‌های تابع نگاشت $\Phi(\vec{x})$ را صراحتاً به دست آورید.

مسئله 15.28: سناریوی کدنویسی: تنظیم فرآپارامترها (Coding: Hyperparameter Tuning) ★

شما باید با استفاده از پایتون و scikit-learn یک طبقه‌بند متن بسازید. کدی (شبه‌کد دقیق) بنویسید که: 1. متون را با TfidfVectorizer برداری کند. 2. با استفاده از GridSearchCV بر روی پارامتر C در مقادیر $10^2, 10^1, 1, 10^{-1}, 10^{-2}$ جستجو کرده و بهترین مدل LinearSVC را با معیار ارزیابی F1-Score پیدا کند. تحلیل کنید که چرا مقدار C بسیار بزرگ ممکن است باعث Overfitting روی داده‌های آموزشی متنی شود.

مسئله 15.29: تحلیل شکست روش نقطه‌ای در LTR (Failure of Pointwise in LTR) ★

موتور جستجوی فعلی شما از روش Pointwise SVM برای امتیازدهی اسناد استفاده می‌کند. متوجه می‌شوید که این سیستم برای کوئری‌های کوتاه (Short Queries) خوب کار می‌کند اما در کوئری‌های طولانی (Long Queries) امتیازات کاملاً به هم می‌ریزد و نتایج نامربوط در بالا قرار می‌گیرند.

با تکیه بر مفاهیم 15.4.2، دلیل بنیادی شکست رویکرد رگرسیون نقطه‌ای در قیاس لیست نتایج کوئری‌های مختلف چیست و رویکرد Listwise چگونه به حل آن کمک می‌کند؟

مسئله 15.30: کدنویسی خط‌لوله و انتخاب ویژگی (Coding Pipeline and Feature Selection) ★

طراحی یک Pipeline در محیط تجاری. شبه‌کدی بنویسید که ابتدا از بین 50,000 کلمه موجود در پیکره متنی، با استفاده از آزمون کای-دو (Chi-Square) تعداد 2000 ویژگی برتر را انتخاب کند (Feature Selection) و سپس خروجی آن را به یک SVC با کرنل RBF بدهد.

چرا در متون، ترکیب انتخاب ویژگی و کرنل غیرخطی به ندرت استفاده می‌شود و معمولاً SVM خطی بدون انتخاب ویژگی ترجیح داده می‌شود؟

مسئله 15.31: معماری یادگیری ماشین برای سیستم پرسش و پاسخ (QA System Architecture) ★

یک سیستم Q&A حقوقی نیاز دارید که کاربر یک سوال متنی می‌پرسد و سیستم باید مرتبط‌ترین پاراگراف را از میان میلیون‌ها سند پیدا کند.

طراحی یک معماری دو مرحله‌ای را بر اساس فصل 15 پیشنهاد دهید: مرحله اول (Recall Phase) با استفاده از روش‌های کلاسیک (مانند TF-IDF / BM25) برای استخراج 100 کاندیدای برتر، و مرحله دوم (Precision Phase) با استفاده از مدل Machine-Learned Scoring پیچیده (مانند Pairwise SVM) برای رتبه‌بندی مجدد (Re-ranking) این 100 سند. جریان داده و تولید ویژگی‌ها را در این معماری رسم/توصیف کنید.

مسئله 15.32: پروژه جامع رتبه‌بند ترکیبی جستجو (Comprehensive Hybrid Ranker Project) ★

به عنوان یک مهندس ارشد، موظف به طراحی سیستم رتبه‌بندی یک فروشگاه اینترنتی هستید. داده‌های شما شامل:

- الف) عنوان محصول
- ب) توضیحات محصول
- ج) تعداد فروش ماه گذشته
- د) امتیاز کاربران

فرمول کاملی طراحی کنید که نشان دهد چگونه امتیازات متنی کلاسیک (از طریق Cosine Similarity) با ویژگی‌های تجاری (فروش و امتیاز) به عنوان یک بردار ویژگی واحد ترکیب می‌شوند. تابع هزینه (Loss Function) مدل SVM خود را بنویسید تا مدلی تربیت کنید که محصولات با ارتباط متنی بالا و فروش زیاد را در صدر نتایج قرار دهد. وزن‌دهی به هر گروه از ویژگی‌ها پیش از آموزش مدل چه اهمیتی دارد؟

پاسخنامه مسئله 15.1: مفهوم حاشیه هندسی در ماشین بردار پشتیبان (SVM)

در مباحث یادگیری ماشین و طبقه‌بندی داده‌ها در بازیابی اطلاعات، الگوریتم «ماشین بردار پشتیبان» (**Support Vector Machine**) یکی از قدرتمندترین ابزارهای موجود برای دسته‌بندی داده‌های خطی تفکیک‌پذیر (**Linearly Separable**) است. هدف از این مسئله، درک عمیق مفهوم «حاشیه هندسی» (**Geometric Margin**) و منطق ریاضی و شهودی نهفته در پس‌زمینه‌سازی الگوریتم **SVM** است.

بخش 1: منظور از «حاشیه هندسی» چیست؟

برای درک حاشیه هندسی، ابتدا باید فرآیند تصمیم‌گیری در **SVM** را تصویرسازی کنیم. فرض کنید مجموعه‌ای از داده‌ها متعلق به دو کلاس متفاوت (مثلاً کلاس **+1** و کلاس **-1**) داریم که می‌توان آن‌ها را با یک خط مستقیم (یا در ابعاد بالاتر، با یک «اب‌صفحه» یا **Hyperplane**) از یکدیگر جدا کرد.

معادله این ابرصفحه جداکننده به صورت زیر تعریف می‌شود:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

که در آن $\mathbf{w}$ بردار وزن‌ها (عمود بر ابرصفحه) و b عرض از مبدأ (سوگیری یا **Bias**) است.

تعریف دقیق حاشیه هندسی:

«حاشیه هندسی» برای یک نقطه داده‌ی مشخص $(\mathbf{x}_i)$ ، دقیقاً برابر با فاصله فیزیکی و عمودی آن نقطه تا ابرصفحه جداکننده است. از نظر ریاضی، اگر نقطه‌ای در فضای برداری داشته باشیم، فاصله عمودی آن تا ابرصفحه از طریق رابطه زیر محاسبه می‌شود:

$$\gamma_i = y_i \left(\frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|} \right)$$

در این فرمول:

y_i : برچسب کلاس نقطه است (**+1** یا **-1**). این متغیر تضمین می‌کند که اگر نقطه در سمت درست ابرصفحه قرار داشته باشد، فاصله همیشه یک عدد مثبت خواهد بود.

$\|\mathbf{w}\|$: اندازه (نرم) بردار وزن است. تقسیم کردن معادله بر این مقدار، باعث می‌شود که فاصله به دست آمده مستقل از مقیاس‌بندی متغیرهای $\mathbf{w}$ و b باشد (به این معنا که اگر کل معادله را در یک عدد ثابت ضرب کنیم، فاصله فیزیکی تغییر نکند).

تحلیل تکمیلی (تفاوت حاشیه تابعی و هندسی):

بدون تقسیم بر $\|\mathbf{w}\|$ ، مقدار به دست آمده $\hat{\gamma}_i = y_i(\mathbf{w}^T \mathbf{x}_i + b)$ «حاشیه تابعی» (**Functional Margin**) نامیده می‌شود. مشکل حاشیه تابعی این است که اگر وزن‌ها را دو برابر کنیم، مقدار آن نیز دو برابر می‌شود، در حالی که خط جداکننده در فضا ثابت مانده است! به همین دلیل، تقسیم بر نرم بردار وزن الزامی است تا هندسه‌ی مسئله مقیاس‌ناپذیر (**Scale-Invariant**) شود.

حاشیه هندسی کل سیستم (Geometric Margin of the Dataset):

در الگوریتم **SVM**، منظور از حاشیه هندسی کل، کوچک‌ترین حاشیه‌های هندسی در میان تمام نقاط داده است. به عبارت ساده‌تر، فاصله نزدیک‌ترین نقطه (یا نقاط) از هر کلاس تا ابرصفحه را حاشیه سیستم می‌گویند. این نقاط نزدیک که حاشیه را تعیین می‌کنند، همان «بردارهای پشتیبان» (**Support Vectors**) نامیده می‌شوند.

بخش 2: چرا هدف اصلی SVM، حداکثرسازی این حاشیه است؟

وقتی داده‌ها به صورت خطی تفکیک‌پذیر باشند، بی‌نهایت خط (یا ابرصفحه) وجود دارد که می‌تواند دو کلاس را با موفقیت و بدون هیچ خطایی از هم جدا کند. اما کدام یک از این خطوط، بهترین انتخاب است؟

الگوریتم **SVM** با استفاده از ایده «حداکثرسازی حاشیه» (**Margin Maximization**) به این سوال پاسخ می‌دهد. دلایل اصلی و علمی این رویکرد به شرح زیر است:

1. بهبود قدرت تعمیم‌پذیری (Generalization) و جلوگیری از بیش‌برازش:

هدف اصلی هر مدل یادگیری ماشین، عملکرد خوب بر روی داده‌های جدید و دیده نشده است، نه صرفاً حفظ کردن داده‌های آموزشی. بر اساس تئوری یادگیری آماری وپنیک-چروننکیس (**VC Theory**)، ابرصفحه‌ای که بیشترین فاصله (حاشیه) را با نزدیک‌ترین داده‌های آموزشی داشته باشد، کمترین خطای تعمیم را روی داده‌های آینده خواهد داشت. حداکثر کردن حاشیه باعث می‌شود که مدل، مرز تصمیم‌گیری را در وسط فضای خالی بین دو کلاس قرار دهد و تمایل به هیچ‌یک از کلاس‌ها نداشته باشد.

2. ایجاد حریم امن (Safety Margin) در برابر نویز:

دنیای واقعی پر از نویز و خطاهای اندازه‌گیری است. اگر مرز تصمیم‌گیری (اب‌صفحه) بیش از حد به داده‌های یک کلاس نزدیک باشد، حتی یک تغییر جزئی در مختصات داده‌های جدید (نویز) ممکن است باعث شود آن داده از مرز عبور کرده و به اشتباه دسته‌بندی شود. با حداکثر کردن حاشیه هندسی، ما در واقع یک «گاردریل» یا حریم امن در دو طرف مرز تصمیم‌گیری ایجاد می‌کنیم تا سیستم در برابر نوسانات جزئی داده‌ها مقاوم (**Robust**) عمل کند.

3. دستیابی به یکتایی جواب بهینه (Uniqueness of Solution):

از نظر ریاضی، با طرح مسئله به عنوان یک مسئله بهینه‌سازی که در آن هدف ما یافتن ماکزیمم حاشیه هندسی با شرط دسته‌بندی صحیح تمام نقاط است، بی‌نهایت انتخاب ممکن برای ابرصفحه، به یک جواب واحد و بهینه سراسری (**Global Optimum**) کاهش می‌یابد.

بخش 3: مقایسه تحلیلی مرزهای تصمیم‌گیری

جهت ارزیابی دقیق تأثیر «اندازه حاشیه» بر رفتار مدل، تفاوت‌های بنیادین یک مدل با حاشیه کوچک در برابر یک مدل با حاشیه حداکثر در جدول زیر خلاصه شده است:

ویژگی مدل	اب‌صفحه با حاشیه کوچک (غیربهینه)	اب‌صفحه با حاشیه حداکثر (SVM ایده آل)
تأثیرپذیری از نویز	بسیار بالا (حساس به کوچکترین تغییرات)	مقاوم (Robust)
خطر بیش‌برازش (Overfitting)	احتمال بسیار زیاد، زیرا مدل صرفاً مرز را به یکی از دسته‌ها چسبانده است.	حداقل ممکن؛ مدل روی فضای خالی متعادل شده است.
اعتماد تصمیم‌گیری (Confidence)	پایین؛ پیش‌بینی نقاط نزدیک به مرز با ابهام همراه است.	بسیار بالا؛ فاصله قطعی و ایمن برای تفکیک دو کلاس وجود دارد.

برای اینکه این مفاهیم را همیشه به خاطر بسپارید، این مراحل ذهنی را طی کنید:

- مشکل اولیه:** بین دو دسته از داده‌ها، می‌توان هزاران خط کشید که آن‌ها را جدا کند.
- معیار انتخاب (حاشیه هندسی):** فاصله نزدیک‌ترین نقاط هر دو گروه تا خط رسم شده را اندازه می‌گیریم. به این فاصله واقعی و فیزیکی، حاشیه هندسی می‌گویند.
- راه حل (SVM):** خطی را انتخاب می‌کنیم که این فاصله را به بیشترین مقدار ممکن برساند (خط را دقیقاً در وسطِ پهن‌ترین جاده‌ی ممکن بین دو کلاس قرار می‌دهیم).
- دلیل (خروجی):** این کار باعث می‌شود مدل روی داده‌های امتحانی آینده، کمترین اشتباه را داشته باشد زیرا فضای خطای بیشتری (تولرانس) در اختیار دارد.

پاسخنامه مسئله 15.2: نقش بردارهای پشتیبان (Support Vectors)

مقدمه و هدف مسئله

در الگوریتم ماشین بردار پشتیبان (SVM)، مفهوم «بردارهای پشتیبان» هسته مرکزی و دلیل نام‌گذاری این الگوریتم است. هدف از این مسئله، شناخت دقیق این نقاط کلیدی در فضای برداری و درک میزان وابستگی مرز تصمیم (Decision Boundary) به تک‌تک داده‌های آموزشی است.

بخش 1: بردارهای پشتیبان دقیقاً به چه نقاطی گفته می‌شود؟

در یک مجموعه داده که توسط الگوریتم SVM دسته‌بندی شده است، بردارهای پشتیبان (Support Vectors) دقیقاً آن دسته از نقاطِ داده‌ی آموزشی هستند که در نزدیک‌ترین فاصله ممکن نسبت به ابرصفحه جداکننده (مرز تصمیم) قرار گرفته‌اند.

از منظر هندسی و ریاضی، این نقاط ویژگی‌های زیر را دارند:

1. **قرارگیری روی حاشیه‌ها:** اگر مرز تصمیم را یک خیابان در نظر بگیریم که دو کلاسِ داده را از هم جدا می‌کند،

بردارهای پشتیبان دقیقاً داده‌هایی هستند که روی لبه‌های این خیابان (جداول کنار خیابان) قرار گرفته‌اند.

2. **معادله ریاضی:** در فرمول‌بندی استاندارد SVM، مرز تصمیم با معادله $w^T x + b = 0$ تعریف می‌شود.

لبه‌های حاشیه دارای معادلات $w^T x + b = +1$ و $w^T x + b = -1$ هستند. بردارهای پشتیبان تنها نقاطی از

مجموعه داده (x_i) هستند که دقیقاً در این دو معادله‌ی لبه صدق می‌کنند.

به بیان ساده، این نقاط ساختار و موقعیت مرز تصمیم را «پشتیبانی» می‌کنند و به همین دلیل این نام برای آن‌ها انتخاب شده است.

بخش 2: اثر حذف سایر داده‌ها بر معادله خط تصمیم

پاسخ صریح:

اگر پس از آموزش اولیه مدل، تمامی داده‌های آموزشی (که فراتر از لبه‌های حاشیه قرار دارند) را حذف کنیم و تنها بردارهای

پشتیبان را نگه داریم، سپس مدل را مجدداً با این داده‌های محدود آموزش دهیم، معادله خط تصمیم (**Decision Boundary**) هیچ‌گونه تغییری نخواهد کرد.

بخش 3: تشریح منطق و چرایی عدم تغییر (اثبات ریاضی و شهودی)

برای درک اینکه چرا مرز تصمیم ثابت می‌ماند، باید به نحوه حل مسئله بهینه‌سازی در **SVM** و معادلات ریاضی آن نگاه کنیم:

1. ضرایب لاگرانژ و وزن‌ها:

در فرآیند بهینه‌سازی ریاضی الگوریتم **SVM** (حل مسئله دوگان یا **Dual Problem**)، بردار عمود بر مرز تصمیم (**w**) به صورت یک ترکیب خطی از داده‌های آموزشی به دست می‌آید:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

در این معادله:

$\mathbf{x}_i$: بردار ویژگی‌های نقطه **i**-ام

y_i : برچسب کلاس (**+1** یا **-1**)

α_i : ضریب لاگرانژ (**Lagrange Multiplier**) اختصاص‌یافته به نقطه **i**-ام است که میزان اهمیت آن نقطه را در ساخت مرز تصمیم نشان می‌دهد.

2. شرایط کاروش-کون-تاکر (KKT Conditions):

طبق اصول ریاضیات بهینه‌سازی (شرایط **KKT**)، الگوریتم **SVM** به‌گونه‌ای عمل می‌کند که برای تمام نقاطی که روی خط حاشیه نیستند (یعنی فاصله‌شان تا مرز تصمیم بیشتر از حریم تعیین‌شده است)، ضریب لاگرانژ دقیقاً برابر با صفر خواهد بود ($\alpha_i = 0$).

در مقابل، تنها برای بردارهای پشتیبان است که این ضریب مقداری غیرصفر و مثبت به خود می‌گیرد ($\alpha_i > 0$).

3. نتیجه‌گیری نهایی:

از آنجا که ضرایب α_i برای تمام نقاط غیر از بردارهای پشتیبان برابر با صفر است، وقتی آن‌ها را در معادله **w** جای‌گذاری می‌کنیم، حاصل ضرب آن‌ها صفر شده و عملاً هیچ تأثیری در محاسبه نهایی بردار **w** (و به تبع آن عرض از مبدأ **b**) ندارند.

بنابراین، حضور یا عدم حضور این داده‌های دورتر از مرز، در معادلات ریاضی کاملاً بی‌اثر است. مدل **SVM** در واقع تمام داده‌های غیرمرزی را پس از یادگیری «فراموش» می‌کند و تمام دانش مدل تنها در مختصات بردارهای پشتیبان ذخیره می‌شود.

بخش 4: مقایسه تحلیلی نقاط داده در SVM

جهت طبقه‌بندی و درک بهتر نقش داده‌ها در شکل‌گیری مرز تصمیم، تفاوت‌های کلیدی میان بردارهای پشتیبان و سایر داده‌های آموزشی در جدول زیر خلاصه شده است:

نوع داده آموزشی	موقعیت هندسی	ضریب لاگرانژ (α_i)	تأثیر در مرز تصمیم
بردارهای پشتیبان	دقیقاً روی خطوط مرزی حاشیه قرار دارند.	$\alpha_i > 0$	تعیین‌کننده و سازنده بردار وزن ($\mathbf{w}$) هستند.
سایر داده‌ها (غیرپشتیبان)	در فضای داخلی (امن) کلاس خود قرار دارند.	$\alpha_i = 0$	هیچ نقشی در شکل‌گیری خط مرزی ندارند.

جمع‌بندی

برای تثبیت این مفهوم، مدل **SVM** را به عنوان یک سیستم کاملاً «مرزگرا» در نظر بگیرید:

- بردارهای پشتیبان:** مرزنشینانی هستند که موقعیت دقیق مرز دو کشور (دو کلاس) را تعیین می‌کنند.
- سایر داده‌ها:** شهرهای داخلی کشورها هستند. جایگاه آن‌ها هرچه قدر هم تغییر کند (تا زمانی که از مرز عبور نکنند)، تأثیری در تعیین خطوط مرزی نخواهد داشت.
- کارایی حافظه (Memory Efficiency):** این ویژگی یکی از بزرگترین نقاط قوت **SVM** است؛ زیرا برای پیش‌بینی داده‌های جدید، مدل نیازی به نگهداری کل داده‌های آموزشی در حافظه ندارد و پردازش‌ها تنها بر اساس تعداد محدودی بردار پشتیبان انجام می‌شود که باعث افزایش چشمگیر سرعت فاز تست (**Testing Phase**) می‌گردد.

پاسخنامه مسئله 15.3: تفاوت حاشیه سخت و نرم (Hard Margin vs Soft Margin)

مقدمه و هدف مسئله

تا به اینجا فرض بر این بود که داده‌ها به صورت کامل و بدون هیچ‌گونه خطایی قابل تفکیک خطی هستند. اما در دنیای واقعی، داده‌ها رفتار ایده‌آلی ندارند. این مسئله با هدف بررسی چالش‌های داده‌های واقعی و معرفی رویکرد انعطاف‌پذیرتر **SVM**، یعنی «حاشیه نرم»، و ضرورت استفاده از آن در حوزه‌هایی مانند بازیابی اطلاعات و پردازش متن طراحی شده است.

بخش 1: تفاوت اساسی بین حاشیه سخت و حاشیه نرم

تفاوت بنیادی این دو رویکرد، در میزان انعطاف‌پذیری آن‌ها نسبت به «خطای دسته‌بندی» (**Misclassification**) و داده‌های پرت (**Outliers**) نهفته است. این تفاوت را از دو منظر مفهومی و ریاضی بررسی می‌کنیم:

1. حاشیه سخت (Hard Margin SVM):

مفهوم: این مدل با قوانین سخت‌گیرانه‌ای عمل می‌کند. در حاشیه سخت، الگوریتم به هیچ نقطه‌ای اجازه

نمی‌دهد که وارد حریم حاشیه (**Margin**) شود یا در سمت اشتباه مرز تصمیم‌گیری قرار گیرد.

شرط ریاضی: برای تمامی داده‌های آموزشی ($i = 1, 2, \dots, n$) شرط زیر باید به طور دقیق برقرار

باشد:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

چالش: اگر حتی یک داده نویزی و پرت در میان داده‌های کلاس مقابل قرار گیرد (یعنی داده‌ها از نظر خطی

تفکیک‌پذیر نباشند)، مسئله بهینه‌سازی هیچ جواب ریاضیاتی نخواهد داشت و الگوریتم با شکست مواجه

می‌شود.

2. حاشیه نرم (Soft Margin SVM):

مفهوم: برای حل مشکل فوق، حاشیه نرم مقداری انعطاف‌پذیری به مدل اضافه می‌کند. این رویکرد می‌پذیرد که برای رسیدن به یک مرز تصمیم‌گیری کلی و پایدار، ممکن است نیاز باشد تعداد کمی از نقاط، حریم حاشیه را نقض کنند یا حتی اشتباه دسته‌بندی شوند.

متغیرهای کمکی (Slack Variables): در مدل‌سازی ریاضی، متغیرهای جدیدی به نام متغیرهای کمکی یا لَقی ($\xi_i \geq 0$) معرفی می‌شوند. این متغیرها به هر نقطه داده اجازه می‌دهند تا به اندازه ξ_i از شرط حاشیه سخت تخطی کند.

(نکته تکمیلی: اگر $0 < \xi_i \leq 1$ باشد، نقطه وارد حریم حاشیه شده اما هنوز درست دسته‌بندی شده است. اگر $\xi_i > 1$ باشد، نقطه در سمت اشتباه مرز قرار گرفته و خطای دسته‌بندی رخ داده است).

شرط ریاضی جدید:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$$

تابع هدف جدید: در حاشیه نرم، تابع هدف بهینه‌سازی به شکل زیر تغییر می‌کند:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

در این فرمول، پارامتر C (یک پارامتر تنظیم‌کننده یا **Hyperparameter**) تعادل بین «حداکثر کردن عرض حاشیه» و «به حداقل رساندن خطای مجموع نقاط متخلف» را برقرار می‌کند.

بخش 2: چرا در کاربردهای واقعی، به‌ویژه در پردازش متن، حاشیه نرم الزامی است؟

استفاده از حاشیه نرم در دنیای واقعی یک انتخاب نیست، بلکه یک الزام محاسباتی و منطقی است. در کاربردهایی مانند پردازش زبان طبیعی (**NLP**) و طبقه‌بندی اسناد (**Document Classification**)، این ضرورت به دلایل زیر تشدید می‌شود:

1. عدم تفکیک‌پذیری مطلق (Non-Linear Separability):

در پردازش متن، کلمات مفاهیم مشترکی دارند. فرض کنید می‌خواهیم اخبار ورزشی را از اخبار حقوقی جدا کنیم. اگر یک خبر ورزشی شامل کلمه «دادگاه» (مثلاً دادگاه رسیدگی به تخلفات باشگاه) باشد، این سند از نظر ویژگی‌های برداری به کلاس اخبار حقوقی نزدیک می‌شود. داده‌های متنی به شدت هم‌پوشانی (**Overlap**) دارند و هیچ ابرصفحه‌ای نمی‌تواند آن‌ها را بدون خطا (حاشیه سخت) جدا کند.

2. مدیریت ماتریس‌های خلوت و ابعاد بالا (High Dimensionality & Sparsity):

در سیستم‌های بازیابی اطلاعات، اسناد معمولاً با مدل کیسه کلمات (**Bag of Words**) یا **TF-IDF** نمایش داده می‌شوند. این کار باعث ایجاد فضایی با ده‌ها هزار بُعد (به تعداد کلمات لغت‌نامه) می‌شود که اکثر مقادیر آن صفر است (ماتریس خلوت). در فضاهای با ابعاد بسیار بالا، خطر بیش‌برازش (**Overfitting**) به شدت افزایش می‌یابد.

حاشیه سخت در چنین فضایی سعی می‌کند خود را با تک‌تک کلمات نادر و نویزی تطبیق دهد، اما حاشیه نرم با چشم‌پوشی از این نویزها، الگوهای کلی‌تر و مقاوم‌تری را یاد می‌گیرد.

3. مصالحه بین بایاس و واریانس (Bias-Variance Tradeoff):

وجود پارامتر C در حاشیه نرم، ابزار کنترل‌کننده‌ای به دست طراح می‌دهد:

مقادیر بزرگ برای C : جریمه سنگین برای خطا (رفتار شبیه به حاشیه سخت، احتمال **Overfitting** بالا).

مقادیر کوچک برای C : تحمل خطای بیشتر برای رسیدن به حاشیه پهن‌تر (مدل تعمیم‌پذیرتر و مقاوم‌تر در برابر کلمات نویزی متون).

بخش 3: مقایسه تحلیلی حاشیه سخت و حاشیه نرم

جهت درک عمیق‌تر و ارزیابی تفاوت‌های این دو معماری، تقابل ویژگی‌های آن‌ها در جدول استاندارد زیر تدوین شده است:

معیار ارزیابی	حاشیه سخت (Hard Margin)	حاشیه نرم (Soft Margin)
فرضیه تفکیک‌پذیری	داده‌ها باید کاملاً و 100% به صورت خطی قابل جداسازی باشند.	می‌پذیرد که داده‌ها هم‌پوشانی دارند و تفکیک مطلق امکان‌پذیر نیست.
تحمل نویز (Noise)	صفر؛ حتی یک نویز باعث شکست کامل الگوریتم می‌شود.	بالا؛ نویزها را به عنوان تخلف می‌پذیرد و مهار می‌کند.
پارامتر کنترلی	فاقد پارامتر تنظیم‌کننده خطاست.	دارای پارامتر جریمه C برای تنظیم رفتار مدل است.
کاربرد عملی	صرفاً تئوری و آموزشی (روی دیتاست‌های ایده‌آل و ساخته‌شده).	صنعتی و واقعی (بسترهای متنی، پردازش تصویر، پزشکی).

جمع‌بندی

برای درک بهتر، این تفاوت را مانند مدیریت یک بزرگراه تصور کنید:

- حاشیه سخت:** یک خط‌کشی بتنی و غیرقابل عبور است. اگر یک ماشین (نقطه داده) به دلیل لغزندگی جاده (نویز) کمی از خط خارج شود، کل سیستم متوقف می‌شود (الگوریتم حل نمی‌شود).
- حاشیه نرم:** خط‌کشی با نوارهای هشداردهنده است. به ماشین‌ها اجازه می‌دهد در مواقع اضطراری کمی از خط خارج شوند (متغیر ξ_i)، اما برای این تخلف، جریمه‌ای (پارامتر C) در نظر می‌گیرد تا مجموع تخلفات در کل سیستم به حداقل برسد.
- در پردازش متن:** چون ذات زبان مبهم و پر از استثناست (ماشین‌ها دائماً از خط خارج می‌شوند)، بدون داشتن این خطوط منعطف (حاشیه نرم)، امکان ساخت هیچ مرز جداکننده‌ای وجود نخواهد داشت.

پاسخنامه مسئله 15.4: نقش متغیرهای کمکی در طبقه‌بندی (Slack Variables)

مقدمه و هدف مسئله

در ادامه مبحث حاشیه نرم (**Soft Margin**)، با پارامترهای جدیدی به نام «متغیرهای کمکی»، لقی یا ξ_i (**Slack Variables**) مواجه می‌شویم. هدف این مسئله، درک دقیق معنای هندسی و محاسباتی این متغیرها و تفسیر مقادیر مختلف آن‌ها در

تعیین موقعیت یک نقطه نسبت به مرز تصمیم‌گیری و حریم حاشیه است.

بخش 1: نقش متغیرهای کمکی (ξ_i) در فرمول‌بندی حاشیه نرم

در حالت ایده‌آل (رویکرد حاشیه سخت)، الگوریتم انتظار دارد تمام داده‌های آموزشی شرط $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ را به طور کامل رعایت کنند؛ یعنی هیچ نقطه‌ای نباید از لبه‌های حاشیه عبور کرده و وارد حریم آن شود. اما در محیط‌های واقعی (مانند طبقه‌بندی متون) که داده‌ها هم‌پوشانی دارند، ما متغیر کمکی $\xi_i \geq 0$ را به هر نمونه داده ($\mathbf{x}_i$) اختصاص می‌دهیم.

نقش‌های اساسی این متغیرها عبارتند از:

1. اندازه‌گیری میزان تخلف هندسی:

متغیر ξ_i به صورت کمی نشان می‌دهد که یک نقطه تا چه اندازه از قانون «حاشیه سخت» تخطی کرده است. به بیان هندسی، این متغیر فاصله قائم‌نقطه را از خط حاشیه‌ی کلاس خودش اندازه‌گیری می‌کند (نه فاصله تا ابرصفحه اصلی).

2. تأمین انعطاف‌پذیری محاسباتی (Error Tolerance):

با وارد کردن این متغیر در نامعادله شرطی به صورت $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ ، به الگوریتم اجازه می‌دهیم نقض قوانین حاشیه را بپذیرد تا مسئله بهینه‌سازی ریاضی، حتی برای داده‌های غیرقابل تفکیک خطی، همگرا شده و حل شود.

3. مدیریت هزینه و بهینه‌سازی:

در تابع هدف الگوریتم **SVM**، مجموع این متغیرها (یعنی $\sum_{i=1}^n \xi_i$) به عنوان «خطای کل تخلفات» در نظر گرفته می‌شود و هدف الگوریتم، کمینه کردن این خطا در کنار بیشینه کردن عرض حاشیه است.

بخش 2: تحلیل وضعیت نمونه‌ها بر اساس مقدار ξ_i

پاسخ صریح به سوال دوم مسئله، نیازمند بررسی موقعیت هندسی نقطه در فضای برداری است.

اگر برای یک نمونه داده مقدار $\xi_i = 0$ باشد:

این نقطه به عنوان یک نمونه کاملاً ایمن و بی‌نقص ارزیابی می‌شود. از نظر موقعیت هندسی، این وضعیت دارای دو ویژگی قطعی است:

1. دسته‌بندی کاملاً صحیح: نقطه در سمت درست ابرصفحه تصمیم‌گیری (Hyperplane) قرار دارد.

2. رعایت کامل حریم حاشیه: نقطه دقیقاً روی مرز حاشیه (لبه) یا در فاصله امنی بیرون از حریم حاشیه قرار گرفته است.

به بیان ساده، این نمونه هیچ‌گونه تجاوزی به منطقه ممنوعه (فاصله بین دو حاشیه) نداشته است.

(نکته تکمیلی: اگر این نقطه دقیقاً روی لبه حاشیه باشد، یک بردار پشتیبان محسوب می‌شود، اما اگر بیرون از آن باشد، یک نقطه معمولی است که در تعیین مرز نقشی ندارد).

بخش 3: تحلیل تکمیلی برای تسلط بر مسئله

برای اینکه به عنوان یک مهندس داده تصویر کاملی از رفتار مدل **SVM** داشته باشید، مقادیر ممکن برای متغیر ξ_i را باید به سه دسته طلایی تقسیم کنید که در جدول تحلیلی زیر به طور دقیق طبقه‌بندی شده‌اند:

مقدار متغیر (ξ_i)	نوع تخلف	موقعیت هندسی نسبت به مرز	وضعیت دسته‌بندی
$\xi_i = 0$	بدون تخلف (حریم کاملاً امن)	روی لبه حاشیه یا کاملاً خارج از منطقه ممنوعه است.	کاملاً صحیح
$0 < \xi_i \leq 1$	تخلف جزئی (Margin Violation)	حریم حاشیه را نقض کرده اما هنوز از ابرصفحه عبور نکرده است. (این نقاط همواره بردار پشتیبان هستند).	صحیح
$\xi_i > 1$	تخلف اساسی (Misclassification)	از ابرصفحه تصمیم‌گیری عبور کرده و وارد حریم کلاس مقابل شده است.	اشتباه (خطا)

جمع‌بندی

متغیر کمکی ξ_i را مانند «جریمه‌ی انحراف از مسیر» تصور کنید:

- بدون جریمه ($\xi_i = 0$):** شما دقیقاً در لاین مختص به خود و با فاصله قانونی از خط‌کشی‌ها حرکت می‌کنید.
- جریمه سبک ($0 < \xi_i \leq 1$):** همچنان در لاین خود هستید، اما روی خطوط ممتد یا حریم وسط جاده انحراف پیدا کرده‌اید.
- جریمه سنگین ($\xi_i > 1$):** به طور کامل وارد لاین مخالف شده‌اید و تصادف (خطای طبقه‌بندی) رخ داده است.

پاسخنامه مسئله 15.5: رویکرد یک-در-برابر-همه (One-vs-All) در طبقه‌بندی چندکلاسه‌ای

مقدمه و هدف مسئله

همان‌طور که در مسائل پیشین بررسی شد، الگوریتم ماشین بردار پشتیبان (**SVM**) در پایه و اساس ریاضی خود، یک الگوریتم «دوکلاسه‌ای» یا باینری (**Binary**) است؛ بدین معنا که تنها می‌تواند یک ابرصفحه رسم کند تا داده‌ها را به دو گروه (مثبت و منفی) تقسیم کند. اما در دنیای واقعی و سیستم‌های بازیابی اطلاعات، ما معمولاً با بیش از دو کلاس سروکار داریم (مانند طبقه‌بندی اخبار به **K** دسته ورزشی، سیاسی، اقتصادی و غیره). هدف از این مسئله، آموزش چگونگی تعمیم یک مدل باینری به یک طبقه‌بند چندکلاسه‌ای (**Multi-class**) با استفاده از استراتژی استاندارد «یک-در-برابر-همه» (**One-vs-All**) یا (**OVA**) است.

بخش 1: منطق و الگوریتم اجرای روش یک-در-برابر-همه (OVA)

ایده اصلی رویکرد **OVA** (که گاهی **One-vs-Rest** نیز نامیده می‌شود) بسیار ساده و در عین حال کارآمد است: تبدیل یک مسئله بزرگ چندکلاسه‌ای، به **K** مسئله کوچک دوکلاسه‌ای.

روند کار به صورت گام‌به‌گام در دو فاز اصلی انجام می‌پذیرد:

گام اول: فاز آموزش (Training Phase)

فرض کنید ما مجموعه‌ای از اسناد متنی داریم که باید به **K** کلاس مختلف طبقه‌بندی شوند.

- الگوریتم به جای آموزش یک مدل پیچیده، دقیقاً **K** مدل **SVM** مجزا و مستقل را آموزش می‌دهد.

2. برای آموزش مدلِ مربوط به کلاس **k**-اُم (f_k)، تمام اسنادی که متعلق به این کلاس هستند را با برچسب مثبت **(+1)** و تمامی اسنادِ متعلق به سایر کلاس‌ها (یعنی **K-1** کلاس باقی‌مانده) را به صورت یک‌جا با برچسب منفی **(-1)** نشانه‌گذاری می‌کنیم.

3. این مدل تلاش می‌کند تا بهترین ابرصفحه جداکننده را صرفاً برای تفکیک «کلاس **k**» از «هر چیز دیگری که کلاس **k** نیست» پیدا کند.

4. در پایان این فاز، ما **K** بردار وزن مجزا ($w_1, w_2, \dots, w_K$) و **K** عرض از مبدأ مجزا ($b_1, b_2, \dots, b_K$) خواهیم داشت.

گام دوم: فاز ارزیابی و قانون تصمیم‌گیری (Testing & Decision Rule)

حال فرض کنید یک سند متنی جدید (بردار x) وارد سیستم شده است و می‌خواهیم کلاس آن را پیش‌بینی کنیم:

- این سند جدید به تمامی **K** مدلِ آموزش‌دیده داده می‌شود.
- هر مدل، به جای یک پاسخ بله یا خیر ساده، یک «امتیاز خام» (**Raw Score**) یا فاصله هندسی بدون علامت را برای این سند محاسبه می‌کند. این امتیاز از طریق معادله خطی زیر به دست می‌آید:

$$f_k(x) = w_k^T x + b_k$$

هرچه این عدد مقدار مثبتِ بزرگ‌تری داشته باشد، یعنی مدل **k**-اُم با اطمینان بیشتری معتقد است که این سند متعلق به کلاس اوست.

3. قانون تصمیم‌گیری (Argmax): در نهایت، کلاسی به عنوان برنده نهایی انتخاب می‌شود که مدلِ مربوط به آن، بیشترین امتیاز مثبت را برای سند جدید تولید کرده باشد. فرمول ریاضی این تصمیم‌گیری به شکل زیر است:

$$\hat{y} = \arg \max_{k \in \{1, 2, \dots, K\}} (w_k^T x + b_k)$$

بخش 2: چالش‌ها، ملاحظات و مزایای محاسباتی در پردازش متن

رویکرد **OVA** به دلیل سادگی و کارایی محاسباتی، به ویژه در فضاهای با ابعاد بالا مانند متن، بسیار محبوب است. مزیت مهم این روش قابلیت پردازش موازی (**Parallel Processing**) آن است؛ زیرا آموزشِ هر یک از **K** مدل کاملاً مستقل از دیگری انجام می‌شود. با این وجود، یک چالش ذاتی در این روش وجود دارد که باید به آن آگاه باشید:

عدم تعادل کلاس‌ها (Class Imbalance):

زمانی که یک مدل **SVM** را برای جدا کردن کلاس **k** از سایر کلاس‌ها آموزش می‌دهیم، معمولاً تعداد نمونه‌های منفی (مجموع **K-1** کلاس دیگر) بسیار بیشتر از نمونه‌های مثبت (فقط کلاس **k**) است. این عدم تعادل می‌تواند باعث شود مرز تصمیم‌گیری به سمت کلاس اقلیت (مثبت) متمایل شود. برای رفع این مشکل در کاربردهای عملی، معمولاً از پارامترهای جریمه وزن‌دار (**Weighted C Parameters**) در فرمول حاشیه نرم استفاده می‌شود تا اهمیت اشتباهات در کلاس مثبت، بیشتر از کلاس منفی در نظر گرفته شود.

بخش 3: معماری برچسب‌گذاری در استراتژی OVA

برای تجسم بهتر فاز آموزش، فرض کنید یک دیتاست دارای 4 کلاس مجزا ($K = 4$) است. جدول زیر نشان می‌دهد که هر یک از 4 مدل ساخته‌شده، چگونه نمونه‌های آموزشی را برچسب‌گذاری و تفسیر می‌کند:

مدل مستقل (SVM)	کلاس هدف (برچسب +1)	سایر کلاس‌ها (برچسب -1)	تابع خروجی
مدل 1	Class 1	Class 2, 3, 4	$f_1(\mathbf{x})$
مدل 2	Class 2	Class 1, 3, 4	$f_2(\mathbf{x})$
مدل 3	Class 3	Class 1, 2, 4	$f_3(\mathbf{x})$
مدل 4	Class 4	Class 1, 2, 3	$f_4(\mathbf{x})$

جمع‌بندی

برای تثبیت الگوریتم در ذهن، سیستم **OVA** را مانند یک پنل از **K** متخصص ارزیاب در نظر بگیرید:

- آموزش:** ما **K** متخصص تربیت می‌کنیم. تخصص فرد اول فقط تشخیص «ورزشی بودن یا نبودن» است، فرد دوم «سیاسی بودن یا نبودن» و الی آخر.
- امتیازدهی:** وقتی خبر جدیدی می‌رسد، همه **K** متخصص خبر را می‌خوانند و هر کدام یک نمره اعتماد (**Confidence Score**) بین $-\infty$ تا $+\infty$ روی برگه می‌نویسند.
- تصمیم‌گیری نهایی:** ما برگه‌ها را جمع‌آوری کرده و صرفاً نمرات را مقایسه می‌کنیم. تخصص فردی که بالاترین نمره مثبت را داده باشد، به عنوان برچسب نهایی خبر در نظر گرفته می‌شود.

پاسخنامه مسئله 15.6: رویکرد یک-در-برابر-یک (One-vs-One) در طبقه‌بندی چندکلاسه‌ای

مقدمه و هدف مسئله

پس از آشنایی با روش «یک-در-برابر-همه» (**OVA**)، اکنون به بررسی استراتژی جایگزین آن، یعنی روش «یک-در-برابر-یک» (**One-vs-One**) یا **OVO** می‌پردازیم. هدف از این مسئله، درک ساختار شبکه‌ای این روش، محاسبه دقیق تعداد مدل‌های مورد نیاز و تحلیل ریاضیاتی برتری‌های آن در مرحله یادگیری (فاز آموزش) نسبت به روش **OVA** است.

بخش 1: تعداد طبقه‌بندهای مستقل در رویکرد OVO

در رویکرد «یک-در-برابر-یک»، برخلاف روش **OVA** که هر کلاس را با تمام کلاس‌های دیگر مقایسه می‌کرد، ما برای هر جفت ممکن از کلاس‌ها، یک مدل مستقل **SVM** آموزش می‌دهیم.

بنابراین، اگر مجموعاً **K** کلاس مختلف داشته باشیم، مسئله پیدا کردن تعداد راه‌های انتخاب 2 کلاس از میان **K** کلاس است. از نظر ریاضی، این مقدار برابر با ترکیب 2 از **K** یا $\binom{K}{2}$ است. فرمول دقیق تعداد طبقه‌بندهای مستقل به صورت زیر محاسبه می‌شود:

$$\text{Number of Classifiers} = \frac{K(K-1)}{2}$$

نحوه عملکرد و تصمیم‌گیری:

در فاز ارزیابی، وقتی یک سند جدید وارد سیستم می‌شود، به تمامی $\frac{K(K-1)}{2}$ مدل ارجاع داده می‌شود. هر مدل بین دو کلاس مختص به خود رأی‌گیری می‌کند. در نهایت، سیستمی شبیه به رأی‌گیری اکثریتی (**Majority Voting**) اعمال شده و کلاسی که بیشترین تعداد پیروزی را در این نبردهای دو به دو (**Duels**) کسب کرده باشد، به عنوان برچسب نهایی سند انتخاب می‌شود.

بخش 2: مزایای OVO نسبت به روش OVA

در نگاه اول، آموزش دادن $\frac{K(K-1)}{2}$ مدل در رویکرد **OVO** (که برای کلاس‌های زیاد، عدد بسیار بزرگی می‌شود) نسبت به آموزش **K** مدل در روش **OVA**، غیرمنطقی و زمان‌بر به نظر می‌رسد؛ اما از نظر ریاضی و محاسباتی، **OVO** در فاز آموزش دارای دو مزیت بنیادین و حیاتی است:

1. کاهش شدید پیچیدگی زمانی در آموزش (Training Time Complexity):

در روش **OVA**: هر یک از **K** مدل باید روی تمامی داده‌های آموزشی (مجموعاً **N** داده) آموزش داده شود. الگوریتم **SVM** در حالت پایه دارای پیچیدگی زمانی غیرخطی، تقریباً از مرتبه $O(N^2)$ تا $O(N^3)$ است. آموزش دادن مدلی با چنین سایز بزرگی از داده‌ها، نیازمند زمان محاسباتی و حافظه عظیمی است.

در روش **OVO**: هر مدل مستقل، تنها روی داده‌های متعلق به همان 2 کلاس خاص آموزش داده می‌شود. اگر توزیع داده‌ها یکنواخت باشد، سایز داده‌های ورودی برای هر مدل **OVO** تنها بخش کوچکی از کل داده‌ها (حدود $\frac{2N}{K}$ نمونه) خواهد بود.

نتیجه: در الگوریتم‌هایی مانند **SVM** که نسبت به حجم داده مقیاس‌پذیری غیرخطی دارند، حل کردن تعداد زیادی مسئله کوچک (**OVO**)، به مراتب سریع‌تر از حل کردن تعداد کمی مسئله بسیار بزرگ (**OVA**) است.

2. غلبه بر مشکل عدم تعادل کلاس‌ها (Class Imbalance Problem):

همان‌طور که پیش‌تر اشاره شد، روش **OVA** در ذات خود یک عدم تعادل شدید ایجاد می‌کند؛ زیرا کلاس هدف در برابر مجموعه عظیمی از سایر کلاس‌ها قرار می‌گیرد (نسبت 1 به **K-1**). این امر باعث کشیده شدن مرز تصمیم‌گیری به سمت کلاس اقلیت می‌شود.

اما در روش **OVO**، چون در هر مرحله فقط 2 کلاس با یکدیگر مقایسه می‌شوند (مثلاً فقط اخبار ورزشی در برابر اخبار سیاسی)، اگر توزیع کلاس‌های اولیه در دیتاست تقریباً برابر باشد، هیچ مشکل عدم تعادلی در فاز آموزش به وجود نخواهد آمد و مرزهای تصمیم‌گیری بسیار دقیق‌تر و بدون سوگیری (**Bias**) رسم می‌شوند.

بخش 3: اثبات ریاضیاتی برتری زمانی OVO

برای اثبات دقیق منطق فوق، فرض کنید پیچیدگی زمانی آموزش **SVM** از مرتبه مکعبی یعنی $O(N^3)$ باشد. اگر داده‌ها به طور مساوی بین **K** کلاس تقسیم شده باشند (هر کلاس $\frac{N}{K}$ نمونه):

زمان کل آموزش **OVA**: شامل **K** مدل است که هر کدام روی کل **N** داده آموزش می‌بینند.

$$T_{OVA} = K \times \mathcal{O}(N^3)$$

زمان کل آموزش **OVO**: شامل $\frac{K(K-1)}{2}$ مدل است که هر کدام صرفاً روی $\frac{2N}{K}$ داده آموزش می‌بینند.

$$T_{OVO} \approx \frac{K^2}{2} \times \mathcal{O}\left(\left(\frac{2N}{K}\right)^3\right) = \frac{K^2}{2} \times \frac{8N^3}{K^3} = \frac{4}{K} \times \mathcal{O}(N^3)$$

همان‌طور که مشاهده می‌شود، زمان آموزش **OVO** با ضریب $\frac{4}{K}$ رشد می‌کند، در حالی که **OVA** با ضریب **K**. این اثبات به وضوح نشان می‌دهد که در مسائل دارای تعداد کلاس بالا ($K > 4$)، روش **OVO** علیرغم نیاز به ساختن مدل‌های بیشتر، از نظر زمانی بسیار سریع‌تر و کارآمدتر از **OVA** بهینه‌سازی می‌شود.

بخش 4: مقایسه جامع استراتژی‌های چندکلاسه‌ای

جهت ارزیابی و انتخاب سریع استراتژی مناسب برای پروژه‌های طبقه‌بندی، مقایسه این دو روش در جدول زیر مدون شده است:

شاخص ارزیابی	یک-در-برابر-همه (OVA)	یک-در-برابر-یک (OVO)
تعداد طبقه‌بندها	K	$K(K-1)/2$
حجم داده هر مدل	تمام مجموعه داده‌ها (N)	فقط داده‌های دو کلاس هدف ($\sim 2N/K$)
مشکل عدم تعادل	شدید (1 در برابر K-1)	حداقل (1 در برابر 1)
قانون تصمیم‌گیری	تابع Argmax (بالاترین امتیاز خام)	رأی‌گیری اکثریتی (بیشترین برد دوئل)

جمع‌بندی

برای درک تفاوت استراتژیک این دو روش، مسابقات ورزشی را تصور کنید:

- روش OVA (مسابقه رکوردی)**: یک تیم به تنهایی باید رکورد تمام تیم‌های دیگر روی هم رفته را بشکند (مقایسه یک نفر با کل جهان). فشار روی آن تیم بسیار زیاد است و تعداد داده‌ها نابرابر است.
- روش OVO (لیگ دوره‌ای یا Round-Robin)**: تیم‌ها به صورت 2 به 2 (تک به تک) با هم مسابقه می‌دهند. تعداد بازی‌ها زیاد است (فرمول $\frac{K(K-1)}{2}$)، اما هر بازی در زمین مسطح و عادلانه (تعداد داده مساوی) و در زمان بسیار کوتاه‌تری برگزار می‌شود. در نهایت، تیمی که بیشترین برد (رأی) را آورده قهرمان می‌شود.

پاسخنامه مسئله 15.7: نمایش فضای برداری خلوت (Sparse Vector Space)

مقدمه و هدف مسئله

در پردازش متن و بازیابی اطلاعات، بازنمایی اسناد به صورت بردارهای عددی (مانند مدل **Bag-of-Words** یا **TF-IDF**) پایه و اساس محاسبات است. در این مدل‌ها، تعداد ویژگی‌ها (ابعاد بردار) برابر با تعداد کلمات موجود در کل واژگان (**Vocabulary**) است. هدف این مسئله، درک پدیده «خلوت بودن» (**Sparsity**) در این بردارها و تحلیل چرایی این ویژگی در بهینه‌سازی مدل‌های خطی مانند **SVM** است.

بخش 1: ماتریس خلوت (Sparse Matrix) چیست؟

یک بردار یا ماتریس «خلوت»، ساختاری است که اکثریت قریب به اتفاقِ درایه‌های آن صفر هستند.

در مدل **Bag-of-Words (BoW)**:

1. ما یک بردار به طول V (اندازه کل واژگان) داریم که هر خانه آن نشان‌دهنده فراوانی یا وزن یک کلمه خاص است.

2. یک سند معمولی معمولاً شامل تعداد انگشت‌شماری کلمه است (مثلاً 100 تا 500 کلمه)، در حالی که لغت‌نامه زبان (و به تبع آن V) ممکن است شامل 50,000 تا 100,000 کلمه باشد.

3. در نتیجه، در بردار مربوط به این سند، بیش از 99 درصد خانه‌ها دارای مقدار **صفر** خواهند بود؛ زیرا اکثر کلمات موجود در لغت‌نامه، در این سند خاص حضور ندارند. به چنین بازنمایی که اکثر مؤلفه‌های آن صفر است، **بردار خلوت** می‌گوییم.

محاسبه نرخ خلوت بودن (Sparsity Ratio):

از منظر ریاضی، ضریب خلوتی یک بردار از تقسیم تعداد عناصر صفر بر کل عناصر بردار به دست می‌آید. در پردازش متن، این ضریب معمولاً عددی نزدیک به 0.999 است که نشان‌دهنده پتانسیل عظیم برای فشرده‌سازی و بهینه‌سازی محاسبات می‌باشد.

بخش 2: چرا خلوت بودن برای مدل‌های خطی یک مزیت است؟

شاید در نگاه اول به نظر برسد که حجم زیاد ابعاد (**Dimensionality**) یک نقطه ضعف است، اما برای مدل‌های خطی (مانند **SVM** یا رگرسیون لجستیک)، خلوت بودن مزایای استراتژیک زیر را به همراه دارد:

1. تفکیک‌پذیری خطی ساده‌تر (Linear Separability):

طبق «قضیه پوشش» (**Cover's Theorem**)، وقتی داده‌ها را به فضایی با ابعاد بسیار بالا نگاشت می‌کنیم، احتمال اینکه داده‌ها به صورت خطی تفکیک‌پذیر شوند، افزایش می‌یابد. در فضای با ابعاد بالا، نقاط (اسناد) از هم دورتر می‌شوند. خلوت بودن کمک می‌کند که هر سند در یک «زیرفضای» منحصربه‌فرد تعریف شود و این کار، یافتن ابرصفحه جداکننده برای **SVM** را بسیار ساده‌تر می‌کند.

2. کاهش هزینه‌های محاسباتی (Computational Efficiency):

مدل‌های خطی بر پایه ضرب داخلی (**Dot Product**) بردار وزن‌ها در بردار ویژگی‌ها ($\mathbf{w}^T \mathbf{x}$) کار می‌کنند.

$$\mathbf{w}^T \mathbf{x} = \sum_{i=1}^V w_i x_i$$

وقتی بردار سند ($\mathbf{x}$) خلوت باشد، بسیاری از x_i ها صفر هستند. در محاسبات عددی، ضرب هر وزنی در صفر برابر با صفر است. الگوریتم‌های بهینه‌سازی که با ماتریس‌های خلوت کار می‌کنند، عملاً از این صفرها صرف‌نظر کرده و فقط روی مؤلفه‌های غیرصفر تمرکز می‌کنند. این کار باعث کاهش چشمگیر تعداد عملیات ریاضی و افزایش سرعت اجرا می‌شود.

3. جلوگیری از «نفرین ابعاد» (Curse of Dimensionality):

به طور معمول، افزایش ابعاد باعث ایجاد مشکلاتی در یادگیری می‌شود، اما در فضای متنی، به دلیل اینکه داده‌ها «خلوت» هستند، حجم واقعی اطلاعاتی که مدل باید یاد بگیرد، بسیار کمتر از تعداد کل ابعاد است. این خلوت بودن اجازه می‌دهد تا مدل بدون نیاز به هزاران نمونه داده برای هر بُعد، ساختار اصلی داده‌ها را کشف کند و از پیچیدگی غیرضروری در امان بماند.

بخش 3: مقایسه تحلیلی ساختارهای ذخیره‌سازی و محاسباتی

برای درک بهتر برتری محاسباتی، مقایسه‌ای بین رفتار سیستم در حالت برخورد چگال (**Dense**) و خلوت (**Sparse**) با یک بردار 100,000 بُعدی که تنها 100 کلمه دارد، در جدول زیر آورده شده است. الگوریتم‌های مدرن از فرمت‌هایی نظیر **CSR** (**Compressed Sparse Row**) استفاده می‌کنند که فقط مقادیر غیرصفر را نگه می‌دارد.

شاخص ارزیابی	نگاه غیربهینه چگال (Dense Array)	نگاه بهینه خلوت (Sparse/CSR)
حافظه مصرفی	ذخیره 100,000 عدد برای هر سند (حجم بالا و اتلاف مموری)	ذخیره صرفاً 100 جفت (اندیس، مقدار)
تعداد ضرب‌های ریاضی	100,000 Multiplications	100 Multiplications
سرعت پردازش	بسیار کند؛ درگیر ضرب کردن وزن‌ها در صفرهای بی‌اثر	بسیار سریع؛ پرش مستقیم روی کلمات کلیدی موجود

جمع‌بندی

برای درک بهتر، سیستم را مانند یک کتابخانه عظیم دیجیتال تصور کنید:

- بدون خلوتی:** اگر قرار بود برای هر کتاب، مشخصات تمام کتاب‌های موجود در جهان را ثبت کنیم، باید هر سند ما میلیون‌ها فیلد پرشده می‌داشت که باعث سردرگمی و کندی شدید می‌شد.
- خلوتی:** هر سند فقط برچسب کلمات خودش را دارد و سایر فیلدها «خالی» (صفر) هستند. این کار مثل داشتن یک فهرست دقیق و بهینه است که فقط روی کلمات کلیدی موجود در متن تمرکز دارد.
- قدرت مدل خطی:** مدل خطی در این فضای خلوت، مثل یک «فیلتر» عمل می‌کند که تنها روی کلمات کلیدی غیرصفر متمرکز شده و با سرعت بسیار بالا، اسناد را طبقه‌بندی می‌کند، بدون اینکه نگران انبوه صفرهای بی‌اثر باشد.

پاسخنامه مسئله 15.8: تناسب الگوریتم با داده متنی (Algorithm Suitability for Text)

مقدمه و هدف مسئله

در درس بازیابی اطلاعات و یادگیری ماشین، انتخاب الگوریتم مناسب به اندازه استخراج ویژگی‌ها اهمیت دارد. دو الگوریتم «ماشین بردار پشتیبان» (**SVM**) و «درخت تصمیم» (**Decision Tree**) رویکردهای کاملاً متفاوتی در مدل‌سازی داده‌ها دارند. هدف این مسئله، کالبدشکافی ساختار هندسی این دو الگوریتم و اثبات این موضوع است که چرا رویکرد **SVM** تطابق بسیار بالاتری با ذات و ویژگی‌های داده‌های متنی دارد.

بخش 1: ماهیت ساختاری و هندسی داده‌های متنی

پیش از مقایسه الگوریتم‌ها، باید بدانیم داده متنی (با بازنمایی **TF-IDF** یا **Bag-of-Words**) در فضای برداری چه شکل هندسی و آماری دارد. این داده‌ها دارای سه ویژگی بنیادین هستند:

1. ابعاد بسیار بالا (High Dimensionality): فضای ویژگی‌ها به اندازه تعداد کلمات لغت‌نامه است (اغلب ده‌ها هزار بُعد).

2. خلوت بودن شدید (Extreme Sparsity): در هر سند، تنها کسر کوچکی از کلمات لغت‌نامه وجود دارد، در نتیجه بیش از 99 درصد مقادیر بردار هر سند، صفر است.

3. تأثیر ترکیبی ویژگی‌ها (Correlated Features): در متون، مفاهیم معمولاً به یک کلمه خاص محدود نمی‌شوند، بلکه ترکیبی خطی از ده‌ها کلمه مرتبط، موضوع یک متن را مشخص می‌کنند.

بخش 2: رویکرد هندسی درخت تصمیم (Decision Tree) و دلایل ضعف آن در متن

الگوریتم درخت تصمیم برای ساخت مرزهای طبقه‌بندی، در هر گره (**Node**) تنها یک ویژگی (یک کلمه) را انتخاب کرده و داده‌ها را بر اساس حضور یا عدم حضور (یا وزن) آن کلمه به دو شاخه تقسیم می‌کند. این رویکرد هندسی در فضای متنی منجر به سه فاجعه‌ی ساختاری می‌شود:

1. مرزهای موازی با محورها (Axis-Parallel Boundaries):

از نظر هندسی، درخت تصمیم فضا را با خطوط (یا صفحات) عمود بر محورهای مختصات برش می‌دهد. در فضای متنی با ده‌ها هزار بُعد، مفاهیم به صورت قطری و ترکیبی در فضا توزیع شده‌اند. تقریب زدن یک مرز مورب بهینه با استفاده از برش‌های عمود بر محورها، منجر به ایجاد یک مرز تصمیم بسیار پیچیده، پلکانی و عمیق می‌شود.

2. تکه‌تکه شدن سریع داده‌ها (Data Fragmentation):

با هر برش (پرسش در مورد یک کلمه)، مجموعه داده‌های آموزشی به دو قسمت تقسیم می‌شود. در متون، چون حضور یک کلمه خاص در اسناد پراکنده است، پس از چند سطح پایین رفتن در درخت، تعداد داده‌های موجود در گره‌ها به شدت کاهش می‌یابد و مدل پیش از آنکه بتواند الگوی کلی را یاد بگیرد، با کمبود داده مواجه می‌شود (به اصطلاح گره‌ها از نظر اطلاعاتی فقیر می‌شوند).

3. مشکل بیش‌برازش (Overfitting):

برای اینکه درخت تصمیم بتواند دقت خوبی روی داده‌های متنی آموزشی به دست آورد، مجبور است بسیار عمیق شود تا کلمات زیادی را بررسی کند. یک درخت عمیق در یک فضای خلوت، شروع به حفظ کردن نویزها و کلمات بی‌اهمیت می‌کند که به شدت قدرت تعمیم‌پذیری (**Generalization**) آن را کاهش می‌دهد.

بخش 3: رویکرد هندسی SVM و دلایل برتری آن در متن

در نقطه مقابل، مدل خطی **SVM** دقیقاً متناسب با ویژگی‌های فضای متنی عمل می‌کند:

1. ترکیب همزمان ویژگی‌ها (Linear Combination):

از نظر هندسی، **SVM** یک ابرصفحه (**Hyperplane**) مورب را در فضا رسم می‌کند که معادله آن به صورت زیر است:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

این یعنی الگوریتم به جای تصمیم‌گیری بر اساس تک‌تک کلمات (مانند درخت)، وزن و تأثیر تمام ده‌ها هزار کلمه را به صورت همزمان (**Simultaneously**) در نظر می‌گیرد. در متن، مجموع وزن کلمات مرتبط است که کلاس سند را تعیین می‌کند، نه فقط حضور یک کلمه خاص.

2. مقاومت ذاتی در برابر ابعاد بالا:

همان‌طور که در تئوری یادگیری آماری (توسط محققانی مانند **Joachims** در سال 1998) اثبات شده است، توانایی **SVM** در جلوگیری از بیش‌برازش، به جای وابسته بودن به تعداد ابعاد ویژگی‌ها (تعداد کلمات)، به حداکثرسازی حاشیه هندسی (**Margin Maximization**) وابسته است. از آنجا که داده‌های متنی به دلیل ابعاد بالا و خلوت بودن، عمدتاً به صورت خطی تفکیک‌پذیر هستند، **SVM** می‌تواند با پیدا کردن حاشیه بهینه، مدلی بسیار قدرتمند و بدون **Overfitting** بسازد.

3. بهره‌برداری از صفرهای ماتریس:

معادله بردار وزن در **SVM** با ماتریس‌های خلوت بسیار سازگار است:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

الگوریتم از صفرهای موجود در بردار متون عبور کرده و محاسبات خود را تنها معطوف به کلمات موجود در اسناد (نقاط غیرصفر) می‌کند که این امر سرعت آموزش و آزمایش را به شدت بالا می‌برد.

بخش 4: مقایسه تحلیلی عملکرد در فضای متنی

جدول زیر به شکل خلاصه تقابل عملکرد این دو الگوریتم در مواجهه با چالش‌های اساسی داده‌های متنی را نشان می‌دهد:

چالش داده‌های متنی	درخت تصمیم (Decision Tree)	ماشین بردار پشتیبان (SVM)
ابعاد بسیار بالا	ضعیف (نیاز به درخت بسیار عمیق)	بسیار عالی (مقاوم در برابر ابعاد)
تأثیر ترکیبی ویژگی‌ها	بررسی تک‌به‌تک (مرزهای پلکانی)	ترکیب خطی همزمان (مرزهای مورب)
مدیریت پراکندگی داده	تکه‌تکه شدن سریع (Data Fragmentation)	استفاده از تمام داده‌ها برای رسم ابرصفحه

جمع‌بندی

برای درک عمیق‌تر تفاوت هندسی، این تمثیل را در نظر بگیرید:

1. **درخت تصمیم** مانند یک بازپرس است که در هر لحظه فقط اجازه دارد درباره **یک کلمه** سوال بپرسد (مثلاً می‌پرسد: "آیا کلمه 'توپ' در متن هست؟"). او مجبور است صدها سوال متوالی بپرسد تا به نتیجه برسد، که در نهایت باعث سردرگمی و حفظ کردن جواب‌های خاص می‌شود.
2. **ماشین بردار پشتیبان (SVM)** مانند یک هیئت منصفه است که **کل متن را به صورت یک‌جا** می‌خواند، به هر کلمه یک وزن (مثبت یا منفی) اختصاص می‌دهد و بر اساس «برآیند نهایی تمام وزن‌ها»، یک تصمیم‌گیری کلی، قطعی و مقاوم اتخاذ می‌کند.

پاسخنامه مسئله 15.9: ارزیابی نقطه‌ای (Pointwise Scoring) در یادگیری برای رتبه‌بندی

مقدمه و هدف مسئله

در سیستم‌های نوین بازیابی اطلاعات (مانند موتورهای جستجو)، برای مرتب‌سازی اسناد از تکنیک‌های یادگیری ماشین استفاده می‌شود که به این حوزه «یادگیری برای رتبه‌بندی» (**Learning to Rank**) یا (**LTR**) می‌گویند. در **LTR**، سه رویکرد اصلی وجود دارد: نقطه‌ای (**Pointwise**)، زوجی (**Pairwise**) و لیستی (**Listwise**). هدف از این مسئله، کالبدشکافی ساده‌ترین و پایه‌ای‌ترین این رویکردها، یعنی روش **ارزیابی نقطه‌ای**، در بستر بازیابی اطلاعات موقت (**Ad hoc IR**) است.

بخش 1: روش ارزیابی نقطه‌ای (Pointwise) چگونه عمل می‌کند؟

در رویکرد نقطه‌ای، مسئله پیچیده‌ی «رتبه‌بندی یک لیست از اسناد»، به یک مسئله‌ی کلاسیک در یادگیری ماشین (یعنی دسته‌بندی یا رگرسیون) تقلیل می‌یابد. نحوه عملکرد این روش در فاز آموزش و آزمون به شرح زیر است:

1. استقلال کامل نمونه‌ها (Independence Assumption):

مهم‌ترین ویژگی روش نقطه‌ای این است که هر جفتِ (پرس‌وجو، سند) یا (q, d) را به عنوان یک نمونه‌ی داده‌ی **کاملاً مستقل** از سایر اسناد در نظر می‌گیرد. در این روش، الگوریتم به هیچ وجه بررسی نمی‌کند که سند **A** نسبت به سند **B** برای یک کوئری خاص بهتر است یا خیر؛ بلکه فقط به ویژگی‌های ذاتی خود سند **A** در ارتباط با آن کوئری نگاه می‌کند.

2. استخراج ویژگی‌ها (Feature Extraction):

برای هر جفت (q, d) ، یک بردار ویژگی x ساخته می‌شود. این بردار شامل مجموعه‌ای از سیگنال‌های ارتباطی است؛ مانند:

امتیاز پایه **BM25** یا **TF-IDF** برای آن جفت.

تطابق کلمات کلیدی در عنوان یا بدنه متن.

ویژگی‌های مستقل از کوئری (مانند **PageRank** سند، طول سند، اعتبار دامنه).

3. فاز آموزش (Training):

الگوریتم یادگیری ماشین، تابعی به فرم $f(x)$ را یاد می‌گیرد. هدف این تابع، پیش‌بینی برچسب ارتباطی واقعی (**Relevance Label**) است که توسط داوران انسانی به آن جفت داده شده است (مثلاً برچسب‌های 0 به معنای نامرتب تا 4 به معنای کاملاً مرتبط). در این فاز، تابع هزینه (**Loss Function**) صرفاً خطای پیش‌بینی برای تک‌تک اسناد را به صورت مجزا محاسبه و کمینه می‌کند.

4. فاز رتبه‌بندی و تولید لیست (Ranking Phase):

هنگامی که یک کوئری جدید در سیستم ثبت می‌شود، الگوریتم به صورت نقطه‌ای (مستقل) به تمامی اسناد نامزدشده یک امتیاز می‌دهد. در نهایت، موتور جستجو اسناد را بر اساس این امتیازات مستقل، به صورت نزولی (از بیشترین به کمترین) مرتب کرده و لیست نهایی را به کاربر نمایش می‌دهد.

بخش 2: ماهیت خروجی مدل برای یک جفت (q, d) چیست؟

بسته به اینکه در روش نقطه‌ای از چه الگوریتم یادگیری ماشینی استفاده کنیم، ماهیت خروجی تابع $f(\mathbf{x})$ برای یک جفت (q, d) متفاوت خواهد بود. به طور کلی، خروجی در این روش دارای **ماهیت مطلق (Absolute)** است، نه ماهیت نسبی (Relative) یا رتبه‌ای (Ordinal Rank).

این خروجی مطلق در سه قالب زیر ظاهر می‌شود:

1. ماهیت پیوسته / رگرسیون (Regression Output):

اگر از الگوریتم‌های رگرسیون (مانند **Linear Regression** یا **SVR**) استفاده کنیم، خروجی مدل یک **عدد حقیقی پیوسته (Continuous Real Number)** خواهد بود. این عدد نشان‌دهنده‌ی «درجه‌ی ارتباط خالص» آن سند به کوئری است (مثلاً امتیاز 2.75).

2. ماهیت طبقه‌بندی باینری / احتمالاتی (Binary Classification Output):

اگر مسئله را به صورت یافتن اسناد مرتبط و غیرمرتبط تعریف کنیم و از مدل‌هایی مانند رگرسیون لجستیک (**Logistic Regression**) استفاده کنیم، خروجی مدل یک **احتمال (Probability)** بین 0 و 1 خواهد بود. مثلاً خروجی 0.85 نشان می‌دهد که مدل با 85 درصد اطمینان، این جفت را «مرتبط» می‌داند.

3. ماهیت طبقه‌بندی ترتیبی (Ordinal Classification Output):

در سیستم‌های پیشرفته‌تر، خروجی مدل یک **برچسب کلاسی گسسته** است (مثلاً یکی از مقادیر $\{0, 1, 2, 3, 4\}$). مدل پیش‌بینی می‌کند که این سند به طور مطلق در کدام سطح از ارتباط کیفی قرار می‌گیرد (مثلاً برچسب 3 به معنای ارتباط خوب).

نکته کلیدی: در تمامی حالت‌های فوق، خروجی مدل هرگز یک «رتبه» (مثل جایگاه اول، دوم یا سوم) در لیست خروجی موتور جستجو نیست؛ بلکه یک نمره خام است که موتور جستجو در گام نهایی برای چیدن لیست از آن استفاده می‌کند.

بخش 3: مقایسه تحلیلی الگوریتم‌ها و توابع هزینه در ارزیابی نقطه‌ای

همان‌طور که اشاره شد، رویکرد نقطه‌ای می‌تواند با استفاده از پارادایم‌های مختلف یادگیری ماشین پیاده‌سازی شود. در جدول زیر، الگوریتم‌های پرکاربرد در این رویکرد به همراه تابع هزینه (**Loss Function**) متناظر آن‌ها مقایسه شده‌اند تا درک بهتری از مکانیزم بهینه‌سازی به دست آید:

ماهیت خروجی $f(\mathbf{x})$	تابع هزینه (Loss Function)	الگوریتم‌های شاخص	رویکرد یادگیری در Pointwise
مقدار پیوسته (مثلاً 2.45)	Mean Squared Error (MSE)	Subset Ranking, PRank	رگرسیون (Regression)
احتمال یا کلاس $\{0, 1\}$	Cross-Entropy / Hinge Loss	Logistic Regression, SVM	طبقه‌بندی باینری (Binary)

رویکرد یادگیری در Pointwise	الگوریتم‌های شاخص	تابع هزینه (Loss (Function	ماهیت خروجی $f(x)$
Classification)			
طبقه‌بندی ترتیبی (Ordinal Classification)	McRank, ODIR	Ordinal Loss / Margin- based Loss	کلاس‌های گسسته (مثلاً 0 تا 4)

جمع‌بندی

- برای درک عمیق روش ارزیابی نقطه‌ای، آن را مانند تصحیح برگه‌های امتحانی توسط یک استاد تصور کنید:
- بررسی مستقل (Pointwise):** استاد هر برگه (سند) را به تنهایی بر می‌دارد، جواب‌ها (ویژگی‌ها) را می‌خواند و به آن برگه یک نمره از 0 تا 20 (امتیاز خروجی) می‌دهد.
 - عدم مقایسه:** استاد در حین نمره‌دهی به برگه علی، اصلاً کاری ندارد که برگه رضا بهتر است یا بدتر. او فقط به برگه علی نمره مطلق می‌دهد.
 - رتبه‌بندی نهایی:** پس از اینکه همه برگه‌ها نمره گرفتند، آموزش کل برگه‌ها را بر اساس نمره‌های کسب‌شده از بیشترین به کمترین مرتب کرده و رتبه‌های اول تا آخر کلاس را اعلام می‌کند.

پاسخنامه مسئله 15.10: نتایج تجربی و ارزیابی (Experimental Results Evaluation)

مقدمه و هدف مسئله

در پایان فرآیند طراحی و آموزش یک مدل یادگیری ماشین برای بازیابی اطلاعات، نیازمند ابزارهایی دقیق برای سنجش عملکرد آن هستیم. هدف از این مسئله، شناخت معیارهای استاندارد ارزیابی در طبقه‌بندی متن و درک یک خطای تحلیلی بسیار رایج (یعنی اتکای صرف به معیار **Accuracy** در فضاهای متنی) است.

بخش 1: معیار اصلی برای ارزیابی کیفیت یک طبقه‌بند متن چیست؟

بر اساس تجربیات علمی و استاندارد کتب مرجع در حوزه بازیابی اطلاعات (**IR**)، ارزیابی کیفیت طبقه‌بندهای متن بر پایه ترکیبی از سه معیار اساسی صورت می‌پذیرد که تمرکز آن‌ها روی «کلاس هدف» (کلاس مثبت) است. برای درک این معیارها، ابتدا باید عناصر ماتریس درهم‌ریختگی (**Confusion Matrix**) شامل **TP** (مثبت واقعی)، **FP** (مثبت کاذب) و **FN** (منفی کاذب) را مبنای محاسبات قرار دهیم:

1. دقت (Precision):

چه نسبتی از اسنادی که مدل ما آن‌ها را متعلق به کلاس **A** تشخیص داده است، واقعاً متعلق به کلاس **A** هستند؟ این معیار حساسیت مدل به خطای مثبت کاذب را می‌سنجد.

$$\text{Precision} = \frac{TP}{TP + FP}$$

2. بازیابی یا فراخوانی (Recall):

از بین تمام اسنادی که در واقعیت متعلق به کلاس **A** بوده‌اند، مدل ما توانسته چه نسبتی از آن‌ها را به درستی پیدا کند؟ این معیار توانایی مدل در جلوگیری از خطای از دست دادن داده‌های مهم (منفی کاذب) را نشان می‌دهد.

$$\text{Recall} = \frac{TP}{TP + FN}$$

3. معیار اف (F-Measure / F1-Score):

از آنجا که بین **Precision** و **Recall** یک تقابل (**Trade-off**) ذاتی وجود دارد (افزایش یکی غالباً به کاهش دیگری منجر می‌شود)، معیار اصلی و نهایی برای ارزیابی کیفیت مدل، **F1-Score** است. این معیار، میانگین هم‌ساز (**Harmonic Mean**) بین دقت و بازیابی است و تنها زمانی مقدار بالایی به خود می‌گیرد که مدل در هر دو فاکتور عملکرد خوبی داشته باشد:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

ارزیابی در مسائل چندکلاسه‌ای :

در مجموعه داده‌های چندکلاسه‌ای، از میانگین‌گیری این معیار روی تمام کلاس‌ها به دو روش زیر به عنوان شاخص نهایی ارزیابی تجربی استفاده می‌شود:

Macro-F1: ابتدا **F1** را برای هر کلاس جداگانه محاسبه کرده و سپس میانگین حسابی آن‌ها را می‌گیرد. این روش به تمام کلاس‌ها (چه اقلیت و چه اکثریت) ارزش کاملاً یکسانی می‌دهد.

Micro-F1: ابتدا مجموع تمام **FP**، **TP** و **FN** ها را در تمام کلاس‌ها محاسبه کرده و سپس یک **F1** کلی به دست می‌آورد. این روش تحت تأثیر کلاس‌های پرجمعیت (اکثریت) قرار دارد.

بخش 2: چرا استفاده از Accuracy به تنهایی گمراه‌کننده است؟

معیار صحت (**Accuracy**) به صورت نسبت «کل پیش‌بینی‌های درست (مثبت و منفی)» به «کل داده‌ها» تعریف می‌شود:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

در دنیای واقعی پردازش متن، مجموعه داده‌ها به شدت **نامتوازن (Imbalanced)** هستند. این نامتوازنی به این معناست که در یک سیستم بازیابی، اسناد نامرتب (**Negative Class**) همیشه بسیار بیشتر از اسناد مرتبط (**Positive Class**) هستند. استفاده از **Accuracy** در چنین شرایطی به دلایل زیر فاجعه‌بار و گمراه‌کننده است:

1. پدیده پارادوکس صحت (Accuracy Paradox):

فرض کنید یک مجموعه داده شامل 10,000 سند داریم که تنها 100 سند آن درباره‌ی موضوع «هوش مصنوعی»

(کلاس مثبت) است و 9,900 سندِ دیگر نامرتب (کلاس منفی) هستند.

اگر ما یک مدل کاملاً احمقانه و ناکارآمد (**Dummy Classifier**) بسازیم که بدون خواندن هیچ متنی، تمامی اسناد را «نامرتب» پیش‌بینی کند، مدل ما 100 اشتباه و 9,900 تشخیص درست خواهد داشت.

با محاسبه **Accuracy** داریم:

$$\text{Accuracy} = \frac{0 + 9900}{10000} = 0.99$$

مدل ما **99 درصد صحت** دارد! یک دانشجو یا محقق بی‌تجربه ممکن است فکر کند به یک دستاورد عظیم علمی رسیده است؛ در حالی که این مدل حتی یک سند مرتبط را هم پیدا نکرده است (در اینجا **Recall = 0** و بنابراین **$F_1 = 0$** خواهد بود).

2. غرق شدنِ خطای اقلیت در اکثریت:

معیار **Accuracy** ارزش مساوی برای کلاس‌ها قائل است. در طبقه‌بندی متن، موفقیت مدل به توانایی آن در یافتن سوزن در انبار کاه (کلاس اقلیت) بستگی دارد، نه در تشخیص خود انبار کاه. **Accuracy** خطاهای رخ داده در کلاس هدف را در میان حجم تشخیص‌های درستِ کلاسِ اکثریت (**TN**) پنهان می‌کند.

بخش 3: کالبدشکافی پارادوکس صحت

برای اثبات ریاضی فاجعه‌ی اتکا به **Accuracy** در مثال بالا، ماتریس درهم‌ریختگی مدل احمقانه (**Dummy Classifier**) در جدول زیر تدوین شده است:

ماتریس درهم‌ریختگی	پیش‌بینی مدل: مثبت (مرتبط)	پیش‌بینی مدل: منفی (نامرتب)
واقعیت: مثبت (مرتبط)	TP = 0	FN = 100 (از دست دادن کل هدف)
واقعیت: منفی (نامرتب)	FP = 0	TN = 9,900 (تشخیص درست بدهیات)
F1-Score = 0 Recall = 0 Precision = 0 Accuracy = 99%		

جمع‌بندی

- سیستم نامتوازن:** یک دستگاه تشخیص بیماری نادر (مثلاً بیماری‌ای که از هر 1,000 نفر فقط 1 نفر به آن مبتلاست).
 - معیار Accuracy:** دکتري که چشم بسته به همه می‌گوید «شما سالم هستید». این دکتري 99.9% دقت کل (**Accuracy**) دارد، اما هیچ بیماری را نجات نمی‌دهد.
 - معیارهای Precision و Recall:** به ما می‌گویند این دکتري از بین کسانی که واقعاً بیمار بوده‌اند، چند نفر را درست تشخیص داده است (**Recall**) و از کسانی که گفته بیمارند، چند نفر واقعاً بیمار بوده‌اند (**Precision**).
- در ارزیابی مدل‌های بازبازی اطلاعات، همیشه به دنبال دکتري باشید که **F_1 بالایی** دارد، نه کسی که فقط آمار شفای عمومی (**Accuracy**) بالایی را به رخ می‌کشد.

پاسخنامه مسئله 15.11: چالش فیلتر اسپم با نویز (Spam Filter Challenge with Noise)

مقدمه و هدف مسئله

در کاربردهای واقعی بازیابی اطلاعات و یادگیری ماشین، داده‌های آموزشی هرگز بی‌نقص نیستند. خطای انسانی در برچسب‌گذاری (مانند برچسب اسپم خوردن اشتباه یک ایمیل کاری) منجر به تولید «داده‌های نویزی» می‌شود. هدف از این مسئله، بررسی رفتار ریاضی الگوریتم ماشین بردار پشتیبان (SVM) در مواجهه با این نویزها و درک عمیق نقش پارامتر تنظیم‌کننده (Regularization Parameter) C در کنترل شکل مرز تصمیم‌گیری است.

بخش 1: چرا حاشیه سخت (Hard Margin) دچار عدم همگرایی می‌شود؟

الگوریتم SVM با حاشیه سخت، بر پایه یک فرض مطلق و غیرقابل انعطاف بنا شده است: تفکیک‌پذیری خطی کامل.

در فرمول‌بندی حاشیه سخت، قید ریاضی زیر باید برای تکتک داده‌های آموزشی برقرار باشد:

$$y_i(w^T x_i + b) \geq 1$$

هنگامی که چند ایمیل کاری (کلاس منفی) به اشتباه برچسب اسپم (کلاس مثبت) دریافت می‌کنند، این نقاط از نظر مختصات ویژگی‌ها در میان داده‌های کلاس خودشان قرار دارند، اما برچسب کلاس مقابل را یدک می‌کشند. از نظر هندسی، این یعنی دو کلاس با یکدیگر «هم‌پوشانی» (Overlap) پیدا کرده‌اند.

تحلیل ریاضیاتی شکست:

در چنین فضایی، کشیدن یک خط مستقیم که تمام داده‌ها را بدون خطا جدا کند، از نظر فیزیکی و ریاضی غیرممکن است. هنگامی که فضای حل مسئله (Feasible Region) به دلیل تضاد در نامعادلات، تهی (خالی) می‌شود، الگوریتم بهینه‌سازی ریاضی که مبتنی بر برنامه‌ریزی درجه دوم (Quadratic Programming) است، نمی‌تواند هیچ پاسخی برای این مجموعه معادلات پیدا کند و اصطلاحاً فرآیند آموزش «همگرا نمی‌شود» (Fails to converge) یا با شکست کامل محاسباتی مواجه می‌گردد.

بخش 2: چگونه حاشیه نرم و پارامتر C از این نویزها عبور می‌کنند؟

برای حل این بن‌بست، مدل به سراغ «حاشیه نرم» (Soft Margin) می‌رود و برای هر نقطه، یک متغیر کمکی یا لَقی ($\xi_i \geq 0$) تعریف می‌کند تا اجازه تخلف از قوانین را صادر کند. قید جدید به شکل زیر در می‌آید:

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

با اضافه شدن این متغیرها، الگوریتم اکنون می‌تواند همگرا شود؛ زیرا برای نقاط نویزی (ایمیل‌های کاری اشتباه برچسب‌خورده)، به سادگی مقدار ξ_i را افزایش می‌دهد تا معادله برقرار شود. اما برای اینکه الگوریتم بی‌دلیل به همه نقاط اجازه تخلف ندهد، این خطاها باید جریمه شوند. اینجاست که پارامتر هزینه (Cost Parameter) C وارد تابع هدف می‌شود:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

پارامتر C یک وزن دهی تعادلی است که به الگوریتم می‌گوید: «تا چه حد باید نگران تخلفات و نویزها باشی؟» این پارامتر در واقع مصالحه‌ای است بین حداکثر کردن حاشیه هندسی (ترم اول) و به حداقل رساندن خطای دسته‌بندی (ترم دوم).

بخش 3: تحلیل تغییرات مرز تصمیم در مقادیر حدی پارامتر C

انتخاب مقدار C مستقیماً روی هندسه مرز تصمیم (**Decision Boundary**) و عرض حاشیه تأثیر می‌گذارد. رفتار مدل را در دو حالت افراطی تحلیل می‌کنیم:

1. حالت اول: C بسیار بزرگ ($C \rightarrow \infty$)

تفسیر ریاضی: جریمه‌ی خطا در تابع هدف به شدت سنگین است. حتی یک تخلف کوچک ($\xi_i > 0$)

هزینه بسیار بالایی برای مدل به همراه خواهد داشت.

رفتار مدل: الگوریتم تلاش می‌کند به هر قیمتی که شده، تمام نقاط آموزشی (حتی آن چند ایمیل نویزی) را مطابق با برچسب‌های غلط آن‌ها دسته‌بندی کند.

تأثیر بر مرز تصمیم: برای دربرگرفتن این نقاط نویزی، مرز تصمیم به شدت کج می‌شود (در حالت خطی) یا دچار پیچ‌خوردگی‌های شدید می‌شود (در صورت استفاده از کرنل‌های غیرخطی). عرض حاشیه (**Margin**) بسیار باریک می‌شود.

نتیجه‌گیری: در این حالت مدل دچار **بیش‌برازش (Overfitting)** روی نویزها شده است. قدرت تعمیم‌پذیری به شدت افت می‌کند و در آینده ایمیل‌های کاری سالم زیادی را به اشتباه اسپم تشخیص خواهد داد.

2. حالت دوم: C بسیار کوچک ($C \rightarrow 0$)

تفسیر ریاضی: جریمه‌ی خطا در تابع هدف بسیار ناچیز است. در این حالت، ترم $\frac{1}{2} \|\mathbf{w}\|^2$ (که مربوط به پهنای حاشیه است) بر تابع هدف غالب می‌شود.

رفتار مدل: الگوریتم تمرکز خود را روی یافتن «عریض‌ترین جاده‌ی ممکن» بین دو کلاس می‌گذارد و به راحتی می‌پذیرد که ایمیل‌های نویزی درون حاشیه یا در سمت اشتباه خط باقی بمانند (تحمل بالای خطا).
تأثیر بر مرز تصمیم: مرز تصمیم به نویزهای محلی بی‌اعتنا می‌شود و شکلی پایدارتر، صاف‌تر و کلی‌نگرتر به خود می‌گیرد. عرض حاشیه به حداکثر خود (بسیار پهن) می‌رسد.

نتیجه‌گیری: در این حالت، مدل روی الگوی کلی و ذاتی داده‌ها (تفاوت واقعی اسپم و غیراسپم) تمرکز کرده است. با چشم‌پوشی از آن چند داده‌ی اشتباه، مدل به **تعمیم‌پذیری (Generalization)** بالا دست می‌یابد و روی داده‌های جدید عملکرد بسیار بهتری خواهد داشت.

بخش 4: مقایسه تحلیلی اثر پارامتر C

جهت ارزیابی و درک سریع‌تر اثر تنظیم پارامتر C بر رفتار تعمیم‌پذیری و مشخصه‌های مدل، مقایسه این دو حالت در جدول استاندارد زیر استخراج شده است:

مشخصه سیستم	پارامتر C بسیار بزرگ	پارامتر C بسیار کوچک
عرض حاشیه (Margin)	بسیار باریک	بسیار پهن
وضعیت بایاس-واریانس	واریانس بالا، بایاس پایین	واریانس پایین، بایاس بالا
خطر مدل سازی	بیش برزش (Overfitting)	احتمال کم برزش (Underfitting) در صورت افراط
بردارهای پشتیبان	تعداد کم (وابستگی شدید به نویزها)	تعداد زیاد (اتکا به ساختار کلی داده ها)

جمع بندی

- C بزرگ (استاد جزئی نگر):** به کوچک ترین غلطها گیر می دهد. برای اینکه هیچ خطایی نداشته باشد، مجبور است مدل را آن قدر پیچیده کند که فرمول اصلی را فراموش کرده و فقط استثنایها (نویزها) را حفظ کند (Overfitting).
- C کوچک (استاد کلی نگر):** می داند که دانشجو ممکن است چند خطای سهوی (نویز) داشته باشد، پس از آن ها چشم پوشی می کند و تمرکزش را روی یادگیری مفاهیم اصلی و کلی (حاشیه پهن تر) می گذارد. در مواجهه با داده های نویزی دنیای واقعی، این رویکرد همیشه موفق تر است.

پاسخنامه مسئله 15.12: مقایسه هزینه های محاسباتی چندکلاسه ای (Multiclass

Computational Costs)

مقدمه و هدف مسئله

در طراحی سیستم های هوشمند مقیاس پذیر، انتخاب استراتژی طبقه بندی نه تنها بر دقت، بلکه بر منابع پردازشی و زمان اجرا تأثیر مستقیم دارد. در این مسئله، با یک سناریوی عملی مواجه هستیم که در آن سیستم باید متون را به 20 دسته مجزا ($K = 20$) تقسیم کند. هدف، محاسبه دقیق بار محاسباتی در دو روش استاندارد (**OVA** و **OVO**) و انتخاب استراتژی بهینه با تمرکز بر سرعت فاز «پیش بینی» (**Inference**) است.

بخش 1: محاسبه تعداد مدل های مورد نیاز

برای کالبدشکافی این سیستم، تعداد کلاس های موضوعی را برابر با $K = 20$ در نظر می گیریم. محاسبات برای هر دو رویکرد به شرح زیر است:

1. روش یک-در-برابر-همه (One-vs-All / OVA):

در این روش، برای هر کلاس موضوعی، دقیقاً یک مدل مستقل آموزش داده می شود که آن کلاس را از مجموع تمام کلاس های دیگر متمایز می کند.

فرمول: تعداد مدل ها $K =$

محاسبه: در این سناریو، به 20 مدل **SVM** مستقل نیاز داریم.

2. روش یک-در-برابر-یک (One-vs-One / OVO):

در این استراتژی، برای هر جفت ممکن از کلاس‌ها، یک مدل متمایز آموزش داده می‌شود (مثلاً مدلی فقط برای تفکیک «اقتصاد» از «ورزش»).

فرمول: تعداد مدل‌ها بر اساس ترکیب 2 از K محاسبه می‌شود:

$$\frac{K(K-1)}{2}$$

محاسبه: با جای‌گذاری $K = 20$ خواهیم داشت:

$$\frac{20 \times 19}{2} = 190$$

در این سناریو، به 190 مدل SVM مستقل نیاز داریم.

بخش 2: انتخاب استراتژی بهینه برای سرعت در فاز «پیش‌بینی» (Inference)

پاسخ صریح:

اگر اولویت مطلق سیستم مدیریت اخبار شما، سرعت ارزیابی و دسته‌بندی مقالات جدید (فاز **Inference**) باشد، روش یک-در-برابر-همه (**OVA**) انتخاب قطعی و مهندسی‌شده خواهد بود.

تحلیل و چرایی این انتخاب (اثبات ریاضی و ساختاری):

فاز پیش‌بینی (**Testing Phase**) با فاز آموزش (**Training Phase**) تفاوت بنیادین دارد. وقتی یک مقاله خبری جدید (به شکل یک بردار ویژگی $\mathbf{x}$) وارد سیستم می‌شود، باید از فیلتر مدل‌های از پیش آموزش‌دیده عبور کند.

1. تفاوت فاحش در تعداد عملیات ریاضی:

در مدل‌های خطی مانند **SVM**، زمان پیش‌بینی برای یک سند، مستقیماً با تعداد مدل‌هایی که باید آن سند را ارزیابی کنند رابطه خطی دارد.

در روش OVA: مقاله جدید تنها به 20 مدل ارسال می‌شود. سیستم باید 20 ضرب داخلی ریاضی ($\mathbf{w}_k^T \mathbf{x} + b_k$) را محاسبه کرده و نهایتاً بزرگ‌ترین مقدار را به عنوان برنده اعلام کند (پیچیدگی زمانی $O(K)$).

در روش OVO: همان مقاله باید از فیلتر 190 مدل عبور کند. یعنی سیستم باید 190 محاسبه‌ی مجزا انجام دهد و سپس نتایج را در یک فرآیند رأی‌گیری (**Majority Voting**) تجمیع کند (پیچیدگی زمانی $O(K^2)$). بدیهی است که انجام 20 محاسبه بسیار سریع‌تر از 190 محاسبه است.

2. تقابل سرعت «آموزش» با سرعت «پیش‌بینی» (Trade-off):

دانشجویان باید به این نکته‌ی ظریف و حیاتی دقت کنند:

چنانچه در مسئله‌های قبلی آموختیم، روش **OVO** در فاز آموزش (**Training**) سریع‌تر است؛ زیرا با وجود اینکه باید 190 مدل بسازد، اما هر مدل را روی کسر بسیار کوچکی از داده‌ها (فقط داده‌های همان دو کلاس) آموزش می‌دهد.

اما در فاز پیش‌بینی (**Inference**)، معادله کاملاً برعکس می‌شود. وقتی مدل‌ها ساخته شدند، ابعاد بردار وزن (**w**) در هر دو روش با هم برابر است (مساوی با طول لغت‌نامه). در این مرحله، حجم داده‌های آموزشی اولیه دیگر اهمیتی ندارد؛ بلکه تنها «تعداد مدل‌های نهایی» است که سرعت سیستم را تعیین می‌کند.

3. ردپای حافظه (Memory Footprint) در محیط عملیاتی:

در زمان استقرار (**Deployment**) روی سرور، سیستم باید بردارهای وزن را در حافظه **RAM** نگه دارد. در روش **OVA** شما نیازمند نگهداری 20 بردار هستید، در حالی که در **OVO** باید 190 بردار هم-اندازه در حافظه بارگذاری شوند. این تفاوت در ابعاد بالا (مانند متن با 100,000 ویژگی)، مصرف حافظه سرور را تقریباً 9.5 برابر افزایش می‌دهد که از نظر مهندسی نرم‌افزار به شدت غیربهرینه است.

بخش 3: مقایسه جامع پارامترهای عملیاتی

برای انتخاب دقیق معماری سیستم در محیط پروداکشن (**Production**)، جدول زیر خلاصه‌ای از تقابل این دو استراتژی را در یک مسئله با 20 کلاس نمایش می‌دهد:

معیار ارزیابی (K=20)	یک-در-برابر-همه (OVA)	یک-در-برابر-یک (OVO)
تعداد محاسبات پیش‌بینی	20 Operations	190 Operations
زمان پیش‌بینی (Inference)	بسیار سریع ($O(K)$)	کُند ($O(K^2)$)
زمان آموزش (Training)	بسیار کُند	سریع و کارآمد
مصرف حافظه (RAM) در اجرا	پایین (فقط 20 بردار)	بالا (190 بردار هم-وزن)

جمع‌بندی

- اگر سیستمی می‌سازید که داده‌هایش دائماً تغییر می‌کند و باید هر روز مدل را از نو آموزش دهید، اما تعداد کاربران کمی دارد $\Leftarrow$ روش **OVO** به دلیل سرعت آموزش بالا پیشنهاد می‌شود.
- اگر سیستمی (مانند موتور جستجو یا پورتال خبری) می‌سازید که مدل آن یک بار در ماه آموزش می‌بیند، اما در هر ثانیه باید هزاران خبر جدید را در کسری از ثانیه دسته‌بندی کند $\Leftarrow$ روش **OVA** به دلیل سرعت استنتاج بسیار بالا و مصرف حافظه کمتر، تنها گزینه منطقی و عملیاتی خواهد بود.

پاسخنامه مسئله 15.13: مشکل انفجار ابعاد در ان‌گرام‌ها

(Dimension Explosion in N-grams)

مقدمه و هدف مسئله

در تحلیل احساسات (**Sentiment Analysis**)، استفاده از تک‌کلمات (**Unigrams**) گاهی برای درک معنای متن کافی نیست. به عنوان مثال، کلمه «خوب» بار مثبت دارد، اما ترکیب «اصلاً خوب نیست» بار کاملاً منفی دارد. برای رفع این مشکل، مهندسان ویژگی‌هایی مانند ترکیب‌های دوتایی (**Bi-grams**) و سه‌تایی (**Tri-grams**) را به مدل اضافه می‌کنند. هدف از این

مسئله، تحلیل تبعات جبر خطی این تصمیم بر روی ماتریس داده‌ها (TF-IDF) و اثبات ریاضی مقاومت و پایداری الگوریتم ماشین بردار پشتیبان (SVM) در برابر این حجم عظیم از ابعاد است.

بخش 1: تأثیر افزودن N-gram ها بر ماتریس TF-IDF

اضافه کردن توالی‌های کلماتی به عنوان ویژگی‌های جدید، ساختار ماتریس TF-IDF را دچار تغییرات بنیادین و شدیدی می‌کند که به آن «پدیده انفجار ابعاد» (Curse/Explosion of Dimensionality) می‌گویند. این تغییرات در دو جنبه اصلی رخ می‌دهند:

1. افزایش نمایی تعداد ستون‌ها (ابعاد فضای ویژگی):

اگر اندازه لغت‌نامه تک‌کلمات (Unigrams) را $|V|$ در نظر بگیریم، تعداد کل ترکیب‌های دوتایی ممکن (Bi-grams) از نظر تئوری برابر با $|V|^2$ و ترکیب‌های سه‌تایی برابر با $|V|^3$ خواهد بود. اگرچه در متون واقعی تمام این ترکیب‌های ممکن رخ نمی‌دهند، اما در عمل با اضافه کردن این ویژگی‌ها، تعداد ستون‌های ماتریس TF-IDF (تعداد ویژگی‌ها) ده‌ها یا صدها برابر می‌شود و به راحتی از مرتبه ده‌ها هزار بُعد به میلیون‌ها بُعد افزایش می‌یابد.

2. تشدید شدید خلوتی ماتریس (Extreme Sparsity):

با بزرگ‌تر شدن N در N-gram ها، احتمال وقوع یک ترکیب خاص در یک سند به شدت کاهش می‌یابد. اگر در ماتریس تک‌کلمات، 99 درصد خانه‌ها صفر بودند، در ماتریس ترکیب‌شده با بای‌گرام و ترای‌گرام، این خلوتی ممکن است به 99.99 درصد برسد. به این معنا که ماتریس شما به شدت پهن‌تر (ستون‌های بیشتر) می‌شود، اما تقریباً تمام محتوای این ستون‌های جدید با عدد «صفر» پر خواهد شد.

بخش 2: کالبدشکافی ریاضیاتی انفجار ابعاد

برای درک ملموس شدت این فاجعه محاسباتی در روش‌های سنتی، فرض کنید لغت‌نامه پایه ما شامل 10,000 کلمه مجزا است. جدول زیر رفتار ابعاد و خلوتی ماتریس را با افزایش N شبیه‌سازی می‌کند:

نوع ویژگی	اندازه پایه ($ V $)	حداکثر ابعاد تئوری (ستون‌ها)	نرخ خلوتی (Sparsity)
Unigrams (1)	10,000	10,000	~ 99.00%
Bi-grams (2)	10,000	$100,000,000 (V ^2)$	~ 99.98%
Tri-grams (3)	10,000	$1,000,000,000,000 (V ^3)$	< 99.99%

بخش 3: چرا SVM خطی با وجود این ابعاد عظیم دچار Overfitting نمی‌شود؟

در یادگیری ماشین کلاسیک، یک قانون کلی وجود دارد که اگر تعداد ویژگی‌ها (ابعاد) از تعداد نمونه‌های آموزشی بیشتر شود، مدل به سرعت تمام نويزها را حفظ کرده و دچار بیش‌برازش (Overfitting) می‌شود. اما الگوریتم SVM خطی یک استثنای درخشان در این زمینه است و به چهار دلیل کلیدی ریاضی و ساختاری، مقاومت بی‌نظیری در برابر ابعاد میلیونی نشان می‌دهد:

1. استقلال خطای تعمیم از تعداد ابعاد (VC Dimension Theory):

بر اساس تئوری یادگیری آماری وپنیک (Vapnik)، کران خطای تعمیم (Generalization Error Bound) در

الگوریتم **SVM**، به طور مستقیم به «تعداد ابعاد» بستگی ندارد؛ بلکه به «اندازه حاشیه هندسی» (**Margin Size**) وابسته است. از آنجا که **SVM** به صورت ذاتی به دنبال پیدا کردن ابرصفحه‌ای با حداکثر حاشیه است، حتی در فضایی با میلیون‌ها بُعد، مدلی را انتخاب می‌کند که کمترین پیچیدگی و بیشترین حریم امن را دارد.

2. تنظیم‌گری ذاتی (Regularization) قدرتمند:

تابع هدف در **SVM** حاشیه نرم به صورت زیر است:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

حضور عبارت $\frac{1}{2} \|\mathbf{w}\|^2$ دقیقاً به عنوان یک جریمه‌کننده یا تنظیم‌گر **L_2 (Ridge Regularization)** عمل می‌کند. این بخش از فرمول، اجازه نمی‌دهد که مقادیر بردار وزن (**w**) برای ویژگی‌های بی‌اهمیت یا نویزی بزرگ شود. در واقع، الگوریتم به طور خودکار وزن **N-gram** های بی‌فایده را به شدت کوچک (نزدیک به صفر) می‌کند و تنها به ترکیب‌های معنادار ضریب می‌دهد.

3. اتکا به بردارهای پشتیبان، نه ابعاد:

در **SVM**، مرز تصمیم‌گیری (**Decision Boundary**) منحصراً توسط تعداد معدودی از داده‌های حیاتی به نام «بردارهای پشتیبان» (**Support Vectors**) ساخته می‌شود. این یعنی مدل تحت تأثیر حضور میلیون‌ها ویژگی قرار نمی‌گیرد، بلکه ساختار خود را بر اساس موقعیت هندسی نقاط مرزی شکل می‌دهد که این امر پایداری ساختاری مدل را در فضاهای بزرگ تضمین می‌کند.

4. بهره‌برداری از قضیه پوشش (Cover's Theorem):

داده‌هایی که در ابعاد پایین ممکن است مرزهای پیچیده و غیرخطی داشته باشند، وقتی به فضایی با میلیون‌ها بُعد نگاشت می‌شوند (به دلیل افزایش **N-gram** ها)، با احتمال بسیار بالایی به صورت **خطی تفکیک‌پذیر (Linearly Separable)** در می‌آیند. **SVM** خطی در این فضای بسیار بزرگ و خلوت، می‌تواند یک ابرصفحه‌ی کاملاً صاف و ساده رسم کند که کلاس‌های احساسات (مثبت و منفی) را بدون نیاز به ساختارهای پیچیده از هم جدا سازد.

جمع‌بندی ساختاری برای دانشجو

برای درک این پارادوکس ظاهری، مدل **SVM** را مانند یک «کارآگاه باتجربه در یک کتابخانه بی‌نهایت بزرگ» تصور کنید:

1. **انفجار ابعاد:** به معنای اضافه شدن میلیون‌ها کتاب جدید (**Bi-grams** و **Tri-grams**) به قفسه‌های کتابخانه است. اکثر این کتاب‌ها کاملاً خالی یا بی‌ربط هستند (**Sparsity**).

2. **مقاومت در برابر Overfitting:** یک کارآگاه تازه‌کار (الگوریتم‌های ضعیف‌تر) سعی می‌کند محتوای تمام این میلیون‌ها کتاب را حفظ کند و در نهایت گیج می‌شود (**Overfitting**). اما کارآگاه ما (**SVM**) مجهز به یک فیلتر تنظیم‌گر قدرتمند ($\frac{1}{2} \|\mathbf{w}\|^2$) است. او در همان نگاه اول متوجه می‌شود که 99 درصد قفسه‌ها اطلاعات مفیدی ندارند؛ بنابراین آن‌ها را نادیده گرفته و فقط روی تعداد انگشت‌شماری از سرنخ‌های مرزی و کلیدی (**Support Vectors**) تمرکز می‌کند تا معمای دسته‌بندی را حل کند.

پاسخنامه مسئله 15.14: ضرورت انتخاب ویژگی (Necessity of Feature Selection)

مقدمه و هدف مسئله

در مسائل پیشین اثبات کردیم که الگوریتم **SVM** به لطف استفاده از حاشیه هندسی و تنظیم‌گرهای ذاتی (**Regularization**)، مقاومت فوق‌العاده‌ای در برابر ابعاد بالا دارد. با این حال، مواجهه با سناریویی که در آن تعداد ویژگی‌ها (100,000 کلمه) بسیار بیشتر از تعداد نمونه‌های آموزشی (1,000 سند) است وضعیتی که در آمار به آن چالش $p \gg n$ می‌گویند یک شرایط افراطی است. هدف از این مسئله، درک این پارادوکس ظاهری است که چرا با وجود قدرت ذاتی **SVM**، تکنیک‌های پیش‌پردازش و «انتخاب ویژگی» (مانند **Mutual Information**) همچنان یک گام الزامی و استاندارد محسوب می‌شوند.

این ضرورت را از دو منظر کاملاً متفاوت کالبدشکافی می‌کنیم:

بخش 1: ضرورت از منظر مهندسی نرم‌افزار و منابع محاسباتی (Software Engineering Perspective)

حتی اگر فرض کنیم یک مدل از نظر ریاضیاتی و آماری هیچ نیازی به کاهش ابعاد ندارد، مهندسی سیستم‌ها ما را ملزم به این کار می‌کند. دلایل زیر این ضرورت سیستماتیک را توجیه می‌کنند:

1. گلوگاه سرعت در فاز پیش‌بینی (Inference Latency):

در یک سیستم واقعی (مانند یک سامانه تشخیص پزشکی یا موتور جستجو)، مدل نهایی باید در کسری از ثانیه روی داده‌های جدید تصمیم‌گیری کند. اگر مدل شما 100,000 ویژگی داشته باشد، سیستم در زمان اجرا (**Run-time**) باید متن ورودی جدید را پردازش کرده و مقادیر 100,000 کلمه را استخراج، وزن‌دهی (مثلاً محاسبه **IDF**) و در بردار وزن w ضرب کند. با انتخاب ویژگی و کاهش ابعاد از 100,000 به مثلاً 5,000 کلمه‌ی کلیدی، سرعت استخراج ویژگی و پیش‌بینی (**Inference**) ده‌ها برابر افزایش می‌یابد.

2. مدیریت حافظه (Memory Constraints):

بردار وزن w در **SVM** خطی دقیقاً هم‌اندازه با تعداد ویژگی‌هاست. علاوه بر این، لغت‌نامه‌ی سیستم (**Vocabulary**) که کلمات را به شناسه‌های عددی (**Index**) نگاشت می‌کند، باید به صورت دائمی در حافظه اصلی (**RAM**) سرور بارگذاری شود. نگهداری و پردازش ماتریس‌ها و دیکشنری‌های 100,000 تایی، به ویژه در سیستم‌های مقیاس‌پذیر، بار سنگینی بر منابع سرور تحمیل می‌کند که با انتخاب ویژگی هوشمندانه، این هزینه به شدت کاهش می‌یابد.

بخش 2: ضرورت از منظر عملکرد مدل و کیفیت آماری (Model Performance Perspective)

با وجود اینکه **SVM** از تنظیم‌گر (مانند **L_2 Penalty**) استفاده می‌کند تا از بزرگ شدن وزن ویژگی‌های بی‌اهمیت جلوگیری کند، اما این مکانیزم مصون از خطا نیست.

1. خطر همبستگی‌های تصادفی یا کاذب (Spurious Correlations):

وقتی شما تنها 1,000 سند آموزشی دارید اما 100,000 کلمه مختلف را رصد می‌کنید، از نظر احتمالات آماری بسیار محتمل است که یک کلمه کاملاً بی‌ربط و نویزی (مثلاً یک غلط املایی خاص، نام یک بیمار، یا یک اصطلاح نامرتبب که فقط 3 بار در کل دیتاست تکرار شده) به طور کاملاً تصادفی فقط در اسناد متعلق به کلاس مثبت ظاهر شود. الگوریتم **SVM** (یا هر الگوریتم دیگری) از درک معنایی کلمات عاجز است؛ بنابراین این همبستگی 100 درصدی

تصادفی را به عنوان یک «سیگنالِ قطعی» در نظر می‌گیرد و به آن کلمه‌ی نویزی، وزنِ بالایی اختصاص می‌دهد. این پدیده باعث می‌شود مدل روی نویزهای تصادفی فریب خورده و در مواجهه با اسناد جدید در دنیای واقعی، دچار خطای فاحش شود.

2. نقش نجات‌بخش Mutual Information (اطلاعات متقابل):

تکنیک‌های انتخاب ویژگی مبتنی بر آمار (نظیر **Mutual Information** یا آزمون χ^2)، پیش از اینکه داده‌ها اصلاً به **SVM** برسند، به عنوان یک **فیلترِ منطق‌سنج** عمل می‌کنند.

معیار **Mutual Information** میزان وابستگی و اشتراک اطلاعاتی بین یک «کلمه» و یک «کلاس هدف» را محاسبه می‌کند. این فیلتر، کلمات که حضورشان در کلاس‌های مختلف صرفاً تصادفی و بر اساس نویز است را شناسایی کرده و آن‌ها را به طور کامل از دیتاست حذف می‌کند. با این کار، ما «نسبت سیگنال به نویز» (**Signal-to-Noise Ratio**) را به شدت بالا می‌بریم.

بخش 3: کالبدشکافی ریاضیاتی اطلاعات متقابل و تأثیر آن

برای درک دقیق‌تر اینکه چگونه **Mutual Information (MI)** کلمات نویزی را فیلتر می‌کند، معادله پایه‌ای آن در تئوری اطلاعات به شکل زیر تعریف می‌شود:

$$MI(U, C) = \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \left(\frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)} \right)$$

در این رابطه، U نشان‌دهنده حضور یا عدم حضور کلمه و C نشان‌دهنده کلاس سند است. اگر حضور یک کلمه کاملاً مستقل از کلاس باشد (یعنی کلمه نویزی باشد)، کسر داخل لگاریتم به عدد 1 میل کرده و در نتیجه لگاریتم آن **صفر** می‌شود. در جدول زیر تقابل سیستم را در دو حالت پیش از فیلتر و پس از فیلتر ارزیابی کرده‌ایم:

معیار ارزیابی سیستم	پیش از انتخاب ویژگی (فضای 100,000 بُعدی)	پس از انتخاب ویژگی (فضای 5,000 بُعدی)
نسبت سیگنال به نویز	بسیار پایین (تجمع همبستگی‌های کاذب)	بسیار بالا (تراکم کلمات معنادار)
زمان تأخیر (Latency)	بالا (محاسبه وزن برای تمام لغت‌نامه)	کسری از ثانیه (استخراج سریع کلمات کلیدی)
فضای اشغالی (RAM)	سنگین (بارگذاری ماتریس‌های عظیم)	سبک و بهینه (کاهش چشمگیر حافظه)

جمع‌بندی

برای درک ترکیب این دو رویکرد (انتخاب ویژگی + الگوریتم **SVM**)، این تمثیل را در ذهن بسپارید:

- الگوریتم **SVM** مانند یک سرآشپز بین‌المللی و بسیار ماهر است که می‌تواند از مواد اولیه، بهترین ترکیب ممکن را بسازد.
- فضای 100,000 ویژگی با نویزهای زیاد، مانند این است که شما یک کامیون زباله که در آن مقداری مواد اولیه تازه هم وجود دارد را مستقیماً در آشپزخانه تخلیه کنید. سرآشپز (**SVM**) تلاش می‌کند با مهارتش (**Regularization**) زباله‌ها را کنار بزند و غذا بپزد، اما هم زمان زیادی از او گرفته می‌شود و هم ممکن است مقداری آلودگی (**Spurious Correlations**) وارد غذا شود.

- **انتخاب ویژگی (مانند Mutual Information)** نقش دستیار آشپز را دارد. او پیش از شروع پخت‌وپز، مواد زائد را دور می‌ریزد و تنها بهترین و مرتبط‌ترین مواد (ویژگی‌های دارای بار اطلاعاتی) را در اختیار سرآشپز قرار می‌دهد. نتیجه‌ی این کار، سیستمی است که هم با سرعت بسیار بالاتری کار می‌کند و هم خروجی خالص‌تر و قابل‌اطمینان‌تری دارد.

پاسخنامه مسئله 15.15: تفکیک‌ناپذیری خطی احساسات و مبانی SVM غیرخطی

مقدمه و هدف مسئله

در تحلیل احساسات (**Sentiment Analysis**)، فرض «استقلال کلمات از یکدیگر» غالباً با چالش مواجه می‌شود. زبان انسانی مملو از پیچیدگی‌هایی نظیر کنایه (**Sarcasm**)، نفی مضاعف (**Double Negation**) و ساختارهای شرطی است. در این حالت، حضور یک کلمه به تنهایی بار معنایی را مشخص نمی‌کند، بلکه تعامل کلمات با یکدیگر تعیین‌کننده است. هدف از این مسئله، درک محدودیت‌های هندسی مدل‌های خطی در مواجهه با چنین الگوهای متقاطعی (معروف به مسئله **XOR**) و آشنایی با راهکار هوشمندانه **SVM** برای عبور از این بن‌بست است.

بخش 1: تصویرسازی ذهنی از فضای 2-بعدی و مسئله XOR

برای درک اینکه چرا یک خط مستقیم در پردازش متن گاهی شکست می‌خورد، بیایید یک فضای هندسی بسیار ساده (2-بعدی) را در ذهن ترسیم کنیم. فرض کنید برای تحلیل نظرات یک محصول، تنها 2 ویژگی (کلمه) را استخراج کرده‌ایم:

محور X: حضور کلمه «خوب» (مقدار 1 یعنی وجود دارد، 0 یعنی وجود ندارد).

محور Y: حضور کلمه «بد» (مقدار 1 یعنی وجود دارد، 0 یعنی وجود ندارد).

حالا 4 نوع نظر مختلف را در این فضای 2-بعدی (مختصات **X, Y**) جانمایی می‌کنیم:

1. **نقطه A در مختصات (0, 1):** کاربر گفته "فقط ویژگی‌های خوب داشت". **کلاس مثبت (سبز)**

2. **نقطه B در مختصات (1, 0):** کاربر گفته "فقط ویژگی‌های بد داشت". **کلاس منفی (قرمز)**

3. **نقطه C در مختصات (0, 0):** نظر خنثی یا نامرتب (فاقد هر دو کلمه). **کلاس** ما سیستم در سیستم ما **کلاس**

منفی (قرمز) دسته‌بندی می‌شود.

4. **نقطه D در مختصات (1, 1):** کاربر از ساختار کنایه‌آمیز یا متناقض استفاده کرده است: "نه تنها خوب نبود، بلکه

خیلی هم بد بود!" **کلاس منفی (قرمز)**

نکته: در مسئله کلاسیک **XOR**، آرایش نقاط به گونه‌ای است که مختصات $(1,0)$ و $(0,1)$ متعلق به یک کلاس و مختصات $(0,0)$ و $(1,1)$ متعلق به کلاس دیگر هستند. در زبان طبیعی، الگوهای کنایه‌آمیز دقیقاً چنین توزیع متقاطعی را در فضای ویژگی‌ها ایجاد می‌کنند.

نقطه داده	کلمه «خوب» (X)	کلمه «بد» (Y)	تحلیل زبانی	برچسب کلاس
A (1, 0)	1	0	مطلقاً مثبت	مثبت (سبز)
B (0, 1)	0	1	مطلقاً منفی	منفی (قرمز)
C (0, 0)	0	0	خنثی / بی‌تفاوت	منفی (قرمز)

نقطه داده	کلمه «خوب» (X)	کلمه «بد» (Y)	تحلیل زبانی	برچسب کلاس
D (1, 1)	1	1	کنایه / نفی متقاطع	منفی (قرمز)

بخش 2: چرا خط مستقیم (مدل خطی) شکست می‌خورد؟

اکنون یک خودکار بردارید و سعی کنید در این فضای ذهنی (که نقاط سبز در مختصات (1,0) و (0,1) و نقاط قرمز در (0,0) و (1,1) قرار دارند)، تنها با رسم یک خط مستقیم، تمام نقاط سبز را در یک سمت خط و تمام نقاط قرمز را در سمت دیگر قرار دهید.

نتیجه هندسی: انجام این کار از نظر ریاضی و هندسی کاملاً غیرممکن است.

هر خط مستقیمی که رسم کنید، حداقل یک نقطه را در سمت اشتباه قرار خواهد داد. معادله خطی به فرم زیر نمی‌تواند الگوهای متقاطع (Non-linear) را مدل‌سازی کند:

$$w_1x + w_2y + b = 0$$

الگوریتم SVM خطی در مواجهه با چنین داده‌هایی، هرگز همگرا نشده و خطای طبقه‌بندی بالایی تولید می‌کند، زیرا مفاهیم زبانی (مانند کنایه) باعث شده‌اند داده‌ها «تفکیک‌ناپذیر خطی» (Linearly Inseparable) شوند.

بخش 3: ایده اولیه SVM غیرخطی و ترفند هسته (Kernel Trick)

چگونه می‌توان این مشکل را حل کرد؟ یک راهکار ابتدایی این است که به جای خط مستقیم، یک مرز منحنی یا دایره‌ای (مثلاً معادله درجه 2) رسم کنیم. اما رسم معادلات پیچیده در فضاهای با ابعاد بالا (مانند متن با ده‌ها هزار کلمه) از نظر محاسباتی غیرممکن و بسیار مستعد Overfitting است.

الگوریتم SVM غیرخطی با یک فلسفه کاملاً متفاوت و شاهکار ریاضی به این مسئله حمله می‌کند: «به جای اینکه خط را کج کنیم تا با داده‌ها منطبق شود، فضا را خم می‌کنیم (ابعاد را افزایش می‌دهیم) تا داده‌ها با یک خط مستقیم جدا شوند!»

گام به گام با ایده غیرخطی:

1. نگاشت به فضای جدید (Feature Mapping): الگوریتم از یک تابع ریاضیاتی به نام $\Phi(\mathbf{x})$ استفاده می‌کند تا

نقاط 2-بعدی را به یک فضای 3-بعدی پرتاب کند.

2. افزودن بُعد تعامل (Interaction Dimension): فرض کنید بُعد سوم (Z) به مختصات اضافه می‌کنیم که

مقدار آن برابر با ضرب 2 ویژگی قبلی است: $Z = X \times Y$.

3. تغییر مختصات در فضای 3-بعدی:

برای نقطه A (1, 0): مقدار $Z = 1 \times 0 = 0$

برای نقطه B (0, 1): مقدار $Z = 0 \times 1 = 0$

برای نقطه D (1, 1): مقدار $Z = 1 \times 1 = 1$. (این نقطه قرمز که مزاحم بود، اکنون در فضای 3-بعدی

به سمت بالا کشیده شده است).

4. جداسازی با یک ابرصفحه تخت (Flat Hyperplane): اکنون در این فضای 3-بعدی، می‌توانیم یک مقوای کاملاً

صاف (ابرفحه 2-بعدی) را دقیقاً از وسط فضا عبور دهیم؛ به طوری که نقاط سبز در پایین مقوا ($Z = 0$) و نقاط قرمز متقاطع در بالای مقوا ($Z = 1$) قرار گیرند.

بخش 4: اثبات ریاضیاتی ترفند هسته (Kernel Trick)

قدرت واقعی SVM در این است که برای انجام این تفکیک، نیازی ندارد داده‌ها را واقعاً و به صورت فیزیکی به فضاهای بی‌نهایت بُعدی ببرد (که حافظه کامپیوتر را منفجر کند). توابع «هسته» (مانند **Polynomial** یا **RBF Kernel**) می‌توانند فاصله و زاویه این نقاط را در آن فضای 3-بعدی، مستقیماً از روی همان مختصات 2-بعدی اولیه محاسبه کنند. معادله اصلی این ترفند جادویی به شکل زیر است:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

به عنوان مثال، جدول زیر نحوه تبدیل شدن مختصات مزاحم را با تابع $\Phi(\mathbf{x}) = (X, Y, X \times Y)$ نشان می‌دهد:

نقطه داده	مختصات 2-بعدی	بُعد تعاملی ($Z=X \times Y$)	مختصات 3-بعدی نهایی	موقعیت ارتفاعی
A	(1, 0)	0	(1, 0, 0)	پایین مقوا ($Z=0$)
B	(0, 1)	0	(0, 1, 0)	پایین مقوا ($Z=0$)
D	(1, 1)	1	(1, 1, 1)	بالای مقوا ($Z=1$)

جمع‌بندی

برای تثبیت این مفهوم انتزاعی، این سناریو را به خاطر بسپارید:

- مسئله XOR:** مانند این است که چند تیلای سبز و قرمز را روی یک میز تخت (فضای 2-بعدی) به صورت شطرنجی چیده باشیم. شما نمی‌توانید با یک چوب صاف (مدل خطی)، بین آن‌ها مرز بکشید.
- راه حل SVM غیرخطی:** میز را با یک ضربه تکان می‌دهد تا تیلایها به هوا پرتاب شوند (نگاشت به بُعد سوم). در همان کسری از ثانیه که تیلایها در هوا در ارتفاع‌های مختلف معلق هستند، یک ورقه فلزی صاف (ابرفحه) را از بین آن‌ها عبور می‌دهد.
- نتیجه‌گیری:** بدین ترتیب، بدون نیاز به کشیدن خطوط درهم و پیچیده، مشکل پیچیده‌ترین الگوهای زبانی با ظرافت هندسی حل می‌شود.

پاسخنامه مسئله 15.16: جادوی ترفند کرنل (The Magic of Kernel Trick)

مقدمه و هدف مسئله

در مسئله‌ی پیشین (15.15) دیدیم که برای حل مشکل داده‌های تفکیک‌ناپذیر خطی، باید داده‌ها را به فضایی با ابعاد بالاتر (مثلاً از 2-بعدی به 3-بعدی یا حتی بی‌نهایت‌بعدی) پرتاب کنیم تا بتوانیم آن‌ها را با یک ابرصفحه صاف جدا کنیم. با این

حال، نگرانی مدیر پروژه در این مسئله کاملاً به جا و علمی است: «انتقال فیزیکی تمام داده‌ها به ابعاد بالا، به توان پردازشی و حافظه‌ی نجومی نیاز دارد». هدف این مسئله، معرفی یکی از زیباترین مفاهیم ریاضیات در یادگیری ماشین یعنی «ترفند کرنل» (**Kernel Trick**) است که این بن‌بست محاسباتی را به شکلی معجزه‌آسا حل می‌کند.

بخش 1: ریشه‌یابی مشکل محاسباتی در نگاشت صریح

برای درک این ترفند، ابتدا باید بدانیم در قلب الگوریتم ماشین بردار پشتیبان (**SVM**) چه عملیاتی رخ می‌دهد.

هنگامی که مدل **SVM** را در فرمول‌بندی دوگان (**Dual Formulation**) آموزش می‌دهیم، الگوریتم برای پیدا کردن مرز تصمیم‌گیری، به دانستن مختصات دقیق نقاط نیازی ندارد؛ بلکه تنها چیزی که نیاز دارد، حاصل «ضرب داخلی» (**Dot Product**) بین جفت نقاط است.

اگر ما تابع نگاشت $\Phi(\mathbf{x})$ را برای انتقال داده‌ها به فضای جدید تعریف کنیم، ضرب داخلی در فضای جدید به شکل زیر خواهد بود:

$$\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

کابوس محاسباتی (نگاشت صریح):

اگر بخواهیم این کار را به صورت عادی (صریح) انجام دهیم، باید 3 مرحله را طی کنیم:

1. بردار $\mathbf{x}_i$ را با استفاده از تابع Φ به فضای جدید (مثلاً 100,000 بعدی) نگاشت کنیم.

2. بردار $\mathbf{x}_j$ را نیز به همان فضای 100,000 بعدی نگاشت کنیم.

3. حالا این 2 بردار عظیم را در هم ضرب داخلی کنیم.

اگر این کار را برای میلیون‌ها جفت داده در دیتاست انجام دهیم، سرورها از شدت بار محاسباتی و کمبود حافظه (**RAM**) از کار خواهند افتاد. در کرنل‌هایی مانند **RBF**، فضای جدید بی‌نهایت‌بعدی است و نگاشت صریح داده به فضایی با بی‌نهایت بُعد، از نظر فیزیکی غیرممکن است!

بخش 2: ترفند کرنل چیست و چگونه عمل می‌کند؟

«ترفند کرنل» یک میان‌بر ریاضی است. ریاضیدانان توابعی به نام «تابع کرنل» $K(\mathbf{x}_i, \mathbf{x}_j)$ کشف کرده‌اند که یک ویژگی شگفت‌انگیز دارند:

تابع کرنل، 2 بردار را در همان فضای اولیه (با ابعاد کم) دریافت می‌کند، یک محاسبه‌ی ریاضی ساده روی آن‌ها انجام می‌دهد و خروجی آن، دقیقاً برابر با عدد حاصل از ضرب داخلی آن 2 بردار در فضای بی‌نهایت‌بعدی است!

از نظر ریاضی، ترفند کرنل یعنی:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

با استفاده از این تابع، ما هرگز نیازی نداریم تابع Φ را محاسبه کنیم و داده‌ها را واقعاً به فضای جدید ببریم. ما فقط از میان‌بر K استفاده می‌کنیم.

پشتوانه تئوریک (قضیه مرسر - Mercer's Theorem):

برای تکمیل بحث آکادمیک، باید بدانیم که هر تابع دلخواهی نمی‌تواند به عنوان کرنل استفاده شود. قضیه مرسر اثبات می‌کند که یک تابع متقارن تنها در صورتی می‌تواند به عنوان تابع کرنل (K) استفاده شود که ماتریس کرنل تولیدشده توسط آن (ماتریس گرام)، یک ماتریس «نیمه‌معین مثبت» (**Positive Semi-Definite**) باشد. این قضیه تضمین می‌کند که فضای ویژگی متناظر با این کرنل، حتماً از نظر ریاضی وجود خارجی دارد.

بخش 3: اثبات جادو با یک مثال ریاضی ساده (کرنل چندجمله‌ای)

برای اینکه به مدیر پروژه نشان دهید این ترفند چگونه در عمل کار می‌کند، این مثال ریاضی ساده اما قدرتمند را ارائه دهید:

فرض کنید 2 نقطه در فضای 2-بعدی داریم: $\mathbf{x} = [x_1, x_2]$ و $\mathbf{y} = [y_1, y_2]$.

می‌خواهیم آن‌ها را به یک فضای 3-بعدی نگاشت کنیم. تابع نگاشت Φ به این شکل است:

$$\Phi(\mathbf{x}) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]$$

حالت اول (بدون ترفند - راه طولانی):

باید ضرب داخلی را در فضای 3-بعدی حساب کنیم:

$$\Phi(\mathbf{x})^T \Phi(\mathbf{y}) = (x_1^2 \times y_1^2) + (\sqrt{2}x_1x_2 \times \sqrt{2}y_1y_2) + (x_2^2 \times y_2^2)$$

$$= x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2$$

(این کار نیازمند تبدیل داده‌ها به 3-بعدی و سپس ضرب آن‌هاست).

حالت دوم (با ترفند کرنل - میان‌بر جادویی):

حالا از تابع کرنل چندجمله‌ای درجه 2 استفاده می‌کنیم که فرمول آن $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2$ است.

ابتدا ضرب داخلی را در همان فضای اولیه (2-بعدی) انجام می‌دهیم و سپس فقط آن را به توان 2 می‌رسانیم:

$$K(\mathbf{x}, \mathbf{y}) = (x_1y_1 + x_2y_2)^2$$

طبق اتحاد مربع دوجمله‌ای، اگر این پرانتز را باز کنیم به دست می‌آوریم:

$$= x_1^2y_1^2 + 2x_1y_1x_2y_2 + x_2^2y_2^2$$

نتیجه‌ی شگفت‌انگیز: همان‌طور که می‌بینید، خروجی هر 2 روش دقیقاً یکسان است!

اما در روش دوم، ما اصلاً نیازی به ساختن بردارهای 3-بعدی نداشتیم. فقط 2 عدد را در فضای 2-بعدی ضرب کردیم و حاصل را به توان 2 رساندیم.

بخش 4: مقایسه مهندسی و محاسباتی

جهت ارائه مستندات دقیق به مدیر پروژه در خصوص بهره‌وری منابع سیستم، جدول زیر هزینه‌های محاسباتی نگاشت صریح را با ترفند کرنل مقایسه می‌کند:

شاخص ارزیابی سیستمی	نگاشت صریح (Explicit Mapping)	ترفند کرنل (Kernel Trick)
پیچیدگی حافظه (Memory)	$\mathcal{O}(D)$ (High)	$\mathcal{O}(d)$ (Low)
پیچیدگی زمانی (Time)	$\mathcal{O}(D)$	$\mathcal{O}(d)$
قابلیت اجرای ابعاد بی‌نهایت (RBF)	کاملاً غیرممکن (کرش سیستم)	کاملاً ممکن و بهینه

جمع‌بندی

برای رفع کامل نگرانی مدیر پروژه، این نتیجه‌گیری را ارائه دهید:

- صرفه‌جویی در حافظه (Memory):** ما هیچ‌گاه ماتریس‌های عظیم مربوط به ابعاد بالاتر (صدها هزار بُعد) را در حافظه سرور نمی‌سازیم و ذخیره نمی‌کنیم. داده‌ها در همان ابعاد کم اولیه باقی می‌مانند.
- صرفه‌جویی در زمان پردازش (CPU/GPU):** پیچیدگی زمانی محاسبه ضرب داخلی، دیگر به ابعاد فضای جدید (D) بستگی ندارد، بلکه تنها به ابعاد فضای اولیه (d) وابسته است.
- تمثیل پایانی:** ترفند کرنل مانند این است که بخواهید مساحت یک زمین کشاورزی بسیار پیچیده در آن سوی دنیا را بدانید. راه اول (نگاشت صریح) این است که با هواپیما به آنجا بروید و متر به متر زمین را اندازه بگیرید. راه دوم (ترفند کرنل) این است که از طریق یک فرمول ماهواره‌ای (تابع کرنل) و مختصات نقشه، بدون اینکه حتی از اتاق خود خارج شوید، مساحت دقیق را به دست آورید. الگوریتم **SVM**، به لطف توابع کرنل، هرگز از فضای کم‌بُعد اولیه خارج نمی‌شود!

پاسخنامه مسئله 15.17: طراحی ویژگی برای ماشین امتیازده

(Feature Design for Machine Scoring)

مقدمه و هدف مسئله

گذر از توابع رتبه‌بندی کلاسیک (مانند **BM25**) به سیستم‌های «یادگیری برای رتبه‌بندی» **(Learning to Rank)** یا **(LTR)**، یک تغییر پارادایم از «فرمول‌های ثابت ریاضی» به «مدل‌های داده‌محور» است. در **LTR**، قلب تپنده‌ی سیستم، بُردار ویژگی‌ها یا **Features** است. این بردار باید بتواند تمام زوایای ارتباط یک پرس‌وجو **(Query)** با یک سند **(Document)** را در قالب اعداد

استخراج کند. هدف این مسئله، پرورش دید مهندسی دانشجو برای طراحی ویژگی‌های ترکیبی (Query-Document Features) است که محدودیت‌های سیستم‌های سنتی را پوشش دهند.

بخش 1: پیشنهاد 5 ویژگی کلیدی و منطق انتخاب آن‌ها

برای ساخت یک بردار ویژگی غنی $\mathbf{x}_{(q,d)}$ که بتواند عملکردی فراتر از **BM25** داشته باشد، 5 ویژگی زیر (شامل ویژگی‌های متنی، ساختاری و اعتباری) پیشنهاد می‌شود:

1. پوشش پرس‌وجو (Query Term Coverage / Overlap Ratio):

تعریف: نسبت تعداد کلمات منحصربه‌فرد پرس‌وجو که در سند ظاهر شده‌اند، به کل کلمات پرس‌وجو. (مثلاً اگر کاربر 3 کلمه جستجو کرده و سند فقط 2 کلمه از آن‌ها را دارد، این نسبت 0.66 خواهد بود). در بیان ریاضی به شکل زیر مدل می‌شود:

$$\text{Overlap}(q, d) = \frac{|q \cap d|}{|q|}$$

دلیل انتخاب: توابع کلاسیک مانند **BM25** گاهی فریب تکرار بیش‌ازحد یک کلمه را می‌خورند. ممکن است سندی تنها یکی از کلمات پرس‌وجو را 50 بار تکرار کرده باشد و امتیاز بالایی بگیرد، اما سندی که تمام کلمات پرس‌وجو را حداقل یک‌بار دارد، برای کاربر به مراتب مفیدتر است. این ویژگی به عنوان یک فیلتر منطقی برای تضمین حضور حداکثری مفاهیم خواسته‌شده عمل می‌کند.

2. امتیاز شباهت پایه (مانند TF-IDF Cosine Similarity):

تعریف: محاسبه شباهت هندسی (کسینوسی) بین بردار پرس‌وجو و بردار سند بر اساس وزن‌دهی **TF-IDF** (Term Frequency-Inverse Document Frequency).

$$\text{Cosine Sim}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|}$$

دلیل انتخاب: حتی در پیشرفته‌ترین سیستم‌های **LTR**، داشتن یک «لنگرگاه آماری» الزامی است. این ویژگی به مدل یادگیری ماشین کمک می‌کند تا بداند کلمات نادر (**Rare Words**) چقدر بااهمیت‌تر از کلمات رایج (**Stop Words**) هستند. مدل‌های **ML** معمولاً توابع سنتی (حتی خود امتیاز خام **BM25**) را به عنوان یکی از ویژگی‌های ورودی خود می‌پذیرند تا نقطه شروع قدرتمندی برای یادگیری داشته باشند.

3. مجاورت و فاصله کلمات (Term Proximity / Window Size):

تعریف: محاسبه کمترین فاصله‌ی بین کلمات پرس‌وجو در داخل متن سند (به دست آوردن کوچک‌ترین پنجره‌ای از کلمات که تمام عبارات جستجو را در بر می‌گیرد).

دلیل انتخاب: مدل **BM25** یک مدل «کیسه کلمات» (**Bag of Words**) است و ترتیب یا فاصله کلمات را کاملاً نادیده می‌گیرد. در حالی که از نظر معنایی، اگر کاربر عبارت «دانشگاه تهران» را جستجو کند، سندی که این 2 کلمه را دقیقاً در کنار هم آورده است، بسیار مرتبط‌تر از سندی است که کلمه «دانشگاه» را در

پاراگراف اول و «تهران» را در پاراگراف دهم آورده است. این ویژگی، نقصِ درکِ بافتار (**Context**) را در مدل‌های سنتی جبران می‌کند.

4. تراکم در مناطق باارزش ساختاری (Zone-based / Field-based Matches):

تعریف: تعداد یا فرکانسِ کلماتِ پرس‌وجو در بخش‌های کلیدی و برجسته‌ی سند متنی (مانند تگ **<title>**، سرتیترهای **<h1>** تا **<h3>**، و آدرس **URL**).

دلیل انتخاب: یک سند متنی در وب، ساختاری مسطح ندارد. کلمه‌ای که در عنوانِ یک مقاله (**Title**) ظاهر می‌شود، بسیار قوی‌تر از همان کلمه در فوتر (**Footer**) یا میانه‌ی متن، موضوعِ اصلی مقاله را نمایندگی می‌کند. با استخراج این ویژگی، به مدل ماشین یاد می‌دهیم که ارزشِ مکانیِ کلمات را درک کرده و به اسنادی که تیتِر آن‌ها تطابقِ دقیقی با پرس‌وجو دارد، وزن بیشتری بدهد.

5. اعتبار ذاتی و مستقل از پرس‌وجوی سند (Document Prior / PageRank):

تعریف: یک امتیاز مطلق و مستقل از جستجو که کیفیت، اعتبار دامنه، یا محبوبیت تاریخی آن سند را نشان می‌دهد (مانند پیج‌رنک، تعداد بک‌لینک‌ها، یا نرخ کلیکِ تاریخی **CTR**).

دلیل انتخاب: ارتباطِ متنی (**Textual Relevance**) تنها نیمی از مسیرِ رضایت کاربر است. فرض کنید کاربر کلمه «اخبار المپیک» را جستجو کرده است. 2 سند متنی کاملاً یکسان با کلمات مشابه وجود دارند؛ اما یکی متعلق به یک وبلاگ نامعتبر و دیگری متعلق به خبرگزاری رسمی ورزشی است. این ویژگی به مدل اجازه می‌دهد که در صورت تساویِ ویژگی‌های متنی، سندی را بالاتر قرار دهد که از نظر شبکه وب و کاربرانِ قبلی، «معتبرتر» و «قابل‌اعتمادتر» ارزیابی شده است.

بخش 2: طبقه‌بندی و معماری ویژگی‌ها در فضای LTR (بخش تکمیلی)

جهت درک معماری این ویژگی‌ها و ادغام آن‌ها درون یک بُردار $\mathbf{x}$ ، طبقه‌بندی استاندارد آن‌ها در مهندسی **LTR** را در جدول زیر تدوین نموده‌ایم:

شناسه ویژگی	دسته‌بندی (Category)	خروجی ریاضی	چالشِ برطرف‌شده
پوشش پرس‌وجو	وابسته به پرس‌وجو (Query-Dependent)	Ratio [0.0, 1.0]	فریب تکرار بی‌مورد واژه
امتیاز TF-IDF	وابسته به پرس‌وجو	Continuous $\mathbb{R}^+$	مدیریت کلمات نایاب
فاصله کلمات	وابسته به پرس‌وجو	Integer Window Size	فقدان درک بافتار (Context)
تطابق مناطق	ساختاری متنی	Frequency Counts	ارزش یکسان تمام نواحی متن
اعتبار ذاتی	مستقل از پرس‌وجو (Query-Independent)	Score / Probabilities	ضعف در تفکیک کیفیت منابع

جمع‌بندی

طراحی ویژگی، هنر ترجمه‌ی «شهود انسانی» به «زبان ماشین» است. توابع سنتی مانند **BM25**، مانند یک ترازوی تک‌کفه‌ای هستند که فقط وزن کلمات (فراوانی) را اندازه می‌گیرند. اما سیستمی که ما در این مسئله طراحی کردیم، مانند یک تیم ارزیابی همه‌جانبه است:

- **ویژگی‌های 1 و 2** $\Leftarrow$ بررسی می‌کنند که آیا کلمات در متن وجود دارند و چقدر نایاب‌اند؟
 - **ویژگی 3** $\Leftarrow$ بررسی می‌کند که این کلمات چگونه و با چه فاصله‌ای در کنار هم نشسته‌اند؟
 - **ویژگی 4** $\Leftarrow$ بررسی می‌کند که این کلمات کجای سند پنهان شده‌اند (در ویتترین تیترا یا در انبار متن)؟
 - **ویژگی 5** $\Leftarrow$ تضمین می‌کند سندی که به کاربر معرفی می‌شود، صرف‌نظر از متنش، قابل‌اعتماد و معتبر باشد.
- ترکیب این 5 بُعد، به سادگی هر الگوریتم سنتی را در سیستم‌های جستجوی واقعی شکست خواهد داد.

پاسخنامه مسئله 15.18: انتخاب نوع طبقه‌بند در عمل (Choosing Classifier in Practice)

مقدمه و هدف مسئله

در پروژه‌های واقعی بازایی اطلاعات و یادگیری ماشین، هیچ الگوریتمی وجود ندارد که در تمام شرایط «بهترین» باشد. انتخاب الگوریتم مناسب، وابستگی شدیدی به حجم داده‌ها (تعداد اسناد) و محدودیت‌های محاسباتی دارد. هدف از این مسئله، آموزش مهارت تصمیم‌گیری مهندسی بر اساس راهنماهای عملی (مبتنی بر بخش 15.3.1 کتب مرجع **IR**) در رویارویی با سناریوهای مختلف توزیع داده است.

بخش 1: تصمیم‌گیری برای مجموعه داده کوچک (500 سند)

برای یک مجموعه داده کوچک با کیفیت بالا، پیشنهاد قطعی و مهندسی‌شده، استفاده از الگوریتم ماشین بردار پشتیبان خطی (**Linear SVM**) است.

دلایل علمی و ساختاری برای این انتخاب:

1. مقاومت در برابر بیش‌برازش (Overfitting):

در پردازش متن، فضای ویژگی‌ها (تعداد کلمات) معمولاً بسیار بزرگ است (مثلاً ده‌ها هزار بُعد). وقتی تنها 500 سند داریم، در وضعیت بحرانی کمبود داده نسبت به ابعاد ($p \gg n$) قرار می‌گیریم. الگوریتم **SVM** به لطف اصل «حداکثرسازی حاشیه هندسی»، به جای حفظ کردن تمام کلمات، تنها روی نقاط مرزی (بردارهای پشتیبان) تمرکز می‌کند و بالاترین قدرت تعمیم‌پذیری را در داده‌های کم‌حجم ارائه می‌دهد.

2. شکست KNN در فضای خلوت و پربُعد:

الگوریتم **KNN** (نزدیک‌ترین همسایه) در این سناریو بدترین انتخاب است. این الگوریتم بر اساس محاسبه فاصله هندسی (مانند فاصله اقلیدسی) کار می‌کند. در متون، به دلیل خلوت بودن شدید بردارها (**Sparsity**) و پدیده «نفرین ابعاد» (**Curse of Dimensionality**)، مفهوم فاصله در ابعاد بالا بی‌معنا می‌شود و تمام اسناد تقریباً فاصله‌ای یکسان از یکدیگر پیدا می‌کنند که منجر به افت شدید دقت خواهد شد.

3. برتری ساختاری بر Naive Bayes:

اگرچه **Naive Bayes** یک مدل پایه (**Baseline**) خوب است، اما فرض «استقلال کلمات» در آن باعث می‌شود

نتواند الگوهای ترکیبی را به خوبی یاد بگیرد. در حجم داده کم اما باکیفیت، **SVM** می‌تواند مرزهای تصمیم‌گیری بسیار دقیق‌تری نسبت به احتمالات ساده‌ی **Naive Bayes** رسم کند.

بخش 2: تصمیم‌گیری در صورت افزایش داده‌ها به 500,000 سند

اگر حجم داده‌ها به شکل چشمگیری افزایش یابد (مثلاً نیم میلیون سند)، پارادایم تصمیم‌گیری تغییر می‌کند و پیشنهاد اول ما به الگوریتم **Naive Bayes** (یا رده‌بندهای خطی با بهینه‌سازی تصادفی) تغییر خواهد یافت.

دلایل تغییر استراتژی در مقیاس کلان:

1. همگرایی دقت (Asymptotic Accuracy):

در یادگیری ماشین، یک اصل تجربی وجود دارد که می‌گوید: «داده‌ی بیشتر، الگوریتم بهتر را شکست می‌دهد». وقتی حجم داده‌ها بسیار عظیم می‌شود، تفاوت دقت پیش‌بینی بین **Linear SVM** و **Naive Bayes** به شدت کاهش یافته و تقریباً به یکدیگر همگرا می‌شوند. بنابراین، برتری دقتی **SVM** در این مقیاس رنگ می‌بازد.

2. پیچیدگی زمانی آموزش (Training Time Complexity):

این مهم‌ترین عامل تغییر تصمیم است. الگوریتم **Naive Bayes** دارای پیچیدگی زمانی خطی، یعنی $O(N)$ است. آموزش این مدل روی 500,000 سند، صرفاً نیازمند یک بار شمارش فرکانس کلمات (پیمایش داده‌ها) است که در چند ثانیه انجام می‌شود.

در مقابل، آموزش الگوریتم **SVM** استاندارد نیازمند حل یک مسئله بهینه‌سازی پیچیده با پیچیدگی زمانی حداقل $O(N^2)$ است. اجرای این محاسبات روی نیم میلیون سند، نیازمند زمان محاسباتی طولانی و حافظه **RAM** بسیار عظیمی است که توجیه اقتصادی و مهندسی ندارد (مگر آنکه از نسخه‌های تقریبی و پیشرفته‌تر مانند **SGD-SVM** استفاده شود).

3. قابلیت بروزرسانی آنلاین (Online Learning):

در سیستم‌های خبری یا شبکه‌های اجتماعی که روزانه اسناد جدیدی تولید می‌شوند، مدل **Naive Bayes** به راحتی قابلیت بروزرسانی دارد (فقط کافی است شمارنده‌ی کلمات را با اسناد جدید آپدیت کنیم). اما در **SVM**، با ورود داده‌های جدید، کل مرز تصمیم‌گیری باید از نو محاسبه و بهینه‌سازی شود.

بخش 3: جمع‌بندی و مقایسه جامع الگوریتم‌ها

ویژگی ارزیابی سیستم	Linear SVM	Naive Bayes	KNN
دقت در داده کم (< 1,000)	بسیار بالا (بهترین انتخاب)	متوسط	بسیار ضعیف
سرعت آموزش در داده عظیم	کند (نیازمند بهینه‌سازی ریاضی)	بسیار سریع (خطی - فقط شمارش)	نیازی به آموزش ندارد (Lazy)
سرعت پیش‌بینی (Inference)	بسیار سریع	بسیار سریع	بسیار کند (مقایسه با کل داده‌ها)
مقاومت در برابر ابعاد بالا	عالی (توسط حاشیه هندسی)	خوب	ضعیف (نفرین ابعاد)

نتیجه‌گیری: مهندسی ماشین لرنینگ، هنر ایجاد تعادل میان دقت و هزینه است. انتخاب الگوریتم نه یک قانون مطلق، بلکه تابعی متغیر از مقیاس داده، امکانات سخت‌افزاری سرور و نیازهای زمان-اجرای (**Run-time**) پروژه است.

پاسخنامه مسئله 15.19: چالش وزن‌دهی نامتقارن (Asymmetric Weighting Challenge)

مقدمه و هدف مسئله

در مسائل استاندارد طبقه‌بندی، فرض بر این است که هزینه و جریمه‌ی تمام اشتباهات مدل با هم برابر است. اما در دنیای واقعی و سیستم‌های حساس (مانند فیلتر محتوای کودکان یا تشخیص بیماری‌ها)، با مفهومی به نام «یادگیری حساس به هزینه» (**Cost-Sensitive Learning**) روبرو هستیم. در این مسئله، فرض می‌کنیم:

کلاس مثبت (+1): محتوای نامناسب (هدف شناسایی)

کلاس منفی (-1): محتوای سالم

در این سناریو، خطای مثبت کاذب (**False Positive**) یعنی محتوای سالمی که مسدود شده است (هزینه کم و قابل چشم‌پوشی). اما خطای منفی کاذب (**False Negative**) یعنی محتوای نامناسبی که سالم تشخیص داده شده و به دست کودک رسیده است (هزینه فاجعه‌بار).

هدف این مسئله، آموزش تکنیک‌های مهندسی برای دستکاری الگوریتم **SVM** جهت به حداقل رساندن خطای فاجعه‌بار (منفی کاذب) در 2 فاز مختلف آموزش و پیش‌بینی است.

بخش 1: استفاده از پارامتر جریمه نامتقارن

در الگوریتم پایه حاشیه نرم (**Soft Margin SVM**)، یک پارامتر جریمه کلی به نام C داریم که تمام خطاها را به یک اندازه جریمه می‌کند. برای حل چالش محتوای کودکان، ما ساختار تابع هدف ریاضی را تغییر داده و از «وزن‌دهی نامتقارن کلاس‌ها» (**Asymmetric Class Weighting**) استفاده می‌کنیم.

1. تفکیک پارامتر جریمه:

به جای یک پارامتر، 2 پارامتر مستقل تعریف می‌کنیم:

C^+ : جریمه‌ی عبور کردن داده‌های کلاس مثبت (محتوای نامناسب) از حریم حاشیه.

C^- : جریمه‌ی عبور کردن داده‌های کلاس منفی (محتوای سالم) از حریم حاشیه.

2. تغییر تابع هدف:

معادله بهینه‌سازی **SVM** به شکل زیر بازنویسی می‌شود:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C^+ \sum_{y_i=+1} \xi_i + C^- \sum_{y_i=-1} \xi_i \right)$$

(نکته ریاضی: در فرمول‌بندی دوگان یا *Dual Formulation*، این تغییر باعث می‌شود که کران بالای ضرایب لاگرانژ α_i برای نقاط

کلاس مثبت به C^+ و برای کلاس منفی به C^- محدود شود، که مستقیماً روی وزن‌دهی بردارهای پشتیبان در هر کلاس اثر

3. استراتژی تنظیم و اثر هندسی آن:

برای اینکه سیستم از خطای **False Negative** به شدت بترسد، ما مقدار C^+ را بسیار بزرگ‌تر از C^- تنظیم می‌کنیم ($C^+ \gg C^-$).

اثر ریاضی: الگوریتم متوجه می‌شود که اشتباه کردن روی داده‌های مثبت، هزینه محاسباتی بسیار سرسام‌آوری در تابع هدف دارد.

اثر هندسی: برای فرار از این جریمه سنگین، الگوریتم ابرصفحه جداکننده (**Hyperplane**) را تا جای ممکن از داده‌های کلاس مثبت دور کرده و به سمت قلمرو کلاس منفی هل می‌دهد. با این کار، حریم امن بزرگی در اطراف محتوای نامناسب ایجاد می‌شود تا هیچ محتوای خطرناکی به اشتباه از مرز عبور نکند.

بخش 2: جابه‌جایی آستانه تصمیم‌گیری

گاهی اوقات ما به مدل آموزش‌دیده دسترسی نداریم یا نمی‌خواهیم فاز زمان‌بر آموزش را تکرار کنیم. در این حالت، می‌توانیم رفتار مدل را در لحظه‌ی استفاده (**Inference Phase**) با تکنیک «جابه‌جایی آستانه» (**Threshold Shifting**) تغییر دهیم.

1. آستانه استاندارد:

در یک **SVM** معمولی، خروجی تابع تصمیم به صورت $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ محاسبه می‌شود. قانون تصمیم‌گیری استاندارد می‌گوید اگر $f(\mathbf{x}) > 0$ باشد، متن نامناسب است و اگر $f(\mathbf{x}) < 0$ باشد، متن سالم است. در اینجا عدد 0، آستانه (**Threshold**) است.

2. تغییر آستانه برای محافظت بیشتر:

برای کاهش خطای **False Negative**، ما این آستانه را از 0 به یک عدد منفی (مثلاً -0.5) تغییر می‌دهیم. قانون تصمیم‌گیری جدید: اگر $f(\mathbf{x}) > -0.5$ بود، متن را نامناسب در نظر بگیر.

3. منطق این کار:

با این کار، ما به سیستم می‌گوییم: «حتی اگر مدل با قطعیت پایین فکر می‌کند متن سالم است (مثلاً خروجی -0.2 شده است)، تو ریسک نکن و آن را نامناسب فرض کن!»

این تکنیک نیازی به آموزش مجدد ندارد و به سادگی مرز تصمیم را به نفع کلاس حساس جابه‌جا می‌کند. (این جابه‌جایی آستانه، در واقع حرکت روی منحنی **ROC** یا منحنی **Precision-Recall** است تا یک نقطه کارکرد (**Operating Point**) بهینه‌تر برای شرایط حساس پیدا شود).

بخش 3: مقایسه تحلیلی رویکردهای مدیریت هزینه

تکنیک مقابله با عدم تقارن هزینه	مکانیزم اجرا	نیاز به آموزش مجدد	مزیت اصلی	محدودیت اصلی
وزن‌دهی نامتقارن کلاس‌ها	تغییر تابع هدف و تفکیک ضرایب C^+ و C^-	بله	مرز تصمیم‌گیری ذاتاً و به صورت ساختاری بهینه می‌شود.	هزینه محاسباتی بالا برای تنظیم پارامترها و آموزش مجدد.
جابه‌جایی آستانه تصمیم	تغییر نقطه 0 در $f(\mathbf{x})$ به سمت اعداد منفی	خیر	اعمال فوری و بدون نیاز به صرف منابع محاسباتی.	عدم تغییر ساختار هندسی مدل (صرفاً تغییر نقطه برش).

طراحی سیستم‌های طبقه‌بندی در دنیای واقعی، همواره نیازمند ایجاد تعادل (**Trade-off**) بین معیارهاست. در یک سیستم محافظت از کودکان:

- هدف اصلی ما:** به حداکثر رساندن معیار **بازیابی (Recall)** برای محتوای نامناسب است؛ یعنی پیدا کردن تک تک محتوای خطرناک.
- هزینه‌ی پرداختی:** با اجرای هر یک از 2 روش بالا (چه تغییر C در آموزش و چه تغییر آستانه در پیش‌بینی)، سیستم سخت‌گیرتر می‌شود. بهای این سخت‌گیری این است که خطای مثبت کاذب (**False Positive**) بالا می‌رود؛ یعنی محتوای سالم بیشتری به اشتباه فیلتر می‌شوند (کاهش **Precision**).
- توجیه مهندسی:** از نظر مهندسی محصول، مسدود شدن اشتباه یک مقاله علمی درباره‌ی زیست‌شناسی، بهای کاملاً قابل قبول و ناچیزی برای جلوگیری از نمایش یک محتوای مخرب به یک کودک است. الگوریتم باید در خدمت منطق کسب و کار و ارزش‌های انسانی تنظیم شود.

پاسخنامه مسئله 15.20: رگرسیون در برابر طبقه‌بندی در رتبه‌بندی

(Regression vs. Classification in Ranking)

مقدمه و هدف مسئله

در طراحی یک سیستم «یادگیری برای رتبه‌بندی» (**Learning to Rank**) با رویکرد نقطه‌ای (**Pointwise**)، هدف نهایی ما مرتب‌سازی اسناد برای نمایش به کاربر است. اما برای آموزش مدل یادگیری ماشین، باید نوع خروجی هدف (**Target Variable**) را تعریف کنیم. در این مسئله، 2 رویکرد بنیادین برای فرمول‌بندی خروجی بررسی می‌شوند: رویکرد طبقه‌بندی (خروجی‌های گسسته و دسته‌ای) و رویکرد رگرسیون (خروجی‌های پیوسته و عددی). هدف، کالبدشکافی مزایا و معایب هر روش از منظر مهندسی بازیابی اطلاعات است.

بخش 1: فرمول‌بندی به عنوان یک مسئله «طبقه‌بندی» (Classification)

در این رویکرد، ما میزان ارتباط سند به پرس‌وجو را به دسته‌های مشخص و گسسته تقسیم می‌کنیم (مثلاً دسته‌بندی باینری: 1=مرتبط، 0=نامرتبط؛ یا دسته‌بندی چندکلاسه‌ای: عالی، خوب، متوسط، ضعیف).

مزایا (Pros):

1. سهولت در تولید داده‌های آموزشی (Annotation):

برای داوران انسانی (**Assessor**)، تصمیم‌گیری 0 و 1 (مرتبط/نامرتبط) یا انتخاب از بین چند برچسب کیفی، بسیار ساده‌تر و کم‌خطاتر از اختصاص دادن یک نمره دقیق و اعشاری به یک سند است.

2. مقاومت در برابر مقیاس (Scale Invariance):

مدل‌های طبقه‌بندی درگیر تفاوت‌های عددی و فاصله‌های دقیق ریاضی نمی‌شوند؛ بلکه فقط تلاش می‌کنند مرزی بین دسته‌ها بکشند. این امر باعث می‌شود مدل تحت تأثیر داده‌های نویزی یا پرت (**Outliers**) قرار نگیرد.

معایب (Cons):

1. فاجعه‌ی «گره‌خوردگی رتبه‌ها» (The Problem of Ties):

این بزرگ‌ترین نقص طبقه‌بندی در رتبه‌بندی است. فرض کنید کاربر کلمه‌ای را جستجو می‌کند و 1000 سند به عنوان «مرتبط» (کلاس 1) طبقه‌بندی می‌شوند. موتور جستجو هیچ ایده‌ای ندارد که در داخل این کلاس 1، کدام سند بهتر از بقیه است تا آن را در رتبه اول نمایش دهد. خروجی طبقه‌بندی خالص، ترتیب (Order) تولید نمی‌کند. (نکته تکمیلی: برای رفع این مشکل معمولاً به جای خروجی قطعی کلاس، از "احتمال تعلق به کلاس" استفاده می‌شود تا یک تابع امتیازدهی پیوسته شبیه‌سازی شود).

2. نادیده گرفتن ماهیت ترتیبی (در حالت چندکلاسه‌ای):

اگر از طبقه‌بندهای استاندارد (مانند SVM چندکلاسه‌ای یا Naive Bayes) استفاده کنیم، الگوریتم درک نمی‌کند که کلاس «عالی» بهتر از «خوب» و «خوب» بهتر از «ضعیف» است. الگوریتم آن‌ها را صرفاً به عنوان دسته‌هایی مستقل و بی‌ارتباط (مثل دسته‌های سیب، پرتقال و موز) می‌بیند و تابع هزینه آن هیچ تفاوتی بین اشتباه گرفتن «عالی» با «خوب» یا «عالی» با «ضعیف» قائل نمی‌شود.

بخش 2: فرمول‌بندی به عنوان یک مسئله «رگرسیون» (Regression)

در این رویکرد، مدل یک تابع $f(x) = y$ را یاد می‌گیرد که در آن y یک عدد حقیقی و پیوسته (مثلاً 2.754) است.

مزایا (Pros):

1. ایجاد یک لیست کاملاً مرتب (Total Order):

برخلاف طبقه‌بندی، خروجی رگرسیون مجموعه‌ای از اعداد اعشاری پیوسته است. احتمال اینکه 2 سند دقیقاً یک نمره اعشاری یکسان (مثلاً 3.14159) بگیرند، تقریباً صفر است. بنابراین، سیستم می‌تواند به راحتی اسناد را از بزرگ‌ترین عدد به کوچک‌ترین عدد مرتب کند و یک لیست نهایی دقیق و بدون گره‌خوردگی (Ties) تحویل دهد.

2. تمایزات ظریف و دقیق (Fine-grained Distinctions):

رگرسیون می‌تواند تفاوت‌های بسیار جزئی بین 2 سند نسبتاً مشابه را مدل‌سازی کرده و به یکی امتیاز 3.8 و به دیگری 3.6 بدهد.

معایب (Cons):

1. تحمیل یک مقیاس خطی اشتباه (False Linear Scale):

در داده‌های آموزشی، ما به اسناد نمراتی مثل 0، 1، 2 و 3 می‌دهیم. الگوریتم رگرسیون (که تلاش می‌کند خطای میانگین مربعات یا MSE را کمینه کند) فرض می‌کند که «فاصله‌ی کیفی بین نمره 3 و 2» دقیقاً برابر با «فاصله‌ی کیفی بین نمره 1 و 0» است. در ارزیابی زبان انسانی، این فرض کاملاً غلط است. تفاوت بین یک سند «عالی» و «خوب»، بسیار متفاوت از تفاوت بین یک سند «ضعیف» و «فاجعه» است.

2. حساسیت شدید به ناهنجاری‌ها (Outliers):

اگر به سندی نمره هدف 4 داده شده باشد اما مدل پیش‌بینی کند که نمره آن 0 است، جریمه‌ی آن در رگرسیون بسیار سنگین (مربع خطا = 16) خواهد بود. این باعث می‌شود مدل در فرآیند گرادیان کاهشی (Gradient Descent)، خط رگرسیون خود را به شدت کج کند تا این اشتباه را جبران نماید، که در نتیجه دقت کلی مدل برای سایر اسناد نرمال کاهش می‌یابد.

بخش 3: مقایسه جامع استراتژی‌های خروجی در LTR

رویکرد مدل سازی	ماهیت خروجی	مزیت اصلی	نقطه ضعف اساسی	تابع هزینه رایج
طبقه بندی قطعی	برچسب های گسسته (0, 1, 2)	مقاوم به نویز (Robust)	گره خوردگی شدید رتبه ها	Cross-Entropy
رگرسیون خطی	اعداد پیوسته ($\mathbb{R}$)	تولید لیست مرتب (بدون گره)	حساسیت بالا به Outlier ها	MSE / RMSE
طبقه بندی احتمالاتی	احتمال تعلق به کلاس [0, 1]	ترکیب دقت طبقه بندی و ترتیب سازی	عدم درک فاصله بین کلاس های کیفی	Log-Loss
رگرسیون ترتیبی	دسته های دارای ارزش گذاری مرتب	کامل ترین انطباق با زبان انسانی	پیچیدگی بالای پیاده سازی	Ordinal Loss

جمع بندی

اگر مجبور باشیم در مدل ارزیابی نقطه ای (**Pointwise**) تنها یکی از این 2 را انتخاب کنیم:

- توصیه نهایی:** استفاده از طبقه بندی احتمالاتی (**Probabilistic Classification**) مانند رگرسیون لجستیک (**Logistic Regression**) است.

چرا؟ این روش در فاز آموزش، مزایای طبقه بندی (مانند درک مرزهای دوگانه و مقاومت در برابر مقیاس خطی) را دارد؛ اما در فاز پیش بینی، به جای یک برچسب خشک و خالی، به ما احتمال (یک عدد پیوسته بین 0 و 1) می دهد (مثلاً با 92% احتمال این سند مرتبط است). ما از این عدد پیوسته برای رتبه بندی (مانند رگرسیون) استفاده می کنیم و مشکل گره خوردگی حل می شود.

- نکته طلایی:** در سیستم های پیشرفته ی امروزی، به جای این 2 رویکرد کلاسیک، از راهکار سومی به نام رگرسیون ترتیبی (**Ordinal Regression**) استفاده می شود (مانند مدل های مبتنی بر **PRank**). این روش، دقیقاً نقطه تلاقی مزایای هر 2 رویکرد است؛ یعنی می داند که خروجی ها طبقه بندی شده و دسته ای هستند، اما هم زمان درک می کند که بین این دسته ها، یک «ترتیب و ارزش گذاری ریاضی» وجود دارد (بدون اینکه فواصل آن ها را برابر فرض کند).

پاسخنامه مسئله 15.21: تحلیل گزارش تجربی و مقایسه کرنل ها در فضای متنی

مقدمه و هدف مسئله

گزارش های تجربی در طبقه بندی متن همواره یک حقیقت جالب را نشان می دهند: کرنل های غیرخطی پیچیده (مانند کرنل پایه شعاعی یا **RBF**) با وجود قدرت ریاضی شگفت انگیزشان، در مواجهه با متون طولانی نه تنها دقت را افزایش نمی دهند، بلکه زمان پردازش را فاجعه بار می کنند. هدف از این مسئله، کالبدشکافی این پدیده از منظر هندسه فضاهای پربعد (**High-dimensional Geometry**) و درک چرایی کفایت مرزهای خطی در پردازش زبان طبیعی است.

بخش 1: هندسه متون در فضاهای با ابعاد بسیار بالا

برای درک این پدیده، ابتدا باید شکل هندسی داده های متنی را در فضای ویژگی ها (**Feature Space**) ترسیم کنیم. هر کلمه مستقل در لغت نامه (**Vocabulary**)، یک بُعد (**Dimension**) در فضای هندسی ایجاد می کند. بنابراین، یک مجموعه داده متنی معمولاً در فضایی با ده ها یا صدها هزار بُعد تعریف می شود.

1. پراکندگی شدید و خلوتی (Sparsity):

در فضایی با 100,000 بُعد، هر سند متنی (که مثلاً شامل 500 کلمه است) به صورت نقطه‌ای نمایش داده می‌شود که بیش از 99.5 درصد مختصات آن دقیقاً برابر با عدد صفر است. این یعنی نقاط داده در این فضای بی‌نهایت وسیع، به شدت از یکدیگر دور و پراکنده هستند.

2. قضیه پوشش (Cover's Theorem):

یک اصل بنیادین در یادگیری ماشین (قضیه توماس کاور) بیان می‌کند که وقتی الگوها و داده‌ها به فضایی با ابعاد بسیار بالا پرتاب می‌شوند، احتمال اینکه به صورت «خطی تفکیک‌پذیر» (**Linearly Separable**) درآیند، به شدت افزایش می‌یابد. در فضای متنی، چون داده‌ها ذاتاً در ابعاد بسیار بالا تولید شده و فضای خالی بین آن‌ها بسیار زیاد است، تقریباً همیشه می‌توان یک ابرصفحه‌ی کاملاً صاف (**Flat Hyperplane**) از بین آن‌ها عبور داد.

بخش 2: چرا کرنل خطی (Linear Kernel) کافی است؟

با توجه به هندسه فوق، کرنل خطی بهترین انتخاب مهندسی است زیرا:

درجه آزادی کافی (Degrees of Freedom): یک خط ساده در فضای 2-بعدی محدودیت‌های زیادی برای جدا کردن نقاط دارد، اما یک ابرصفحه‌ی خطی در فضای 100,000 بعدی دارای 100,000 درجه آزادی (متغیر جهت‌دهی) برای چرخش و تنظیم شدن است. این انعطاف‌پذیری عظیم به ابرصفحه خطی اجازه می‌دهد تا بدون نیاز به هیچ‌گونه انحنا، خمیدگی یا پیچ‌خوردگی، کلاس‌های مثبت و منفی را به طور کامل و بی‌نقص از هم تفکیک کند.

بخش 3: چرا کرنل RBF در متن شکست می‌خورد (زمان و دقت)؟

کرنل **RBF** با استفاده از تابع نمایی $\exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ داده‌ها را به فضایی با ابعاد بی‌نهایت پرتاب می‌کند تا مرزهای منحنی و غیرخطی پیچیده بسازد. تحمیل این ساختار بر داده‌های متنی پیامدهای مخرب زیر را دارد:

1. خطر بیش‌برازش در مرزهای پیچیده (Overfitting Risk):

وقتی داده‌ها از قبل (در فضای خطی) تفکیک‌پذیر و دور از هم هستند، استفاده از یک مرز غیرخطی مانند **RBF**، مدل را گمراه می‌کند. الگوریتم به جای یادگیری الگوی کلی کلمات، شروع به کشیدن مرزهای منحنی و پیچیده به دور تک‌نویزها، غلط‌های املائی و کلمات استثنایی در داده‌های آموزشی می‌کند. به همین دلیل، مدل در مواجهه با داده‌های جدید (**Test Data**) نمی‌تواند تفاوت معناداری ایجاد کند و حتی گاهی به دلیل **Overfitting**، دقت آن نسبت به مدل خطی افت می‌کند.

2. فاجعه محاسباتی و از بین رفتن خلوتی ماتریس:

کرنل خطی که بر پایه ضرب داخلی $(\mathbf{x}_i^T \mathbf{x}_j)$ کار می‌کند، با ماتریس‌های خلوت بسیار سازگار است؛ ضرب هر عدد در 0 برابر 0 می‌شود و الگوریتم با سرعت بالا از روی صفرهای ماتریس پرش می‌کند. در مقابل، در تابع کرنل **RBF** محاسبه فاصله اقلیدسی $(\|\mathbf{x}_i - \mathbf{x}_j\|^2)$ بین تمامی جفت‌اسناد الزامی است. این عملیات تفریق و به توان رساندن، مزیت «خلوت بودن» (**Sparsity**) را از بین می‌برد و ماتریس کرنل به یک ماتریس کاملاً متراکم و چگال (**Dense**) تبدیل می‌شود. محاسبه و حل مسئله بهینه‌سازی روی چنین ماتریس عظیمی، پیچیدگی زمانی و مصرف حافظه را به شدت بالا برده و زمان آموزش را ده‌ها تا صدها برابر افزایش می‌دهد.

3. سربار جستجوی ابرپارامترها (Hyperparameter Tuning Burden)

در کرنل خطی، مهندس ماشین‌لرنینگ تنها نیازمند تنظیم یک پارامتر جریمه (C) است. اما کرنل **RBF** نیازمند

تنظیم همزمان دو پارامتر به شدت حساس است: C و شعاع کرنل (γ). جستجوی شبکه‌ای (Grid Search) روی فضای 2-بعدی این پارامترها در مجموعه داده‌های متنی بزرگ، زمان آزمایشات و اعتبارسنجی متقابل (Cross-Validation) را به شکل تصاعدی غیرمعقولی طولانی می‌کند.

بخش 4: مقایسه جامع کرنل‌ها در بستر پردازش زبان طبیعی

به منظور نتیجه‌گیری عملیاتی، جدول زیر به طور مشخص عملکرد این 2 کرنل را منحصراً در محیط‌های پربُعد و متنی مقایسه می‌نماید:

شاخص ارزیابی عملیاتی	کرنل خطی (Linear Kernel)	کرنل شعاعی (RBF Kernel)
بهره‌برداری از Sparsity	بسیار عالی (جهش از روی صفرها)	تخریب کامل (تولید ماتریس متراکم)
ریسک Overfitting	پایین (به دلیل محدودیت به مرز صاف)	بسیار بالا (حفظ نویزها در مرز منحنی)
زمان محاسبات (Training)	مقرون‌به‌صرفه و سریع	فاجعه‌بار در حجم بالای داده‌ها
تنظیم ابرپارامترها	ساده (فقط نیاز به جستجوی C)	پیچیده (فضای جستجوی 2-بعدی C, γ)

جمع‌بندی

برای تثبیت این تفاوت در طراحی سیستم‌های عملیاتی، این تمثیل هندسی را به خاطر بسپارید:

- فضای کم‌بُعد:** وقتی در یک اتاق کوچک و شلوغ (ویژگی‌های کم، مانند اطلاعات یک بیمار با 5 پارامتر خونی) هستید، برای عبور از بین افراد باید مسیرتان را دائماً **کج و راست** کنید. در اینجا کرنل **RBF** و مرزهای منحنی به شدت نیاز است.
- فضای پربُعد (متن):** وقتی در یک بیابان بی‌انتهای (فضای متنی با صدها هزار کلمه) هستید و افراد (اسناد) کیلومترها از هم فاصله دارند، برای جدا کردن 2 گروه از آدم‌ها، کافی است یک **خط کاملاً صاف و مستقیم** روی زمین بکشید.
- نتیجه‌گیری:** کشیدن خطوط مارپیچ و حصارهای پیچیده در بیابان (استفاده از **RBF** در متن)، چیزی جز اتلاف شدید زمان، هدررفت انرژی پردازشی و سردرگمی سیستم به همراه نخواهد داشت.

پاسخنامه مسئله 15.22: مهندسی ویژگی‌های ساختاری (Structural Feature Engineering)

مقدمه و هدف مسئله

در نسل اول موتورهای جستجو، رتبه‌بندی اسناد صرفاً بر اساس «ارتباط متنی» (مانند فراوانی کلمات و **TF-IDF**) انجام می‌شد. اما با گسترش وب، مشکل اسپم و صفحات بی‌کیفیت نمایان شد. موتورهای جستجوی مدرن متوجه شدند که باید بین «ارتباط» (**Relevance**) و «اعتبار» (**Authority**) تفاوت قائل شوند. الگوریتم‌هایی مانند **PageRank** و **HITS** اعتبار یک صفحه را بر اساس ساختار گراف لینک‌های وب محاسبه می‌کنند.

هدف این مسئله، درک چگونگی ادغام این 2 دنیای متفاوت (متن و گراف وب) در قالب یک مدل واحد با استفاده از رویکرد «یادگیری برای رتبه‌بندی» (**Learning to Rank**) است.

بخش 1: چالش فرمول‌های سنتی در ترکیب ویژگی‌ها

در سیستم‌های بازیابی اطلاعات کلاسیک، مهندسان سعی می‌کردند امتیاز متنی و امتیاز گرافی را به صورت دستی و با فرمول‌های ابتکاری (**Heuristic**) ترکیب کنند. مثلاً فرمولی شبیه به این می‌نوشتند:

$$\text{Final Score} = (\alpha \times \text{BM25}) + ((1 - \alpha) \times \text{PageRank})$$

مشکلات این رویکرد:

1. تفاوت مقیاس (**Scale Mismatch**):

امتیاز **TF-IDF** یا **BM25** معمولاً یک عدد بزرگ و بدون سقف است، در حالی که **PageRank** یک احتمال (بین 0 و 1) است. جمع کردن این 2 عدد به صورت مستقیم از نظر ریاضی بی‌معناست و همواره ویژگی دارای مقیاس بزرگ‌تر، تأثیر ویژگی کوچک‌تر را می‌بلعد.

2. تنظیم دستی پارامترها:

پیدا کردن وزن دقیق α نیازمند آزمون و خطای انسانی است. اگر ده‌ها ویژگی مختلف داشته باشیم (مانند **HITS**، طول **URL**، سن دامنه)، تنظیم دستی ضرایب برای انسان کاملاً غیرممکن و غیرمقیاس‌پذیر می‌شود.

بخش 2: راهکار یادگیری ماشین (بردار ویژگی‌های یکپارچه)

یادگیری ماشین این مشکل را با تغییر زاویه دید به طور کامل حل می‌کند. در این پارادایم، ما دیگر فرمول دستی نمی‌نویسیم؛ بلکه تمام اطلاعات را به عنوان مؤلفه‌های یک «بردار ویژگی» (**Feature Vector**) به مدل می‌دهیم و اجازه می‌دهیم الگوریتم، وزن بهینه را بر اساس داده‌های آموزشی (مبتنی بر رفتار کاربر) پیدا کند.

مراحل مهندسی ویژگی (**Feature Engineering**) در این رویکرد:

1. استخراج ویژگی‌های متنی (**Textual Features**):

ابتدا ویژگی‌های وابسته به پرس‌وجو محاسبه می‌شوند. این ویژگی‌ها معمولاً در قالب یک ماتریس خلوت (**Sparse**) هستند.

x_1 : امتیاز **TF-IDF** برای کلمات کوئری

x_2 : امتیاز **BM25**

2. استخراج ویژگی‌های ساختاری و گرافی (**Graph Features**):

سپس ویژگی‌های مستقل از پرس‌وجو (**Query-Independent**) که از گراف وب استخراج شده‌اند، محاسبه می‌شوند. این مقادیر برای هر سند ثابت و چگال (**Dense**) هستند.

x_3 : نمره **PageRank** سند (نشان‌دهنده اعتبار عمومی صفحه)

x_4 : نمره **Hub** در الگوریتم **HITS** (ارزش ارجاعی صفحه)

x_5 : نمره **Authority** در الگوریتم **HITS** (ارزش محتوایی صفحه)

3. الحاق و تشکیل بردار نهایی (Concatenation):

تمام این ویژگی‌ها در یک بردار واحد $\mathbf{x}$ در کنار هم قرار می‌گیرند:

$$\mathbf{x} = [x_1, x_2, x_3, x_4, x_5, \dots, x_n]^T$$

اکنون این بردار نماینده‌ی کاملِ وضعیتِ یک جفت (کوئری، سند) است که هم محتوا و هم اعتبار ساختاری را در خود جای داده است.

بخش 3: استانداردسازی و پیاده‌سازی محاسباتی (به همراه بسط ریاضی)

برای اینکه مدل یادگیری ماشین (مانند **SVM** خطی یا رگرسیون لجستیک) بتواند این ویژگی‌های ناهمگون را به درستی پردازش کند و ویژگی‌هایی با مقادیر بزرگ‌تر (مانند **TF-IDF**) اثر ویژگی‌های کوچک‌تر (مانند **PageRank**) را خنثی نکنند، مرحله **نرمال‌سازی (Normalization)** الزامی است.

معمولاً تمام مقادیر با استفاده از روش‌هایی مانند نمره استاندارد (**Z-score**) به میانگین 0 و واریانس 1 مقیاس‌بندی می‌شوند. فرمول این تبدیل ساختاری برای هر ویژگی j در نمونه i به شرح زیر است:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

پیاده‌سازی بهینه محاسباتی در سیستم‌های واقعی:

در معماری سیستم‌های بازیابی اطلاعات، الحاق ویژگی‌های خلوتِ متنی (**Sparse**) با ویژگی‌های چگالِ ساختاری (**Dense**) باید با دقت انجام شود تا با خطای کمبود حافظه (**Out of Memory**) مواجه نشویم. در زبان‌هایی مانند پایتون، این عملیات از طریق ساختارهای ماتریسی بهینه‌سازی می‌شود.

برای درک بهتر فرآیند نرم‌افزاری این الحاق در بک‌اند موتور جستجو، نمونه کد بهینه‌ی زیر فرآیند ترکیب را از نظر منطقی نشان می‌دهد:

```
feature_fusion.py

import numpy as np
import scipy.sparse as sp

def build_unified_feature_vector(tfidf_sparse_matrix, pagerank_array, hits_auth_array):
    """
    الحاق بهینه ماتریس خلوت متنی با بردار ویژگی‌های چگال ساختاری گراف
    """
    # تبدیل بردارهای گرافی به فرمت ستونی و چگال
    pr_col = np.array(pagerank_array).reshape(-1, 1)
    hits_col = np.array(hits_auth_array).reshape(-1, 1)

    # چسباندن افقی ویژگی‌های ساختاری به انتهای ویژگی‌های متنی
    # برای حفظ ساختار خلوت و جلوگیری از اشغال حافظه اضافی hstack استفاده از
    unified_features = sp.hstack([tfidf_sparse_matrix, pr_col, hits_col], format='csr')

    return unified_features
```

بخش 4: مقایسه تحلیلی سیستم‌های کلاسیک و ماشین‌لرنینگ

ویژگی معماری	سیستم‌های کلاسیک (فرمول‌محور)	سیستم‌های مدرن (LTR)
تخصیص اوزان (ضرایب)	تنظیم دستی و حدسی (Heuristic)	بهینه‌سازی خودکار توسط الگوریتم
مقیاس‌پذیری ویژگی‌ها	محدود؛ افزودن بیش از 3-4 ویژگی فرمول را غیرقابل مدیریت می‌کند.	نامحدود؛ قابلیت الحاق صدها ویژگی ساختاری و متنی به بردار x .
مدیریت اختلاف مقیاس	نیازمند ترفندهای پیچیده ریاضی	مدیریت یکپارچه با Z-score

جمع‌بندی

- سیستم سنتی (مثل یک آشپز تازه‌کار): سعی می‌کند ادویه‌ها (TF-IDF) و نمک (PageRank) را با حدس و گمان ترکیب کند؛ که معمولاً در مقیاس بزرگ به نتیجه‌ای نامتوازن و گاهی فاجعه‌بار ختم می‌شود.
- سیستم یادگیری ماشین (ML): شما فقط مواد اولیه (بردار یکپارچه ویژگی‌ها) را به همراه نمونه‌هایی از غذاهای خوشمزه (داده‌های آموزشی و کلیک‌های کاربر) به سیستم می‌دهید. الگوریتم از طریق ریاضیات بهینه‌سازی، دقیقاً متوجه می‌شود که چه مقدار از «ارتباط متنی» باید با چه مقدار از «اعتبار شبکه» ترکیب شود تا بهترین رتبه‌بندی ممکن حاصل گردد.

پاسخنامه مسئله 15.23: محاسبات ریاضی ابرصفحه در ماشین بردار پشتیبان

مقدمه و هدف مسئله

این مسئله یک تمرین کلاسیک و پایه‌ای برای درک هندسه و جبر خطی نهفته در الگوریتم ماشین بردار پشتیبان (SVM) است. در اینجا با داشتن مختصات دقیق نقاط، می‌خواهیم به جای تکیه بر نرم‌افزار، پارامترهای اصلی مدل شامل بردار وزن (w)، عرض از مبدأ (b) و اندازه حاشیه (Margin) را به صورت دستی و تحلیلی محاسبه کنیم.

الف) یافتن معادله خط جداکننده (ابرصفحه) بهینه

برای یافتن معادله خط، ابتدا باید موقعیت نقاط را در یک دستگاه مختصات دکارتی 2-بعدی تحلیل کنیم:

داده‌های کلاس مثبت (+1): نقاط (2, 2) و (2, 0)

داده‌های کلاس منفی (-1): نقاط (0, 0) و (0, 2)

تحلیل هندسی:

اگر این 4 نقطه را روی محور مختصات رسم کنید، متوجه می‌شوید که تمام داده‌های کلاس مثبت روی خط عمودی $x = 2$ و تمام داده‌های کلاس منفی روی محور عرض‌ها یعنی خط عمودی $x = 0$ قرار گرفته‌اند.

در الگوریتم SVM، ابرصفحه بهینه (Optimal Hyperplane) خطی است که در دقیق‌ترین نقطه میانی بین این 2 دسته قرار می‌گیرد تا فاصله (حاشیه) خود را از هر 2 طرف به حداکثر برساند. وسط فاصله بین خط $x = 0$ و $x = 2$ ، خط عمودی $x = 1$ است. بنابراین، معادله ابرصفحه بهینه به سادگی به شکل زیر خواهد بود:

$$x - 1 = 0$$

ب) محاسبه بردارهای $\mathbf{w}$ ، مقدار b و اندازه حاشیه

اکنون این شهود هندسی را از طریق معادلات جبری **SVM** اثبات کرده و مقادیر دقیق را به دست می‌آوریم.

1. استخراج معادلات پایه‌ای:

معادله کلی ابرصفحه در فضای 2-بعدی به فرم زیر است (با فرض بردارهای $\mathbf{x} = (x, y)$ و $\mathbf{w} = (w_1, w_2)$):

$$\mathbf{w} \cdot \mathbf{x} + b = 0 \implies w_1x + w_2y + b = 0$$

نقاط مرزی (بردارهای پشتیبان) دقیقاً روی خطوط حاشیه قرار دارند. معادلات خطوط حاشیه برای 2 کلاس به شرح زیر است:

برای بردارهای پشتیبان کلاس مثبت: $w_1x + w_2y + b = +1$

برای بردارهای پشتیبان کلاس منفی: $w_1x + w_2y + b = -1$

2. جای‌گذاری مختصات و حل دستگاه معادلات:

برای یافتن مجهولات، مختصات 2 نقطه (یکی از هر کلاس) را در معادلات فوق قرار می‌دهیم:

با استفاده از نقطه $X_3(0, 0)$ از کلاس منفی:

$$w_1(0) + w_2(0) + b = -1 \implies b = -1$$

مقدار b مستقیماً برابر با -1 به دست آمد.

با استفاده از نقطه $X_2(2, 0)$ از کلاس مثبت (و جای‌گذاری $b = -1$):

$$w_1(2) + w_2(0) - 1 = 1 \implies 2w_1 = 2 \implies w_1 = 1$$

از آنجا که تغییرات روی محور y هیچ تأثیری در کلاس داده‌ها ندارد (خط ما عمودی است)، مقدار w_2 برابر

با صفر خواهد بود. (می‌توانید با جای‌گذاری نقطه $(2, 2)$ نیز این موضوع را اثبات کنید:

$$1(2) + w_2(2) - 1 = 1 \implies 2w_2 = 0 \implies w_2 = 0$$

3. نتیجه‌گیری مقادیر بردارها:

بردار وزن:

$$\mathbf{w} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

عرض از مبدأ (سوگیری): $b = -1$

بررسی صحت: اگر این مقادیر را در معادله اصلی جای‌گذاری کنیم، به $x = 1 \implies 1(x) + 0(y) - 1 = 0$

می‌رسیم که دقیقاً با تحلیل هندسی بخش (الف) تطابق دارد.

4. محاسبه اندازه حاشیه (Margin Size):

عرض کل حاشیه در الگوریتم **SVM** از طریق فرمول $\frac{2}{\|\mathbf{w}\|}$ محاسبه می‌شود.

ابتدا اندازه (نُرم) بردار وزن را محاسبه می‌کنیم:

$$\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2} = \sqrt{1^2 + 0^2} = 1$$

اکنون این مقدار را در فرمول حاشیه قرار می‌دهیم:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{1} = 2$$

ج) ارزیابی تحلیلی بردارهای پشتیبان

برای بسط و تثبیت این مفهوم ریاضی، می‌توانیم موقعیت تک‌تک نقاط را نسبت به مرزهای حاشیه آزمایش کنیم. نقطه داده‌ای در **SVM** یک «بردار پشتیبان» محسوب می‌شود که شرط $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$ برای آن صادق باشد. جدول زیر نشان می‌دهد که در این مثال خاص، تمامی 4 نقطه داده‌ی ما بردار پشتیبان هستند:

نقطه داده	مختصات (x, y)	برچسب (y_i)	معادله حاشیه $(w_1x + w_2y + b)$	بررسی صحت
X_1	(2, 2)	+1	$1(2) + 0(2) - 1 = +1$	بردار پشتیبان
X_2	(2, 0)	+1	$1(2) + 0(0) - 1 = +1$	بردار پشتیبان
X_3	(0, 0)	-1	$1(0) + 0(0) - 1 = -1$	بردار پشتیبان
X_4	(0, 2)	-1	$1(0) + 0(2) - 1 = -1$	بردار پشتیبان

نتیجه هندسی نهایی

اندازه کل حاشیه برابر با 2 است. این مقدار از نظر فیزیکی کاملاً منطقی است؛ زیرا پهنای خیابانی که مرز چپ آن روی خط $x = 0$ و مرز راست آن روی خط $x = 2$ قرار دارد، دقیقاً برابر با 2 واحد است.

پاسخنامه مسئله 15.24: بهینه‌سازی فرمول‌بندی دوگان (Dual Optimization Formulation)

مقدمه و هدف مسئله

قلب تپنده و جادوی ریاضیاتی الگوریتم ماشین بردار پشتیبان (**SVM**)، نحوه حل مسئله بهینه‌سازی آن است. فرمول‌بندی اولیه (**Primal**) که به شکل زیر تعریف می‌شود، یک مسئله «بهینه‌سازی مقید» (**Constrained Optimization**) است:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

حل مستقیم چنین مسائلی، به ویژه در ابعاد بالا، بسیار دشوار است. هدف از این مسئله، درک نقش بنیادین «ضرایب لاگرانژ» در انتقال از فضای مسئله اولیه به فضای «دوگان» (**Dual Problem**) و کشف این حقیقت است که چرا بدون این انتقال ریاضی، استفاده از «ترفند کرنل» (**Kernel Trick**) کاملاً غیرممکن بود.

بخش 1: نقش عملگرهای لاگرانژ در تبدیل به فرمول بندی دوگان

در ریاضیات بهینه سازی، زمانی که می‌خواهیم یک تابع را با وجود محدودیت‌هایی (شروطی که باید برقرار باشند) کمینه یا بیشینه کنیم، از روش «ضرایب لاگرانژ» (**Lagrange Multipliers**) استفاده می‌کنیم. نقش و روند کار این ضرایب به شرح زیر است:

1. ادغام محدودیت‌ها در تابع هدف (ساخت تابع لاگرانژین):

مسئله اولیه **SVM** دارای محدودیت‌هایی است (هر نقطه باید در سمت درست حاشیه باشد):
 $(\mathbf{y}_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i)$. عملگرهای لاگرانژ (متغیرهای $\alpha_i \geq 0$ و $\mu_i \geq 0$) به عنوان «جریمه‌کننده» عمل می‌کنند. آن‌ها این محدودیت‌های سخت را به عنوان یک ترم جریمه، مستقیماً وارد خود تابع هدف می‌کنند. تابع جدیدی که به دست می‌آید «لاگرانژین» (**Lagrangian**) نام دارد.
مزیت این کار چیست؟ ما یک مسئله «مقید» (سخت) را به یک مسئله «نامقید» (ساده‌تر برای مشتق‌گیری) تبدیل کرده‌ایم.

2. حذف متغیرهای اولیه (Primal Variables):

در مرحله بعد، برای پیدا کردن نقطه بهینه، از تابع لاگرانژین نسبت به متغیرهای اولیه (یعنی بردار وزن $\mathbf{w}$ ، عرض از مبدأ b و متغیرهای کمکی ξ_i) مشتق جزئی گرفته و برابر با 0 قرار می‌دهیم (بر اساس شرایط **KKT**). این مشتق‌گیری یک نتیجه‌ی ریاضی خیره‌کننده و بسیار مهم به همراه دارد:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

این معادله نشان می‌دهد که بردار وزن ($\mathbf{w}$)، چیزی جز یک ترکیب خطی از داده‌های آموزشی ($\mathbf{x}_i$) ضرب در ضرایب لاگرانژ آن‌ها (α_i) نیست.

3. تولد مسئله دوگان (The Dual Problem):

وقتی معادله‌ی به دست آمده برای $\mathbf{w}$ را دوباره در تابع لاگرانژین جای‌گذاری کنیم، تمام متغیرهای اولیه ($\mathbf{w}$, b , ξ) به طور کامل حذف می‌شوند. آنچه باقی می‌ماند، یک تابع هدف جدید است که تنها بر اساس ضرایب لاگرانژ (α) و داده‌های آموزشی ($\mathbf{x}$) نوشته شده است. به این فرمول بندی جدید، مسئله دوگان (**Dual Problem**) می‌گویند که به جای کمینه سازی، باید آن را بیشینه (**Maximize**) کنیم.

(بسط ریاضی: در فرمول دوگان، محدودیت‌های جدید بسیار ساده‌تر و صرفاً شامل $0 \leq \alpha_i \leq C$ و $\sum_{i=1}^n \alpha_i y_i = 0$ خواهند بود).

بخش 2: چرا فرم دوگان برای اعمال ترفند کرنل (Kernel Trick) ضروری است؟

اینجاست که شکوه معماری **SVM** خود را نشان می‌دهد. دلیل اینکه ما این همه محاسبات ریاضی انجام دادیم تا به فرم دوگان برسیم، یک کشف استراتژیک درباره تابع هدف دوگان است:

1. ظهور ضرب داخلی (Dot Product) به عنوان تنها عملگر:

اگر به فرمول نهایی مسئله دوگان نگاه کنیم، داده‌های آموزشی ($\mathbf{x}$) به صورت مستقل در هیچ کجای فرمول وجود ندارند؛ بلکه فقط و فقط به شکل ضرب داخلی جفت داده‌ها ظاهر می‌شوند: $(\mathbf{x}_i^T \mathbf{x}_j)$. همچنین در فاز پیش‌بینی (**Inference**) برای یک سند جدید مانند $\mathbf{z}$ ، معادله خط تصمیم به شکل زیر در می‌آید:

$$f(\mathbf{z}) = \sum_{i=1}^n \alpha_i y_i (\mathbf{x}_i^T \mathbf{z}) + b$$

2. بن‌بست مسئله اولیه در فضاهای غیرخطی:

اگر می‌خواستیم از مسئله اولیه (**Primal**) برای تفکیک غیرخطی استفاده کنیم، باید داده‌ها را با تابعی مانند $\Phi(\mathbf{x})$ به فضایی با ابعاد بی‌نهایت می‌بردیم. در مسئله اولیه، الگوریتم مستقیماً بردار $\mathbf{w}$ را یاد می‌گیرد. از آنجا که بُعد $\mathbf{w}$ برابر با بُعد فضای داده‌هاست، در یک فضای بی‌نهایت‌بعدی، بردار $\mathbf{w}$ نیز بی‌نهایت‌بعدی می‌شد. ذخیره و محاسبه‌ی یک بردار بی‌نهایت‌بعدی در حافظه کامپیوتر کاملاً غیرممکن است.

3. رهایی به کمک مسئله دوگان و کرنل:

اما در فرم دوگان، ما اصلاً متغیری به نام $\mathbf{w}$ را یاد نمی‌گیریم! ما متغیر α_i را یاد می‌گیریم که برای هر سند، فقط یک عدد اسکالر (1-بعدی) است. و چون داده‌ها فقط به شکل ضرب داخلی $\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$ با هم تعامل دارند، می‌توانیم به سادگی این ضرب داخلی سنگین را با یک «تابع کرنل» (مانند **RBF**) جایگزین کنیم:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

کرنل این محاسبه را در همان فضای کم‌بُعد اولیه انجام می‌دهد و خروجی ضرب داخلی در فضای بی‌نهایت‌بعدی را به ما تحویل می‌دهد.

بخش 3: مقایسه تحلیلی فرمول‌بندی اولیه و دوگان

جهت ارزیابی دقیق تفاوت‌های ریاضیاتی و محاسباتی این دو فرمول‌بندی، ویژگی‌های آن‌ها در جدول زیر سازمان‌دهی شده‌اند:

ویژگی مورد بررسی	مسئله اولیه (Primal Problem)	مسئله دوگان (Dual Problem)
متغیرهای بهینه‌سازی	$\mathbf{w}, b, \xi$	α
تعداد مجهولات	$d + 1 + n$ (بستگی به تعداد ویژگی‌ها و اسناد دارد)	n (دقیقاً برابر با تعداد اسناد آموزشی است)

ویژگی مورد بررسی	مسئله اولیه (Primal Problem)	مسئله دوگان (Dual Problem)
وابستگی به ابعاد داده‌ها (d)	مستقیم و شدید	کاملاً مستقل
قابلیت اجرای هسته غیرخطی	غیرممکن (در صورت بی‌نهایت بودن بُعد)	ایده‌آل (صرفاً از طریق ضرب داخلی)

جمع‌بندی

- مسئله اولیه (Primal):** سعی دارد مختصات دقیق یک مرز در فضاها را ترسیم کند ($\mathbf{w}$). این کار در ابعاد بالا منجر به فروپاشی محاسباتی می‌شود.
- ضرایب لاگرانژ (Lagrange):** مانند مترجمانی عمل می‌کنند که زبان «مختصات مستقیم فضا» ($\mathbf{w}$) را به زبان «روابط و فاصله‌ی بین نقاط» (α) ترجمه می‌کنند.
- مسئله دوگان (Dual):** فرمولی است که به دست می‌آید. در این فرمول، الگوریتم دیگر به نقشه کل فضا نیازی ندارد، بلکه فقط نیازمند دانستن زاویه و فاصله (ضرب داخلی) بین جفت نقاط است.
- ترفند کرنل (Kernel):** چون سیستم اکنون فقط زبان «ضرب داخلی» را می‌فهمد، کرنل وارد می‌شود و مانند یک میان‌بر محاسباتی، فاصله‌ی نقاط در پیچیده‌ترین فضاها را در کسری از 1 ثانیه محاسبه می‌کند. بدون فرم دوگان، جایگاهی برای ورود این میان‌بر وجود نداشت.

پاسخنامه مسئله 15.25: طراحی سیستم با توزیع به شدت نامتوازن

(Extreme Imbalanced Data System)

مقدمه و هدف مسئله

در بسیاری از کاربردهای واقعی ماشین لرنینگ در حوزه امنیت و بازبازی اطلاعات (مانند تشخیص کلاهبرداری، تشخیص اسپم، یا بیماری‌های نادر)، با پدیده‌ی «توزیع به شدت نامتوازن» (**Extreme Class Imbalance**) مواجه هستیم. در این مسئله، نسبت کلاس هدف (کلاهبرداری) به کلاس عادی، 10 به 1,000,000 (یعنی یک در صد هزار) است. الگوریتم **SVM** استاندارد، تلاش می‌کند «خطای کلی» را به حداقل برساند؛ بنابراین با فدا کردن آن 10 پیامک، به یک حاشیه بسیار امن دست می‌یابد و **Accuracy** معادل 99.999% تولید می‌کند، در حالی که در عمل سیستم کاملاً بی‌فایده است.

هدف این مسئله، آموزش 3 رویکرد مهندسی و استاندارد (در 3 سطح داده، الگوریتم و تصمیم‌گیری) برای غلبه بر این چالش است.

بخش 1: راهکار اول (سطح داده): نمونه‌برداری مجدد و کاهش اکثریت (Undersampling)

ساده‌ترین و گاهی مؤثرترین روش برای حل مشکل نامتوازن، دخالت فیزیکی در مجموعه داده‌های آموزشی پیش از تزریق آن‌ها به الگوریتم است.

نحوه اجرای استراتژی:

در سناریوهایی با این شدت از نامتوازن، استفاده از روش‌های تولید داده مصنوعی (**Oversampling**) مانند **SMOTE** (برای تولید 999,990 پیامک کلاهبرداری جعلی، باعث ایجاد نویز شدید و **Overfitting** روی متن می‌شود.

به جای این کار، از تکنیک **Undersampling** (کاهش نمونه‌های اکثریت) استفاده می‌کنیم. ما تمام 10 پیامک کلاهبرداری را نگه می‌داریم، اما از بین 1,000,000 پیامک عادی، تنها یک زیرمجموعه تصادفی کوچک (مثلاً 1,000 یا حداکثر 10,000 پیامک) را استخراج می‌کنیم تا نسبت داده‌ها از 1:100000 به یک نسبت قابل هضم‌تر مانند 1:100 یا 1:1000 برسد.

دلیل علمی اثربخشی:

این کار مانع از آن می‌شود که انبوه داده‌های کلاس اکثریت، تابع هزینه را «غرق» (**Swamp**) کنند. با کاهش تعداد داده‌های عادی، مرز تصمیم‌گیری (**Hyperplane**) فضای تنفس پیدا می‌کند و به سمت کلاس اقلیت هل داده نمی‌شود. همچنین زمان آموزش مدل روی سرور به شدت کاهش می‌یابد.

بخش 2: راهکار دوم (سطح الگوریتم): یادگیری حساس به هزینه (Cost-Sensitive SVM)

اگر نخواهیم هیچ‌یک از داده‌های عادی را حذف کنیم، باید منطق ریاضیات الگوریتم **SVM** را دستکاری کنیم تا ارزش پیامک‌های کلاهبرداری را به صورت مصنوعی بالا ببرد. این رویکرد به نام «وزن‌دهی نامتقارن کلاس‌ها» (**Asymmetric Class Weighting**) شناخته می‌شود.

نحوه اجرای استراتژی:

در تابع هدف **SVM** با حاشیه نرم، پارامتر جریمه C را به 2 پارامتر مستقل تفکیک می‌کنیم:

C^+ (جریمه خطای کلاس کلاهبرداری)

C^- (جریمه خطای کلاس عادی)

معادله به شکل زیر تغییر می‌کند:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C^+ \sum_{y_i=+1} \xi_i + C^- \sum_{y_i=-1} \xi_i \right)$$

برای جبران توزیع 1 در صدهزار، ما پارامتر C^+ را صدهزار برابر بزرگ‌تر از C^- تنظیم می‌کنیم ($C^+ = 100,000 \times C^-$).

(بسط علمی: از نظر ریاضیات بهینه‌سازی، تنظیم نسبت جریمه‌ها دقیقاً معکوس نسبت توزیع کلاس‌ها **Inverse Class**)

Frequency، ثابت‌شده‌ترین و استانداردترین رویکرد اکتشافی (**Heuristic**) برای خنثی‌سازی اثر نامتوازنی در توابع هزینه است).

دلیل علمی اثربخشی:

با این وزن‌دهی نامتقارن، به الگوریتم می‌فهمانیم که «اشتباه دسته‌بندی کردن حتی یک پیامک کلاهبرداری، به اندازه اشتباه کردن در صدهزار پیامک عادی برای سیستم هزینه دارد». الگوریتم برای فرار از این جریمه‌ی نجومی، مرز تصمیم‌گیری را تا جای ممکن از داده‌های کلاس مثبت (کلاهبرداری) دور می‌کند تا حریم امنی برای آن‌ها بسازد.

بخش 3: راهکار سوم (سطح تصمیم‌گیری): جابه‌جایی آستانه تصمیم‌گیری (Threshold Shifting)

این روش زمانی استفاده می‌شود که مدل قبلاً آموزش دیده است و ما می‌خواهیم در فاز زمان اجرا (**Inference**) جلوی خطای سیستم را بگیریم.

نحوه اجرای استراتژی:

تابع خروجی مدل **SVM** به شکل $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ است. در حالت استاندارد، آستانه تصمیم‌گیری (**Threshold**) روی عدد 0 تنظیم شده است (اگر حاصل مثبت بود $\rightarrow$ کلاهبرداری، اگر منفی بود $\rightarrow$ عادی). برای افزایش حساسیت سیستم به کلاهبرداری، ما این آستانه را به سمت اعداد منفی جابه‌جا می‌کنیم (مثلاً آستانه را روی 1.5- قرار می‌دهیم). یعنی از این پس، حتی اگر تابع خروجی مقداری مثل 1.2- را تولید کرد (که در حالت عادی سالم تلقی می‌شد)، سیستم آن را به عنوان «کلاهبرداری مشکوک» علامت‌گذاری می‌کند.

دلیل علمی اثربخشی:

این کار باعث می‌شود معیار بازیابی (**Recall**) به شدت افزایش یابد و شانس عبور پیامک کلاهبرداری به 0 نزدیک شود. البته تاوان این دستکاری این است که معیار **Precision** افت می‌کند (تعداد زیادی پیامک عادی به اشتباه به عنوان کلاهبرداری پرچم‌گذاری می‌شوند). اما در سیستم‌های امنیتی بانک، بررسی دستی چند پیامک عادی توسط اپراتور (**False Positive**)، بسیار ارزان‌تر از خالی شدن حساب یک مشتری (**False Negative**) است.

(بسط علمی: این عمل معادل است با حرکت روی منحنی **Precision-Recall Curve** برای یافتن نقطه کارکردی که در آن **Recall** نزدیک به 100% است).

بخش 4: مقایسه جامع استراتژی‌های مقابله با نامتوازن

جهت ارزیابی و انتخاب دقیق استراتژی در محیط‌های عملیاتی، جدول زیر مزایا و محدودیت‌های مهندسی این 3 رویکرد را طبقه‌بندی می‌کند:

استراتژی (سطح)	تکنیک اجرایی	تأثیر بر مدل	مزیت کلیدی	چالش / ریسک
داده محور (Data-Level)	Undersampling	تغییر فیزیکی نسبت‌ها پیش از آموزش	افزایش شدید سرعت آموزش	حذف اطلاعات مفید
الگوریتم محور (Algorithm-Level)	Cost-Sensitive SVM	تغییر تابع جریمه ($C^+ \gg C^-$)	بهینه‌سازی ذاتی و دقیق مرزها	یافتن ضرایب بهینه زمان‌بر است
تصمیم محور (Decision-Level)	Threshold Shifting	جابه‌جایی آستانه در فاز Inference	بدون نیاز به آموزش مجدد	افت شدید Precision

جمع‌بندی

طراحی یک سیستم در شرایط توزیع به شدت نامتوازن، نیازمند تغییر نگاه از «صحت» به «امنیت» است. 3 گام طلایی که یک مهندس در این شرایط طی می‌کند:

- داده‌ها (Data):** هرگز اجازه ندهید لشکر عظیم کلاس اکثریت، داده‌های ارزشمند اقلیت را در فاز آموزش مدفون کند (**Undersampling**).
- الگوریتم (Algorithm):** قوانین ریاضی را به نفع کلاس مهم‌تر تغییر دهید تا اشتباه روی آن‌ها تاوان سنگینی برای الگوریتم داشته باشد (**Cost-Sensitive Learning**).

3. ارزیابی و تصمیم (Evaluation & Decision): معیار Accuracy را به طور کامل دور بریزید و سیستم را بر اساس معیار Recall ارزیابی و تنظیم کنید (Threshold Shifting). (در ارزیابی نهایی، معیار F_β با $\beta > 1$ که وزن بیشتری به Recall می‌دهد، استانداردترین معیار است).

پاسخنامه مسئله 15.26: رتبه‌بندی مبتنی بر یادگیری زوجی (Pairwise Learning to Rank)

مقدمه و هدف مسئله

در رویکردهای نقطه‌ای (Pointwise)، سیستم تلاش می‌کند تا به هر سند یک «نمره مطلق» بدهد. اما در بازیابی اطلاعات، نمره مطلق اهمیتی ندارد؛ بلکه آنچه مهم است، «ترتیب و اولویت» اسناد نسبت به یکدیگر است (اینکه سند الف بالاتر از سند ب قرار گیرد). رویکرد زوجی (Pairwise) و به طور خاص الگوریتم RankSVM، این مشکل را با تغییر زاویه دید به شکل هوشمندانه‌ای حل می‌کند. هدف این مسئله، کالبدشکافی دقیق ریاضیات این تغییر شکل (Transformation) است تا ببینیم چگونه یک مسئله رتبه‌بندی به یک مسئله طبقه‌بندی باینری (2-کلاسه‌ای) تبدیل می‌شود.

بخش 1: ساختار ریاضی ورودی جدید (The Mathematical Construction of Input)

فرض کنید برای یک پرس‌وجوی مشخص (q)، کاربر روی سند A کلیک کرده اما روی سند B کلیک نکرده است، یا داور انسانی صراحتاً اعلام کرده است که سند A مرتبط‌تر از سند B است. این یعنی ما یک ترجیح و اولویت داریم: $A \succ B$.

هر سند در ارتباط با کوئری، دارای یک بردار ویژگی است ($\mathbf{x}_A$ برای سند A و $\mathbf{x}_B$ برای سند B). هدف مدل خطی ما پیدا کردن یک بردار وزن ($\mathbf{w}$) است که بتواند به هر سند امتیازی به فرم $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ اختصاص دهد.

منطق ریاضی برای تبدیل رتبه‌بندی به طبقه‌بندی:

1. اگر سند A بهتر از B است، پس امتیازی که مدل به A می‌دهد باید اکیداً بیشتر از امتیاز B باشد:

$$f(\mathbf{x}_A) > f(\mathbf{x}_B)$$

2. با جای‌گذاری فرمول امتیازدهی خطی، خواهیم داشت:

$$\mathbf{w}^T \mathbf{x}_A > \mathbf{w}^T \mathbf{x}_B$$

3. حالا با انتقال هر 2 عبارت به یک سمت نامعادله و فاکتورگیری از بردار $\mathbf{w}$ ، به معادله بنیادین RankSVM می‌رسیم:

$$\mathbf{w}^T (\mathbf{x}_A - \mathbf{x}_B) > 0$$

پاسخ بخش اول:

ورودی جدیدی که به مدل SVM داده می‌شود، دیگر بردار ویژگی یک سند مجزا نیست؛ بلکه «بردار تفاضل» (Difference Vector) است. از نظر ریاضی، ویژگی نمونه جدید برابر است با:

$$\mathbf{x}_{new} = \mathbf{x}_A - \mathbf{x}_B$$

یعنی ما درایه به درایه‌ی ویژگی‌های سند B را از سند A کم می‌کنیم تا یک بردار جدید بسازیم.

بخش 2: برچسب نمونه جدید (The Label of the New Instance)

اکنون که بردار ویژگی جدید $(\mathbf{x}_A - \mathbf{x}_B)$ را ساختیم، باید به الگوریتم **SVM** بگوییم که این نمونه متعلق به کدام کلاس است.

با نگاه به نامعادله‌ی $\mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) > 0$ ، متوجه می‌شویم که هدف ما این است که حاصل ضرب داخلی بردار وزن در این نمونه‌ی جدید، یک عدد مثبت بشود. در ماشین بردار پشتیبان (**SVM**)، نمونه‌هایی که خروجی تابع تصمیم آن‌ها بزرگ‌تر از 0 است، متعلق به کلاس مثبت هستند.

پاسخ بخش دوم:

برچسب (**Label**) این نمونه‌ی جدید، **مثبت یک (+1)** است. مدل **SVM** این بردار تفاضل را به عنوان یک نمونه آموزشی از کلاس مثبت دریافت می‌کند.

(نکته تکمیلی مهندسی: برای حفظ تقارن در فضای ریاضی و جلوگیری از سوگیری، معمولاً بردار تفاضل معکوس یعنی $\mathbf{x}_{new} = \mathbf{x}_B - \mathbf{x}_A$ نیز ساخته می‌شود و برچسب **منفی یک (-1)** به آن اختصاص می‌یابد تا مدل یاد بگیرد که این ترکیب باید نمره کمتری بگیرد).

بخش 3: بسط ریاضیاتی تابع هدف RankSVM (بخش تکمیلی)

برای درک چگونگی یادگیری بردار $\mathbf{w}$ روی این داده‌های جدید، باید فرمول‌بندی بهینه‌سازی را ارتقا دهیم. مشابه **SVM** کلاسیک، ما به دنبال حداکثرسازی حاشیه بین اسناد بهتر و بدتر هستیم، در حالی که خطای رتبه‌بندی را با استفاده از متغیر کمکی $(\xi_{A,B})$ جریمه می‌کنیم.

اگر P مجموعه تمام جفت‌اسنادی باشد که $A \succ B$ برای آن‌ها صادق است، مسئله بهینه‌سازی **RankSVM** به فرم زیر در می‌آید:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{(A,B) \in P} \xi_{A,B} \right)$$

$$\text{به شرطی که: } \mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) \geq 1 - \xi_{A,B}$$

در این فرمول، هر جفت سندی که ترتیب آن‌ها توسط الگوریتم نقض شود (یعنی فاصله آن‌ها کمتر از حاشیه امن 1 باشد)، هزینه‌ای معادل $\xi_{A,B}$ به تابع هدف اضافه می‌کند که الگوریتم برای کمینه‌سازی خطا، آن را جریمه خواهد کرد.

بخش 4: مقایسه تحلیلی رویکرد نقطه‌ای و زوجی

به منظور ارزیابی دقیق تفاوت‌های این دو استراتژی در معماری سیستم‌های **Learning to Rank**، مقایسه آن‌ها در جدول استاندارد زیر ارائه شده است:

شاخص ارزیابی سیستم	رویکرد نقطه‌ای (Pointwise)	رویکرد زوجی (Pairwise / RankSVM)
ماهیت فضای ورودی	بردار ویژگی یک تک‌سند مجزا ($\mathbf{x}_i$)	بردار تفاضلی 2 سند ($\mathbf{x}_A - \mathbf{x}_B$)
ماهیت برچسب (Label)	امتیاز مطلق و مقیاس‌دار (مثلاً 0 تا 4)	ترجیح باینری و نسبی (1+ یا 1-)
تابع هزینه (Loss Function)	کاهش خطای پیش‌بینی برای تک‌سند	کاهش خطای وارونگی (جابجایی 2 سند در لیست)
چالش و محدودیت اصلی	گره‌خوردگی رتبه‌ها (Ties)	حجم بالای جفت‌ها ($O(N^2)$) در آموزش

جمع‌بندی

الگوریتم **RankSVM** یکی از زیباترین ترفندهای مهندسی در یادگیری ماشین است. این الگوریتم به جای اینکه الگوریتم پایه **SVM** را تغییر دهد، داده‌ها را تغییر می‌دهد. برای درک بهتر، این سناریو را تصور کنید:

- 1. **رویکرد نقطه‌ای (تلاش بی‌فایده):** از مدل می‌خواهیم قد دقیق شخص **A** و شخص **B** را حدس بزند (رگرسیون). این کار بسیار سخت است و خطای بالایی دارد.
- 2. **رویکرد زوجی RankSVM:** مدل اصلاً کاری به قد مطلق این 2 نفر ندارد. ما شخص **B** را کنار شخص **A** می‌ایستادنیم، اختلاف قد آن‌ها را اندازه می‌گیریم (بردار تفاضل $\mathbf{x}_A - \mathbf{x}_B$) و به مدل می‌گوییم: «نگاه کن، این اختلاف باید یک عدد مثبت باشد (برچسب +1)، یعنی **A** بلندتر است».

با این ترفندِ جبری ساده، یک مسئله‌ی پیچیده‌ی «رتبه‌بندی لیست»، به یک مسئله‌ی بسیار ساده‌ی «طبقه‌بندی باینری روی بردارهای تفاضل» تبدیل می‌شود و می‌توانیم از تمام قدرت و امکانات SVM استاندارد (مانند ترفند کرنل و حاشیه نرم) برای مرتب‌سازی اسناد استفاده کنیم.

پاسخنامه مسئله 15.27: اثبات ریاضیاتی کرنل چندجمله‌ای (Polynomial Kernel Proof)

مقدمه و هدف مسئله

در مباحث پیشین، با مفهوم شگفت‌انگیز «ترفند کرنل» (**Kernel Trick**) آشنا شدیم و بیان کردیم که یک تابع کرنل می‌تواند بدون نیاز به نگاشت صریح داده‌ها به فضاها، حاصل ضرب داخلی آن‌ها را در آن فضای جدید محاسبه کند. هدف از این مسئله، اثبات دقیق و جبری این ادعا برای یکی از پرکاربردترین توابع، یعنی «کرنل چندجمله‌ای درجه 2» (**Quadratic Polynomial Kernel**) است. در این فرآیند، نشان خواهیم داد که چگونه یک فرمول ساده در فضای 2-بعدی، دقیقاً معادل با یک ضرب داخلی در فضایی 6-بعدی است.

بخش 1: بسط جبری عبارت کرنل

گام نخست برای اثبات، باز کردن فرمول کرنل بر اساس مختصات بردارهای ورودی است. فرض مسئله به این شرح است:

بردار اول: $\vec{x} = (x_1, x_2)$

بردار دوم: $\vec{z} = (z_1, z_2)$

$$K(\vec{x}, \vec{z}) = (\vec{x} \cdot \vec{z} + c)^2 \text{ تابع کرنل:}$$

ابتدا ضرب داخلی 2 بردار را در همان فضای 2-بعدی اولیه محاسبه می‌کنیم:

$$\vec{x} \cdot \vec{z} = x_1 z_1 + x_2 z_2$$

حالا این مقدار را در تابع کرنل جای‌گذاری می‌کنیم:

$$K(\vec{x}, \vec{z}) = (x_1 z_1 + x_2 z_2 + c)^2$$

اکنون باید این عبارت را به کمک اتحاد مربع 3-جمله‌ای بسط دهیم. یادآوری اتحاد:

$$(a + b + d)^2 = a^2 + b^2 + d^2 + 2ab + 2ad + 2bd$$

با اعمال این اتحاد بر روی تابع کرنل، خواهیم داشت:

$$\begin{aligned} K(\vec{x}, \vec{z}) &= (x_1 z_1)^2 + (x_2 z_2)^2 + c^2 + 2(x_1 z_1)(x_2 z_2) + 2(x_1 z_1)(c) + \\ &= x_1^2 z_1^2 + x_2^2 z_2^2 + c^2 + 2x_1 x_2 z_1 z_2 + 2c x_1 z_1 + 2c x_2 z_2 \end{aligned}$$

بخش 2: بازآرایی و جداسازی متغیرها برای تشکیل ضرب داخلی

هدف ما این است که ثابت کنیم عبارت به دست آمده در بالا، در واقع حاصل ضرب داخلی 2 بردار مجزا است (یک بردار فقط شامل مولفه‌های x و بردار دیگر فقط شامل مولفه‌های z).

برای این کار، عبارت بسط‌یافته را طوری بازنویسی و دسته‌بندی می‌کنیم که مولفه‌های x و z به عنوان ضریب‌های تفکیک‌شده در کنار هم قرار گیرند:

$$K(\vec{x}, \vec{z}) = (x_1^2)(z_1^2) + (x_2^2)(z_2^2) + (\sqrt{2}x_1 x_2)(\sqrt{2}z_1 z_2) + (\sqrt{2c}x_1)(\sqrt{2c}z_1)$$

اگر با دقت به این آرایش جدید نگاه کنید، متوجه می‌شوید که این ساختار، دقیقاً فرمول استاندارد ضرب داخلی بین 2 بردار است که هر کدام دارای 6 مولفه هستند.

بخش 3: استخراج صریح تابع نگاشت $\Phi(\vec{x})$

از آنجا که فرمول به دست آمده دقیقاً معادل با ساختار $\Phi(\vec{x}) \cdot \Phi(\vec{z})$ است، می‌توانیم با استخراج جملاتِ مربوط به x ، بردار ویژگی جدید $\Phi(\vec{x})$ را به صورت صریح تعریف کنیم.

نگاشت Φ داده‌های ورودیِ ما را از یک فضای 2-بعدی، به یک فضای 6-بعدی (**6D Feature Space**) منتقل می‌کند. مولفه‌های این بردار 6-بعدی عبارتند از:

$$\Phi(\vec{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2c}x_1, \sqrt{2c}x_2, c)$$

به طور مشابه، برای بردار $\vec{z}$ نیز خواهیم داشت:

$$\Phi(\vec{z}) = (z_1^2, z_2^2, \sqrt{2}z_1z_2, \sqrt{2c}z_1, \sqrt{2c}z_2, c)$$

اثبات نهایی:

اگر شما بردارهای $\Phi(\vec{x})$ و $\Phi(\vec{z})$ را که در بالا به دست آمدند، به صورت سنتی در هم ضربِ داخلی کنید، دقیقاً به همان بسط جبری خواهید رسید:

$$\Phi(\vec{x}) \cdot \Phi(\vec{z}) = (x_1^2z_1^2) + (x_2^2z_2^2) + (\sqrt{2}x_1x_2 \times \sqrt{2}z_1z_2) + \dots = (\vec{x} \cdot$$

بخش 4: مقایسه تحلیلی پیچیدگی محاسباتی

مزیت استراتژیک	تعداد عملیات ریاضی (تقریبی)	ابعاد فضای پردازش	رویکرد محاسباتی
سادگی و سرعت، اما ناتوان در تفکیک غیرخطی	2 Multiplications + 1 Addition	2-بعدی	فضای اصلی خطی
قدرت تفکیک غیرخطی، اما هزینه محاسباتی بالا	6 Multiplications + 5 Additions + Feature Mapping	6-بعدی	نگاشت صریح (Φ)
ترکیب جادویی سرعتِ مبدأ و قدرت تفکیکِ مقصد	2 Multiplications + 1 Addition + 1 Exponentiation	2-بعدی (مجازی 6-بعدی)	ترفند کرنل (K)

جمع‌بندی

این اثبات ریاضیاتی، «قلب تپنده‌ی» موفقیت الگوریتم ماشین بردار پشتیبان (**SVM**) را آشکار می‌کند. لطفاً به این نکته مهندسی عمیقاً توجه کنید:

- عدم تولید فیزیکی:** برای محاسبه‌ی زاویه و فاصله بین 2 نقطه در آن فضای پیچیده‌ی 6-بعدی، ما هیچ‌گاه بردارهای 6-بعدی $\Phi(\vec{x})$ را در حافظه کامپیوتر تولید (**Allocate**) نکردیم.
- سادگی محاسباتی:** ما تنها 2 عدد را در فضای 2-بعدی در هم ضرب کردیم $(\vec{x} \cdot \vec{z})$ ، با مقدار ثابت c جمع بستیم و نتیجه را به توان 2 رساندیم.

- **نتیجه‌گیری:** این اثبات به ما اطمینان می‌دهد که آن محاسبه‌ی بسیار ساده و کم‌هزینه، پاسخی دقیقاً معادل با ضرب داخلی در یک فضای 6-بعدی تولید می‌کند. در پردازش متن (جایی که توان‌ها و ابعاد بسیار عظیم هستند)، همین صرفه‌جویی ریاضیاتی است که امکان یادگیری روی ده‌ها هزار کلمه را برای سرورها فراهم می‌سازد. (طبق قضیه مرسر **Mercer's Theorem**، این ویژگی برای تمام توابع معتبر کرنل صادق است).

پاسخنامه مسئله 15.28: سناریوی کدنویسی و تنظیم فرآپارامترها (Hyperparameter Tuning)

مقدمه و هدف مسئله

در محیط‌های عملیاتی و مهندسی نرم‌افزار، تئوری‌های یادگیری ماشین باید به کدهای بهینه و مقیاس‌پذیر تبدیل شوند. کتابخانه **scikit-learn** در پایتون، استاندارد صنعتی برای این کار است. هدف از این مسئله، آموزش نحوه پیاده‌سازی یک خط لوله (**Pipeline**) استاندارد برای پردازش متن، جستجوی خودکارِ بهترین فرآپارامتر (**Hyperparameter**) و در نهایت، تحلیل تئوریکِ رفتار مدل در مواجهه با پارامتر تنظیم‌گر C در فضای متنی است.

بخش 1: پیاده‌سازی کد پایتون با scikit-learn

در مهندسی یادگیری ماشین، ترکیب برداری‌ساز (**Vectorizer**) و طبقه‌بند (**Classifier**) در قالب یک «خط لوله» (**Pipeline**) یک الزام است؛ زیرا از پدیده‌ی خطرناکِ «نشت داده» (**Data Leakage**) در طول فرآیند اعتبارسنجی متقاطع (**Cross-Validation**) جلوگیری می‌کند.

در ادامه، ساختار دقیق و بهینه‌سازی‌شده برای پاسخ به الزامات این مسئله در زبان پایتون آورده شده است. جهت دستیابی به حداکثر کارایی محاسباتی در فضاهاى متنی و جلوگیری از تخصیص مازاد حافظه، از ماژول **LinearSVC** (مبتنی بر **liblinear**) استفاده شده است که نسبت به هسته استاندارد، مراتب پردازشی بسیار سریع‌تری دارد:

```
hyperparameter_tuning.py

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

# 1. تعریف خط لوله (Pipeline) برای یکپارچه‌سازی فرآیند
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('svm', LinearSVC(dual=True, max_iter=10000))
])

# 2. بر اساس صورت مسئله C تعریف فضای جستجوی پارامتر
param_grid = {
    'svm__C': [0.01, 0.1, 1.0, 10.0, 100.0] # معادل با 2^-10 تا 2^10
}

# 3. پیکربندی جستجوی شبکه‌ای (GridSearchCV)
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring='f1', # معیار ارزیابی F1-Score (برای حالت باینری)
    cv=5, # اعتبارسنجی متقاطع 5 مرحله‌ای (K-Fold)
    n_jobs=-1 # استفاده از تمام هسته‌های پردازنده برای تسریع محاسبات
)

# 4. اجرای فرآیند آموزش و تنظیم پارامتر (برچسب‌ها هستند y_train و X_train فرض بر این است که)
grid_search.fit(X_train, y_train)

# 5. استخراج نتایج نهایی
print("Best C:", grid_search.best_params_['svm__C'])
print("Best F1:", grid_search.best_score_)
```

نکات کلیدی کد:

Pipeline: استفاده از خط لوله ضروری است تا ماتریس **TF-IDF** برای داده‌های ارزیابی (**Validation Data**) در

طول **GridSearchCV** به اشتباه ویژگی‌های جدیدی را استخراج نکند.

Scoring: اگر دیتاست شما چندکلاسه‌ای (**Multi-class**) است، باید پارامتر `scoring='f1_macro'` را جایگزین

`'f1'` کنید تا میانگین امتیاز اف-1 برای همه کلاس‌ها به درستی محاسبه شود.

بخش 2: تحلیل تئوریک رفتار پارامتر C و پدیده بیش‌برازش (Overfitting)

صورت مسئله یک سوال تحلیلی بسیار مهم مطرح کرده است: «چرا مقدار C بسیار بزرگ (مثلاً $C = 100$) ممکن است

باعث **Overfitting** روی داده‌های متنی شود؟»

برای پاسخ به این سوال، باید به تابع هدف ریاضی در **SVM** حاشیه نرم (**Soft Margin**) رجوع کنیم:

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

1. نقش ریاضی پارامتر C:

پارامتر C ضریب متغیرهای کمکی (ξ_i) است. این متغیرها نشان‌دهنده «خطاها» یا «نقض حریم حاشیه» هستند.

وقتی شما C را یک عدد بسیار بزرگ در نظر می‌گیرید (مثلاً 10^2)، به الگوریتم دستور می‌دهید که: «تحت هیچ

شرایطی اجازه نداری حتی یک داده‌ی آموزشی را اشتباه دسته‌بندی کنی، زیرا جریمه‌ی آن در تابع هدف بی‌نهایت

سنگین خواهد بود.»

2. ساختار هندسی و نویز در داده‌های متنی:

داده‌های متنی معمولاً فضایی با ده‌ها هزار بُعد ایجاد می‌کنند. در متون واقعی، نویزهای فراوانی وجود دارد؛ مانند:

- غلط‌های املایی.
- کلمات بسیار نادر که به طور کاملاً تصادفی فقط در یک کلاس خاص از اسناد آموزشی ظاهر شده‌اند.
- برچسب‌گذاری‌های اشتباه انسانی.

3. مکانیسم وقوع Overfitting:

وقتی C بسیار بزرگ است، الگوریتم برای اینکه از جریمه فرار کند و دقت روی داده‌های آموزشی (**Training**)

(**Accuracy**) را به 100% برساند، مجبور می‌شود به این کلمات نویزی و بی‌اهمیت، وزن‌های بسیار بالایی اختصاص

دهد.

- اثر هندسی:** حاشیه هندسی (**Margin**) به شدت باریک می‌شود و ابرصفحه (**Hyperplane**) برای

دربرگرفتن استثناها، حالت پیچیده و شکننده‌ای به خود می‌گیرد.

- نتیجه نهایی:** مدل، به جای کشف الگوهای کلی زبان، شروع به حفظ کردن تکتک اسناد و نویزهای

مجموعه آموزشی می‌کند. هنگامی که این مدل سخت‌گیر با اسناد جدید و دیده نشده (**Test Data**) مواجه

می‌شود، به دلیل اتکای بیش از حد به کلمات نادر، دچار خطای فاحش شده و **F1-Score** آن به شدت

افت می‌کند.

بخش 3: مقایسه تحلیلی اثر پارامتر C در فضای متنی (جدول تکمیلی)

جهت بررسی دقیق تاثیر تنظیم این پارامتر در خط لوله یادگیری ماشین، رفتار مدل به ازای مقادیر مختلف C در جدول استاندارد زیر تحلیل شده است:

مقدار پارامتر C	رویکرد الگوریتم (Penalty)	عرض حاشیه	خطای آموزش	خطای آزمون	وضعیت نهایی مدل
$C = 0.01$ (کوچک)	جریمه بسیار کم برای تخلف	بسیار پهن	بالا	بالا	Underfitting
$C = 1.0$ (متعادل)	تعادل بین حاشیه و خطا	بهینه	قابل قبول	حداقل	Optimal
$C = 100.0$ (بزرگ)	جریمه سنگین برای تخلف	بسیار باریک	صفر (مطلق)	بالا	Overfitting

جمع‌بندی

- در پروژه‌های پردازش زبان طبیعی (NLP)، تنظیم پارامتر C مانند تنظیم «حساسیت یک کارآگاه» است:
- اگر C خیلی کوچک باشد (10^{-2}): کارآگاه بیش از حد بی‌خیال است و تفاوت‌های مهم کلاس‌ها را نادیده می‌گیرد (Underfitting).
- اگر C خیلی بزرگ باشد (10^2): کارآگاه دچار وسواس شده و هر غلط املایی ساده‌ای را به عنوان یک سرخ قطعی برای جرم تلقی می‌کند (Overfitting).
- نتیجه‌گیری: به همین دلیل است که ما به ابزاری مانند GridSearchCV تکیه می‌کنیم تا به صورت خودکار، آن مقدار متعادل (C بهینه) را پیدا کند که بالاترین قابلیت تعمیم‌پذیری (Generalization) را روی داده‌های جدید به همراه داشته باشد.

پاسخنامه مسئله 15.29: تحلیل شکست روش نقطه‌ای در LTR (Failure of Pointwise in LTR)

مقدمه و هدف مسئله

در تکامل سیستم‌های «یادگیری برای رتبه‌بندی» (Learning to Rank)، روش نقطه‌ای (Pointwise) به عنوان ساده‌ترین رویکرد شناخته می‌شود. اما در محیط‌های عملیاتی پیچیده، مانند موتورهای جستجوی واقعی که با طیف وسیعی از پرس‌وجوها (از تک‌کلمه‌ای تا جملات طولانی) مواجه هستند، این روش به شدت دچار افت کیفیت می‌شود. هدف از این مسئله، کالبدشکافی بنیادی محدودیت‌های رویکرد نقطه‌ای از منظر توزیع ویژگی‌ها و معرفی رویکرد پیشرفته‌ی لیستی (Listwise) به عنوان راهکار قطعی این چالش است.

بخش 1: دلیل بنیادی شکست روش نقطه‌ای در قیاس پرس‌وجوها

شکست الگوریتم‌های نقطه‌ای (مانند Pointwise SVM یا رگرسیون خطی) در مدیریت همزمان کوئری‌های کوتاه و طولانی، ریشه در 3 نقص بنیادین ساختاری و ریاضی دارد:

1. نادیده گرفتن «مرز پرس‌وجوها» (Query Boundary Ignorance):

الگوریتم نقطه‌ای هر جفت (q, d) را به عنوان یک نمونه‌ی کاملاً مستقل در نظر می‌گیرد. تابع هزینه (Loss)

Function در این مدل، تلاش می‌کند خطای پیش‌بینی «مطلق» را روی کل مجموعه داده‌ها (شامل میلیون‌ها جفت از هزاران کوئری مختلف) کمینه کند. مدل اصلاً درک نمی‌کند که اسنادِ مربوط به کوئری **A**، فقط باید با اسنادِ کوئری **A** رقابت کنند، نه با اسنادِ کوئری **B**.

2. عدم تطابق مقیاس ویژگی‌ها (Feature Scale Discrepancy):

این عامل، دلیل اصلی به هم ریختن امتیازات در کوئری‌های طولانی است.

- در یک کوئری کوتاه (مثلاً 2 کلمه)، تطابق دقیق هر 2 کلمه در متن، امتیازاتی نظیر **TF-IDF** یا **BM25** را به شدت بالا می‌برد (ایجاد ویژگی‌های متراکم و قدرتمند).
- در یک کوئری طولانی (مثلاً 10 کلمه)، بعید است یک سند تمام 10 کلمه را داشته باشد. بنابراین ویژگی‌های آماری به دست آمده برای اسنادِ مرتبط، در مقیاس‌های عددی بسیار پایین‌تری قرار می‌گیرند. الگوریتم نقطه‌ای سعی می‌کند تمام این مقیاس‌های ناهمگون را با یک فرمول ثابت، به یک نمره مطلق (مثلاً برچسب 3 به معنای مرتبط) نگاشت کند. در نتیجه، مدل در تطبیق دادن مقیاس‌های کوئری طولانی با خروجی‌های هدف دچار سردرگمی شده و اسناد نامربوط (که به طور تصادفی چند کلمه پرارزش دارند) را در رتبه‌های بالا قرار می‌دهد.

3. بهینه‌سازی خطای اشتباه (MSE به جای NDCG):

روش نقطه‌ای خطای رگرسیون (مانند **Mean Squared Error**) را کمینه می‌کند. اما در بازیابی اطلاعات، کاربر به نمره مطلق اهمیت نمی‌دهد؛ کاربر فقط به این اهمیت می‌دهد که «بهترین سند در رتبه اول باشد». روش نقطه‌ای هیچ مکانیزمی برای تمرکز بر بالای لیست (**Top-k**) ندارد و خطای رتبه دهم را با خطای رتبه اول یکسان جریمه می‌کند.

بخش 2: رویکرد لیستی (Listwise) چگونه این مشکل را حل می‌کند؟

رویکرد لیستی (مانند الگوریتم‌های **ListNet** یا **LambdaMART**) یک تغییر پارادایم کامل در نحوه نگاه به داده‌های آموزشی است و مشکلات فوق را به شکل زیر برطرف می‌سازد:

1. تغییر واحد یادگیری (Learning Instance):

در روش لیستی، ورودی مدل دیگر یک تک‌سند (نقطه‌ای) یا یک جفت‌سند (زوجی) نیست؛ بلکه ورودی مدل، «یک پرس‌وجو به همراه تمام لیست اسناد نامزد آن» است. مدل کل لیست را به صورت یک‌جا پردازش می‌کند.

2. نرمال‌سازی ذاتی بافتار (Contextual Normalization):

چون مدل تمام اسناد یک کوئری را با هم ارزیابی می‌کند، تفاوت مقیاس ویژگی‌ها بین کوئری‌های طولانی و کوتاه بی‌اثر می‌شود. مدل یاد می‌گیرد که در یک کوئری طولانی، حتی نمرات **BM25** پایین نیز ممکن است برای برنده شدن کافی باشد (زیرا رقابت فقط بین اسناد همان کوئری است). به بیان ساده، ارزیابی از حالت «مطلق» به حالت «نسبی» در درون همان لیست تبدیل می‌شود.

3. بهینه‌سازی مستقیم معیارهای رتبه‌بندی (Direct Ranking Optimization):

توابع هزینه در روش لیستی به گونه‌ای طراحی شده‌اند که مستقیماً معیارهای ارزیابی لیست (مانند **NDCG** یا **MAP**) را بهینه‌سازی کنند. این توابع درک می‌کنند که جابه‌جا شدن اسناد در رتبه‌های اول و دوم، بسیار پرهزینه‌تر و حساس‌تر از جابه‌جایی در رتبه‌های صدم و صد و یکم است.

شاخص ارزیابی سیستم	رویکرد نقطه‌ای (Pointwise)	رویکرد لیستی (Listwise)
واحد یادگیری 모델	تک‌سند مستقل از بافتار	یک پرس‌وجو به همراه کل اسناد نامزد
حساسیت به مقیاس کوئری	بسیار بالا (آسیب‌پذیر در کوئری طولانی)	خنثی شده (نرمال‌سازی ذاتی درون لیستی)
تابع هزینه (Loss)	MSE / Cross-Entropy	NDCG Loss / MAP
تمرکز بهینه‌سازی	روی کل داده‌ها به طور یکسان	تمرکز هوشمندانه روی اسناد بالای لیست (Top-k)

جمع‌بندی

برای درک عمیق این تفاوت، سیستم نمره‌دهی دانشگاه را تصور کنید:

- روش نقطه‌ای (Pointwise):** مانند استادی است که به تمام دانشجویان در تمام دروس (ریاضی، هنر، فیزیک) با یک معیار ثابت سخت‌گیرانه نمره از 100 می‌دهد. اگر امتحان فیزیک (کوئری طولانی) بسیار سخت باشد، بهترین دانشجوی فیزیک نمره 50 می‌گیرد و در مقایسه با دانشجویان هنر، ضعیف به نظر می‌رسد.
- روش لیستی (Listwise):** مانند استادی است که نمرات را روی «نمودار کلاس» (Curve) می‌برد. او کاری ندارد که امتحان سخت بوده یا آسان؛ او فقط بررسی می‌کند که چه کسی در همان کلاس بالاترین عملکرد را داشته و رتبه اول را به او می‌دهد. این روش به طور خودکار، تفاوت سختی دروس (تفاوت کوئری‌های کوتاه و طولانی) را خنثی می‌کند.

پاسخنامه مسئله 15.30: کدنویسی خطلوله و تحلیل انتخاب ویژگی

مقدمه و هدف مسئله

در محیط‌های عملیاتی، پیاده‌سازی گسسته‌ی مراحل استخراج ویژگی و آموزش مدل می‌تواند منجر به نشت داده (Data Leakage) شود. این مسئله نحوه کپسوله‌سازی صحیح فرآیندها در یک خطلوله (Pipeline) استاندارد را نشان داده و سپس یک ترکیب به ظاهر قدرتمند اما در عمل مخرب (انتخاب ویژگی + کرنل شعاعی) را در فضای متنی کالبدشکافی می‌کند.

بخش 1: پیاده‌سازی خطلوله (Pipeline)

در ادامه، پیاده‌سازی بهینه و ساختاریافته‌ی این خطلوله با استفاده از کتابخانه **scikit-learn** آورده شده است. این ساختار تضمین می‌کند که فیلتر **Chi-Square** صرفاً بر اساس توزیع داده‌های آموزشی تنظیم گردد:

```
nlp_pipeline.py

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.svm import SVC

# ساختاردهی خطلوله یکپارچه
pipeline = Pipeline([
    ('vectorizer', TfidfVectorizer(max_features=50000)),
    ('chi2_selector', SelectKBest(score_func=chi2, k=2000)),
    ('classifier', SVC(kernel='rbf'))
])
```

بخش 2: تحلیل تئوریک و مهندسی

در مهندسی سیستم‌های پردازش زبان طبیعی (NLP)، ترکیب انتخاب ویژگی شدید (مانند تقلیل 50,000 به 2,000 ویژگی) با کرنل‌های غیرخطی نظیر **RBF** به ندرت استفاده می‌شود. این موضوع ریشه در ساختار هندسی داده‌های متنی و رفتار الگوریتم‌ها دارد:

1. چرا ترکیب «انتخاب ویژگی + کرنل RBF» در متن نامطلوب است؟

پارادوکس ابعادی (Dimensionality Paradox):

کرنل **RBF** با استفاده از توابع نمایی، داده‌ها را به فضایی با ابعاد بی‌نهایت پرتاب می‌کند تا مرزهای غیرخطی پیدا کند. وقتی شما با انتخاب ویژگی، فضای غنی 50,000 بعدی را به 2,000 بُعد محدود می‌کنید، در واقع سیگنال‌های ضعیف اما پرتعداد زبانی را دور می‌ریزید. تغذیه یک کرنل تشنه‌ی ابعاد با یک بردار به شدت فشرده‌شده، پتانسیل کشف الگوهای پیچیده را از بین می‌برد.

از دست رفتن خلوتی و انفجار محاسباتی:

ماتریس‌های متنی به شدت خلوت (**Sparse**) هستند. کرنل **RBF** برای محاسبه فاصله اقلیدسی بین اسناد، این خلوتی را از بین می‌برد و ماتریس را چگال (**Dense**) می‌کند. حتی با کاهش ابعاد به 2,000، محاسبه فاصله اقلیدسی برای تمام جفت‌اسناد در کرنل **RBF** دارای پیچیدگی زمانی حداقل $O(n^2)$ تا $O(n^3)$ است که برای دیتاست‌های متنی تجاری، یک فاجعه محاسباتی محسوب می‌شود.

2. چرا «SVM خطی بدون انتخاب ویژگی» استاندارد صنعتی است؟

تفکیک‌پذیری ذاتی در ابعاد بالا (Cover's Theorem):

فضای متنی ذاتاً دارای ابعاد بسیار بالاست (50,000 بُعد در این سناریو). در چنین فضای وسیعی، داده‌ها به احتمال بسیار زیاد به صورت **خطی تفکیک‌پذیر (Linearly Separable)** هستند. بنابراین یک ابرصفحه‌ی تخت (**Flat Hyperplane**) به راحتی می‌تواند کلاس‌ها را جدا کند و نیازی به خم کردن فضا توسط **RBF** نیست.

تنظیم‌گری ذاتی (Built-in Regularization):

مدل **LinearSVC** دارای جریمه‌گر L_2 است. این تنظیم‌گر به طور خودکار وزن کلمات بی‌اهمیت و نویزی را به 0 (یا بسیار نزدیک به 0) می‌رساند. در واقع، **SVM** خطی فرآیند «انتخاب ویژگی نرم» (**Soft Feature Selection**) را در دل فرآیند بهینه‌سازی خود انجام می‌دهد و نیازی به فیلتر کردنِ خشن (**Hard**) ویژگی‌ها پیش از آموزش ندارد.

مقیاس‌پذیری و سرعت (Scalability):

کرنل خطی صرفاً با ضرب داخلی $(\mathbf{x}_i^T \mathbf{x}_j)$ کار می‌کند. این عملیات روی ماتریس‌های **Sparse** بسیار سریع انجام می‌شود (پیچیدگی زمانی آموزش تقریباً $O(n)$ است). الگوریتم می‌تواند میلیون‌ها ویژگی و سند را در کسری از زمان مورد نیاز برای **RBF** پردازش کند، در حالی که دقت تعمیم‌پذیری (**Generalization**) بالاتری نیز ارائه می‌دهد.

بخش 3: ارزیابی تحلیلی خطوط لوله متنی

جهت جمع‌بندی تصمیمات معماری، جدول زیر رفتار این 2 رویکرد را در پردازش زبان طبیعی مستقیماً مقایسه می‌کند:

معيار ارزیابی معماری	SelectKBest + RBF SVM	LinearSVC (All Features)
تراکم اطلاعاتی	از دست رفتن شدید سیگنال‌های ضعیف	حفظ کامل واژگان و الگوهای نادر
خلوتی ماتریس (Sparsity)	تخریب کامل ساختار خلوت	بهره‌برداری کامل از ساختار Sparse
پیچیدگی محاسباتی	$O(n_{samples}^2 \times k)$	$\sim O(n_{samples} \times n_{features})$
مدیریت ویژگی‌های نویزی	نیازمند پیش‌فیلتر قوی (Hard Truncation)	تنظیم‌گری خودکار (L2 Penalty)

نتیجه‌گیری: در مهندسی نرم‌افزار برای طبقه‌بندی متون، خط‌لوله استاندارد و بهینه، عبور مستقیم تمامی ویژگی‌های متنی (بدون کاهش ابعاد) به یک طبقه‌بند خطی (Linear SVM / Logistic Regression) است. این ساختار نه تنها از نظر ریاضی با هندسه متن همخوانی دارد، بلکه از فروپاشی منابع حافظه و پردازنده در محیط‌های **Production** جلوگیری می‌کند.

پاسخنامه مسئله 15.31: معماری یادگیری ماشین برای سیستم پرسش و پاسخ (QA System Architecture)

مقدمه و هدف مسئله

در طراحی یک سیستم پرسش و پاسخ (QA) حقوقی، با یک تضاد مهندسی بزرگ روبرو هستیم: از یک سو، متون حقوقی نیازمند درک عمیق معنایی و دقت فوق‌العاده بالا (**Precision**) هستند که تنها از عهده مدل‌های پیچیده یادگیری ماشین ساخته است. از سوی دیگر، اجرای این مدل‌های سنگین (مانند **SVM** با ویژگی‌های غنی) روی پایگاه داده‌ای با میلیون‌ها پاراگراف، نیازمند زمان پردازش غیرقابل‌قبولی است که سیستم را از حالت «بی‌درنگ» (**Real-time**) خارج می‌کند.

برای حل این تنگنای محاسباتی، از الگوی استاندارد صنعتی گذرگاه 2-مرحله‌ای (**Retrieve-and-Rerank Pipeline**) استفاده می‌کنیم. این معماری مانند یک «قیف» عمل می‌کند که در مرحله اول سرعت را به حداکثر و در مرحله دوم دقت را به اوج می‌رساند.

بخش 1: جریان داده و معماری سیستم (Data Flow Architecture)

در ادامه، مسیر حرکت یک پرس‌وجو از لحظه ورود تا استخراج دقیق‌ترین پاراگراف حقوقی به شکل ساختاریافته مدل‌سازی شده است:

1. دریافت و پیش‌پردازش پرس‌وجو (ورودی سیستم):

کاربر یک سوال حقوقی (مثلاً: "جریمه نقض قرارداد عدم افشا در حقوق تجارت چیست؟") را وارد می‌کند. سیستم کوئری را توکنایز کرده و ریشه‌یابی کلمات حقوقی را انجام می‌دهد.

2. فاز اول: بازیابی سریع (Recall Phase):

کاهش فضا از 1,000,000 به 100 سند

تکنولوژی: موتور جستجوی مبتنی بر نمایه وارونه (**Inverted Index**) مانند **Elasticsearch**.

منطق عملیاتی: سیستم در کسری از 1 ثانیه پایگاه داده میلیونی را اسکن کرده و 100 پاراگرافی که بیشترین اشتراک کلماتی را با سوال کاربر دارند، برمی‌گرداند. هدف در اینجا «دقت» نیست؛ هدف صرفاً این است که پاراگراف جواب، به دلیل نداشتن کلمات کلیدی، جا نماند (حداکثر کردن Recall).

3. پل ارتباطی: تولید بردارهای ویژگی (Feature Generation):

آماده‌سازی برای یادگیری ماشین

مدل‌های یادگیری ماشین متن خام را نمی‌فهمند. برای هر یک از 100 پاراگراف نامزد شده در مرحله قبل، یک بردار ویژگی $x_{(q,d)}$ ساخته می‌شود. این بردار، کیفیت ارتباط آن پاراگراف با پرس‌وجو را از ده‌ها بُعد مختلف اندازه‌گیری می‌کند. در نهایت، ما 100 بردار ویژگی متراکم داریم.

4. فاز دوم: رتبه‌بندی مجدد (Precision Phase):

استفاده از مدل Machine-Learned Scoring

تکنولوژی: مدل یادگیری زوجی Pairwise SVM (مانند RankSVM).

منطق عملیاتی: این مدل سنگین که روی داده‌های رفتار کاربران و داوران حقوقی آموزش دیده است، وارد عمل می‌شود. الگوریتم RankSVM بردارهای ویژگی این 100 سند را به صورت جفت‌جفت (مثلاً سند A در برابر سند B) با هم مقایسه می‌کند تا ببیند احتمال برتری کدامیک بیشتر است. سپس یک لیست مرتب‌شده‌ی جدید از این 100 سند تولید می‌کند که مبتنی بر «کیفیت و بافتار» است، نه صرفاً تکرار کلمات.

5. خروجی نهایی (QA Output):

سیستم، پاراگرافی که در مرحله رتبه‌بندی مجدد به رتبه اول (Rank 1) رسیده است را استخراج کرده و به عنوان پاسخ نهایی و دقیق به کاربر نمایش می‌دهد.

بخش 2: کالبدشکافی مرحله تولید ویژگی (Feature Engineering)

قلب تپنده‌ی این سیستم در فاز دوم، کیفیت ویژگی‌هایی است که برای RankSVM تولید می‌کنیم. برای یک سیستم Q&A حقوقی، ویژگی‌ها باید فراتر از آمار ساده باشند. بردار ویژگی ما شامل 4 دسته اطلاعات است:

نوع ویژگی	نمونه ویژگی در سیستم	هدف از استخراج
Statistical	امتیاز خام BM25 و TF-IDF	حفظ لنگرگاه کلاسیک و درک کلمات نادر
Structural	مجاورت کلمات در متن	تضمین پیوستگی کلمات کوئری در متن
Domain-Specific	تطابق موجودیت‌های حقوقی	بررسی همخوانی مواد قانونی و تبصره‌ها
Quality / Prior	PageRank یا اعتبار منبع	ترجیح منابع معتبر (مانند دیوان عالی)

بخش 3: چرا Pairwise SVM برای فاز دوم بهترین انتخاب است؟

در یک سیستم پرسش و پاسخ، کاربر به نمره خام سند (مثلاً اینکه سند نمره 3.8 گرفته است) اهمیتی نمی‌دهد؛ کاربر فقط می‌خواهد «بهترین» پاراگراف در صدر نتایج باشد.

اگر از روش نقطه‌ای (Pointwise) استفاده می‌کردیم، سیستم تلاش می‌کرد نمره مطلق تک‌تک اسناد را تخمین بزند که به دلیل تفاوت مقیاس کوئری‌های کوتاه و بلند، دچار خطای فاحش می‌شد (پدیده گره‌خوردگی رتبه‌ها). اما در روش زوجی

(مانند RankSVM)، سیستم مستقیماً اختلاف بردارهای ویژگی ($\mathbf{x}_A - \mathbf{x}_B$) را ارزیابی می‌کند.

نتیجه‌گیری: به عبارت دیگر، الگوریتم RankSVM در فاز دوم دائماً از خود می‌پرسد: «آیا پاراگراف A پاسخ بهتری از پاراگراف B برای این سوال خاص است؟» این رویکرد مقایسه‌ای در یک فضای محدود (100 سند)، دقیقاً همان مکانیزمی است که به موتورهای جستجوی مدرن اجازه می‌دهد مرتبط‌ترین پاسخ را در کسری از ثانیه شناسایی کنند.

پاسخنامه مسئله 15.32: طراحی جامع رتبه‌بند ترکیبی جستجو

(Comprehensive Hybrid Ranker)

مقدمه و هدف مسئله

در یک فروشگاه اینترنتی واقعی، رتبه‌بندی نتایج جستجو نمی‌تواند تنها بر پایه تطابق متنی (ارتباط) یا تنها بر پایه محبوبیت (فروش) باشد. اگر فقط متن را در نظر بگیریم، ممکن است یک محصول نامعتبر اما با توضیحات طولانی در صدر قرار گیرد. اگر فقط فروش را در نظر بگیریم، محصولات جدید و مرتبط شانس برای دیده شدن ندارند. هدف این مسئله، طراحی یک سیستم «یادگیری برای رتبه‌بندی» (Learning to Rank) است که سیگنال‌های متنی کلاسیک را با سیگنال‌های تجاری به صورت ریاضی ترکیب کرده و از الگوریتم RankSVM برای یادگیری وزن بهینه هر ویژگی استفاده کند.

بخش 1: طراحی بردار ویژگی ترکیبی (Hybrid Feature Vector)

برای اینکه مدل یادگیری ماشین بتواند ارتباط یک محصول (d) با جستجوی کاربر (q) را درک کند، تمام اطلاعات ناهمگون باید در قالب یک بردار ویژگی واحد $\mathbf{x}^{(q,d)}$ ترجمه شوند.

در این سیستم، ما یک بردار 4-بعدی طراحی می‌کنیم که 2 بُعد آن وابسته به کوئری (Query-dependent) و 2 بُعد دیگر مستقل از کوئری و متکی بر اعتبار تجاری محصول (Query-independent) هستند:

$$\mathbf{x}^{(q,d)} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

بُعد	نام ویژگی	نحوه محاسبه ریاضی	ماهیت
x_1	تطابق عنوان	شباهت کسینوسی بین بردار TF-IDF کوئری و عنوان	متنی (بازه 0 تا 1)
x_2	تطابق توضیحات	شباهت کسینوسی بین بردار TF-IDF کوئری و توضیحات	متنی (بازه 0 تا 1)
x_3	حجم فروش	$\log(1 + \text{Sales})$	تجاری (اعداد پیوسته)
x_4	رضایت کاربر	میانگین امتیازات (ستاره‌ها) تقسیم بر 5	تجاری (بازه 0 تا 1)

نکته: استفاده از لگاریتم برای میزان فروش (ویژگی x_3) الزامی است. تفاوت ارزش بین 10 فروش و 100 فروش بسیار مهم است، اما تفاوت بین 10,010 فروش و 10,100 فروش اهمیت چندانی در رتبه‌بندی ندارد. لگاریتم این توزیع دم‌سنگین (**Heavy-tailed Distribution**) را متعادل می‌کند.

بخش 2: تابع هزینه (Loss Function) در مدل RankSVM

برای اینکه محصولاتی که هم ارتباط متنی بالا و هم فروش زیادی دارند در صدر قرار گیرند، ما مسئله را به صورت یادگیری زوجی (**Pairwise**) فرمول‌بندی می‌کنیم.

فرض کنید بر اساس داده‌های کلیک کاربران، می‌دانیم که برای یک جستجوی خاص، محصول A به محصول B ترجیح داده شده است ($A \succ B$). مدل باید بردار وزنی ($\mathbf{w}$) پیدا کند که امتیاز محصول A اکیداً بیشتر از B شود:

$$\mathbf{w}^T \mathbf{x}_A > \mathbf{w}^T \mathbf{x}_B \implies \mathbf{w}^T (\mathbf{x}_A - \mathbf{x}_B) > 0$$

برای آموزش این مدل، تابع هزینه حاشیه نرم (**Hinge Loss**) در الگوریتم RankSVM به صورت زیر تعریف می‌شود:

$$L(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{(i,j) \in P} \max(0, 1 - \mathbf{w}^T (\mathbf{x}_i - \mathbf{x}_j))$$

تحلیل اجزای فرمول:

(تنظیم‌گر) $\frac{1}{2} \|\mathbf{w}\|^2$: از بزرگ شدن بیش از حد وزن یک ویژگی (مثلاً اتکای مطلق سیستم فقط به تعداد فروش) جلوگیری کرده و مدل را در برابر **Overfitting** مقاوم می‌کند.

(زیان لولا) $\max(0, 1 - \dots)$: اگر مدل به درستی محصول i را حداقل با اختلاف حاشیه‌ی 1 واحد بالاتر از محصول j رتبه‌بندی کند، مقدار زیان 0 می‌شود. اما اگر محصول j (محصول نامربوط یا کم‌فروش) امتیاز بالاتری بگیرد، تابع به شدت جریمه می‌شود.

$\sum_{(i,j) \in P}$: یادگیری بر روی تمام جفت‌محصولاتی که ترجیح کاربر در آن‌ها مشخص است (مثلاً محصول کلیک‌شده در برابر محصول نادیده‌گرفته‌شده) انجام می‌شود.

بخش 3: اهمیت حیاتی استانداردسازی ویژگی‌ها (Feature Scaling) پیش از آموزش

پیش از اینکه بردار $\mathbf{x}^{(q,d)}$ را به تابع هزینه SVM تزریق کنیم، یک گام الزامی وجود دارد: استانداردسازی (**Normalization** / **Standardization**). عدم انجام این کار، سیستم رتبه‌بندی را به طور کامل فلج خواهد کرد.

دلیل این اهمیت بنیادین چیست؟

الگوریتم ماشین بردار پشتیبان (**SVM**) نسبت به «مقیاس ویژگی‌ها» (**Scale of Features**) به شدت حساس است؛ زیرا تابع هزینه آن مبتنی بر محاسبه فاصله هندسی و ضرب داخلی است.

در بردار طراحی‌شده‌ی ما، مقیاس ویژگی‌ها بسیار متفاوت است:

- تطابق کسینوسی عنوان (x_1) عددی بین 0 و 1 است.

- امتیاز کاربران (x_4) عددی بین 0 و 1 است (پس از تقسیم بر 5).
- اما لگاریتم فروش (x_3) ممکن است عددی بین 0 تا 10 یا بیشتر باشد. در صورت عدم استفاده از لگاریتم، این عدد می‌توانست هزاران باشد.

جمع‌بندی:

اگر این مقیاس‌ها هم‌سان نشوند (مثلاً با تبدیل آن‌ها به میانگین 0 و واریانس 1 از طریق **Z-score**)، ویژگی «فروش» به دلیل داشتن مقادیر عددی بزرگ‌تر، در محاسبات ریاضی ضرب داخلی غلبه پیدا می‌کند. در نتیجه، الگوریتم تصور می‌کند که x_3 مهم‌ترین ویژگی است و بردار وزن $\mathbf{w}$ را به گونه‌ای تنظیم می‌کند که سیستم تبدیل به یک ماشین «مرتب‌سازی صرفاً بر اساس فروش» شود و سیگنال‌های ظریف متنی (ارتباط کوئری با عنوان) به طور کامل در تابع هزینه محو شوند. **استانداردسازی، شرایط عادلانه‌ای فراهم می‌کند تا الگوریتم ارزش واقعی هر ویژگی را کشف کند.**